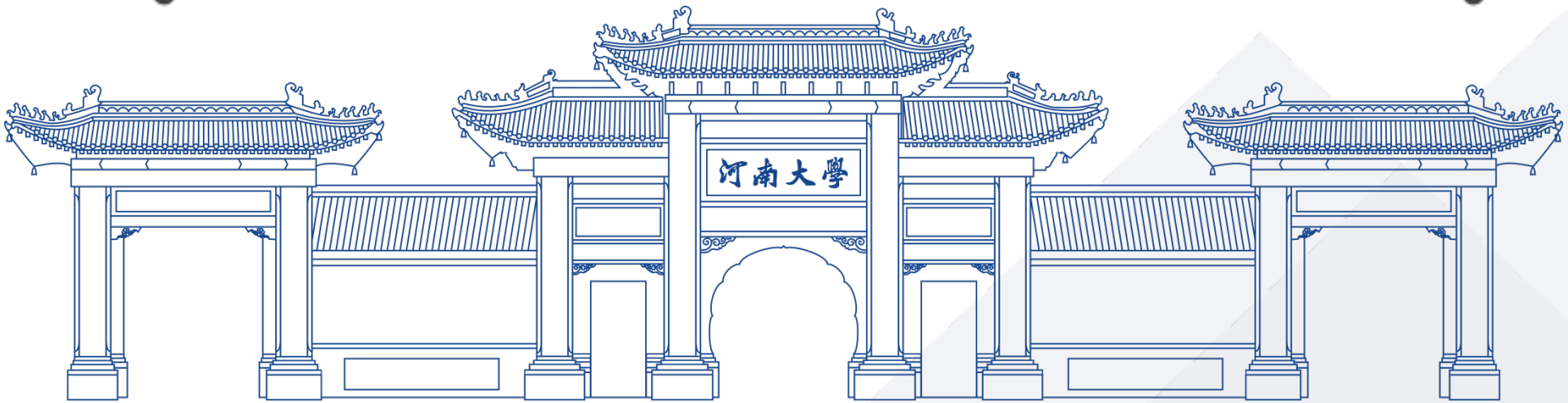
河南大學
Henan University

第1章 移动应用开发技术基础入门

软件学院 · 刘俊男

2025年2月18日



学习目标/Target

- 了解1G~5G的通信技术，能够说出1G~5G技术的发展内容
- 掌握DevEco Studio开发环境的搭建步骤，能够独立搭建Harmony开发环境
- 掌握编写简单OpenHarmony程序的步骤，能够编写一个Hello World程序
- 掌握资源的管理与使用方式，能够灵活使用程序中的资源
- 掌握单元测试与Logcat的使用，能够完成对程序的调试

章节概述/ Summary

Harmony与Android是基于LiteOS与Linux平台开发的手机及平板电脑的操作系统，它们自问世以来，受到了前所未有的关注，并迅速成为国内外移动平台最受欢迎的操作系统。Harmony与Android手机随处可见，如果能加入Harmony与Android开发者行列，编写自己的应用程序供别人使用，想必是件诱人的事情。那么从今天开始，我们将开启Harmony与Android开发之旅，并逐渐成为一名出色的Harmony与Android开发者。

目录/Contents

01 Harmony与OpenHarmony简介

02 OpenHarmony开发环境搭建

03 开发第一个OpenHarmony程序

04 OpenHarmony程序结构

目录/Contents

05 资源的管理与使用

06 程序调试

河南大學
Henan University

1.1 移动应用开发技术简介

1.1 Harmony与Android简介

- 了解1G~5G的通信技术，能够说出1G~5G技术的发展内容
- 了解OpenHarmony与Android的发展历史，能够说出OpenHarmony与Android各版本对应的系统名称和图标
- 了解OpenHarmony的体系结构，能够说出OpenHarmony系统的4种分层结构
- 了解虚拟机，能够说出虚拟机编译文件的过程

1.1.1 通信技术

第一代通信技术（1G）：是指最初的模拟、仅限语音的蜂窝电话标准。

第二代通信技术（2G）：是指第2代移动通信技术，代表为GSM，以数字语音传输技术为核心。传输速度9.6k/s。

第三代通信技术（3G）：是指将无线通信与国际互联网等多媒体通信结合的新一代移动通信系统。3G通信网在室内、室外和行车的环境中能够分别支持至少2M/s、384K/s以及144K/s的传输速度。

第四代通信技术（4G）：又称IMT-Advanced技术，它包括了TD-LTE 和FDD-LTE。4G通信网最高甚至可以达到100M/s的传输速度。

第五代通信技术（5G）：传输速度可达20Gbps。

1.1.2 Harmony与Android发展历史

Android操作系统最初是由安迪·鲁宾（Andy Rubin）开发出的，后来被Google收购，并于2007年11月5日正式向外界展示了这款系统。随后Google以Apache开源许可证的授权方式，发布了Android操作系统的源代码。

开放原子开源基金会
OPENATOM FOUNDATION

开源鸿蒙是鸿蒙操作系统的开源项目，由开放原子开源基金会运作，华为是OpenHarmony项目的主要贡献者，也是项目的支持者和使用者。一个智能终端设备操作系统的框架和平台，实现不同设备之间的互联互通和数据共享，旨在促进万物互联产业的繁荣发展。
开源四年多来，开源鸿蒙已汇聚8100多名贡献者和70多家单位持续共建，项目代码量累计超过1.2亿行。目前，以开源鸿蒙为底座的生态设备突破10亿。

1.1.2 Android发展历史

2008年9月发布Harmony与Android**第1个**版本Harmony与Android1.1。

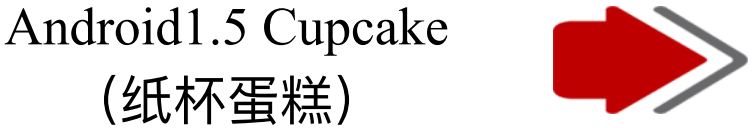
2009年4月30日，Harmony与Android1.5 **Cupcake**（纸杯蛋糕）正式发布。

2009年9月5日，Harmony与Android1.6 **Donut**（甜甜圈）版本发布。

• • •

.....

Android1.5 Cupcake
（纸杯蛋糕）



Android9.0 Android pie
（派）

1.1.2 OpenHarmony发展历史

鸿蒙操作系统 1.0

OpenHarmony最初被称为鸿蒙系统， 在2019年8月，鸿蒙操作系统 1.0 版本首次亮相。

OpenHarmony 1.0

2020年9月份，华为向开放原子开源基金会捐赠了鸿蒙操作系统的首批基础能力相关代码并随后开源。

OpenHarmony 2.0

2021年5月份，华为再次把鸿蒙操作系统的核心基础能力全部捐献给了开放原子开源基金会并由基金会对外开源。

OpenHarmony 3.0

2021年9月份，开放原子开源基金会发布了 OpenHarmony 3.0 版本，这个版本开放了 OpenHarmony 标志性的分布式能力。

OpenHarmony 3.1 Release

2022年3月份，开放原子开源基金会正式发布 OpenHarmony 3.1 Release 版本，该版本的发布是一个里程碑事件，标志着 OpenHarmony 基本具备了带屏设备的产品能力，OpenHarmony 3.1 版本为上手机、PC 做好了准备。

OpenHarmony 3.2 Release

2023年2月正式发布，该版本系统已经足够流畅支持各行各业开发复杂带屏设备。

1.1.2 OpenHarmony发展历史

OpenHarmony 4.0

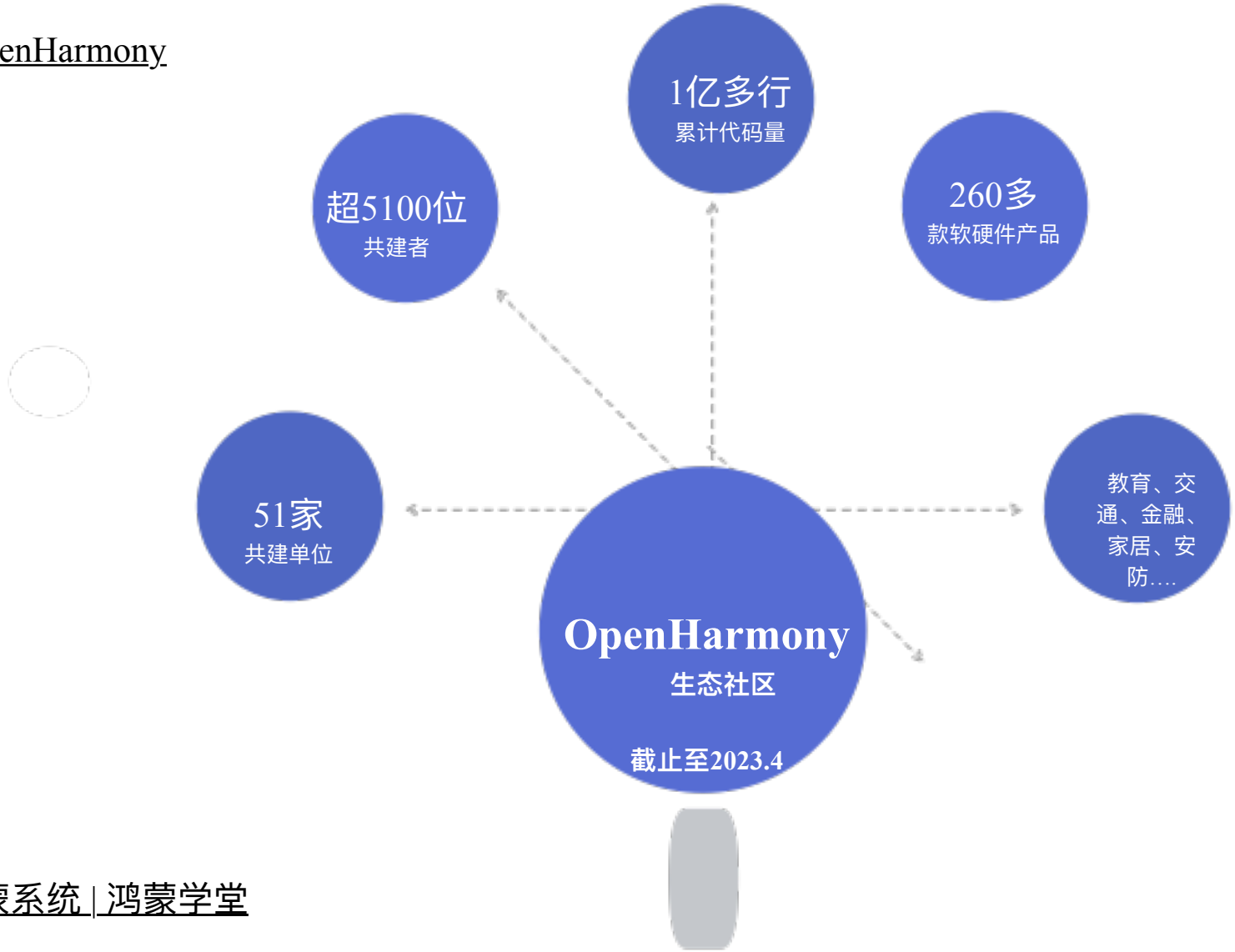
2023年6月，华为发布HarmonyOS 4.0开发者版本。2023年10月26日，OpenHarmony在 Gitee 发布了OpenHarmony 4.0 Release 版本更新，带来了标准系统能力的持续完善。4.0版本的推出，将为用户带来更加流畅、个性化的使用体验，同时也为开发者提供了更加丰富的开发资源和工具，我们有理由相信，OpenHarmony将在未来取得更加辉煌的成绩。

开源鸿蒙操作系统5.0 Release版重磅发布

开源鸿蒙操作系统5.0 Release版本在系统完备度、分布式创新、开发者体验以及系统稳定性等方面均实现了显著提升，标志着开源鸿蒙操作系统进入一个全新的发展阶段，将加速推动生态共建，助推应用落地，赋能各地方产业创新升级发展，加速实现万物互联。

OpenHarmony生态社区

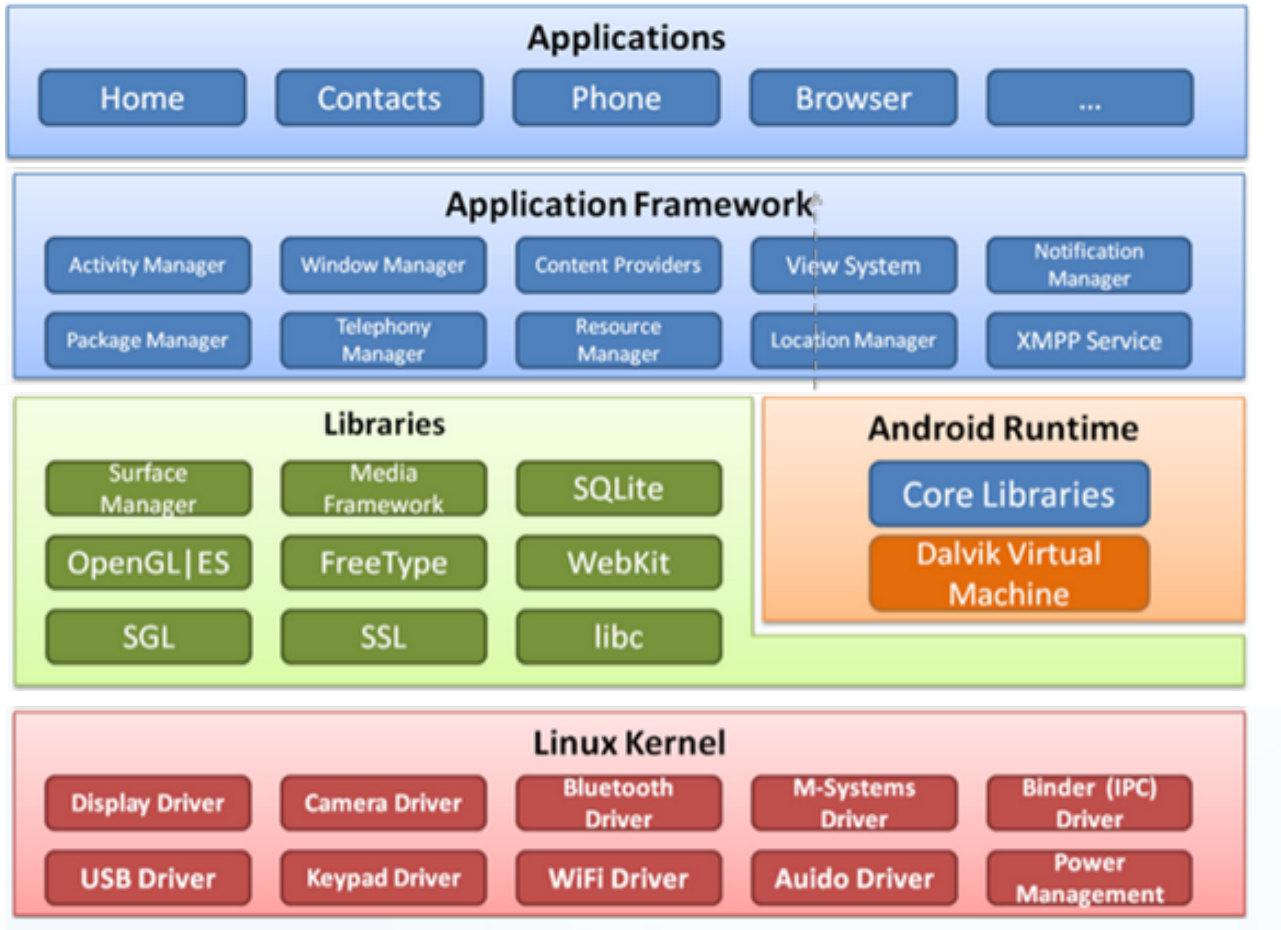
OpenAtom OpenHarmony



HarmonyOS 鸿蒙系统 | 鸿蒙学堂

OpenHarmony生态

1.1.3 Android体系结构



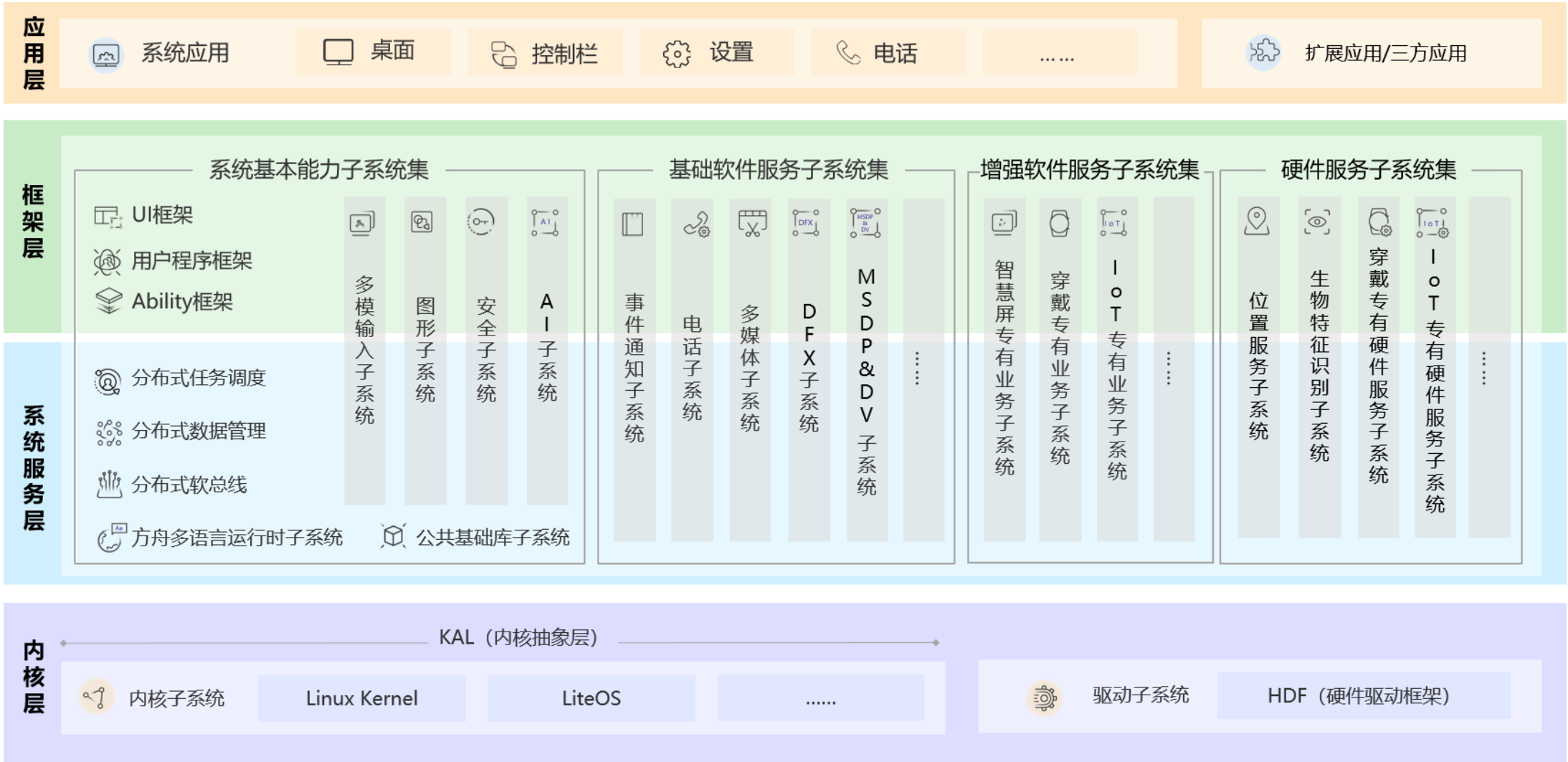
应用程序层

应用程序框架层

核心类库层

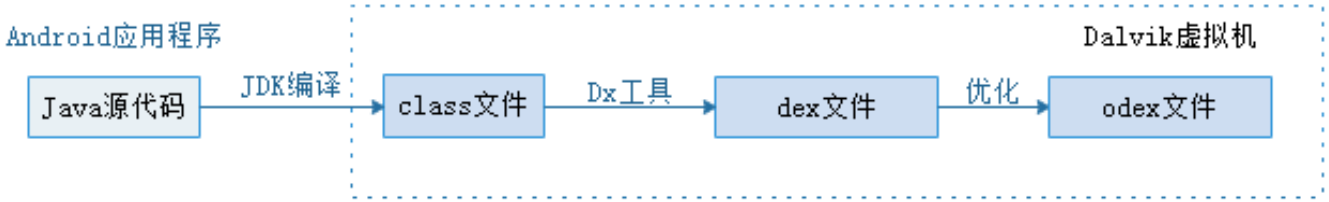
Linux内核层

1.1.3 HarmonyOS与OpenHarmony的技术架构



1.1.4 Dalvik虚拟机

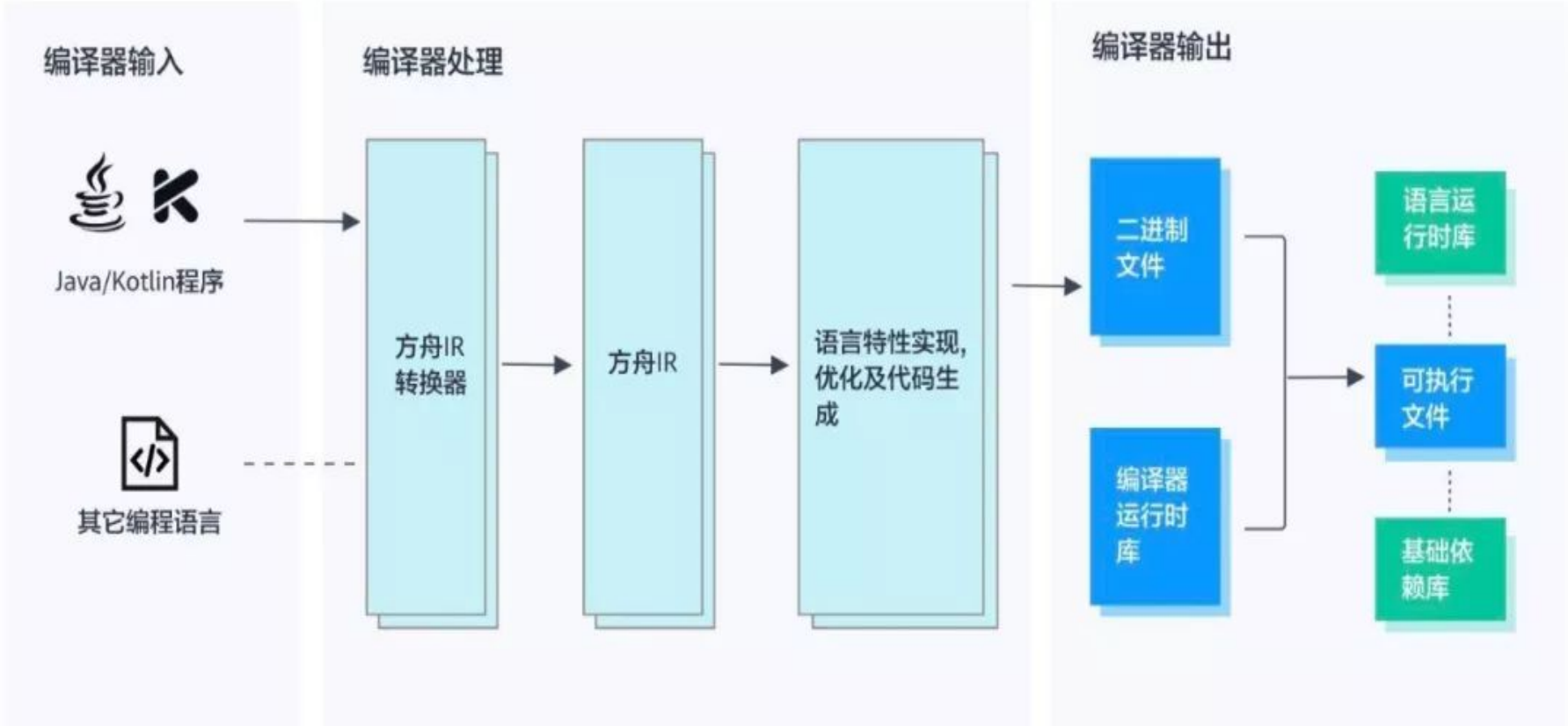
Dalvik是Google公司设计的，用于在Android平台上运行的虚拟机，其指令集基于寄存器架构，执行其特有的dex文件来完成对象生命周期管理、堆栈管理、线程管理、安全异常管理、垃圾回收等重要功能。每一个Android应用在底层都会对应一个独立的Dalvik虚拟机实例，其代码在虚拟机的解释下得以执行，Dalvik虚拟机编译文件的过程如下图所示。



Dalvik虚拟机编译文件过程

1.1.4 方舟编译器

方舟基础架构与 IR 中间表示目前开源的方舟基础架构一部分是关于 Java/Kotlin 的编程语言实现，因为这两类语言强依赖虚拟机，方舟编译器去掉了虚拟机之后，补全了一些功能在里面；另一部分是后续优化和分析的基础支持，比如开源了 SSA



河南大學
Henan University

1.2

OpenHarmony开发环境搭建

1.2 OpenHarmony开发环境搭建

技能目标

1. 能安装与配置DecEco Studio 4.0 Release开发工具
2. 能创建/预览并对应用进行签名运行
3. 能安装与配置OpenHarmony 4.0 Release
4. 能修改SDK的版本为OpenHarmony(API 10) 4.0 Release
5. 能修改项目的运行操作系统为OpenHarmony
6. 能直接创建API 10的OpenHarmony工程

1.2.1 OpenHarmony 安装

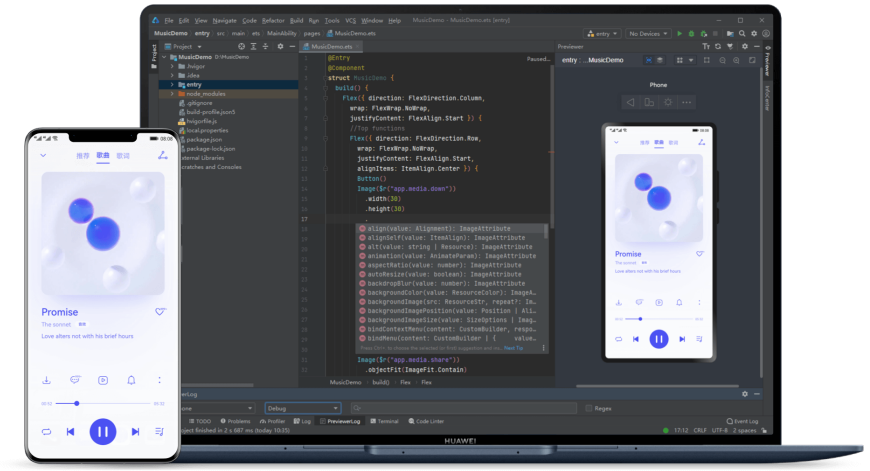
俗话说，“工欲善其事，必先利其器”。在开发Harmony与Android程序之前，先要搭建开发环境。OpenHarmony使用HUAWEI DevEco Studio（简称DevEco Studio），它是面向华为终端全场景多设备的一站式集成开发环境（IDE）。了创建工程、开发、编译、调试、发布等功能外，DevEco Studio还可以支持多设备开发，实时预览器/模拟器等。接下来，本节将针对OpenHarmony开发工具的环境搭建进行讲解。

1.2.1 OpenHarmony 安装

HUAWEI DevEco Studio（简称DevEco Studio）是面向华为终端全场景多设备的一站式集成开发环境（IDE），除了创建工程、开发、编译、调试、发布等功能外，DevEco Studio还可以支持多设备开发，实时预览器/模拟器等。

基于IntelliJ IDEA Community开源版本打造，为运行在HarmonyOS系统上的应用和元服务（以下简称应用/元服务）提供一站式的开发平台。

DevEco Studio支持多设备预览、模拟器运行、真机运行三种方式。



1.2.1 OpenHarmony 安装步骤

步骤1

步骤2

步骤3

下载OpenHarmony

OpenHarmony安装包可以从中文社区进行下载。这里我们以Windows 64系统为例，下载DevEco Studio 5.0.2版本。OpenHarmony下载页面如下图所示。

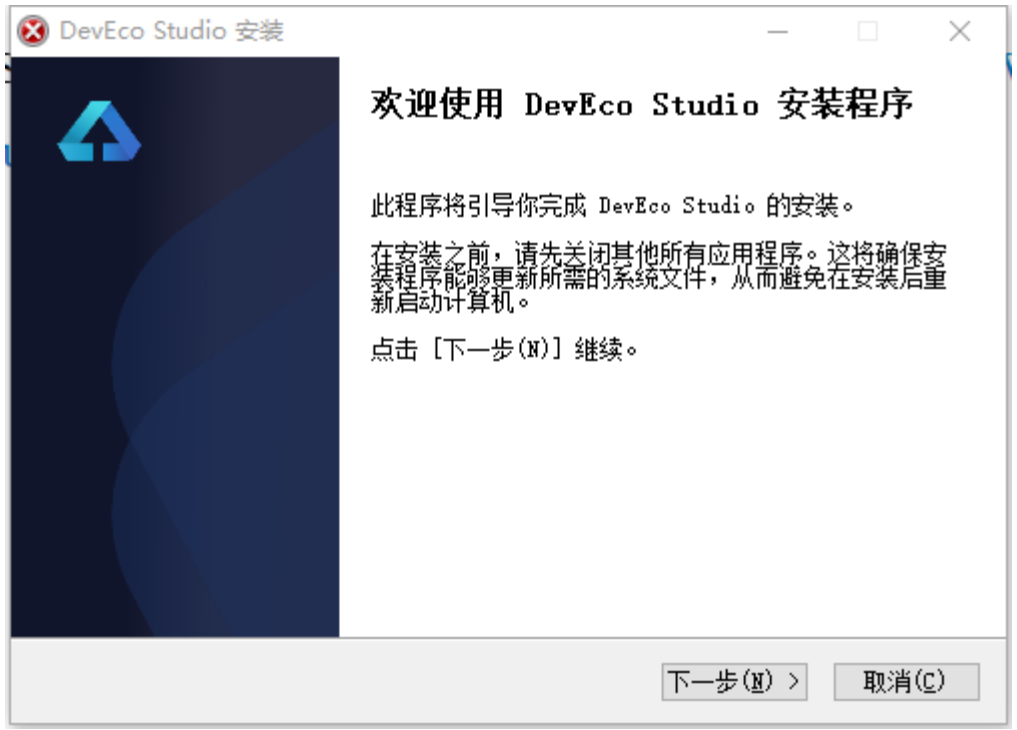
安装DevEco Studio

步骤1

成功下载DevEco Studio安装包后，双击后缀名为.exe的文件，进入Welcome to DevEco Studio Setup页面，如下图所示。

步骤2

步骤3



安装DevEco Studio

步骤1

步骤2

步骤3

单击上一页图中的“Next” 按钮，进入Choose Components页面，如下图所示。

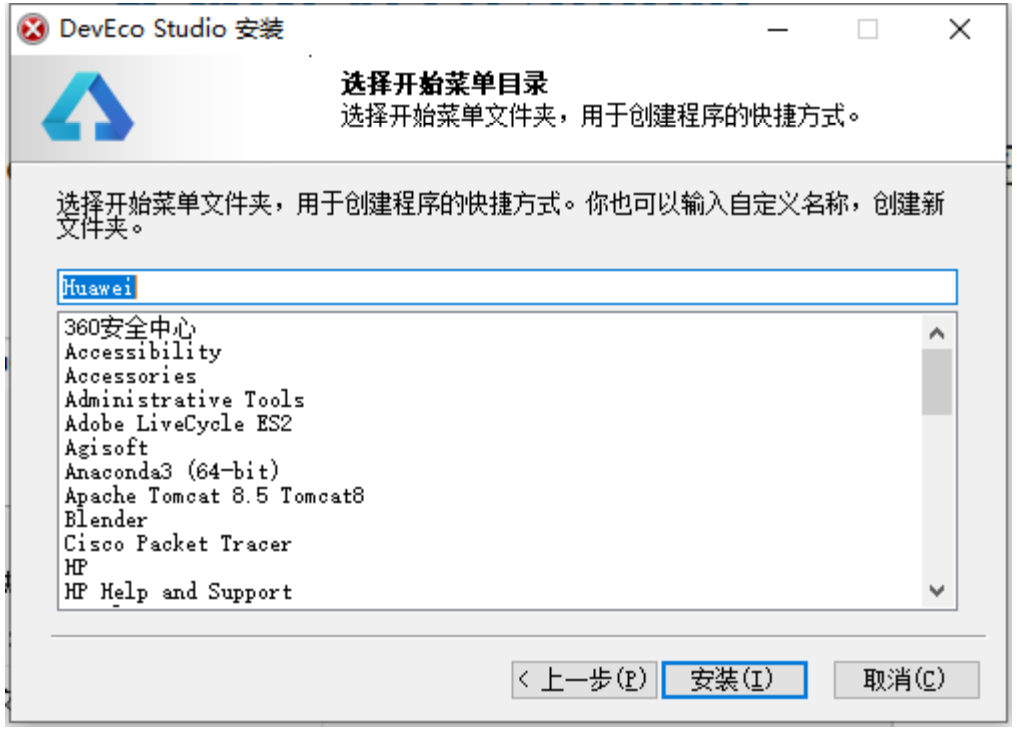
安装DevEco Studio

步骤1

步骤2

步骤3

单击上一页图中的“Next” 按钮，进入Choose Components页面，如下图所示。

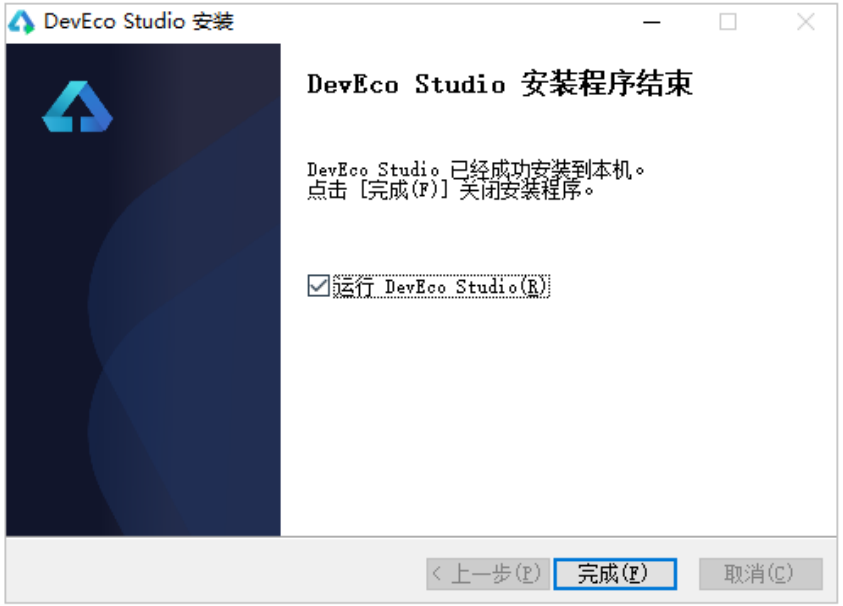


安装DevEco Studio

步骤1

安装完成后，单击上一页图中的“Next”按钮进入Completing DevEco Studio Setup页面，如下图所示。

步骤2



步骤3

单击图中的“完成”按钮，至此，DevEco Studio的安装全部完成。

步骤1

步骤2

步骤3

配置DevEco Studio

我们选择选应用程序图标进入之后会进入DevEco Studio的开启窗口，如下图所示。

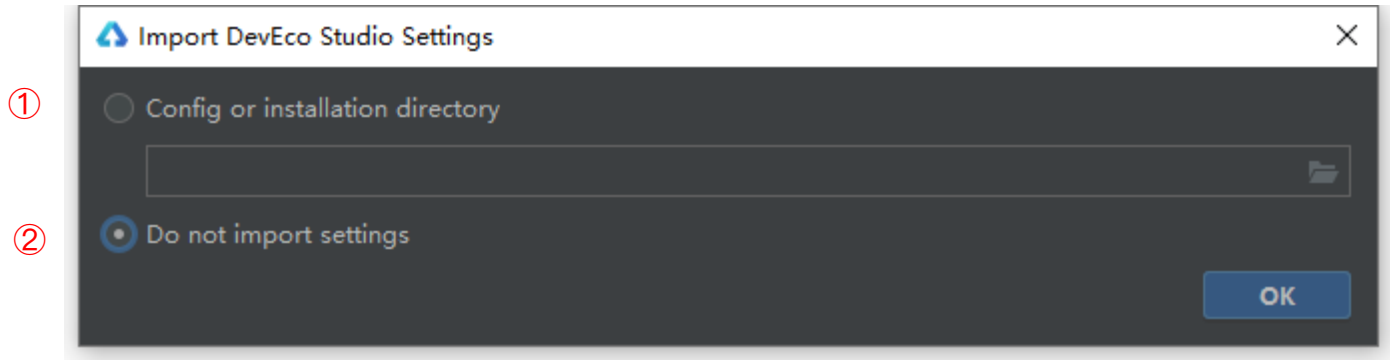
配置DevEco Studio

步骤1

图中包含2个选项，其中选项①表示自定义DevEco Studio配置文件的位置，选项②表示不导入配置文件的位置。如果之前安装过DevEco Studio，想要导入之前的配置文件，则可以选择选项①，否则，选择选项②，此处可以根据实际情况进行选择。

步骤2

步骤3



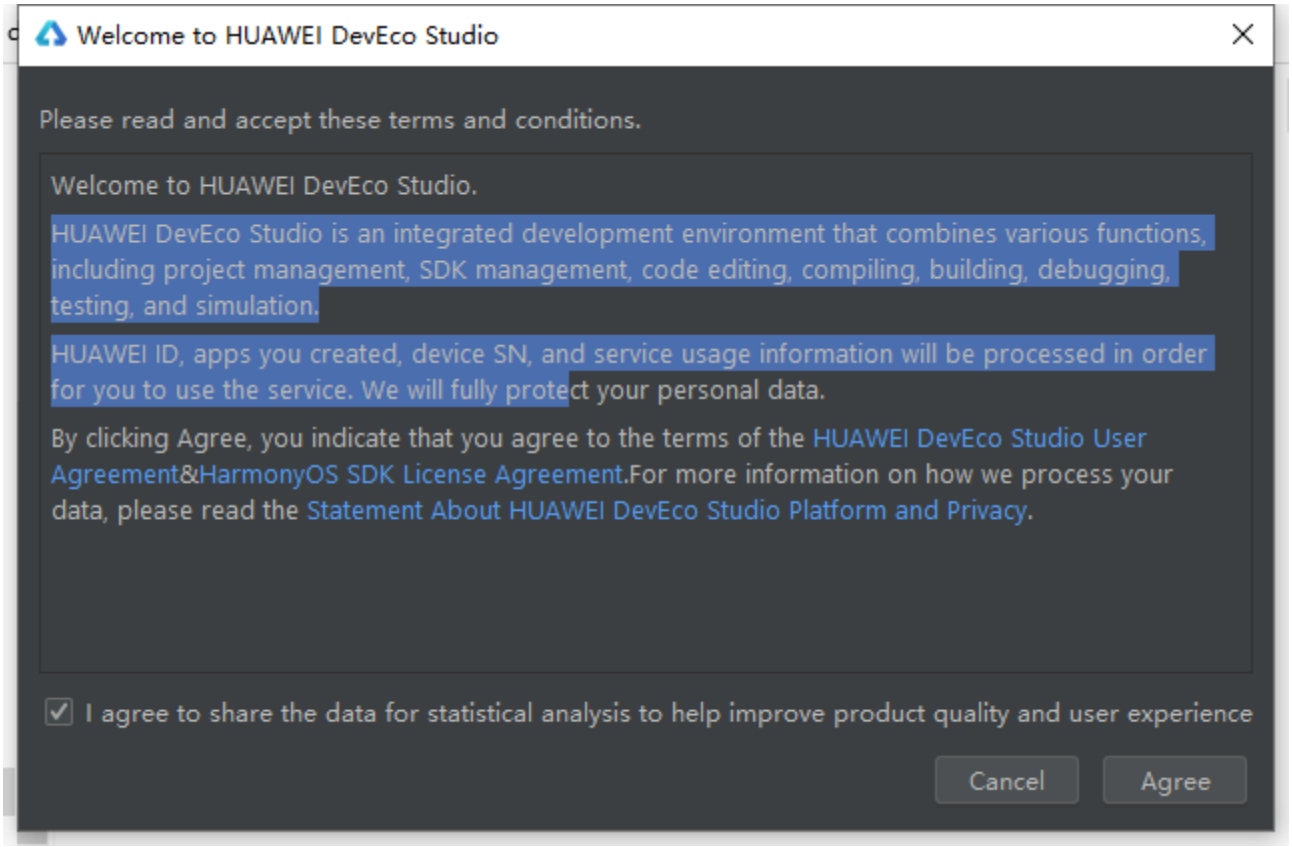
配置DevEco Studio

步骤1

步骤2

步骤3

上一页图中的进度完成之后，会弹出DevEco Studio First Run对话框，如下图所示。



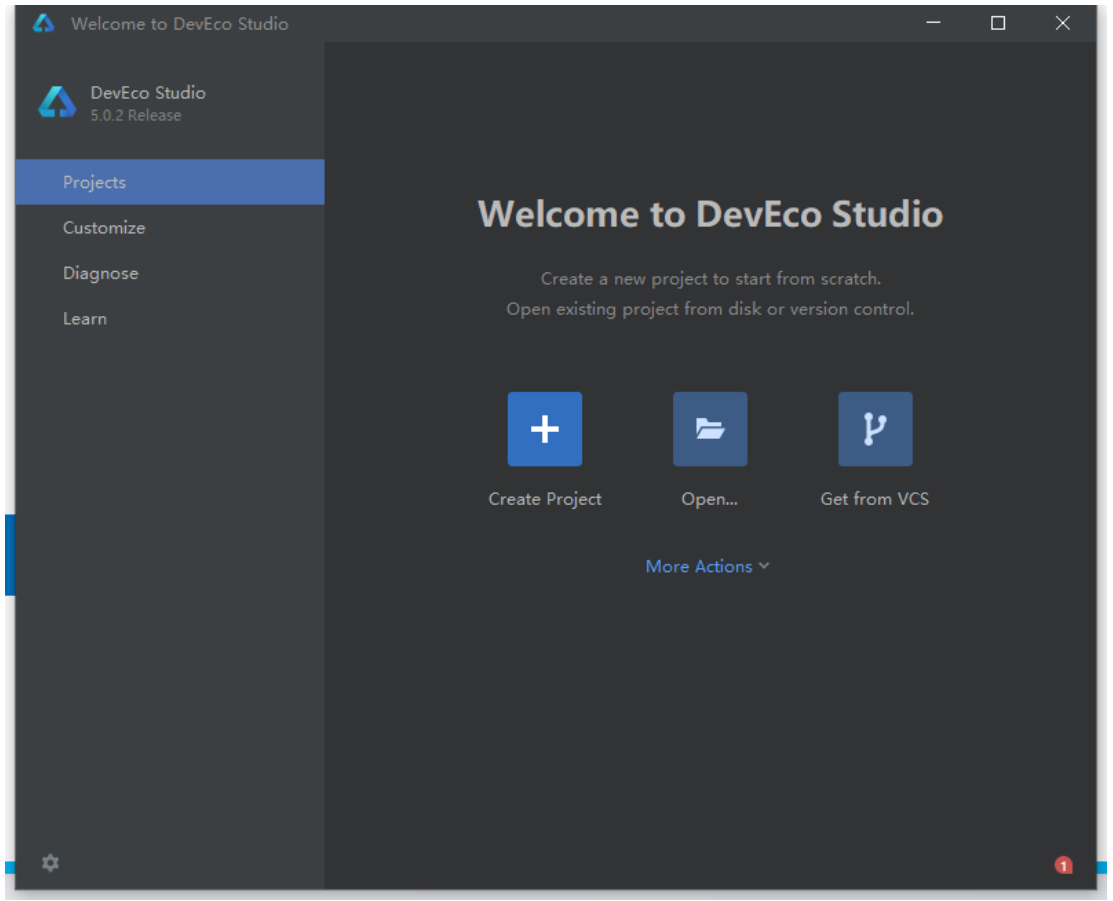
配置DevEco Studio

步骤1

步骤2

步骤3

上一页图中的进度完成之后，会弹出DevEco Studio First Run对话框，如下图所示。



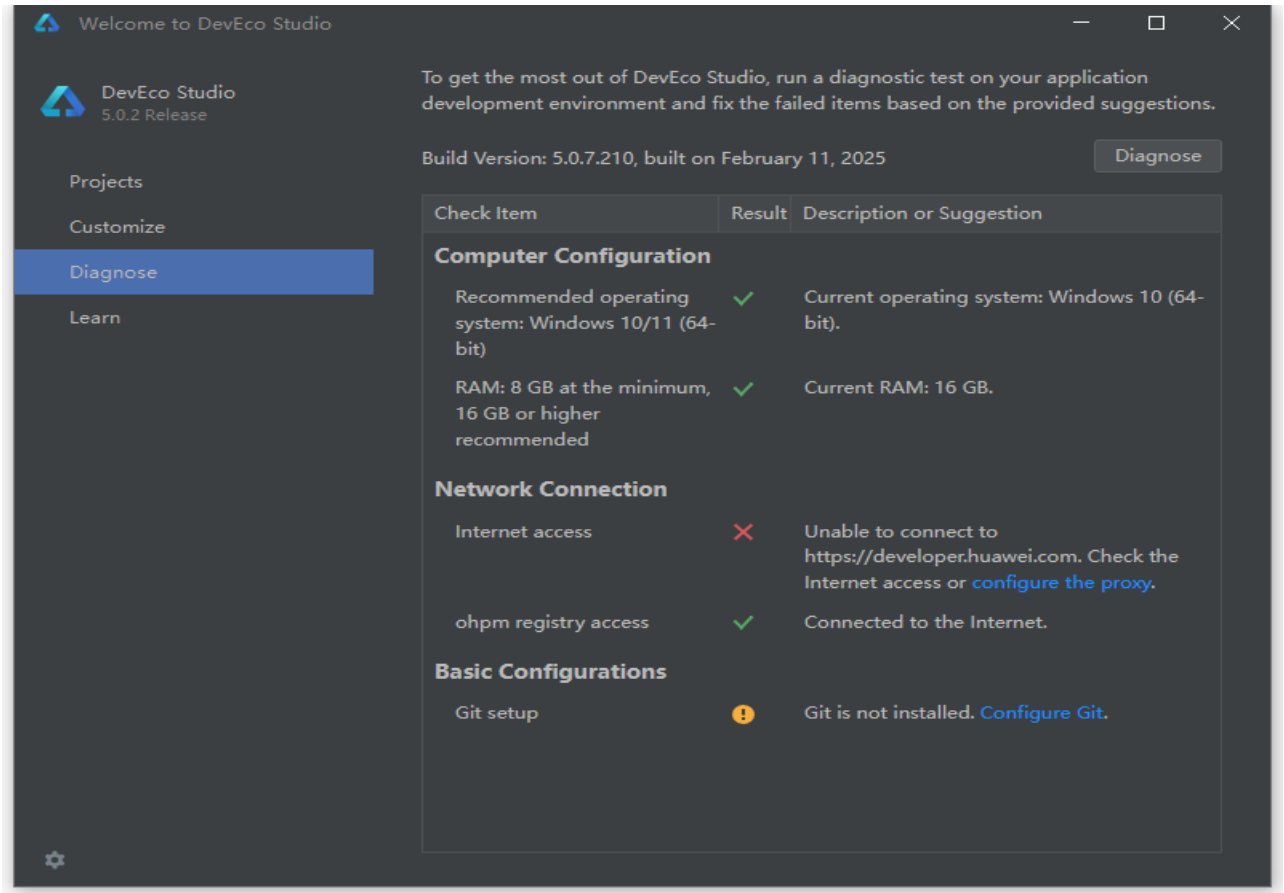
配置DevEco Studio

步骤1

步骤2

步骤3

DevEco Studio开发环境依赖于网络环境，需要连接上网络才能确保工具的正常使用。DevEco Studio提供了开发环境诊断的功能，帮助您识别开发环境是否完备。可以在欢迎页面单击Diagnose进行诊断。



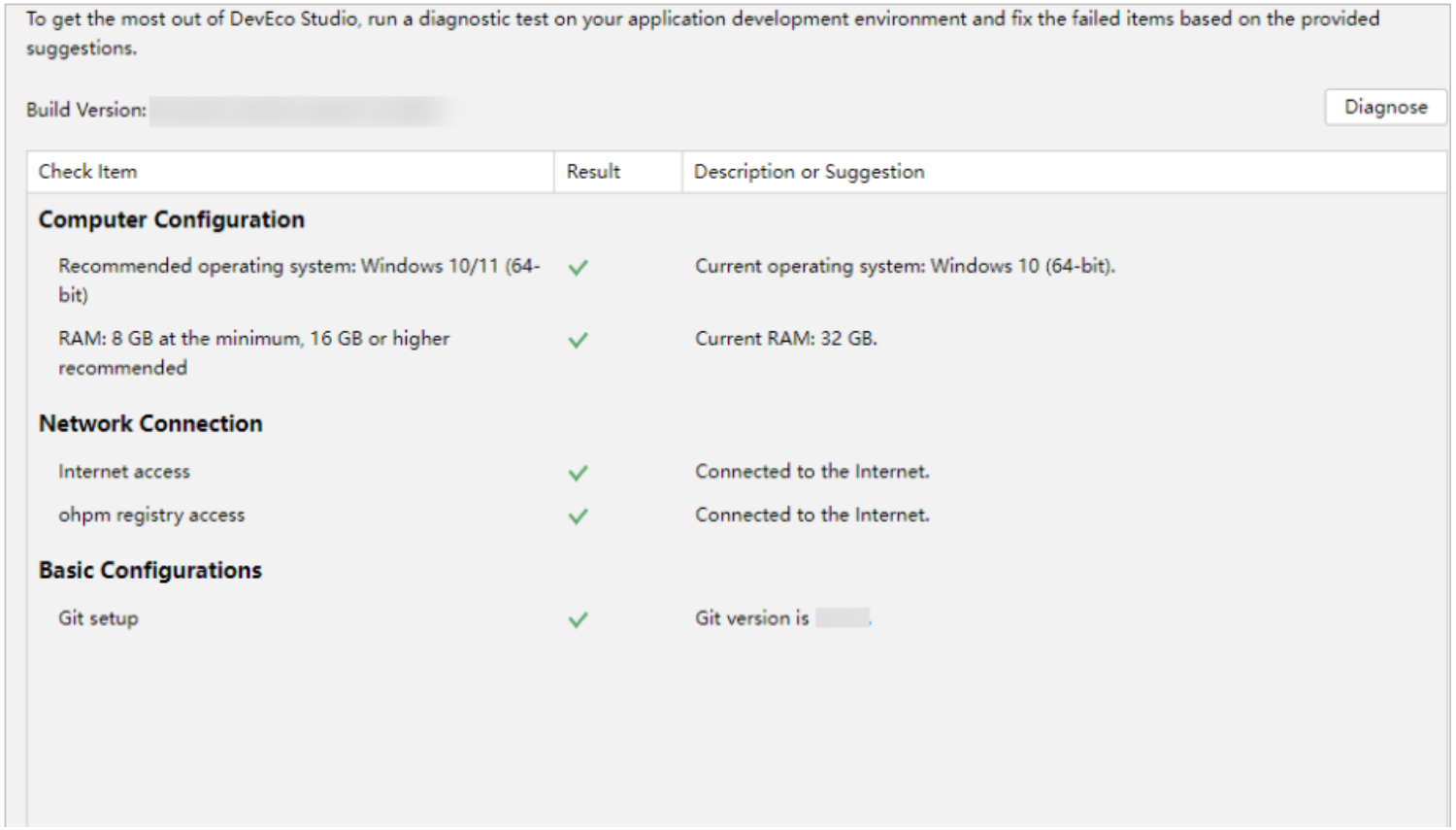
配置DevEco Studio

步骤1

步骤2

步骤3

如果您已经打开了工程开发界面，也可以在菜单栏单击Help > Diagnostic Tools > Diagnose Development Environment进行诊断。如下图所示。



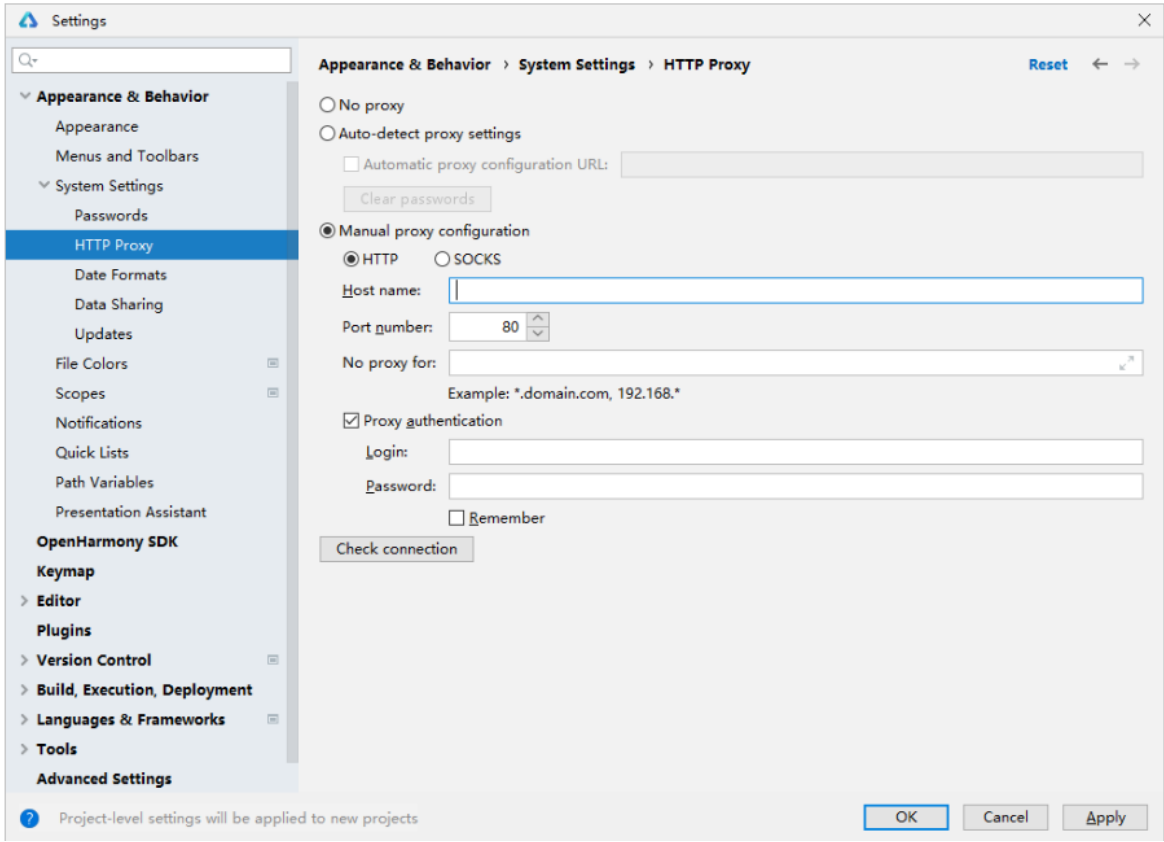
步骤1

步骤2

步骤3

配置DevEco Studio

在欢迎页单击Customize > All settings… > Appearance & Behavior > System Settings > HTTP Proxy进入HTTP Proxy设置界面。如果已经打开了工程，可以单击File > Settings > Appearance & Behavior > System Settings > HTTP Proxy进入HTTP Proxy设置界面



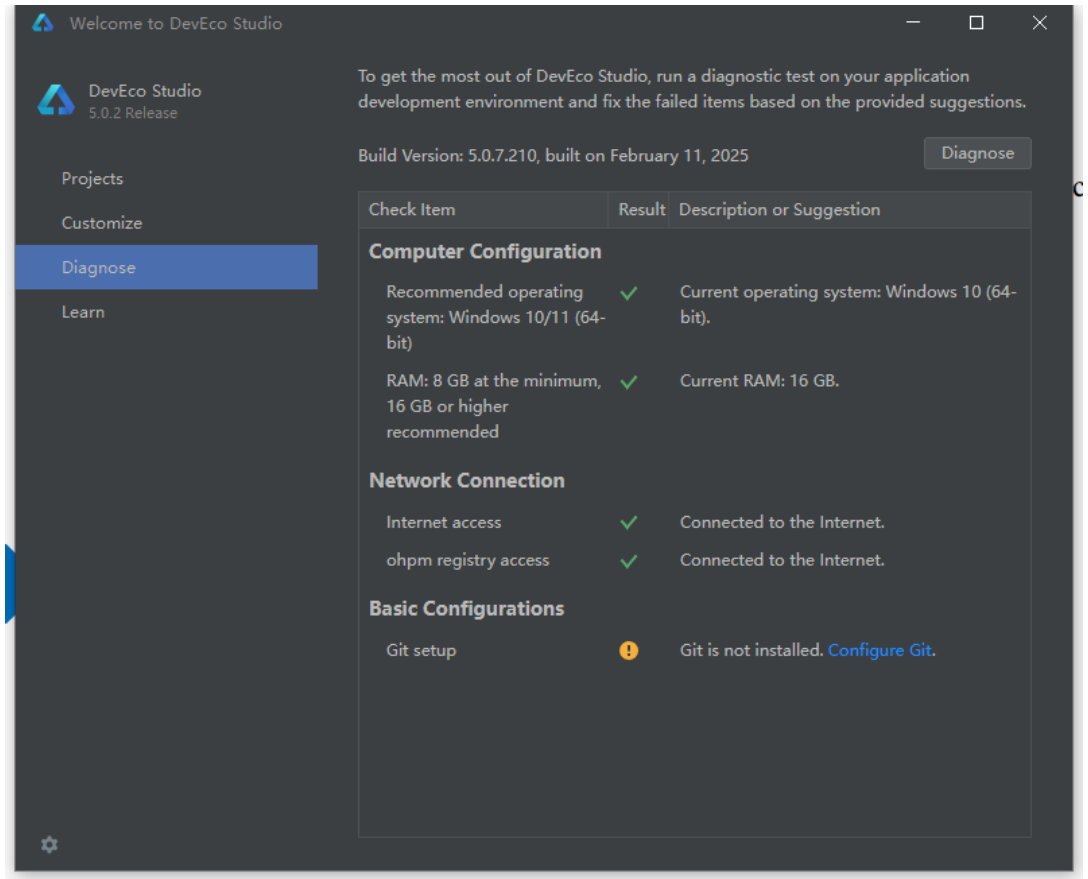
步骤1

步骤2

步骤3

配置DevEco Studio

配置完成后，单击Check connection，输入网络地址（如：https://developer.huawei.com），检查网络连通性。提示“Connection successful”表示代理设置成功



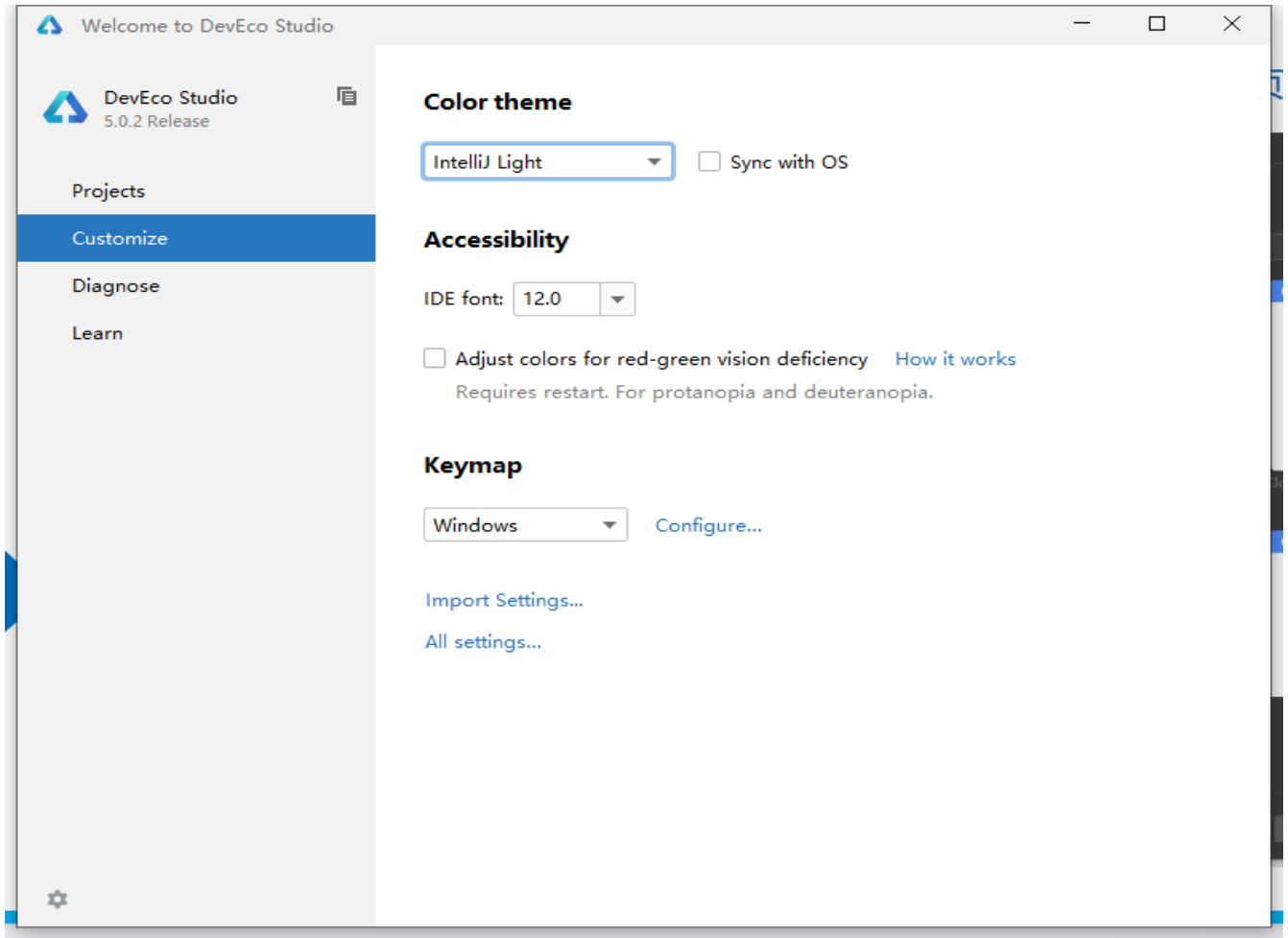
步骤1

步骤2

步骤3

配置DevEco Studio

配置完成后，单击Customize，选择Color theme 可以调整主题颜色。



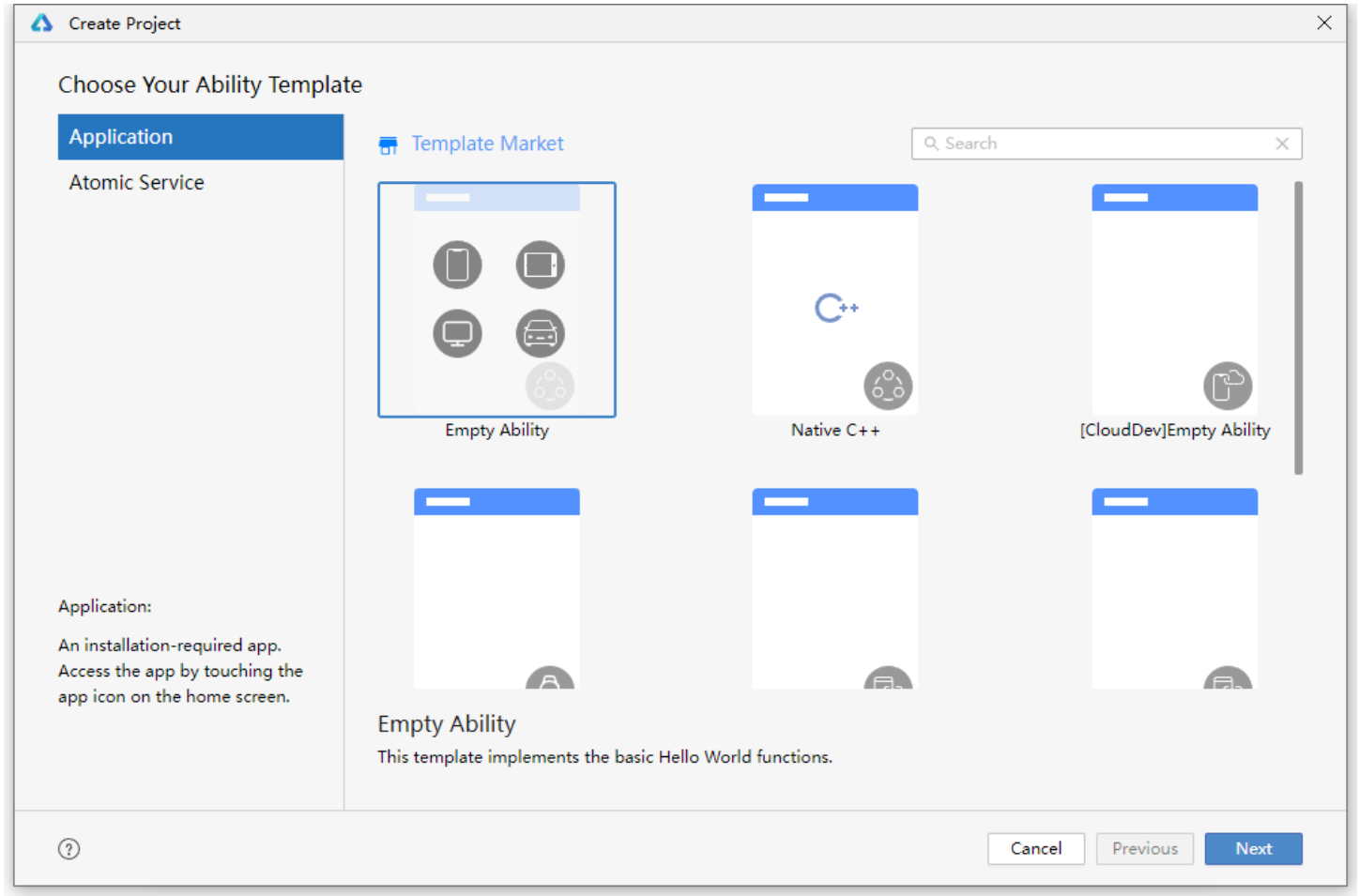
配置DevEco Studio

步骤1

步骤2

步骤3

单击上一页图中的“Cancel” 按钮之后进入Welcome DevEco Studio页面，如下图所示。



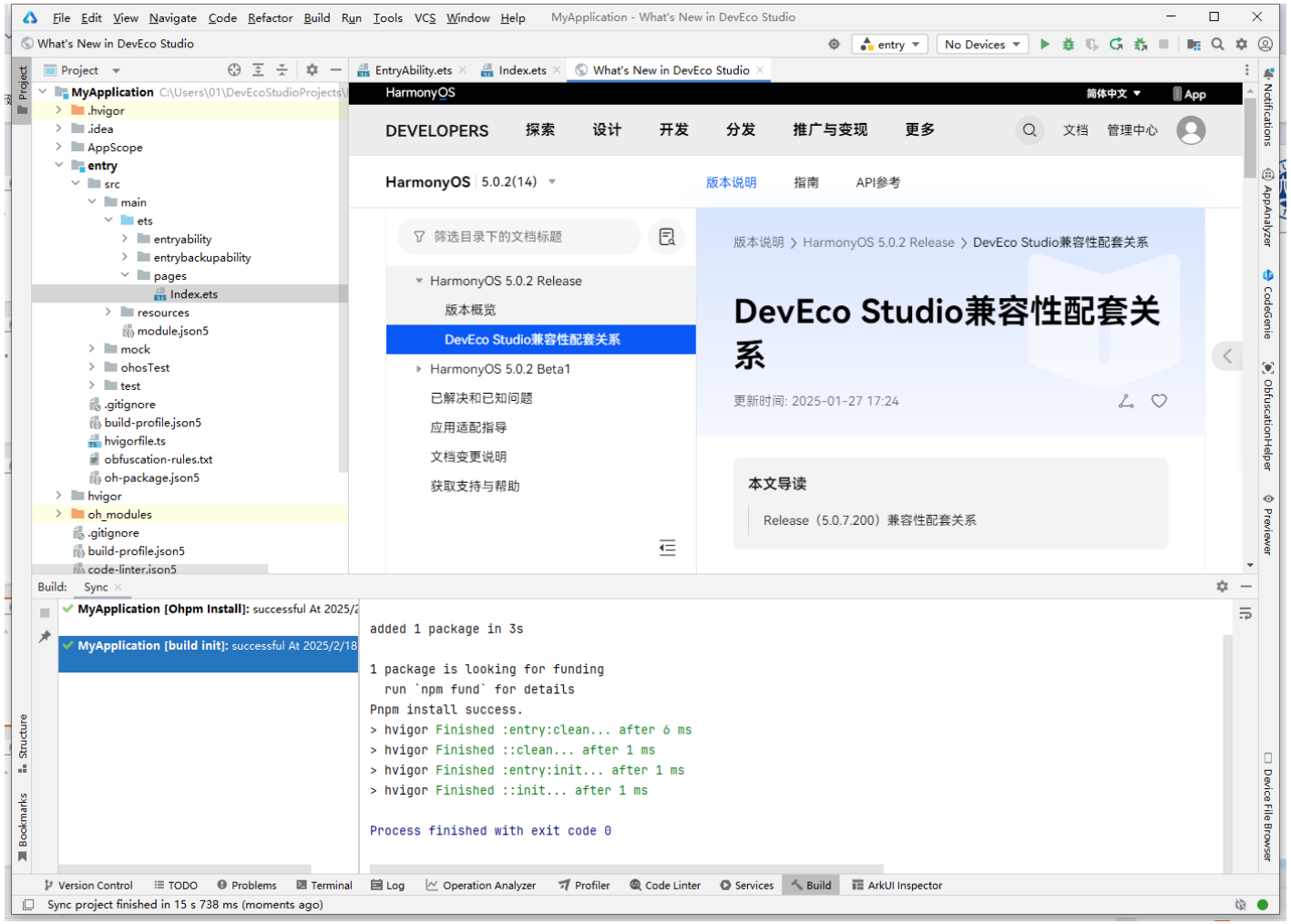
步骤1

步骤2

步骤3

配置DevEco Studio

单击上一页图中的“Next” 按钮之后进入主页面，如下图所示。



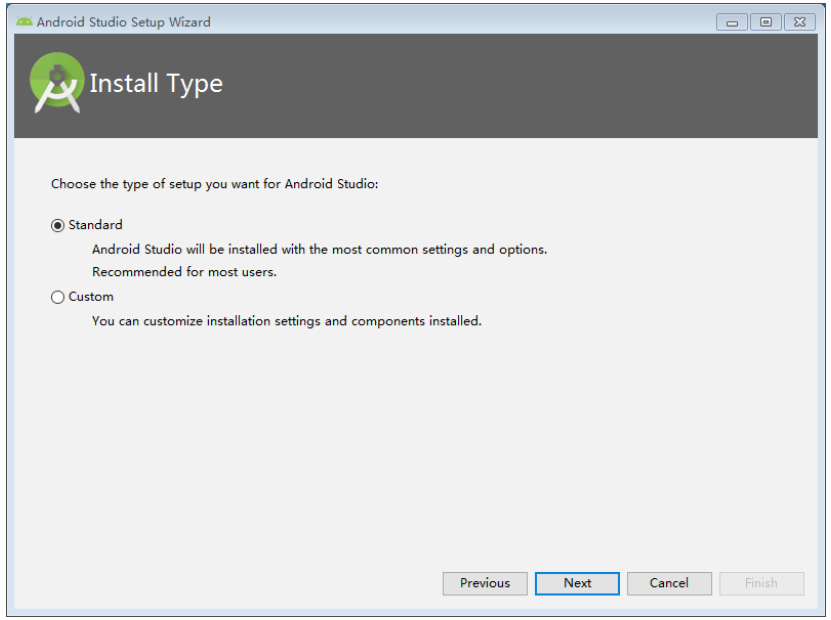
配置DevEco Studio

步骤1

步骤2

步骤3

单击上一页图中的“Next”按钮进入Install Type页面，如下图所示。



1.2.2 模拟器创建

OpenHarmony程序可以运行到手机和平板等物理设备上，当运行OpenHarmony程序时，没有手机或平板等物理设备，可以使用OpenHarmony系统提供的模拟器。模拟器是一个可以运行在电脑上的虚拟设备。在模拟器上可预览和测试OpenHarmony应用程序。

单击ADV Manager标签

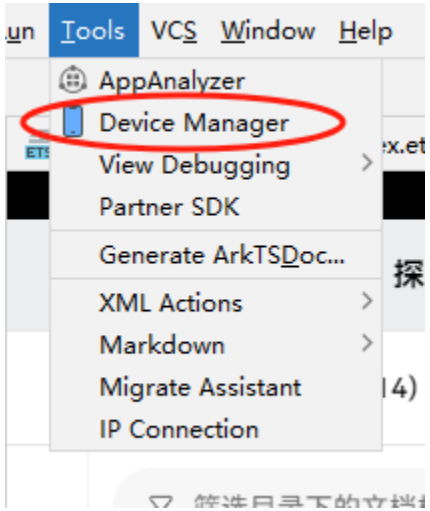
步骤1

当创建完第一个OpenHarmony程序时，在DevEco Studio中，单击导航栏中的

下拉图标或者菜单栏Tools中的Device Manager会进入Your Devices页面，如下图所示。

步骤2

步骤3



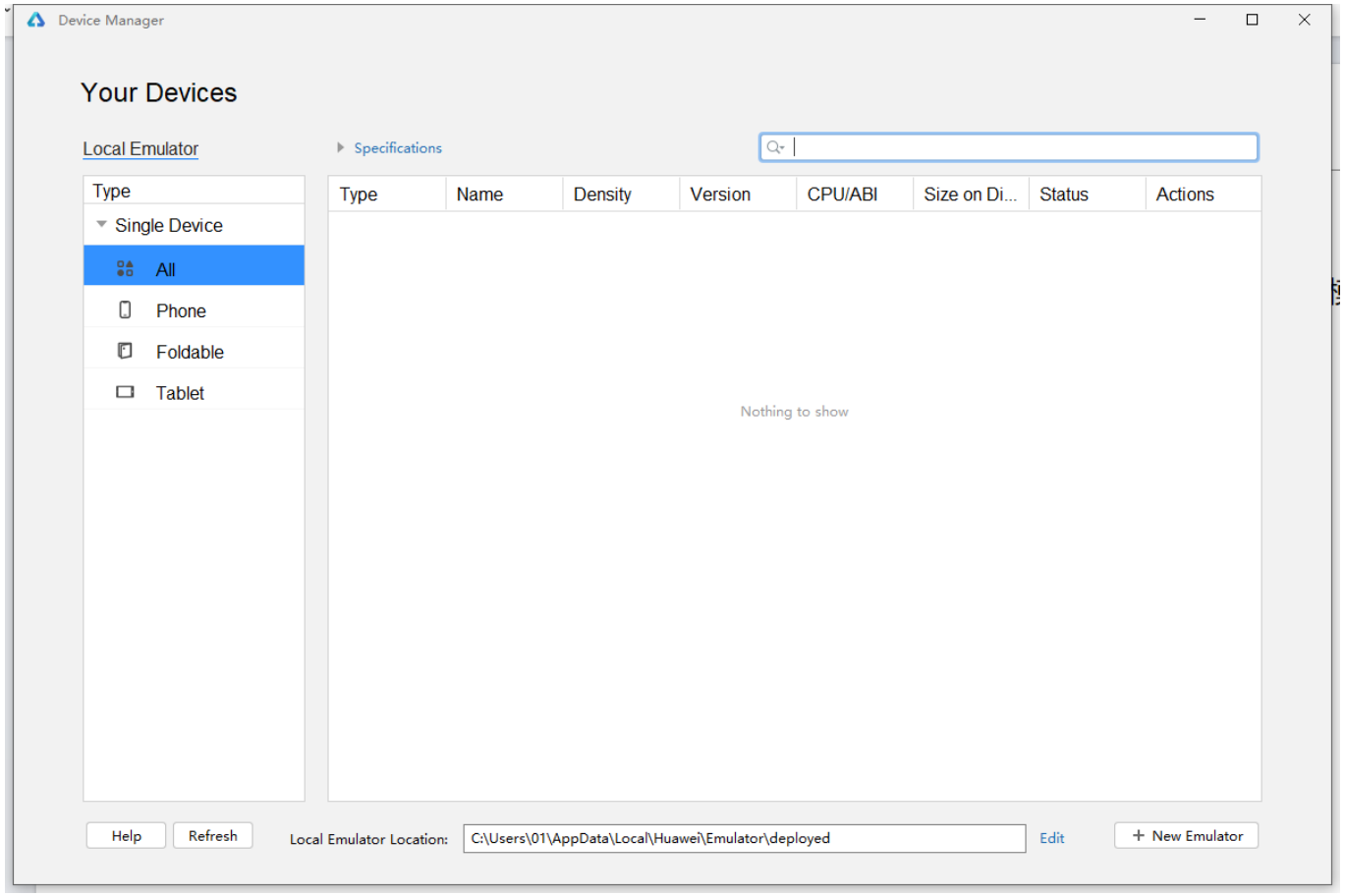
选择模拟设备

步骤1

步骤2

步骤3

当创建完第一个OpenHarmony程序时，在DevEco Studio中，单击导航栏中的下拉图标或者菜单栏Tools中的Device Manager会进入Your Devices页面，如下图所示。



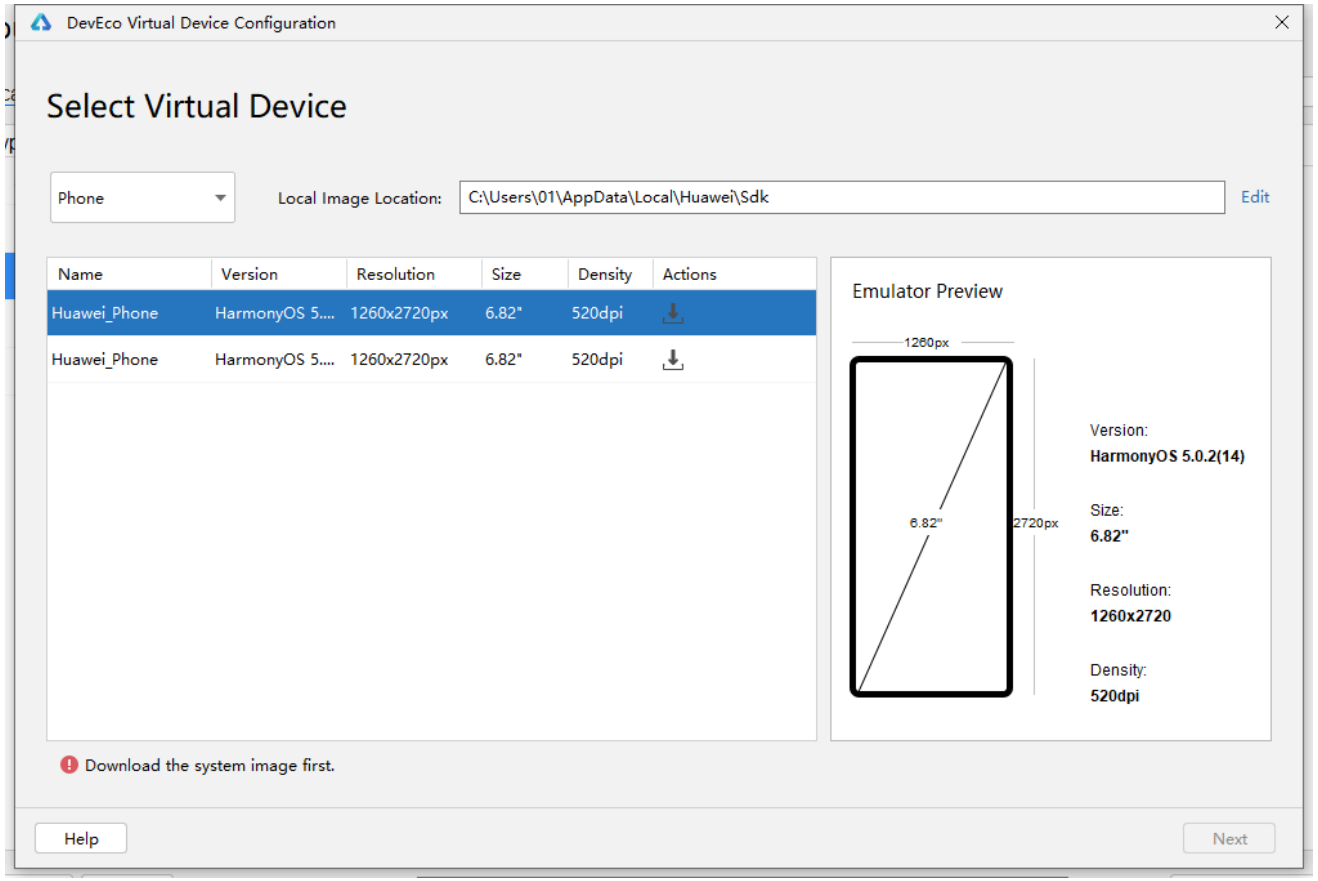
步骤1

步骤2

步骤3

选择模拟设备

单击上图中左侧的模拟设备类型，点击“+ New Emulator”按钮，此时会进入选择模拟设备的Select Virtual Device页面，如下图所示。



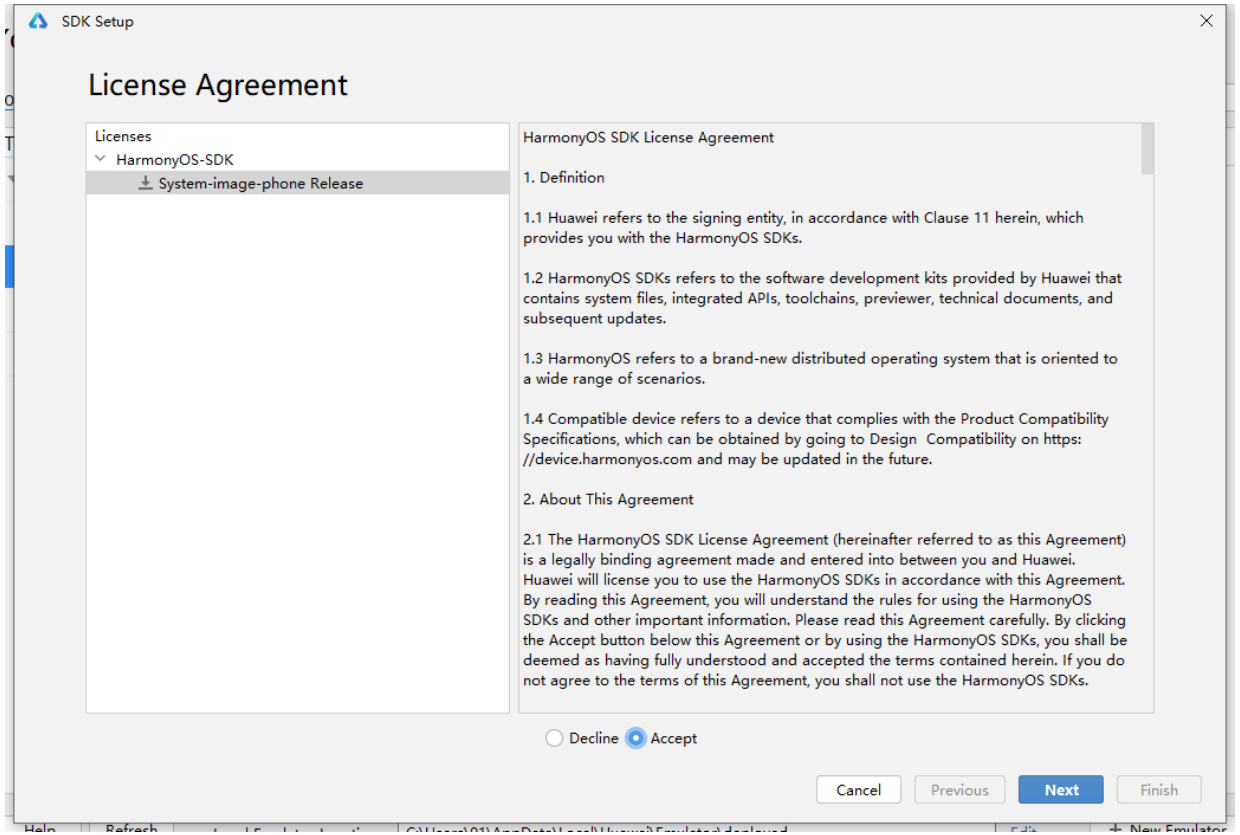
选择模拟设备

步骤1

步骤2

步骤3

我们选择上一页图中的【Phone】□【Huawei_Phone】选项（此选项可根据自己需求选择不同HarmonyOS版本，屏幕分辨率的模拟器），选择“Accept”，单击“Next”按钮进入，如下图所示。



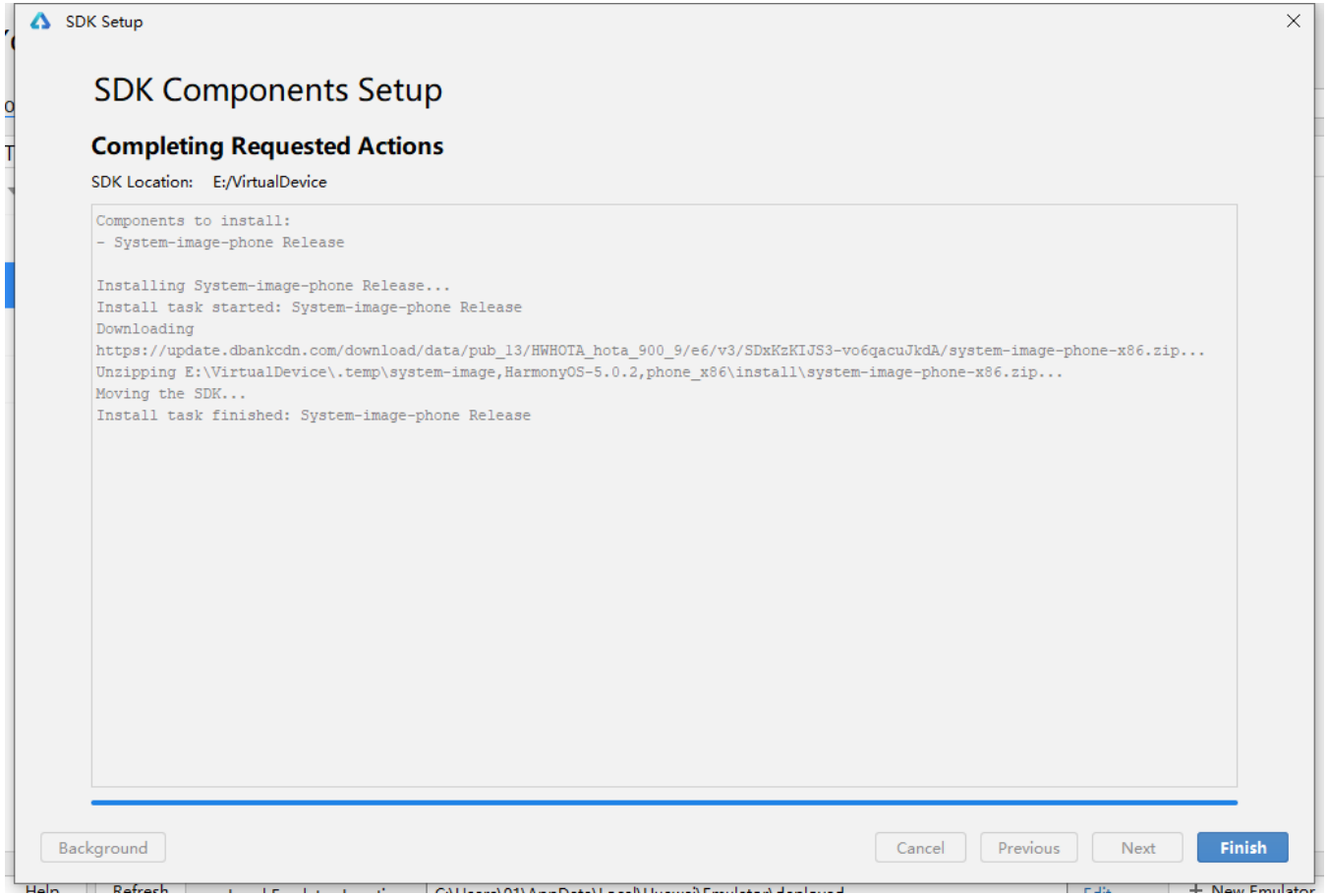
步骤1

步骤2

步骤3

下载SDK System Image

选中上一页图中的Harmony5.0.2系统版本，单击“Next”进入SDK Componets Setup 页面，如下图所示。



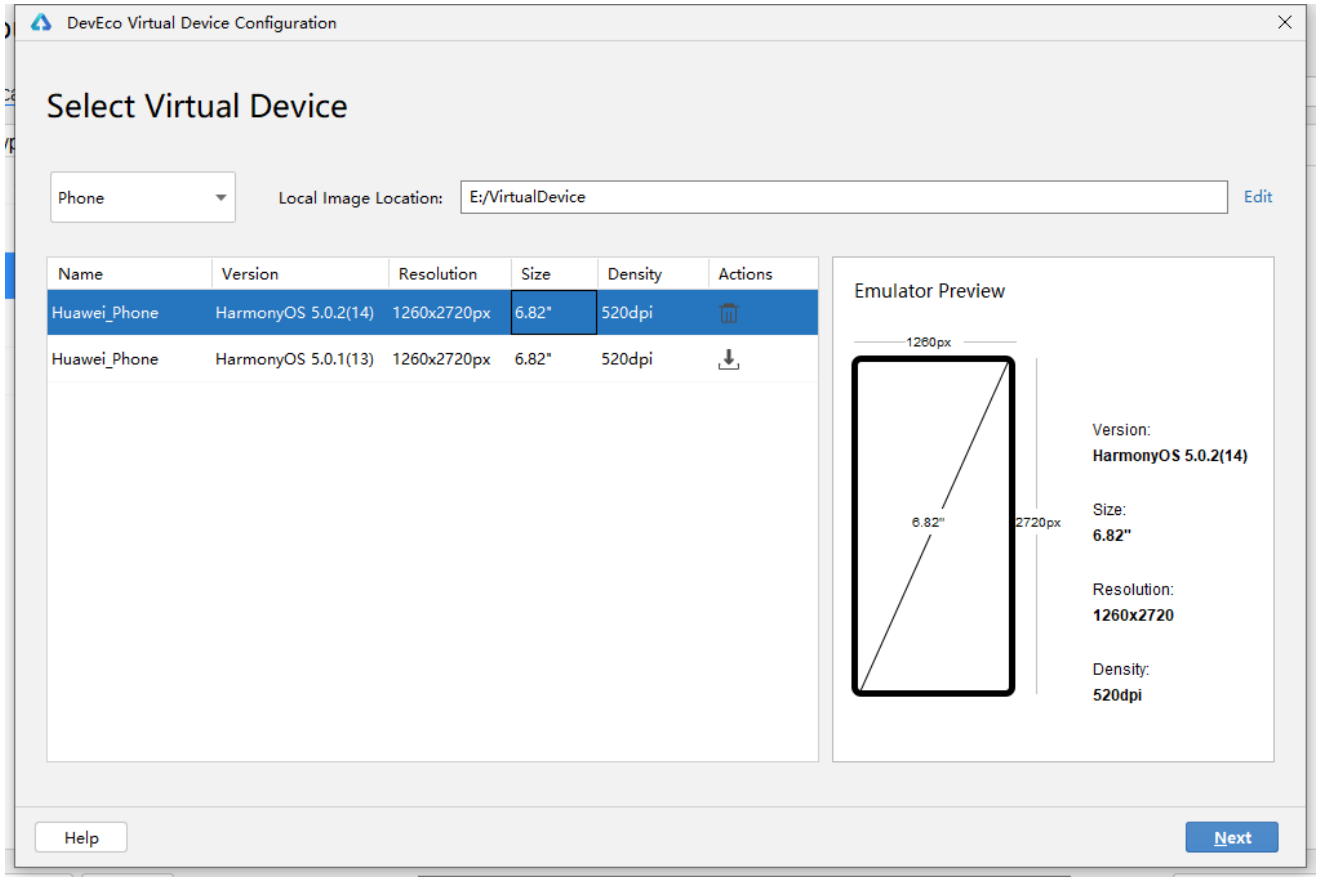
创建模拟设备

步骤1

步骤2

步骤3

我们选择上一页图中的【Finish】选项，返回Select Virtual Device页面，如下图所示。



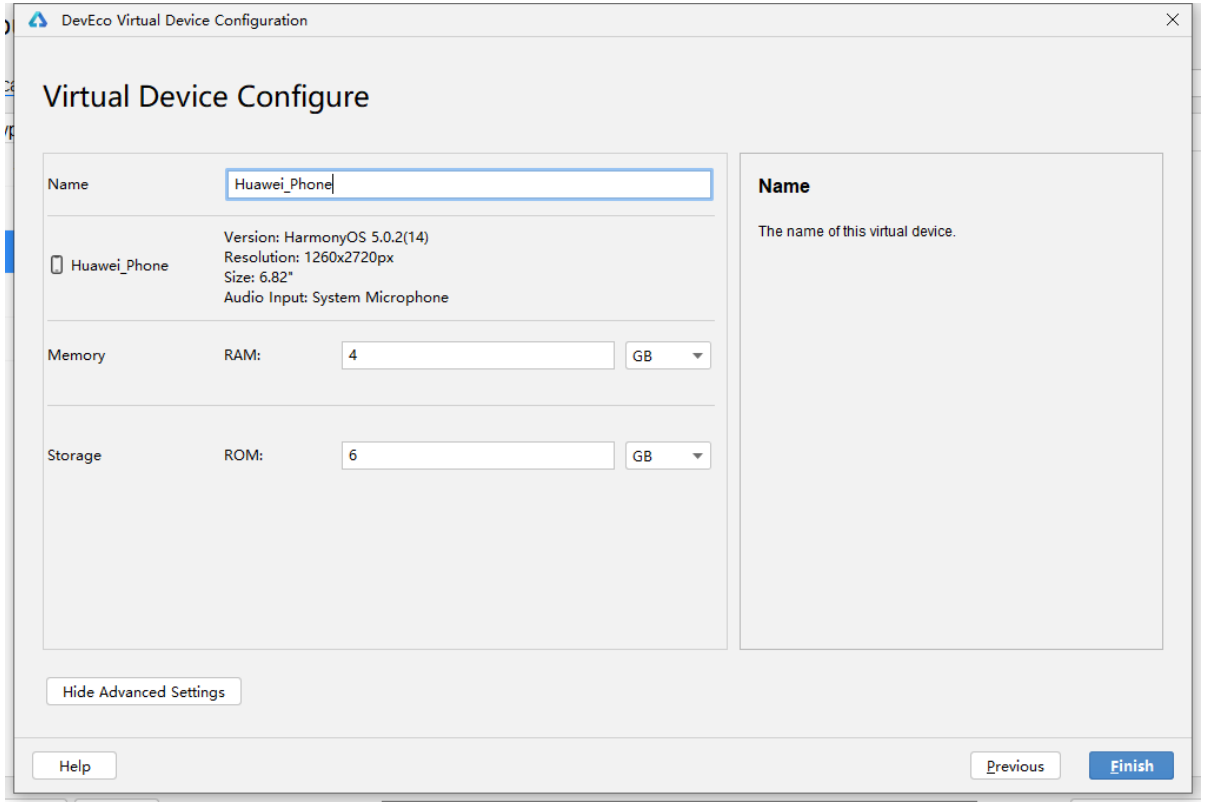
创建模拟设备

步骤1

步骤2

步骤3

我们选择上一页图中的“Next”选项进入如下图 Virtual Device Configure页面，填写设备名称【Name】,内存【Memory】存储【Storage】,单击“Finish”选项进入 Your Devices。



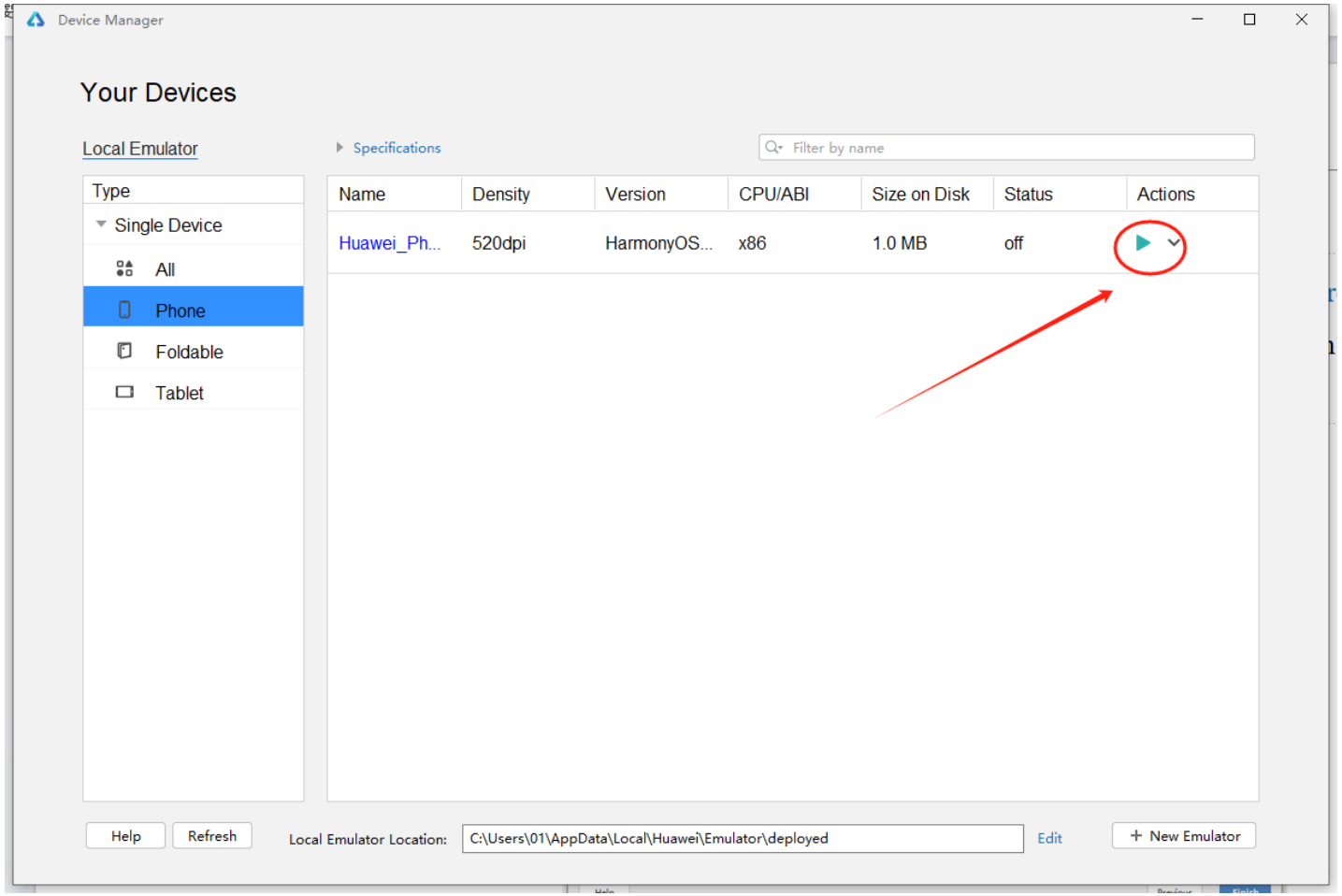
创建模拟设备

Your Devices页面中会显示创建完成的模拟机。

步骤1

步骤2

步骤3

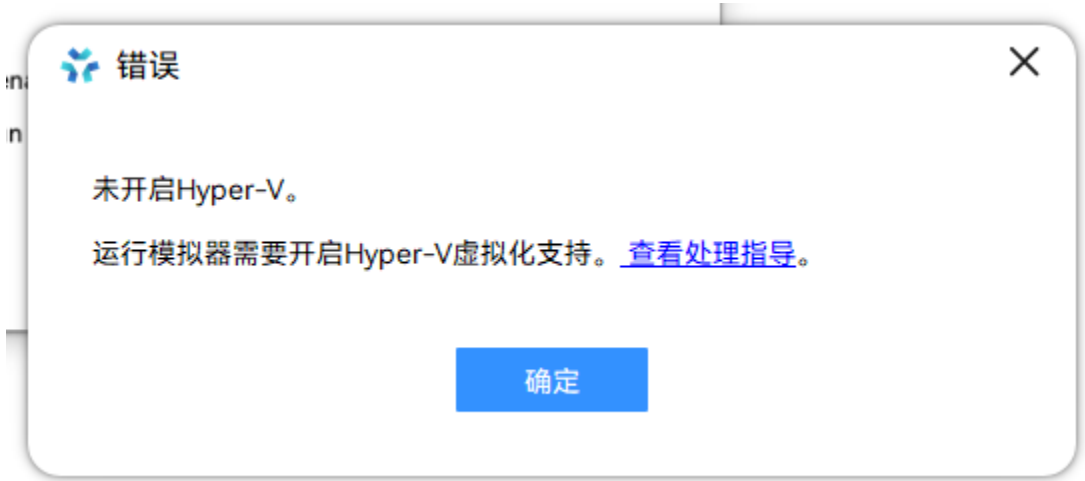


打开模拟设备

步骤4

单击上一页图中的“启动”按钮启动模拟器，启动如出现如下图所示错误。CPU支持虚拟化，打开控制面板 > 程序 > 程序与功能 > 启动或关闭Windows功能，找到并勾选“Hyper-V”、“Windows虚拟机监控程序平台”、“虚拟机平台”，点击确定并重启电脑。若勾选后启动模拟器仍然提示该错误，需要以管理员权限打开命令行窗口并执行：`bcdedit /set hypervisorlaunchtype auto`并重启电脑。

步骤5



步骤4

步骤5

打开模拟设备

单击上一页图中的“启动”按钮正常启动模拟器，启动完成后的Harmony模拟器界面。

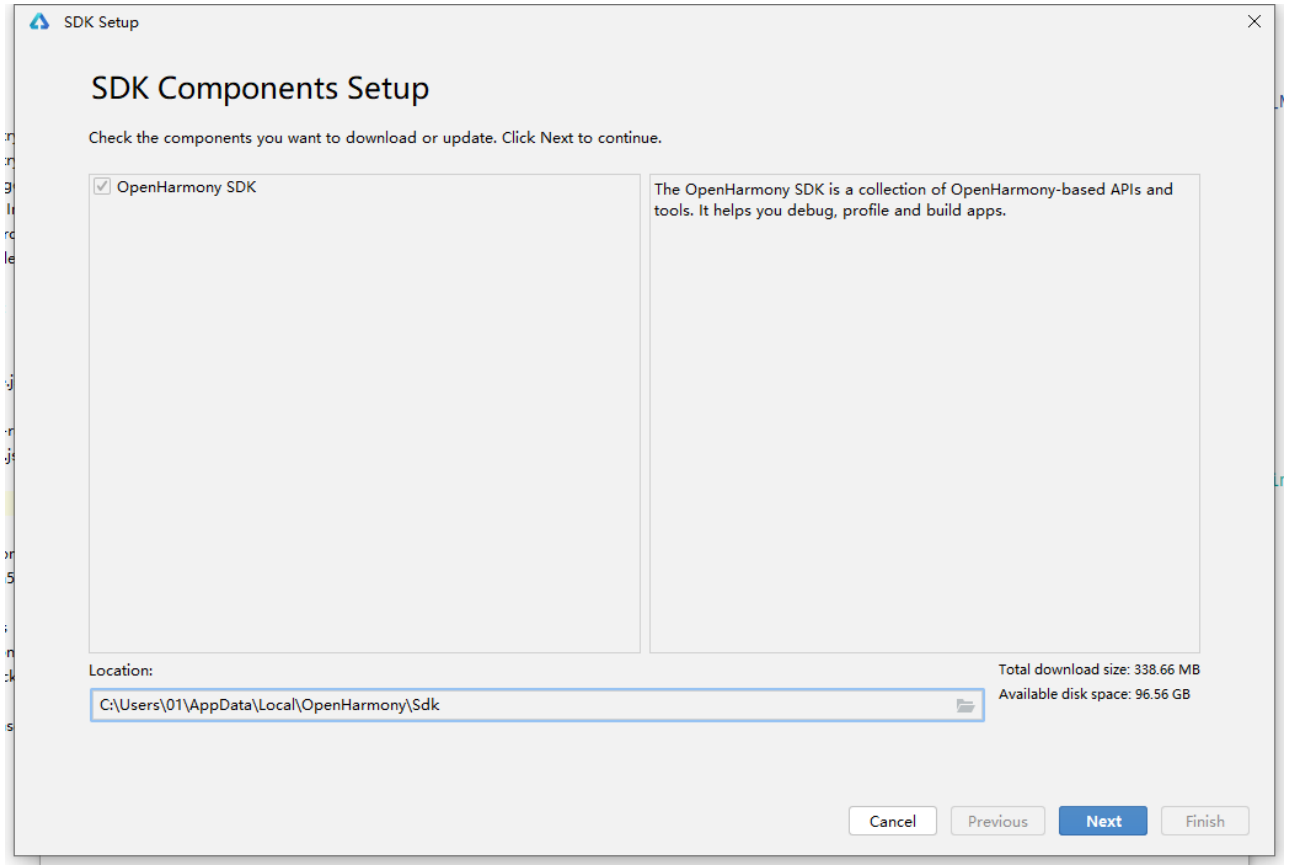
1.2.3 在DevEco Studio中下载OpenHarmony SDK

下载SDK版本

步骤1

打开DevEco Studio，通过菜单栏点击“文件”>“设置”，找到并点击“SDK和工具”选项，依次选择保存路径，执行下图所示“Next”选项。

步骤2



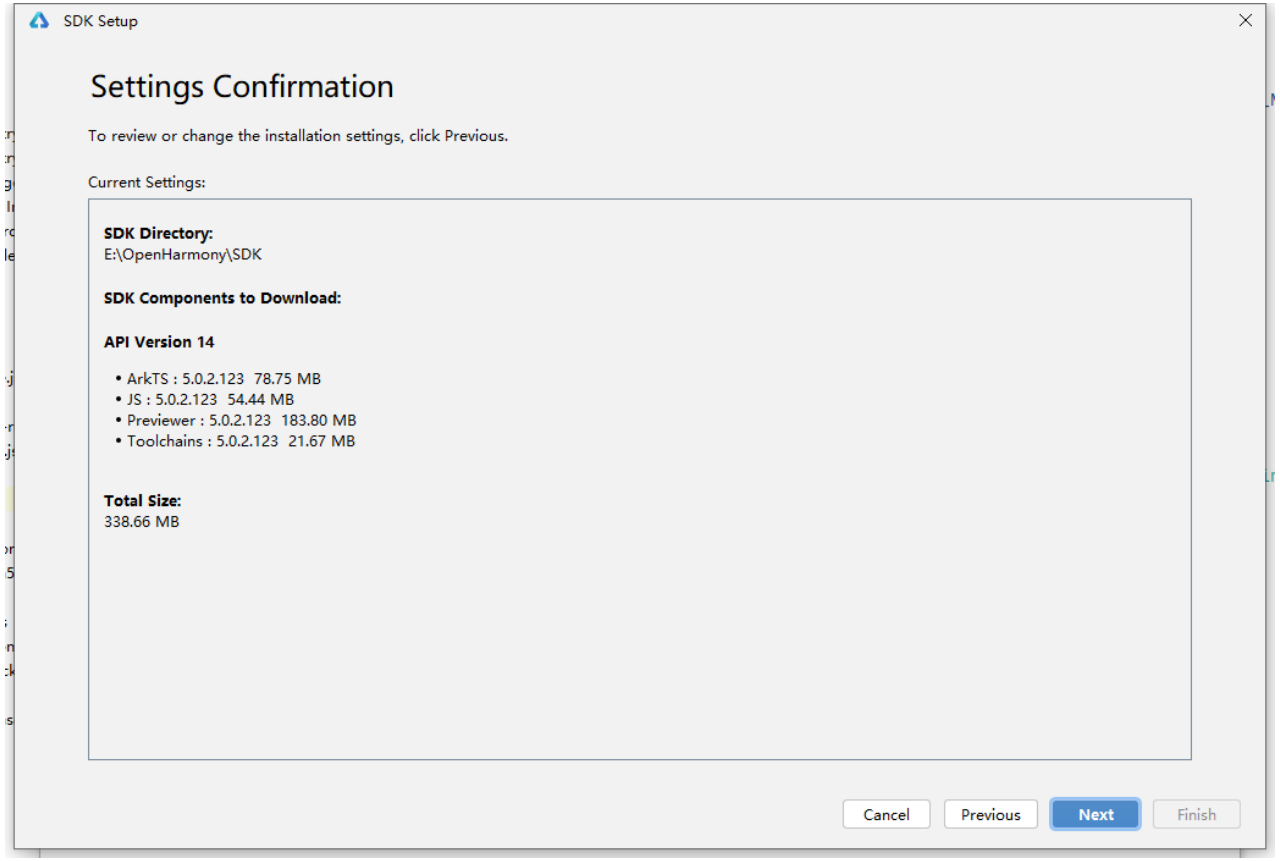
1.2.3 在DevEco Studio中下载OpenHarmony SDK

下载SDK版本

步骤1

打开DevEco Studio，通过菜单栏点击“文件”>“设置”，找到并点击“SDK和工具”选项，依次执行下图“Next”选项。

步骤2



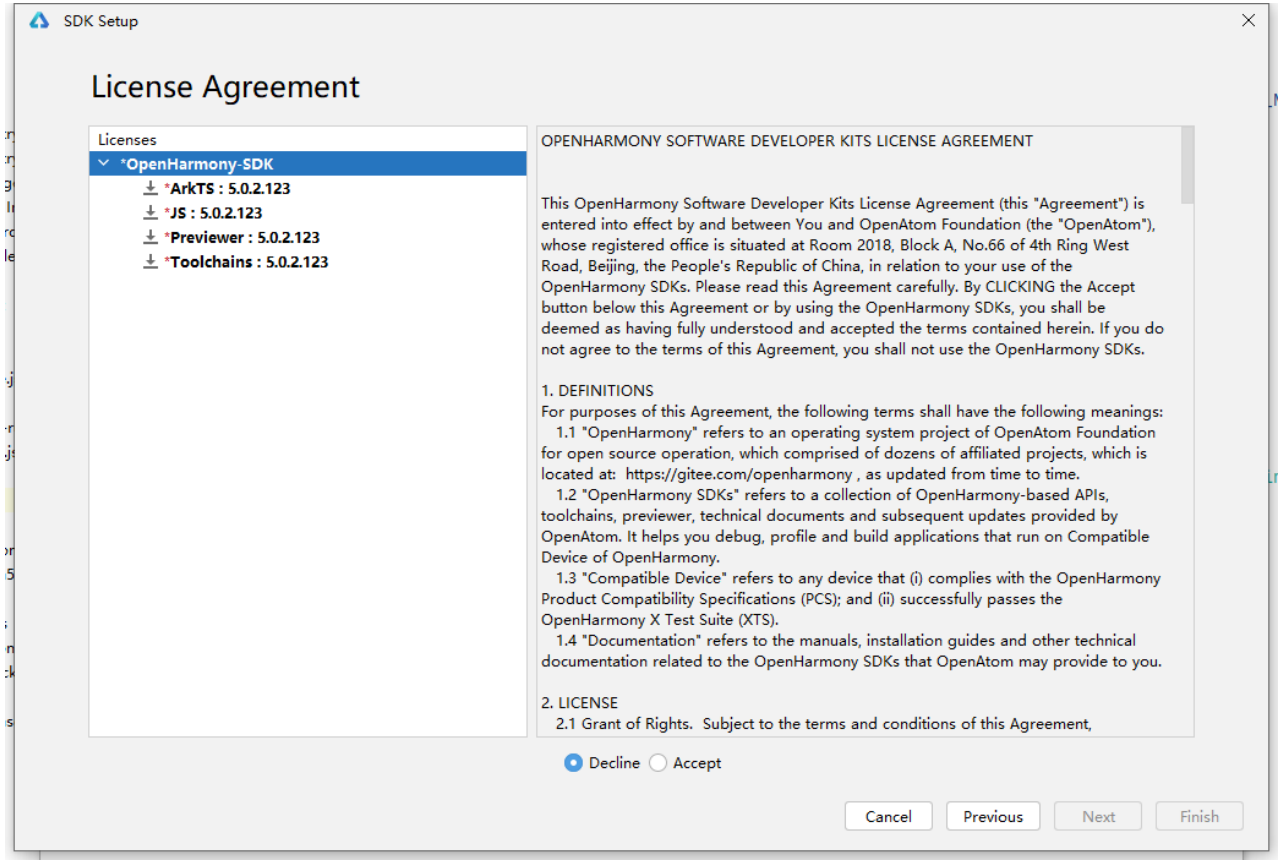
1.2.3 在DevEco Studio中下载SDK

下载SDK版本

在SDK Platforms选项卡下选择OpenHarmony SDK条目，单击图中的“Accpect”选项确认安装SDK组件，如下图所示。

步骤1

步骤2



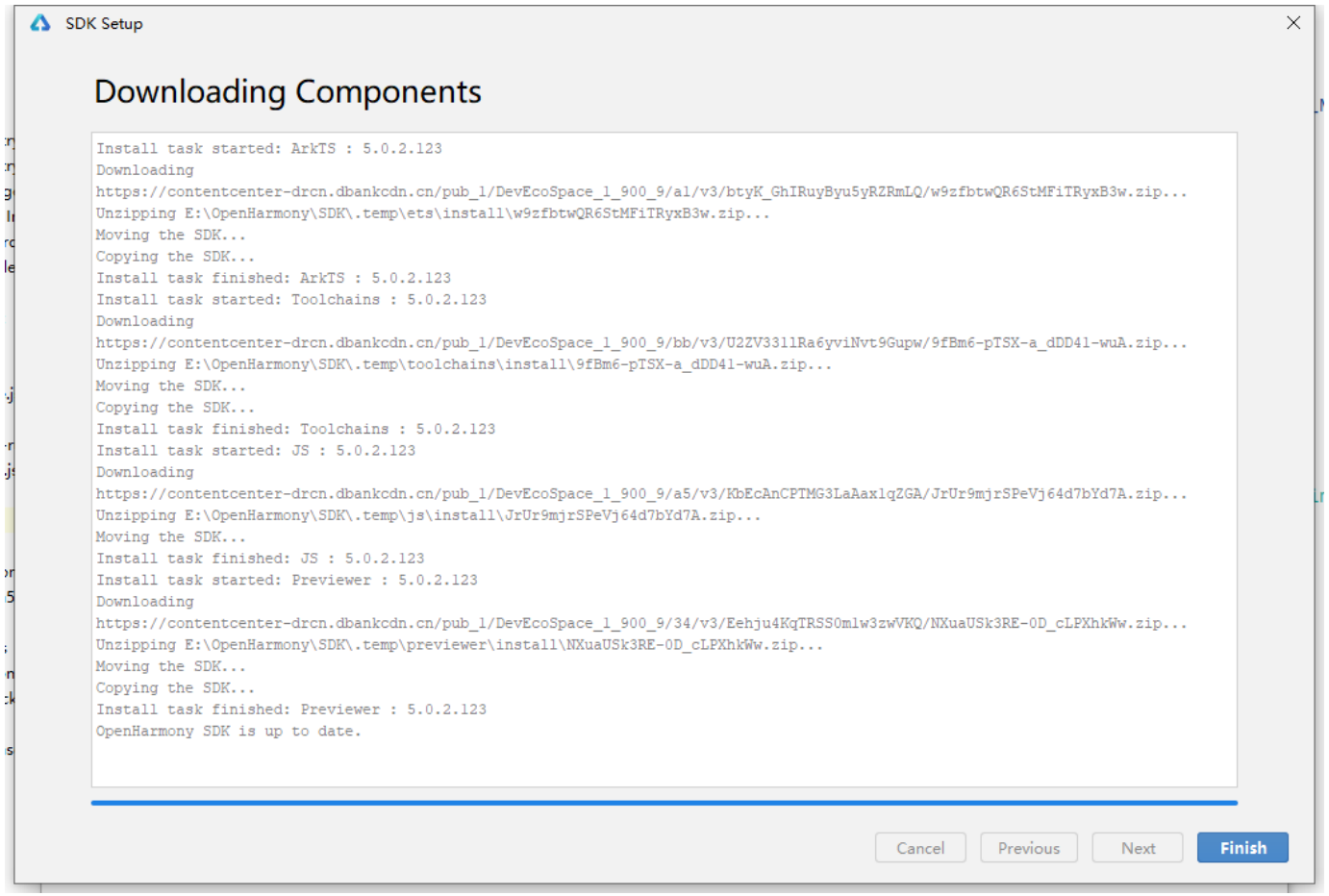
1.2.3 在DevEco Studio中下载SDK

下载SDK版本

步骤1

单击上一页图中的“next”按钮，进入Downloading Components下载页面，如下图所示。

步骤2



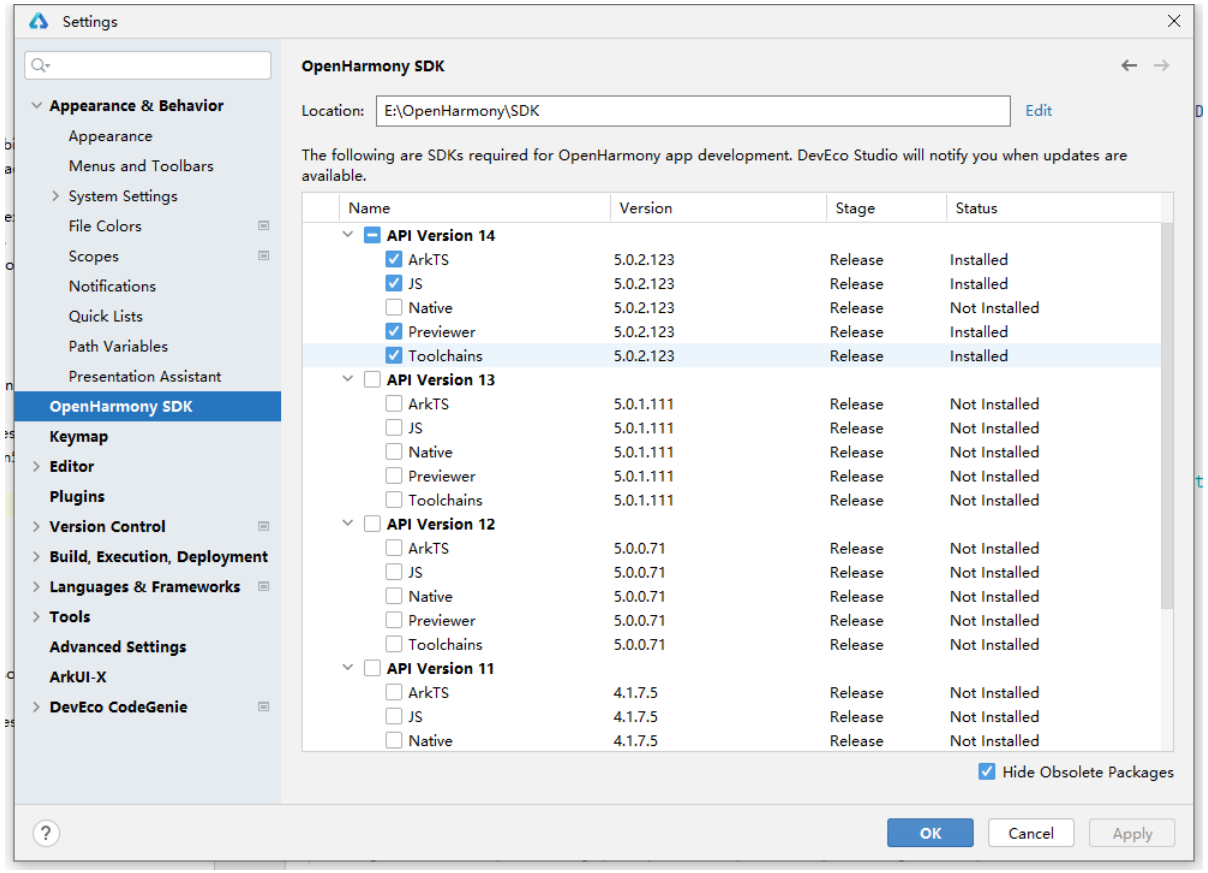
1.2.3 在DevEco Studio中下载SDK

下载SDK版本

步骤1

下载完成后的Component Installer页面，如下图所示。

步骤2



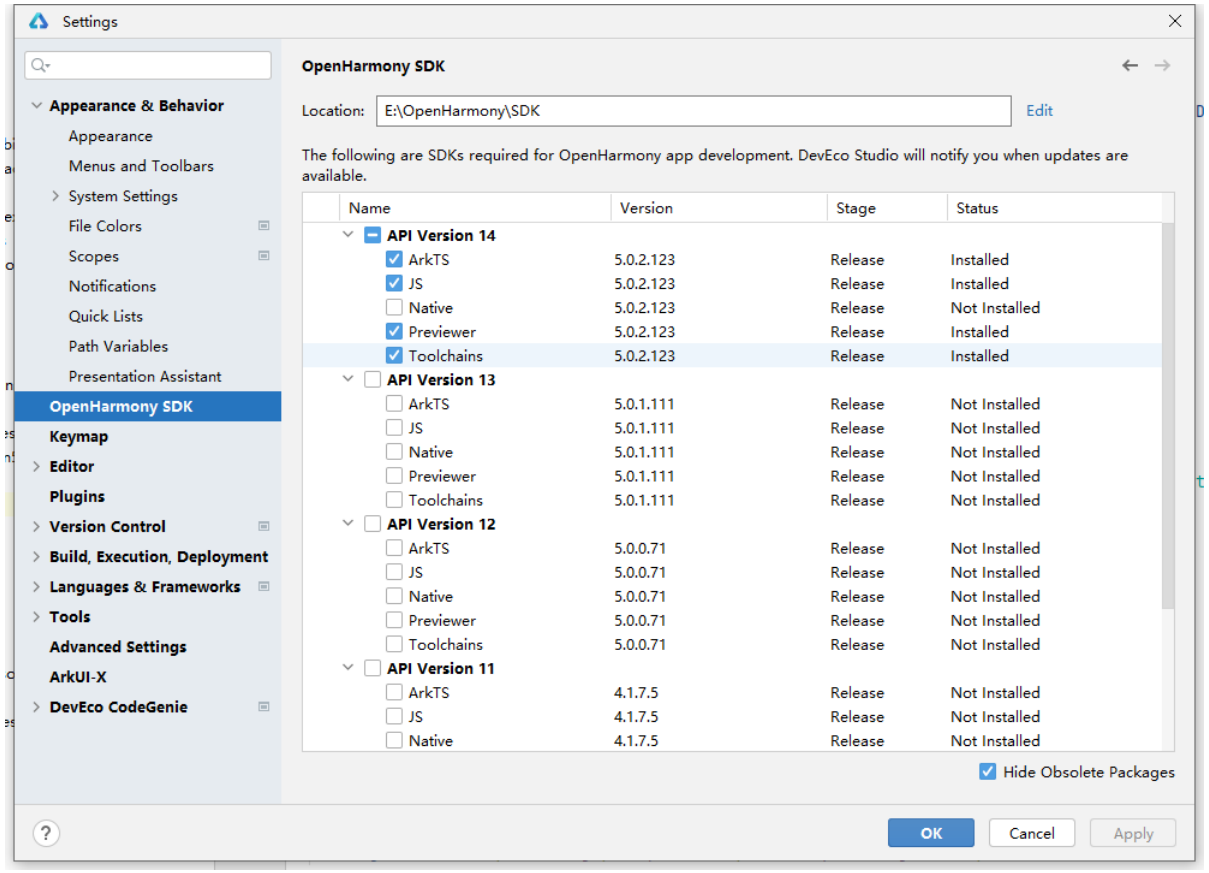
1.2.3 在DevEco Studio中下载SDK

选择使用SDK版本

步骤1

步骤2

在Settings窗口中的OpenHarmony SDK选项卡下，勾选API Version 14选项，点击“OK”如下图所示。

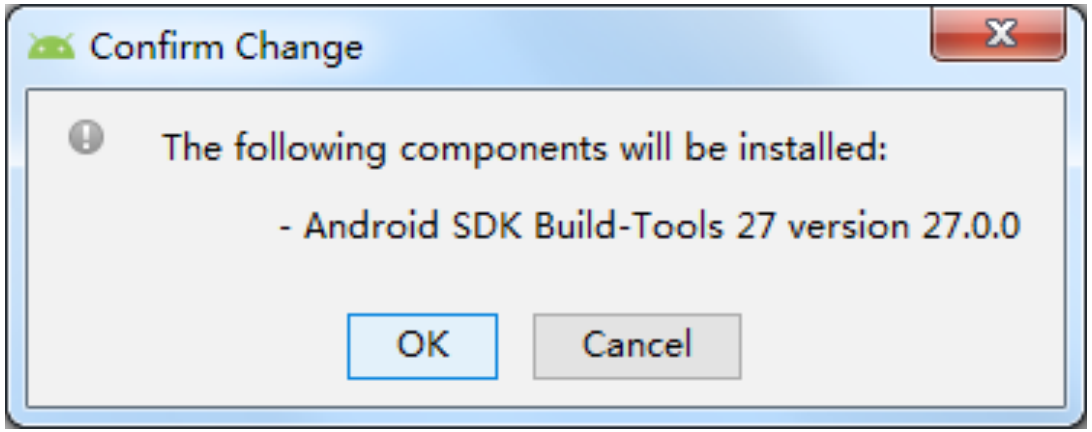


下载Tools工具

步骤1

接着勾选Default Settings窗口右下角的Show Package Details选项，会打开Harmony与Android SDK Build-Tools中的SDK版本列表信息，在列表中勾选27.0.0条目，单击“OK”按钮会弹出Confirm Change窗口，如下图所示。

步骤2



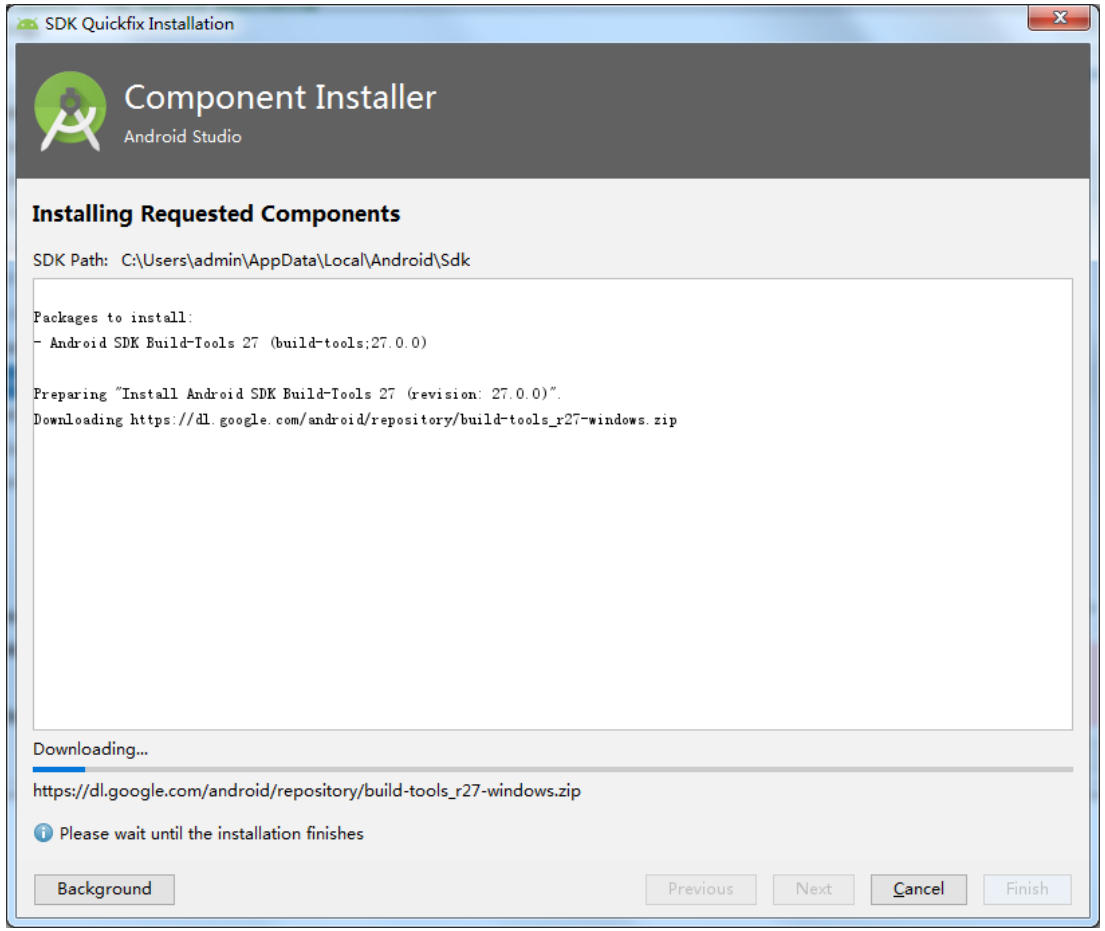
1.2.3 在DevEco Studio中下载SDK

下载Tools工具

步骤1

步骤2

单击上一页图中的“OK”按钮进入Component Installer下载页面，如下图所示。



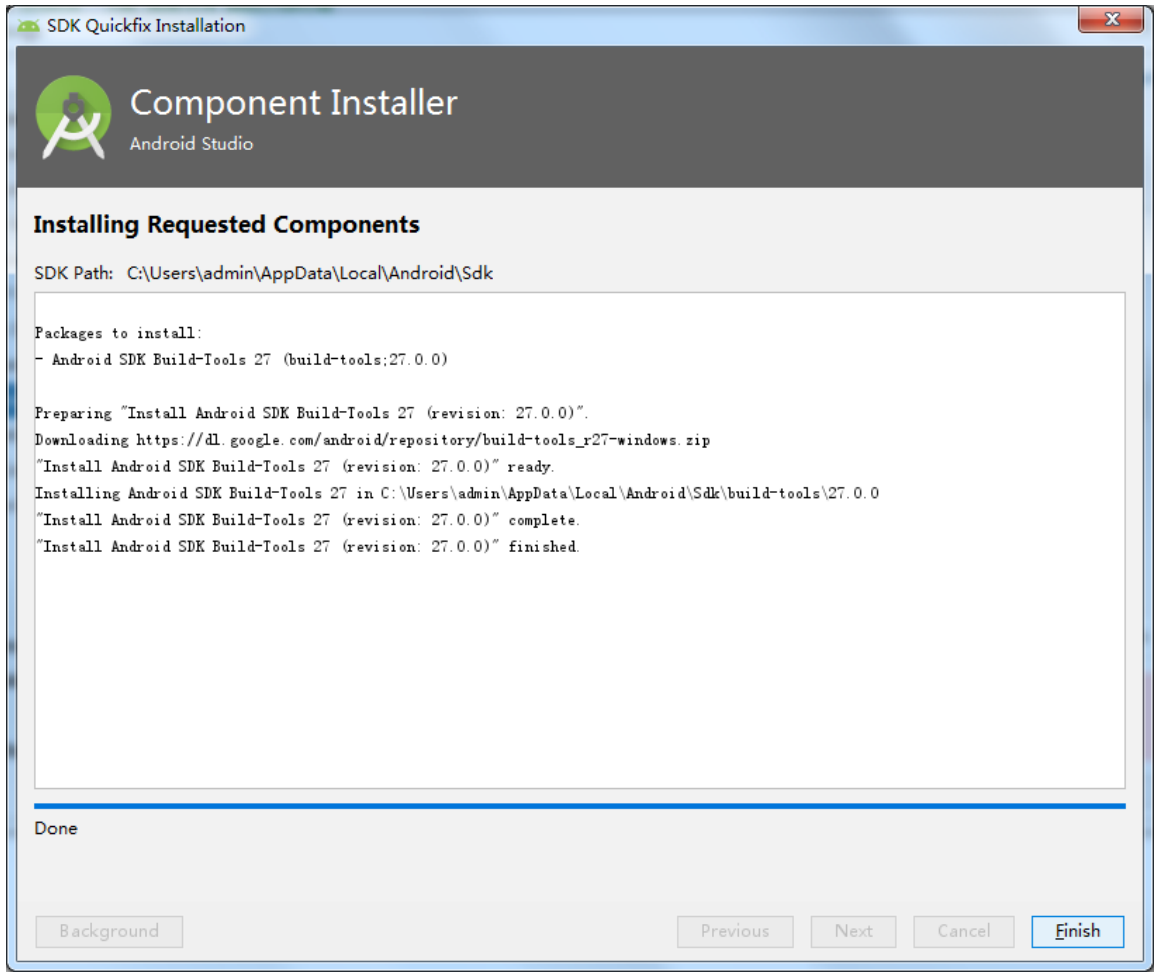
1.2.3 在DevEco Studio中下载SDK

下载Tools工具

步骤1

步骤2

一段时间之后，SDK下载完成，Component Installer下载完成页面的显示如下图所示。

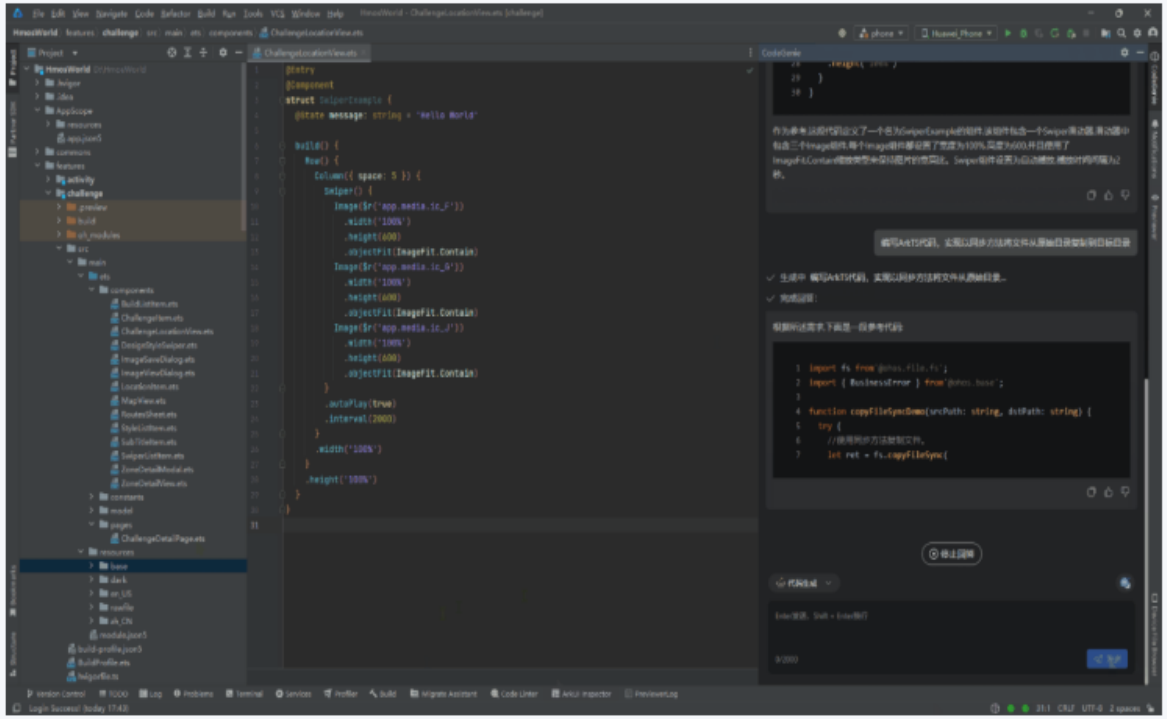


AI 辅助编程（Beta）

AI 辅助编程工具包括 CodeGenie 插件和 UI Generator 插件。DevEco CodeGenie 尝鲜版本支持 HarmonyOS 应用开发领域的智能知识问答、ArkTS 代码生成&补全、万能卡片生成等能力。DevEco UI Generator 尝鲜版本支持一键生成可编译、可运行的 HarmonyOS UI 工程。

资料介绍

下载入口



河南大學
Henan University

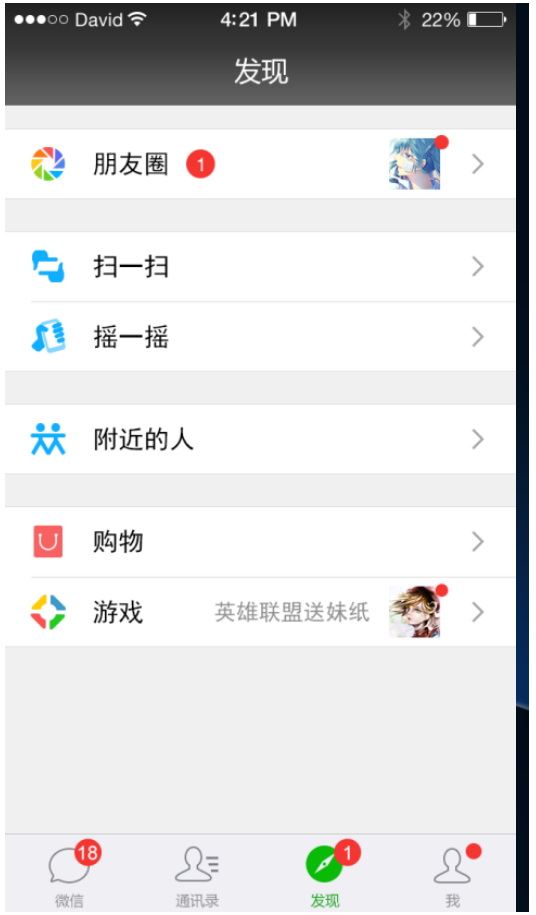
1.3 开发第一个OpenHarmony程序

1.3 开发第一个Harmony与Android程序

- 掌握编写简单Harmony的步骤，能够编写一个Hello World程序

1.3 开发第一个OpenHarmony程序

学习OpenHarmony可以开发出精美的APP，比如我们常见的QQ、微信和淘宝APP等。



1.3 开发第一个OpenHarmony程序

前面小节中已经完成了Harmony开发环境的搭建，接下来使用OpenHarmony工具开发第一个Harmony程序，具体步骤如下：

创建程序：

(1) 创建名为Hello World的程序

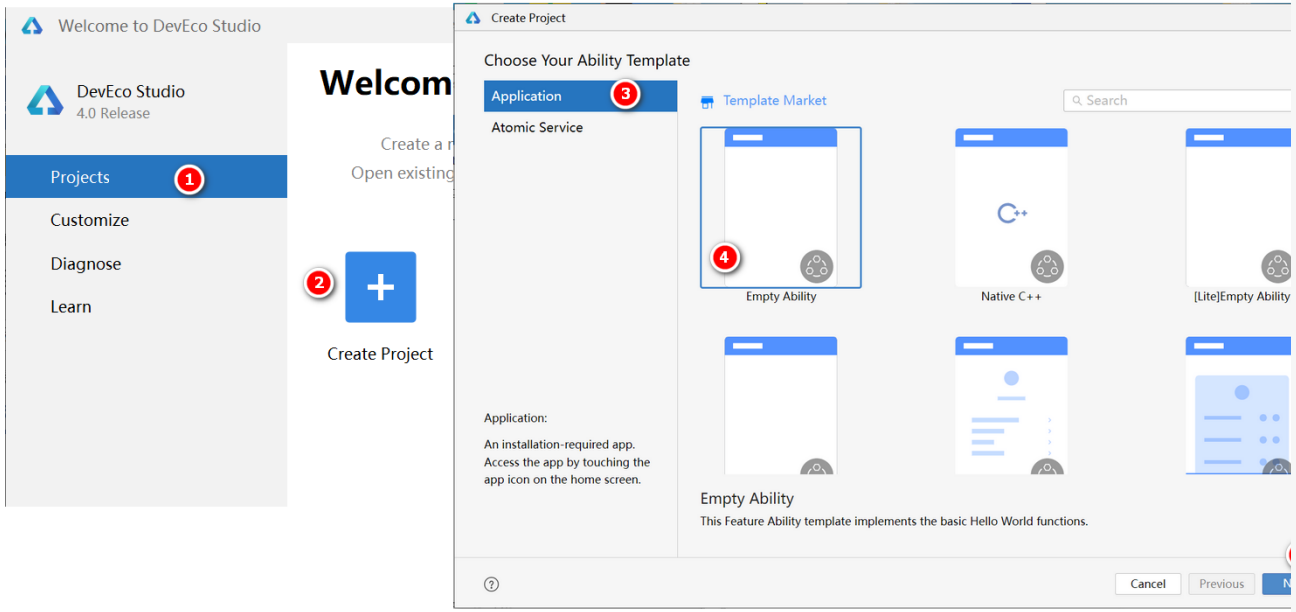
(2) 指定包名为com.example.helloworld

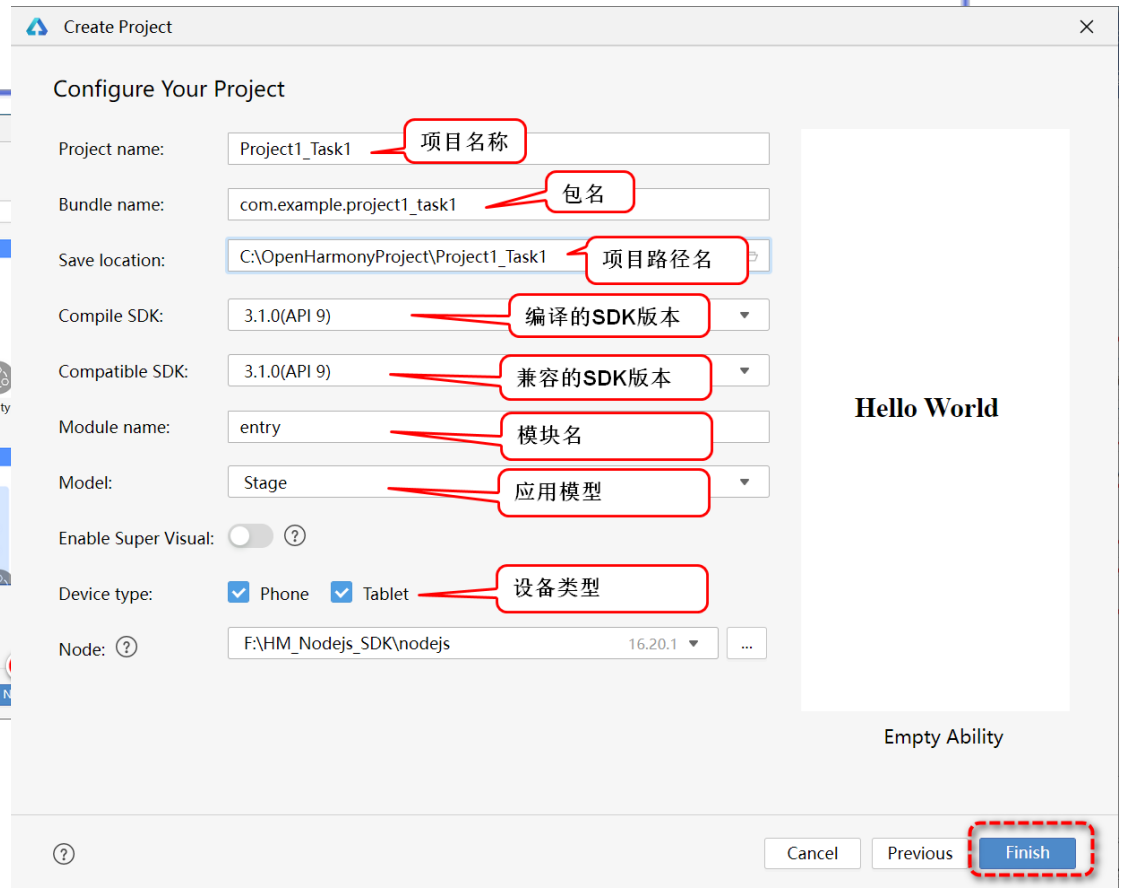
2 **启动模拟器：** 点击工具栏中【AVD Manager】标签启动模拟器

3 **运行程序：** 点击工具栏中的运行按钮运行程序

ArkTS是鸿蒙生态的一种应用开发语言，创建工程时默认创建的就是ArkTS工程。

打开DevEco Studio，创建ArkTS工程，依次选择Projects->Create Project->Application->Empty Ability->Next，输入项目信息后进行创建。





Project name：项目名称，全英文

Bundle name：包名，全英文

Save location：项目存放路径，必须为空文件夹（建议不含中文字符）

Compile SDK：当前使用的SDK版本（目前暂不支持创建工程时选择API 10，需要工程创建好后在配置文件里修改）

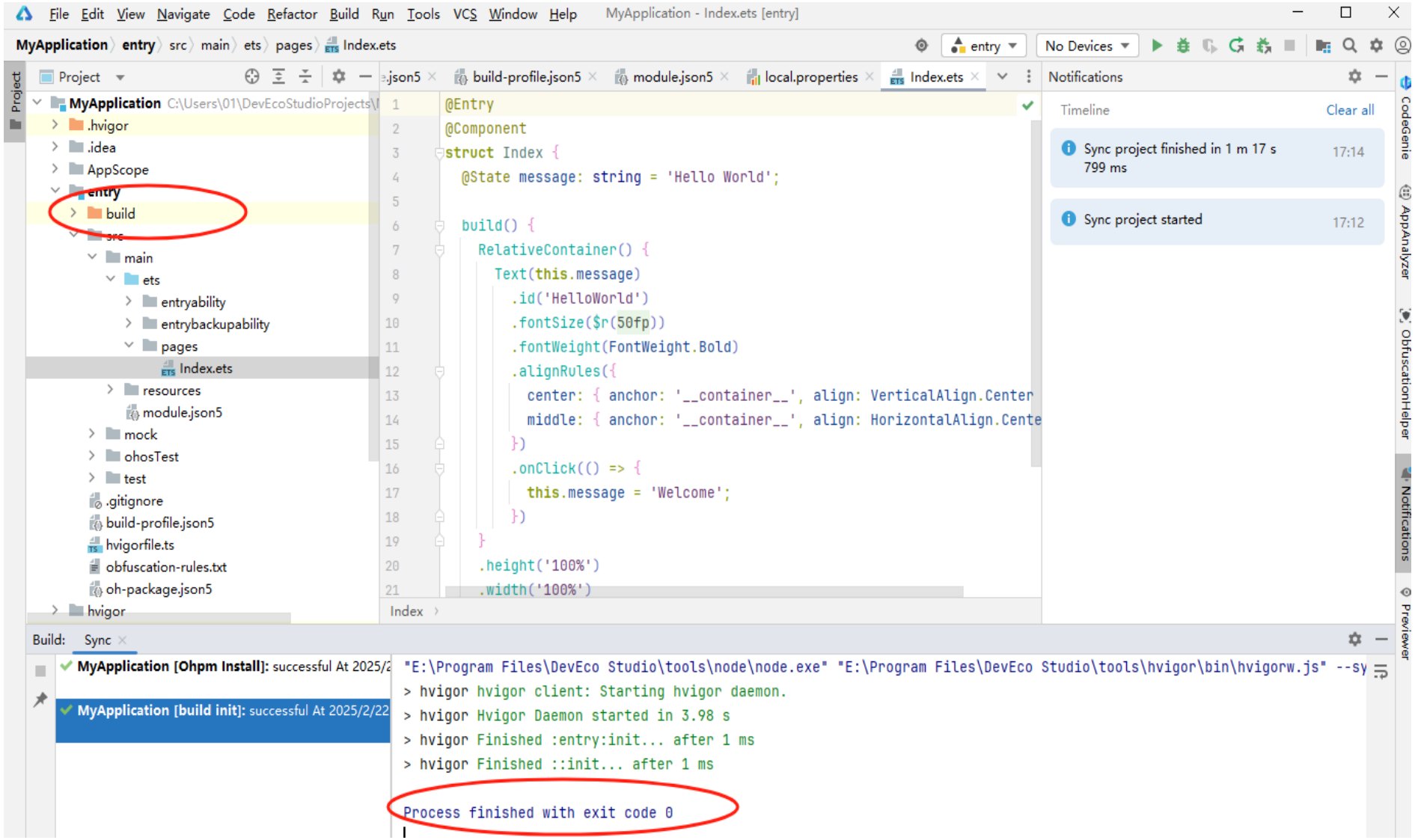
Compatible SDK：适配最低SDK版本

Model：应用模型（API 9之后官方推荐使用Stage）

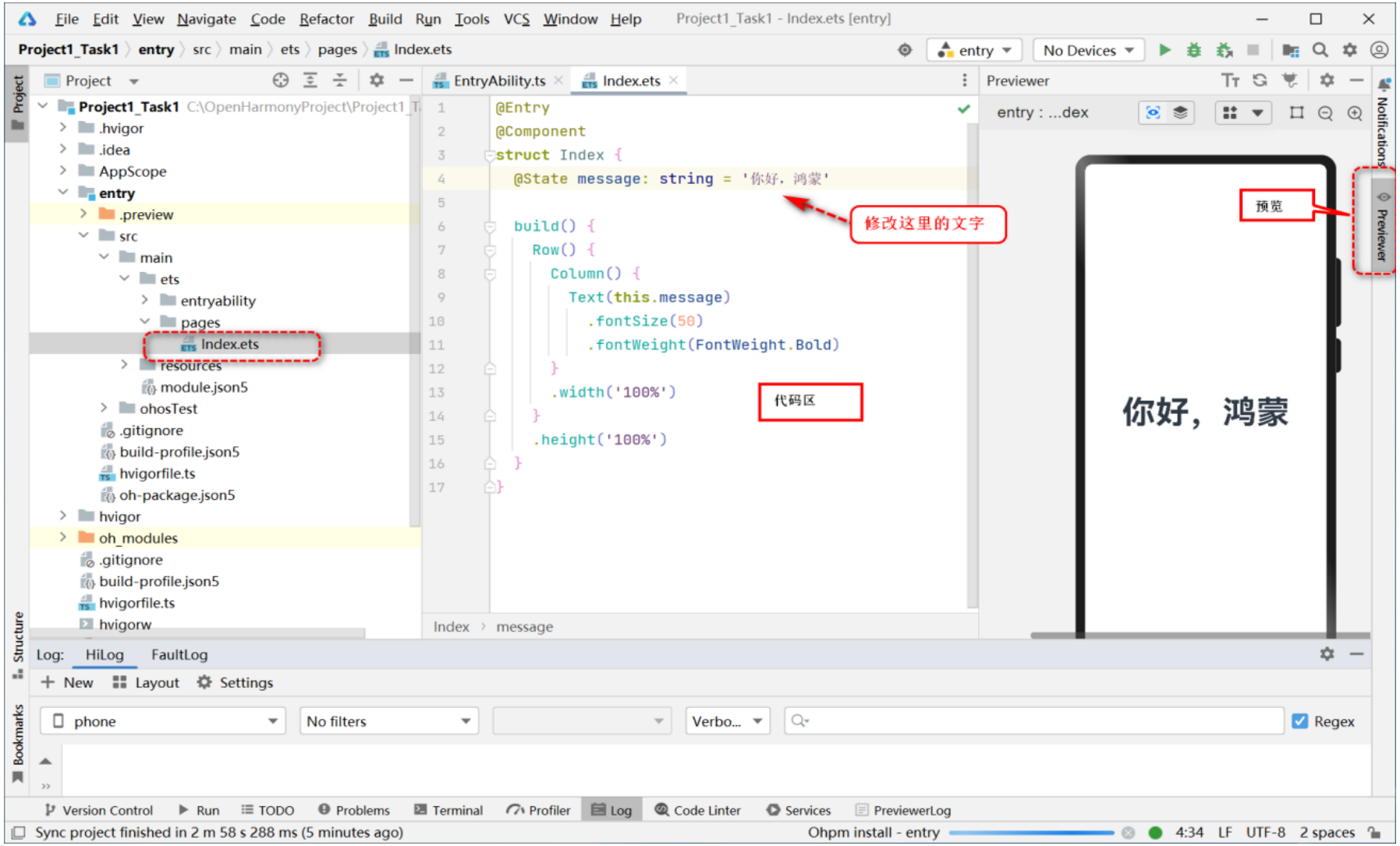
Enable Super Visual：是否启用低代码（默认false，也不推荐使用）

Device type：适配的设备类型（建议全部勾选）

项目生成后会通过node下载一些资源，并进行一些工程化配置。待看到控制台打印Process finished with exit code 0后表示工程已创建完成。（如果没有默认执行工程化配置，可以手动依照控制台的指令手动进行配置）

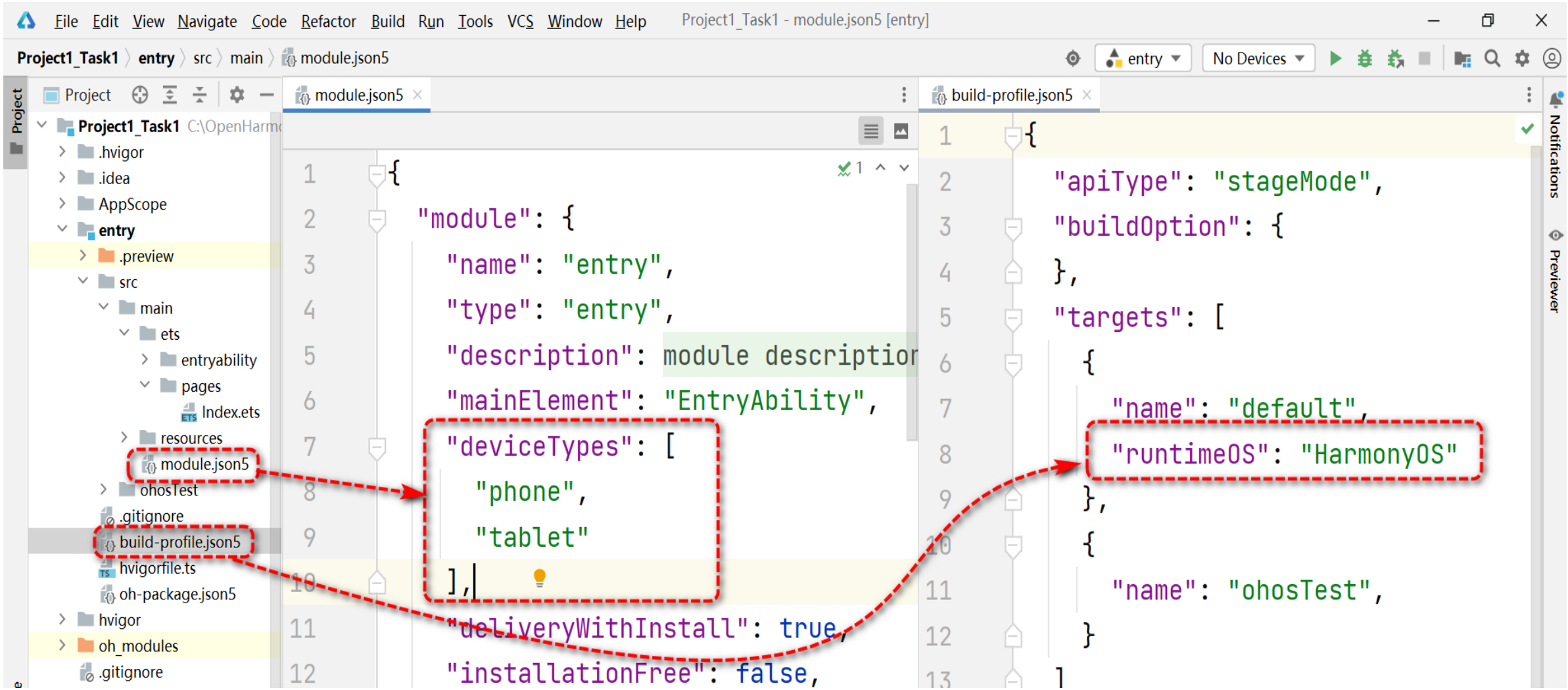


项目构建完成后，默认会打开Index.ets页，可以点击“Preview”进行页面的预览。



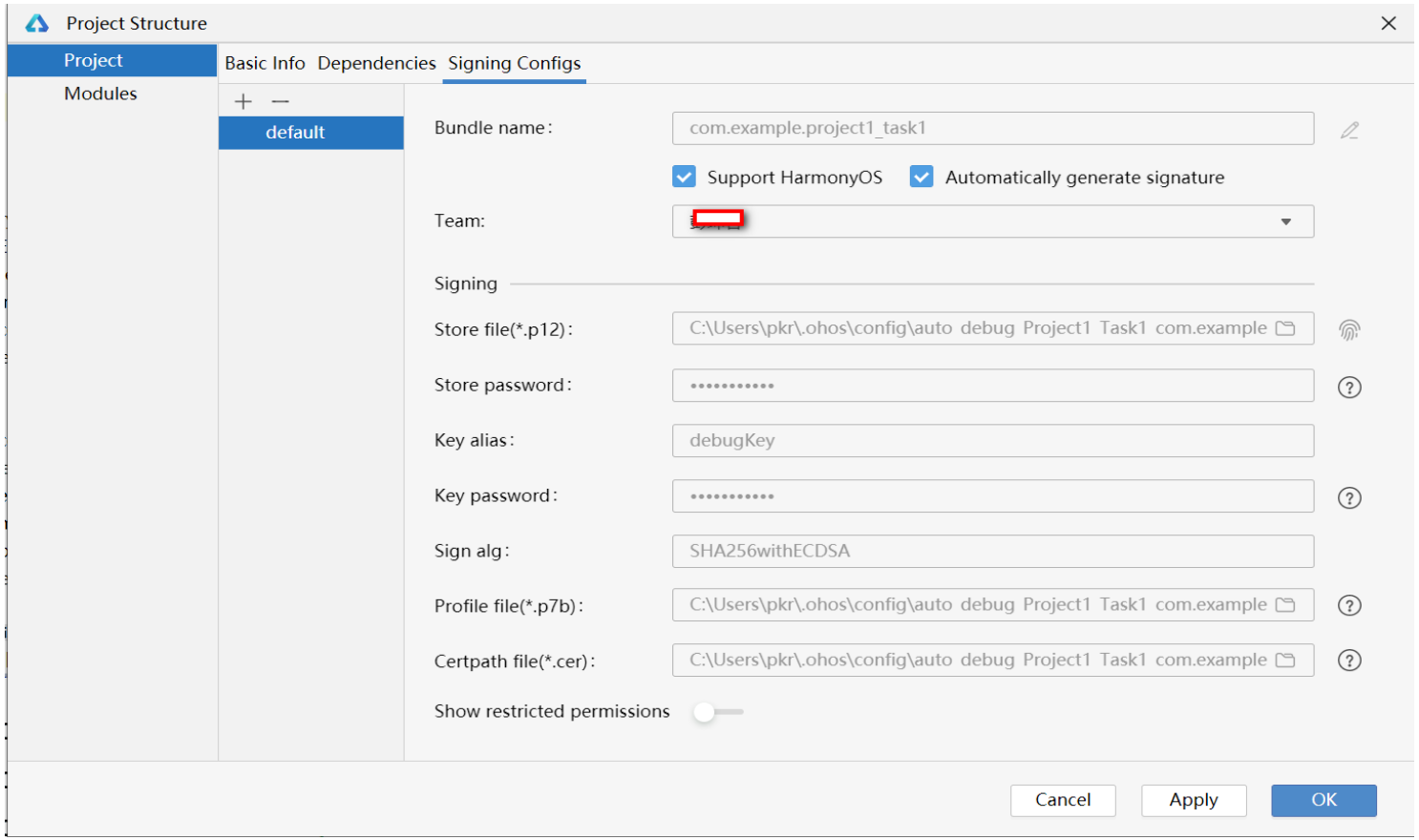
默认的运行操作系统

默认创建的ArkTS工程，运行的设备类型是phone/tablet，运行的操作系统是HarmonyOS。



应用签名并在鸿蒙富设备开发终端上运行

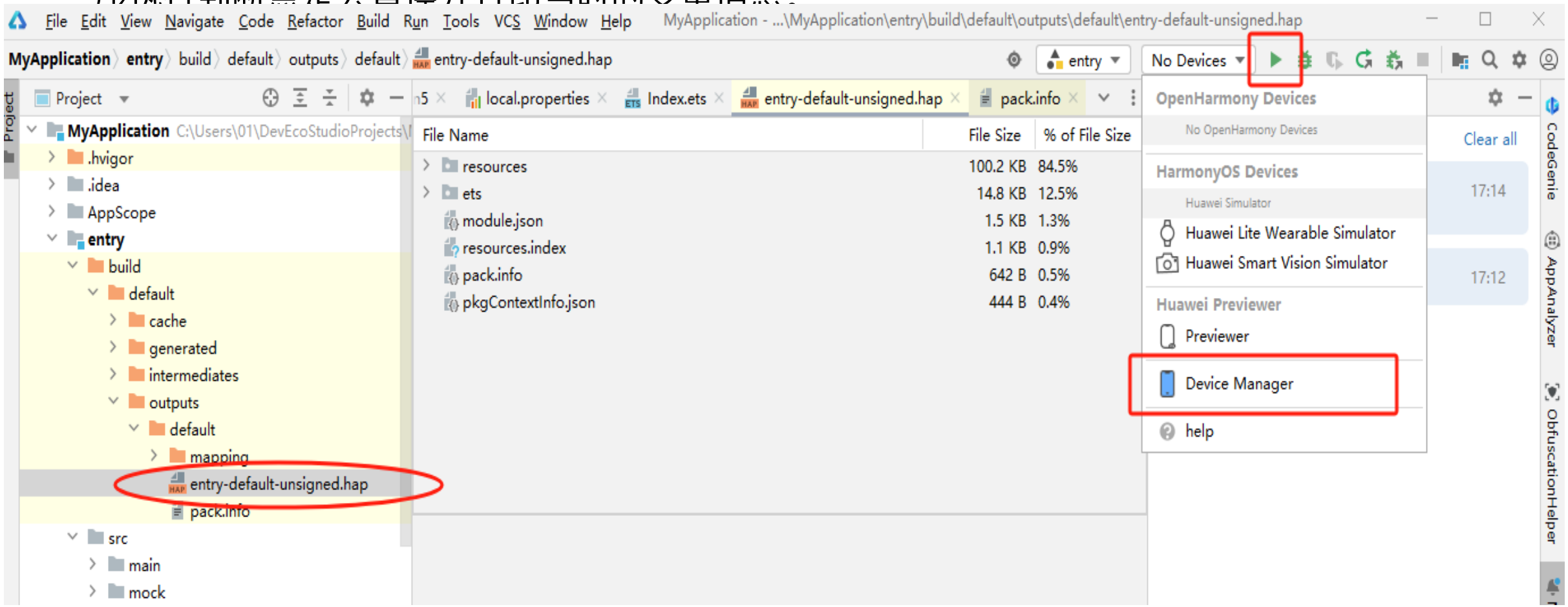
应用要在鸿蒙富设备开发终端设备上运行，需要先将鸿蒙富设备开发终端通过USB调试口连接到PC的USB口。应用签名的流程是：依次选择File->Project Structure->Project->Signing Configs界面，按默认勾选“Support HarmonyOS”“Automatically generate signature”，按提示登录华为账号后，等待自动签名完成即可，完成签名后可将应用发布并运行。



1.电脑通过USB连接上设备，可以在右上角看到设备信息。

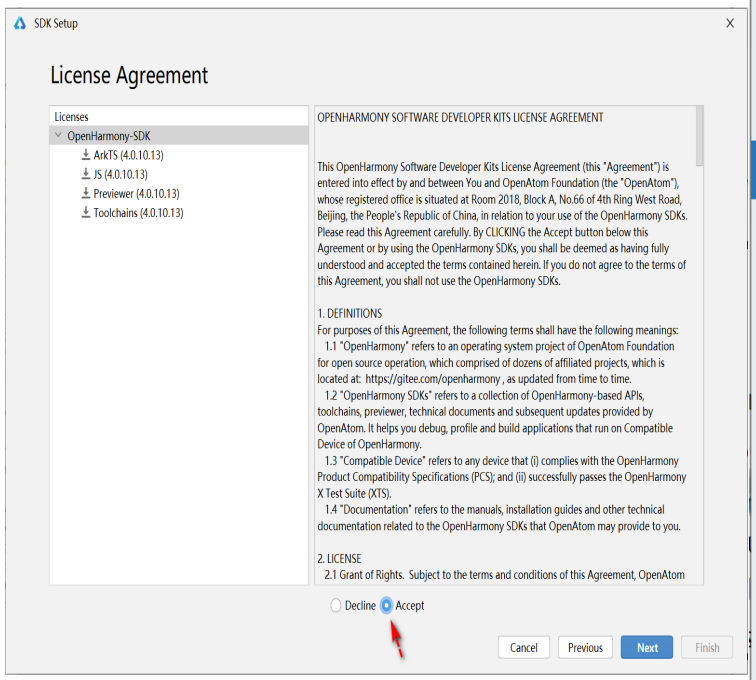
2.点击设备旁的绿色三角，IDE会自动编译构建hap并安装到设备上。

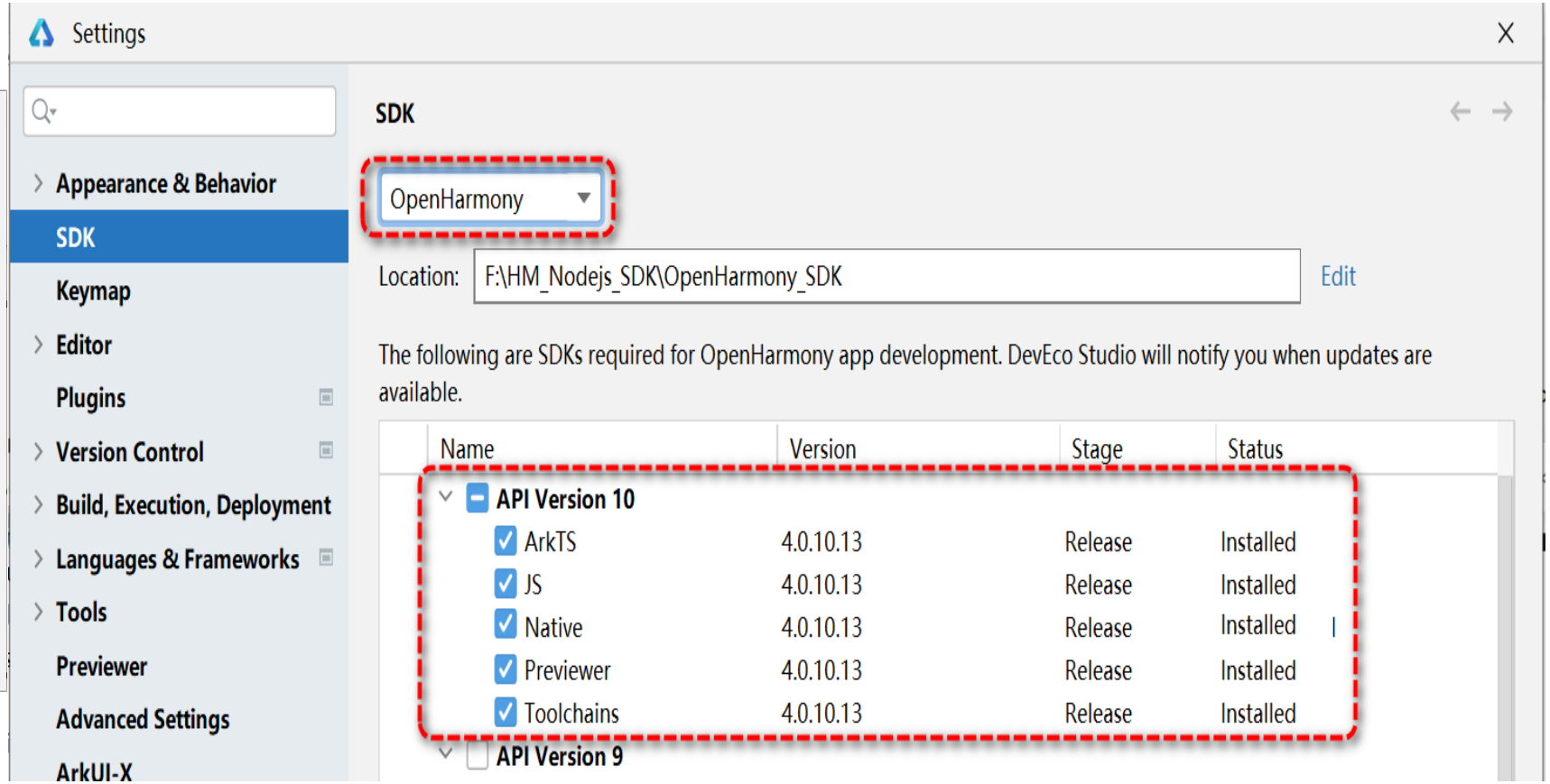
3.点击绿色三角旁的绿色甲虫，并在需要调试的代码左侧打上断点。当程序执行到断点是会暂停并打印当时的变量信息。

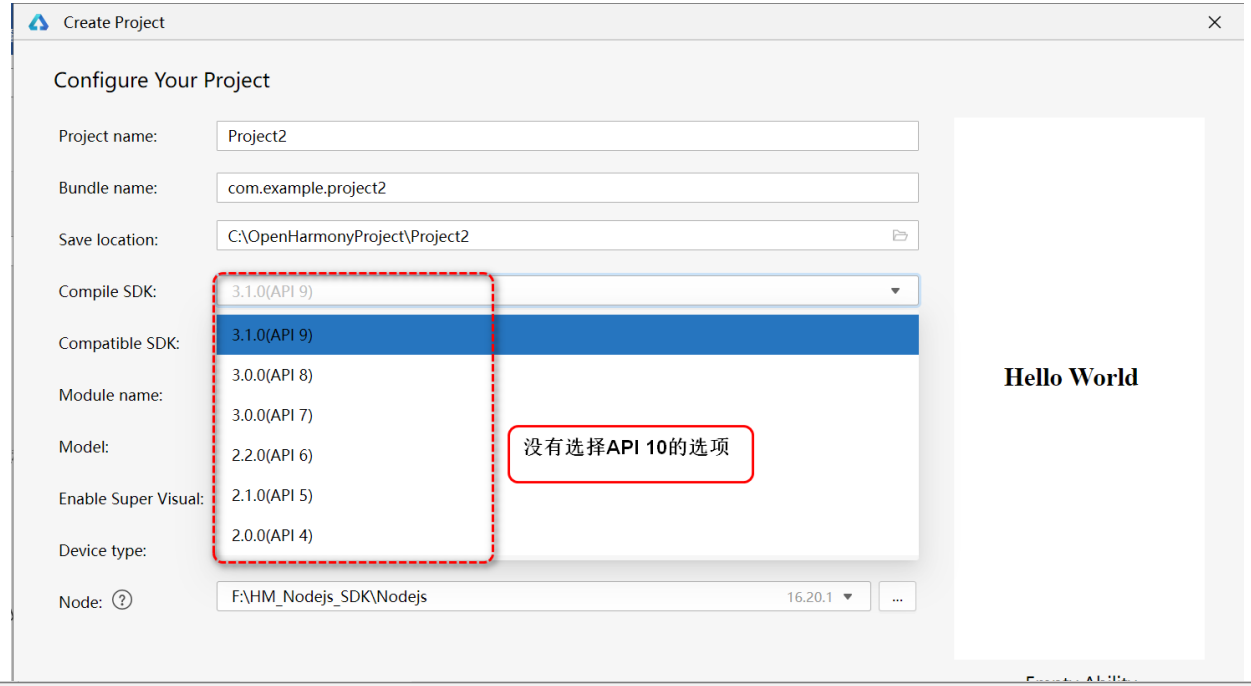


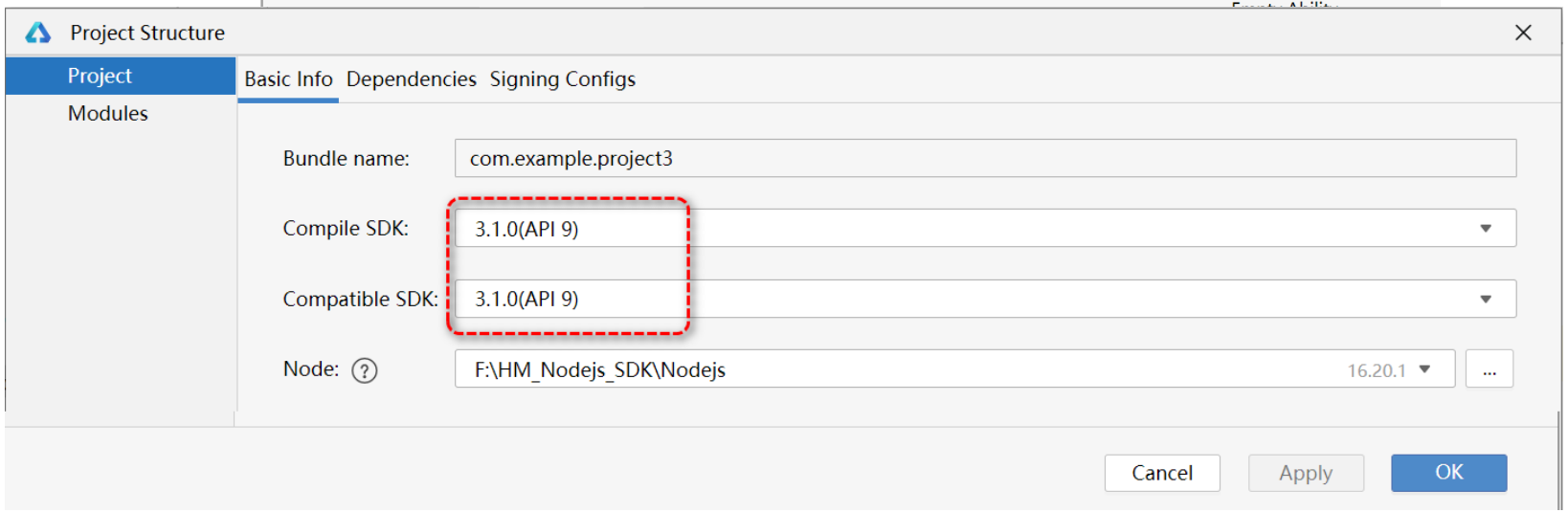
安装与配置OpenHarmony 4.0 Release

将SDK切换为OpenHarmony时，安装路径是空白的，需要点击Edit进行安装OpenHarmony 4.0 Release版本，API版本为10。






Create Project
Configure Your Project
Project name:
Project2
Bundle name:
com.example.project2
Save location:
C:\OpenHarmonyProject\Project2
Compile SDK:
3.1.0(API 9)
Compatible SDK:
3.1.0(API 9)
3.0.0(API 8)
3.0.0(API 7)
2.2.0(API 6)
2.1.0(API 5)
2.0.0(API 4)
Module name:
Model:
Enable Super Visual:
Device type:
Node:
F:\HM_Nodejs_SDK\Nodejs
16.20.1
没有选择API 10的选项
Hello World

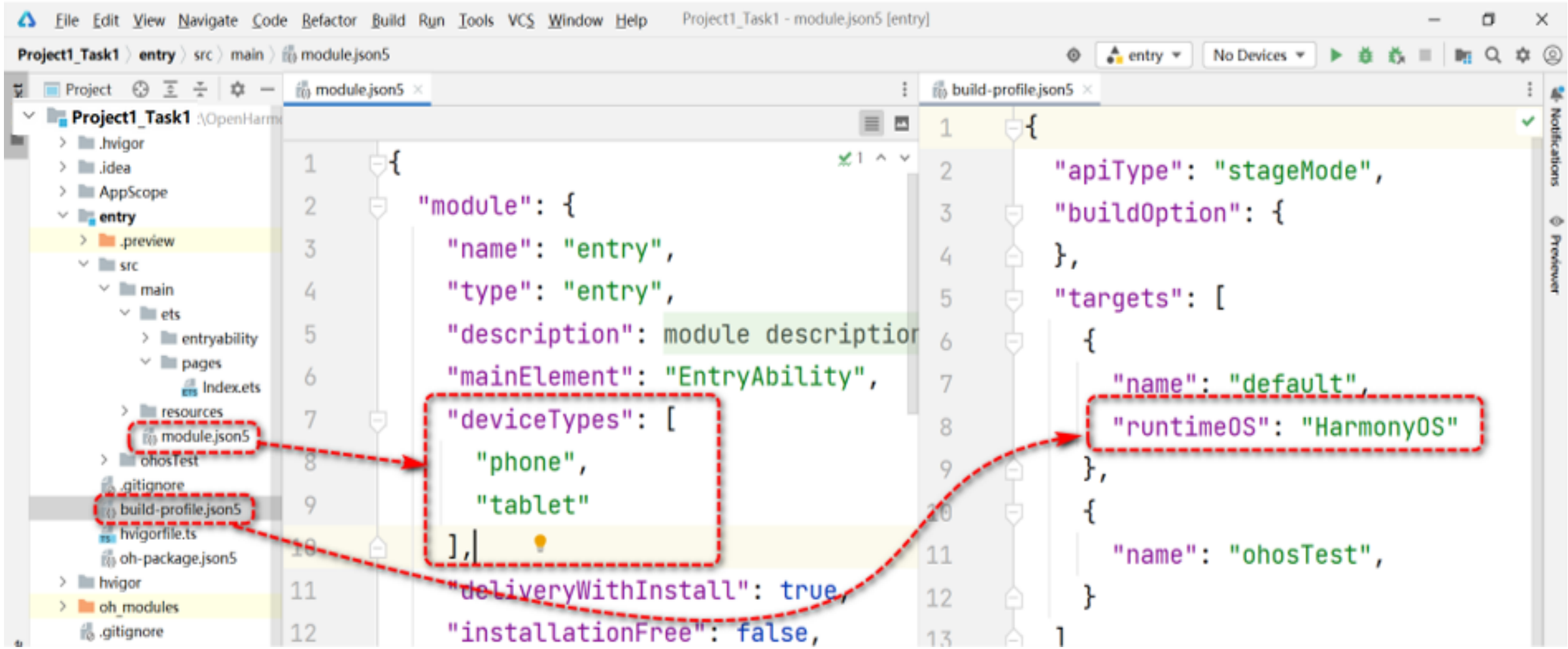

Project Structure
Project
Modules
Basic Info
Dependencies
Signing Configs
Bundle name:
com.example.project3
Compile SDK:
3.1.0(API 9)
Compatible SDK:
3.1.0(API 9)
Node:
F:\HM_Nodejs_SDK\Nodejs
16.20.1
Cancel
Apply
OK

修改项目为OpenHarmony(API 10)

第1处修改是在项目级别的build-profile.json5配置文件中，将编译API的版本配置信息由app节点移动到products节点，并修改版本为10；在products节点中新增目标版本为10和指定运行的操作系统为OpenHarmony，最后进行数据同步。

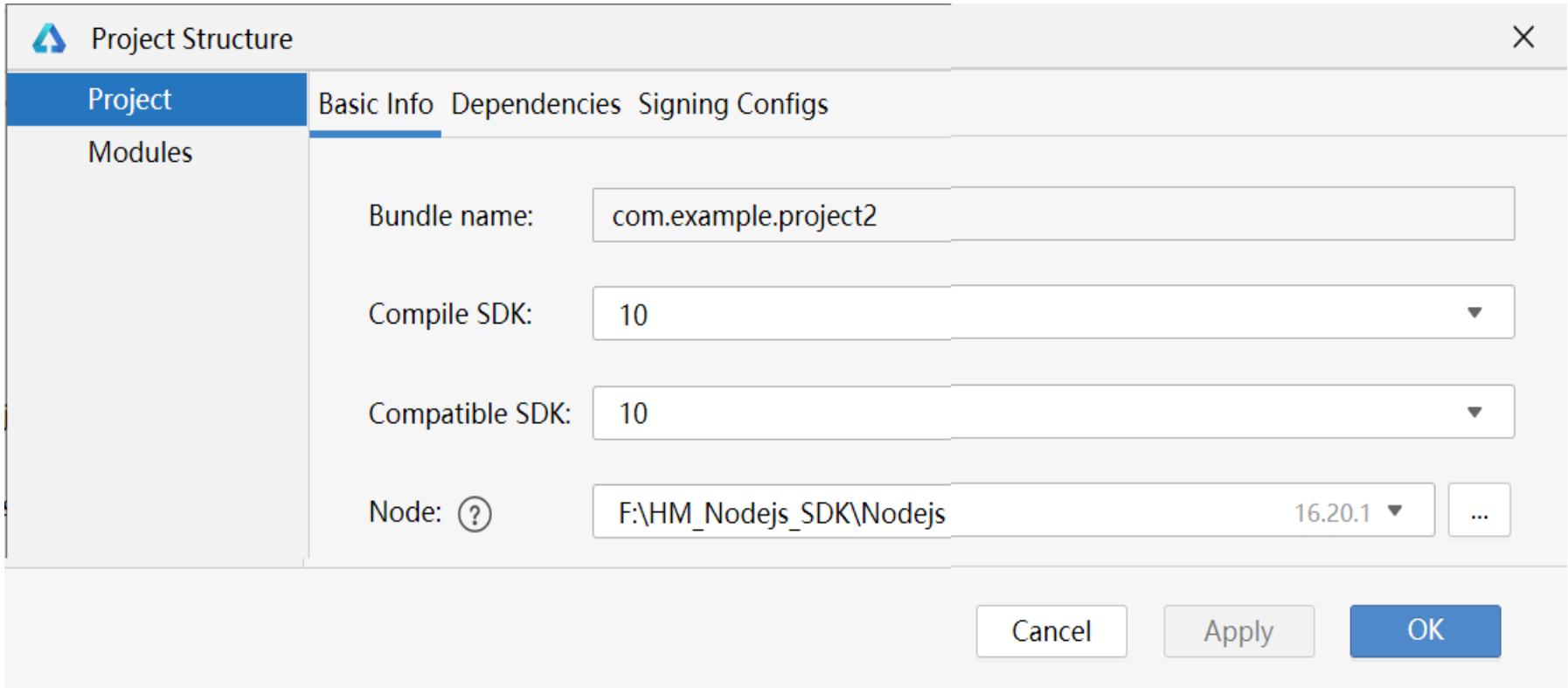
修改项目为OpenHarmony(API 10)

第2处修改是在entry模块级别的build-profile.json5配置文件中，删除targets节点中的runtimeOS这一行代码，进行数据同步。



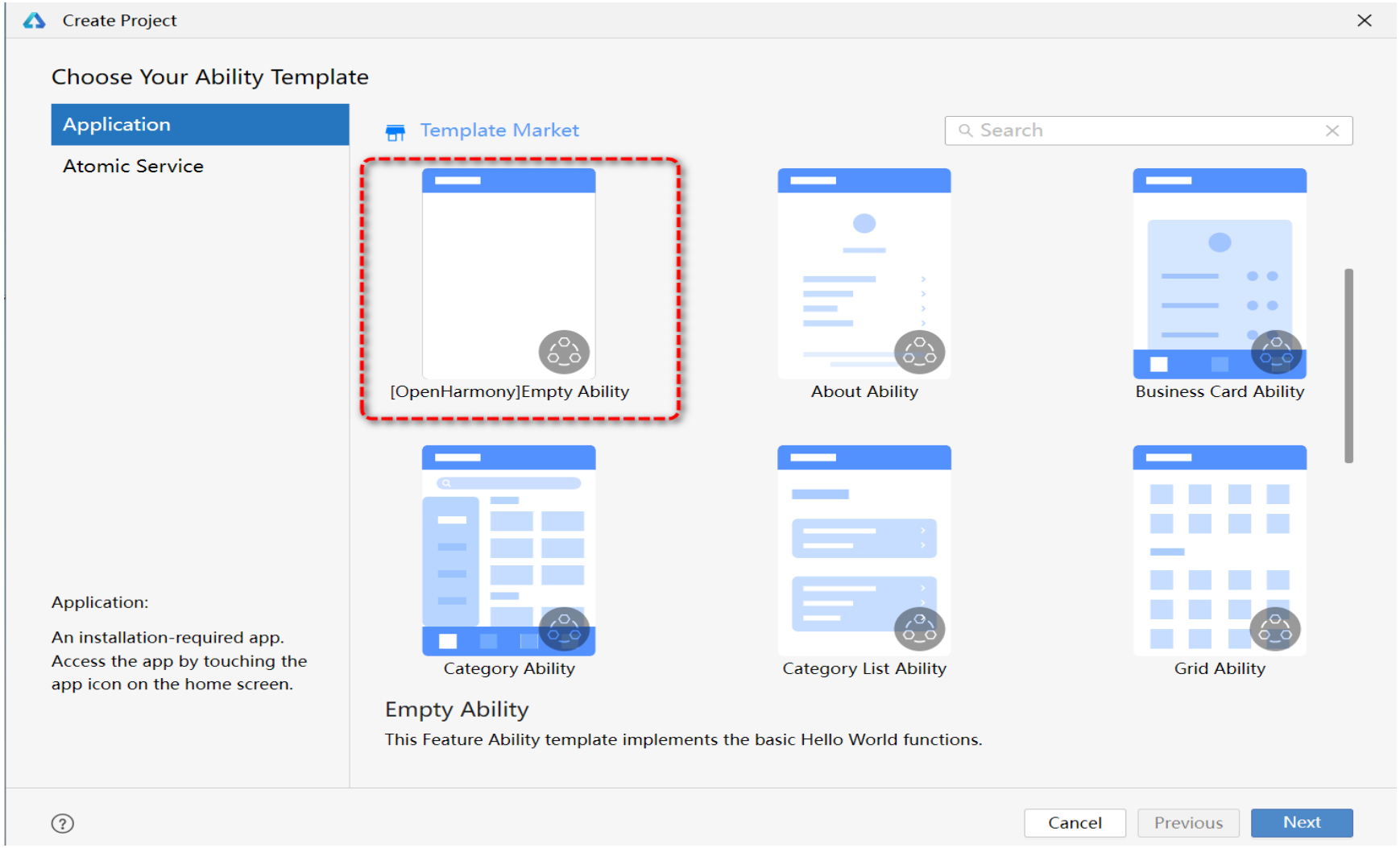
验证项目改为OpenHarmony(API 10)

打开项目的基本信息，验证项目的API版本：



直接创建API 10的OpenHarmony工程

在创建工程时，也可以在模版中选择“[OpenHarmony] Empty Ability”，这样创建出来的工程直接就是API 10的OpenHarmony工程。



需要注意的是，修改项目为API 10和运行操作系统为OpenHarmony，目前对于DevEco Studio 4.0 Release开发工具来说，都是针对单个项目的修改，即便单个项目修改完毕，当通过DevEco Studio 4.0 Release创建新项目时，默认还是API 9 ，运行操作系统还是HarmonyOS。

由于DevEco Studio一直在迭代更新，随着新版本的推出，有可能项目修改为API 10的过程有所变化，读者需要关注DevEco Studio的变化以便使用新的方式进行OpenHarmony的应用开发。

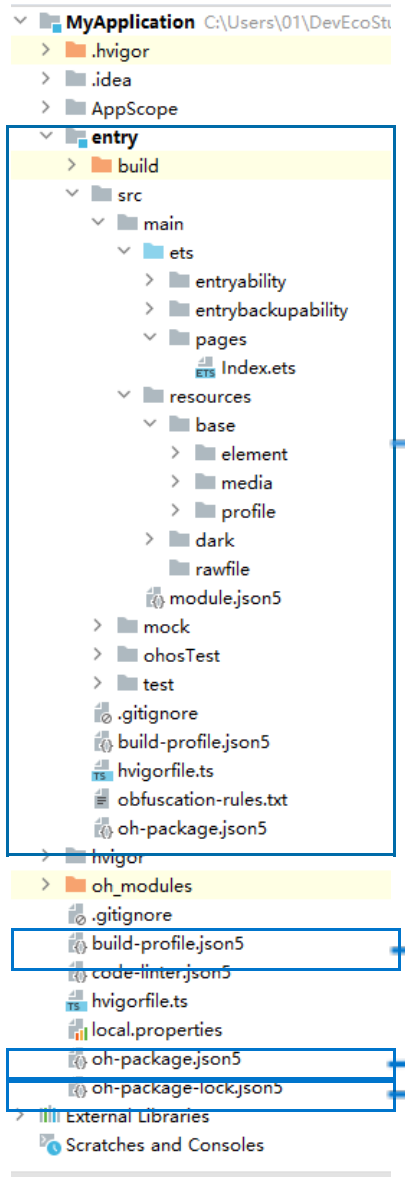
河南大學
Henan University

1.4 OpenHarmony程序结构

1.4 OpenHarmony程序结构

- 熟悉OpenHarmony程序结构，能够归纳OpenHarmony程序中常用的文件和文件夹的作用

1.4 OpenHarmony程序结构



存放程序的代码和资源等文件

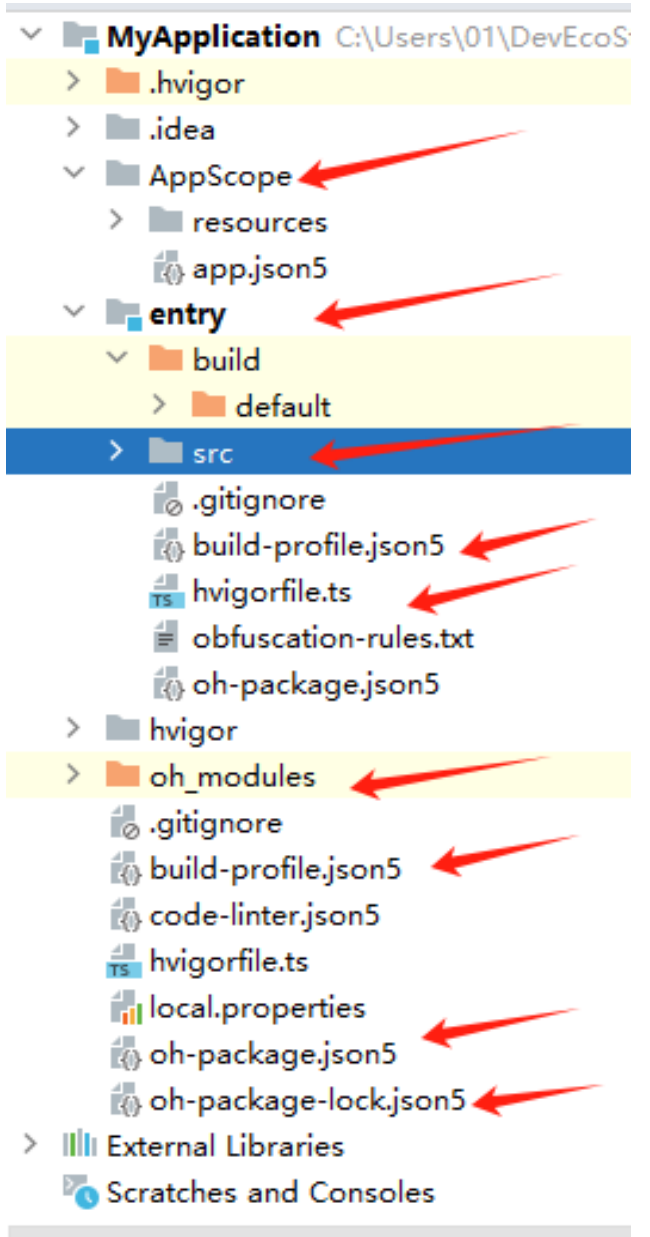
程序的profile构建脚本

配置在OpenHarmony中使用的子项目(Moudle)

1.4 OpenHarmony程序结构

AppScope 中存放应用全局所需要的资源文件。

- entry 是应用的主模块，存放 HarmonyOS 应用的代码、资源等。
- oh_modules 是工程的依赖包，存放工程依赖的源文件。
- build-profile.json5 是工程级配置信息，包括签名、产品配置等。
- hvigorfile.ts 是工程级编译构建任务脚本，hvigor 是基于任务管理机制实现的一款全新的自动化构建工具，主要提供任务注册编排，工程模型管理、配置管理等核心能力。
- oh-package.json5 是工程级依赖配置文件，用于记录引入包的配置信息。



在 AppScope，其中有 resources 文件夹和配置文件 app.json5。AppScope>resources>base 中包含 element 和 media 两个文件夹。

- element 文件夹主要存放公共的字符串、布局文件等资源

entry程序结构

entry>src 目录中主要包含总的 main 文件夹，单元测试目录 ohosTest，以及模块级的配置文件。

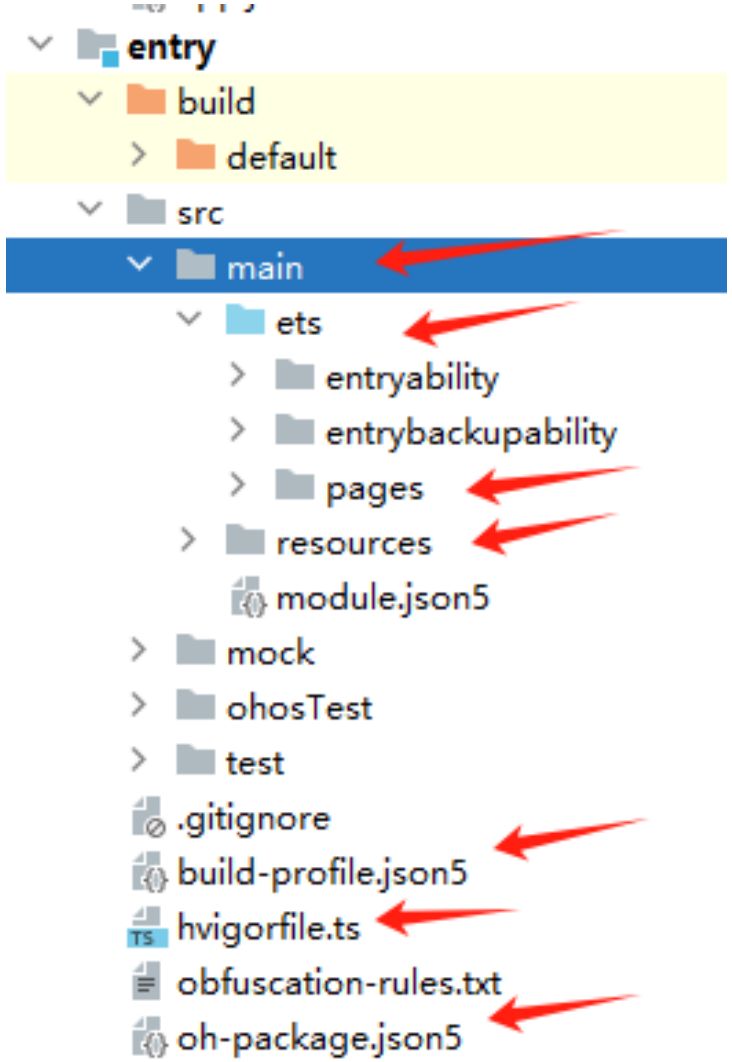
- main 文件夹中，ets 文件夹用于存放 ets 代码，resources 文件存放模块内的多媒体及布局文件

等，module.json5 文件为模块的配置文件。

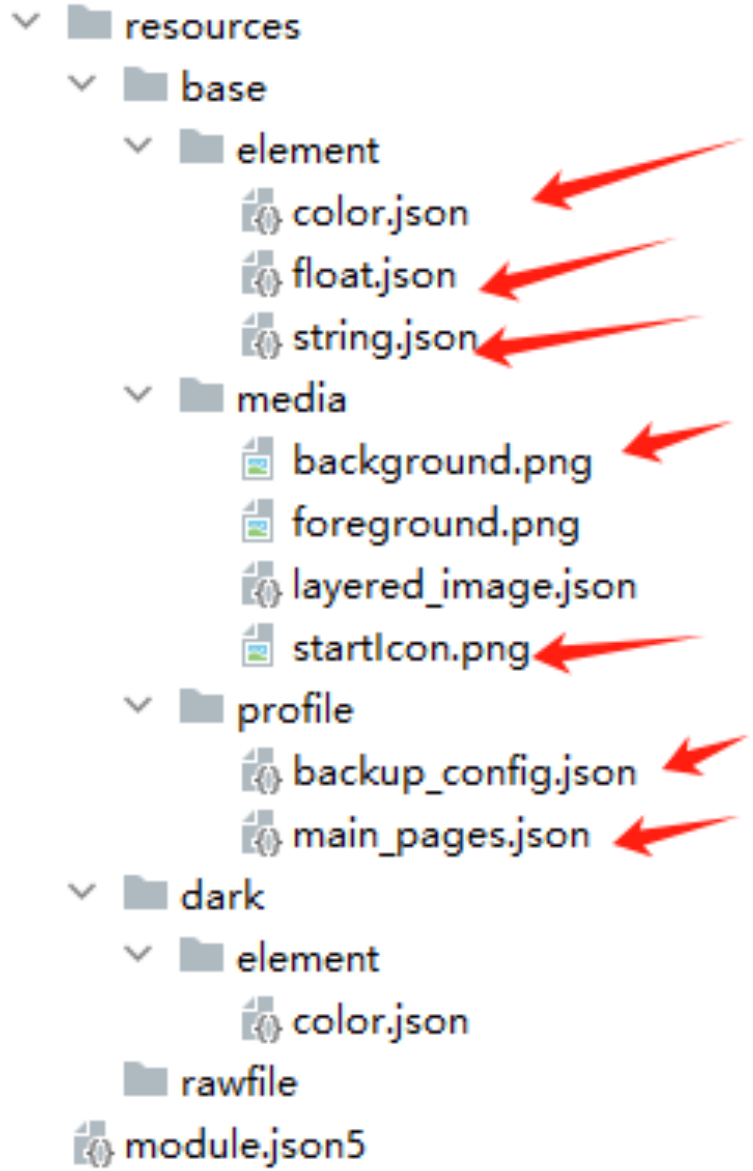
- ohosTest 是单元测试目录。
- build-profile.json5 是模块级配置信息，包括编译构建配置项。
- hvigorfile.ts 文件是模块级构建脚本。
- oh-package.json5 是模块级依赖配置信息文件。

进入 src>main>ets 目录中，其分为 entryability、pages 两个文件夹。

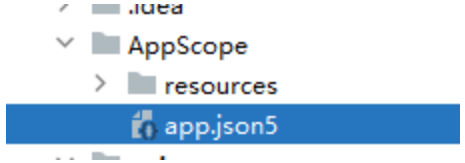
- entryability 存放 ability 文件，用于当前 ability 应用逻辑和生命周期管理。



resources 目录下存放模块公共的多媒体、字符串及布局文件等资源，分别存放在 element、media 文件夹中。



AppScope>app.json5 是应用的全局的配置文件，用于存放应用公共的配置信息



其中配置信息如下：

bundleName 是包名。

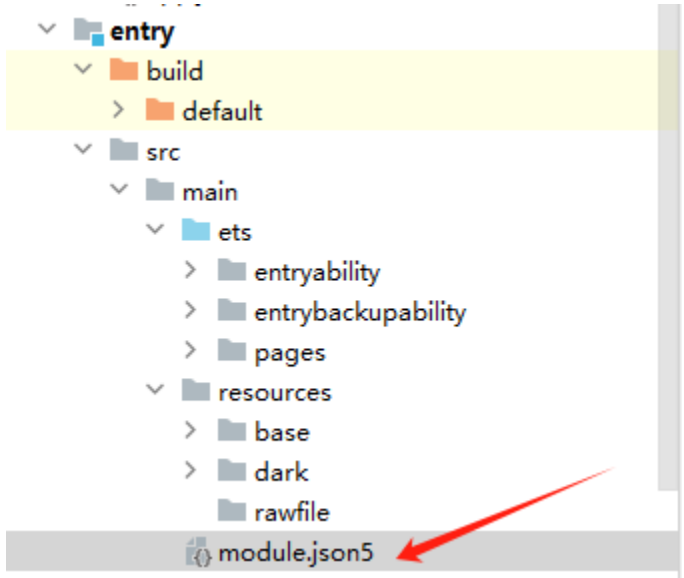
vendor 是应用程序供应商。

versionCode 是用于区分应用版本。

versionName 是版本号。

```
"app": {
  "bundleName": "com.example.myapplication",
  "vendor": "example",
  "versionCode": 1000000,
  "versionName": "1.0.0",
  "icon": "$media:app_icon",
  "label": "$string:app_name"
}
```

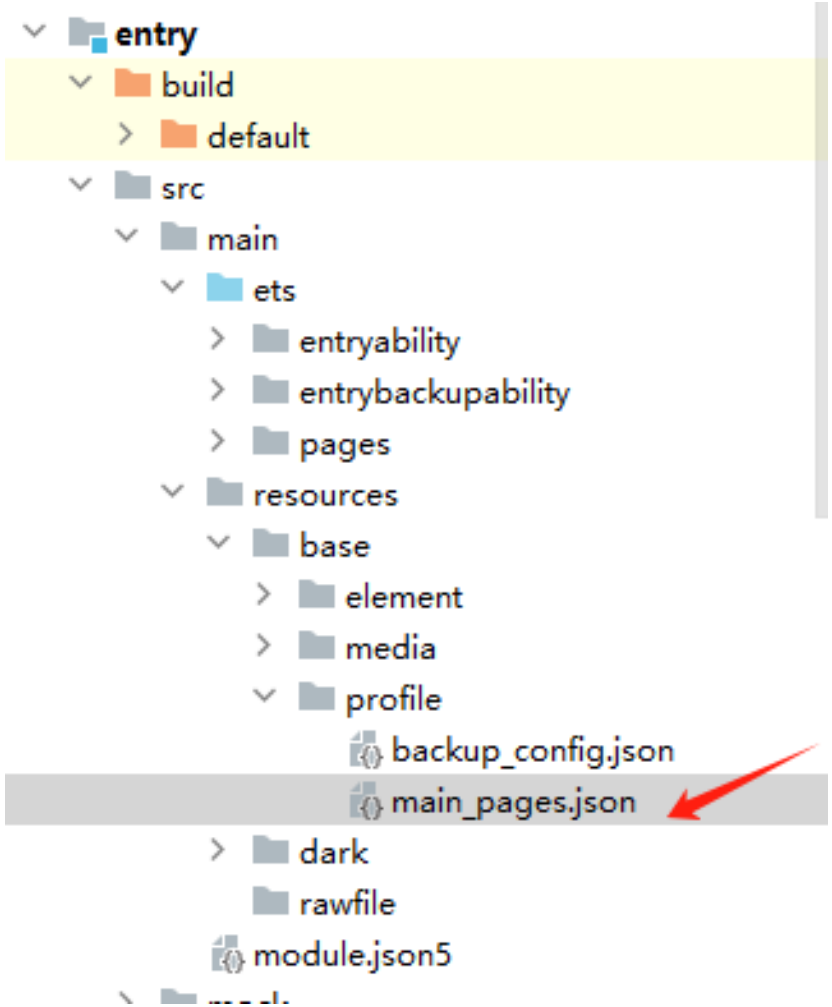
entry>src>main>module.json5 是模块的配置文件，包含当前模块的配置信息。




```
"module": {
  "name": "entry",
  "type": "entry",
  "description": "$string:module_desc",
  "mainElement": "EntryAbility",
  "deviceTypes": [
    "phone",
    "tablet",
    "2in1"
  ],
  "deliveryWithInstall": true,
  "installationFree": false,
  "pages": "$profile:main_pages",
  "abilities": [
    {
      "name": "EntryAbility",
      "srcEntry": "./ets/entryability/EntryAbility.ets",
      "description": "$string:EntryAbility_desc",
      "icon": "$media:layered_image",
      "label": "$string:EntryAbility_label",
      "startWindowIcon": "$media:startIcon",
      "startWindowBackground": "$color:start_window_background",
      "exported": true,
      "skills": [
        {
          "entities": [
            "entity.system.home"
          ],
```

src/main/resources/base/profile/main_pages.json 文件保存的是页面 page 的路径配置信息，所有需要进行路由跳转的 page 页面都要在这里进行配置。

```
{
 "src": [
   "pages/Index"
 ]
}
```



河南大學
Henan University

1.5 资源的管理与使用

1.5 资源的管理与使用

- 掌握资源的管理与使用方式，能够灵活使用程序中的资源

1.5 资源的管理与使用

应用开发过程中，经常需要用到颜色、字体、间距、图片等资源，在不同的设备或配置中，这些资源的值可能不同，开发者们需要根据设备类型、功能需求来设置不同的参数。

资源类型

开发者在工程中自主添加的资源

应用资源

能够直接使用的系统预置的资源

系统资源

资源类别

base目录

当应用的resources目录中没有与设备状态匹配的限定词目录时，会自动引用该目录中的资源文件。

base目录的二级子目录为资源组目录，用于存放字符串、颜色、布尔值等基础元素，以及媒体、动画、布局等资源文件

rawfile目录

支持创建多层子目录，目录名称可以自定义，文件夹内可以自由放置各类资源文件。

rawfile目录的文件不会根据设备状态去匹配不同的资源。

限定词目录

en_US和zh_CN是默认存在的两个限定词目录，其余限定词目录需要开发者自行创建。目录名称由一个或多个表征应用场景或设备特征的限定词组合而成。

河南大學
Henan University

1.6 程序调试

1.6 程序调试

- 掌握单元测试与HiLog的使用，能够完成对程序的调试

1.6.1 单元测试

单元测试是指在OpenHarmony程序开发过程中对最小的功能模块进行测试，单元测试包括OpenHarmony单元测试和LocalUnit.test.ets单元测试。

OpenHarmony单元测试

1. 该测试方式执行测试的时候需要连接OpenHarmony设备。
2. 速度比较慢。
3. 适合需要调用OpenHarmony API的单元测试。

LocalUnit.test.ets单元测试

1. 该测试方式不需要依赖OpenHarmony设备，在本地即可运行。
2. 速度快。
3. 适合只对Java代码功能进行的单元测试。

1.6.1 单元测试

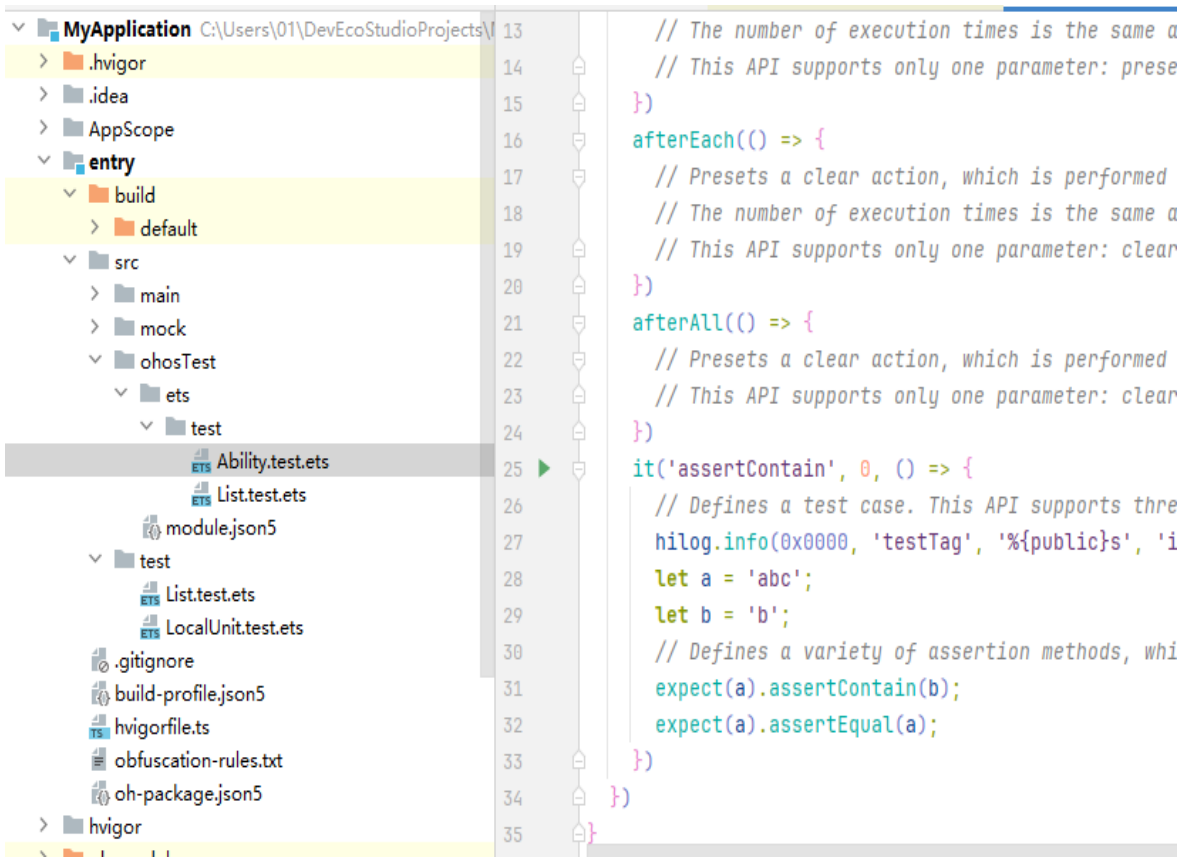
OpenHarmony 4.0版本在创建项目时，会默认在entry/src/ets/ohosTest和entry/src/test文件夹中创建单元测试类abilityTest和LocalUnit单元测试类localUnitTest。

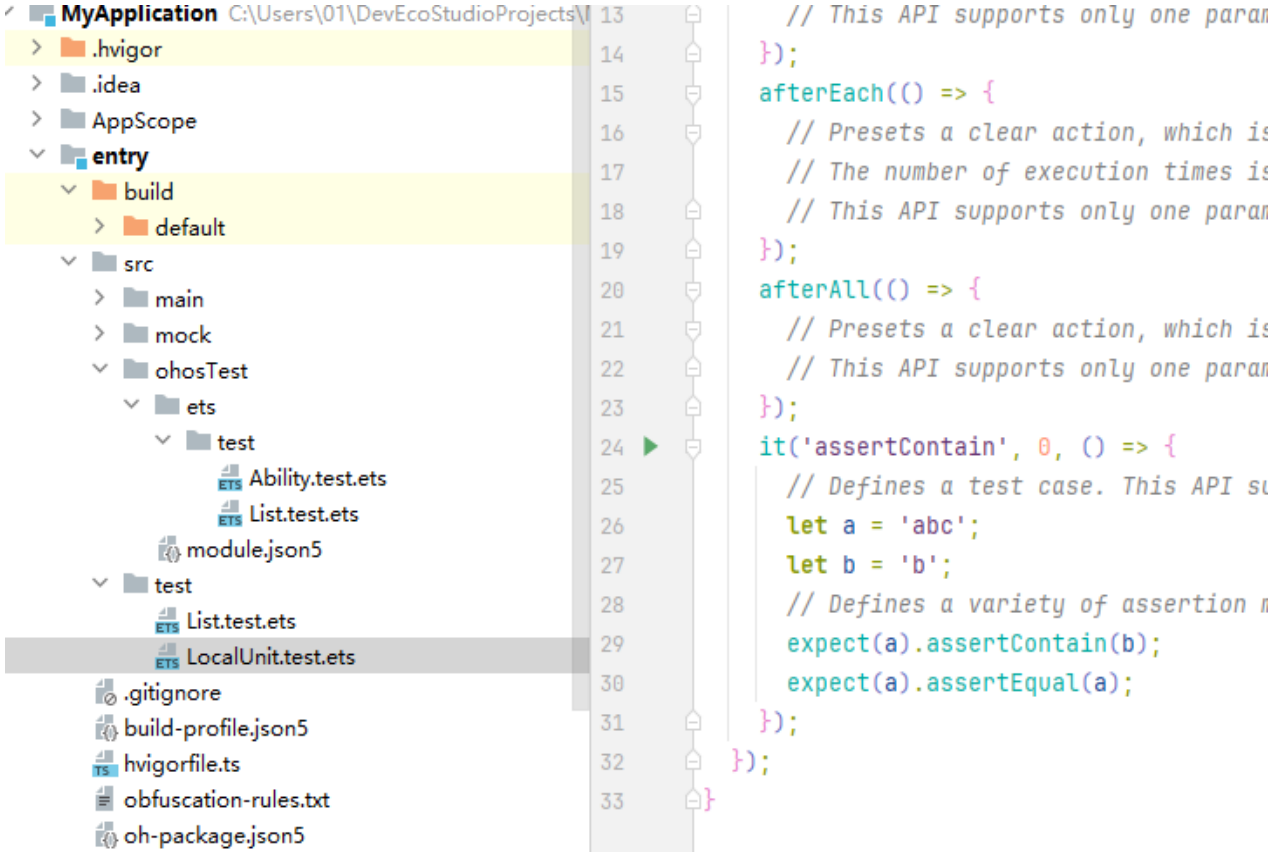
（1） OpenHarmony单元测试类abilityTest

- import { hilog } from '@kit.PerformanceAnalysisKit';
- import { describe, beforeAll, beforeEach, afterEach, afterAll, it, expect } from '@ohos/hypium';

（2）Junit单元测试类LocalUnit

- import { describe, beforeAll, beforeEach, afterEach, afterAll, it, expect } from '@ohos/hypium';


MyApplication C:\Users\01\DevEcoStudioProjects\
.hvigor
.idea
AppScope
entry
build
default
src
main
mock
ohosTest
ets
test
Ability.test.ets
List.test.ets
module.json5
test
List.test.ets
LocalUnit.test.ets
.gitignore
build-profile.json5
hvigorfile.ts
obfuscation-rules.txt
oh-package.json5
hvigor
13 // The number of execution times is the same a
14 // This API supports only one parameter: prese
15 })
16 afterEach(() => {
17 // Presets a clear action, which is performed
18 // The number of execution times is the same a
19 // This API supports only one parameter: clear
20 })
21 afterAll(() => {
22 // Presets a clear action, which is performed
23 // This API supports only one parameter: clear
24 })
25 it('assertContain', 0, () => {
26 // Defines a test case. This API supports thre
27 hilog.info(0x0000, 'testTag', '%{public}s', 'i
28 let a = 'abc';
29 let b = 'b';
30 // Defines a variety of assertion methods, whi
31 expect(a).assertContain(b);
32 expect(a).assertEqual(a);
33 })
34 })
35 }


MyApplication C:\Users\01\DevEcoStudioProjects\
.hvigor
.idea
AppScope
entry
build
default
src
main
mock
ohosTest
ets
test
Ability.test.ets
List.test.ets
module.json5
test
List.test.ets
LocalUnit.test.ets
.gitignore
build-profile.json5
hvigorfile.ts
obfuscation-rules.txt
oh-package.json5
13 // This API supports only one param
14 });
15 afterEach(() => {
16 // Presets a clear action, which is
17 // The number of execution times is
18 // This API supports only one param
19 });
20 afterAll(() => {
21 // Presets a clear action, which is
22 // This API supports only one param
23 });
24 it('assertContain', 0, () => {
25 // Defines a test case. This API su
26 let a = 'abc';
27 let b = 'b';
28 // Defines a variety of assertion m
29 expect(a).assertContain(b);
30 expect(a).assertEqual(a);
31 });
32 });
33 }

单元测试

```
it('assertContain', 0, () => {
    hilog.info(0x0000, 'testTag', '%{public}s', 'it begin');
    let a = 'abc';
    let b = 'b';
    // Defines a variety of assertion methods, which are used to
declare expected boolean conditions.
    expect(a).assertContain(b);
    expect(a).assertEqual(a);
})
```

a变量赋值

b变量赋值

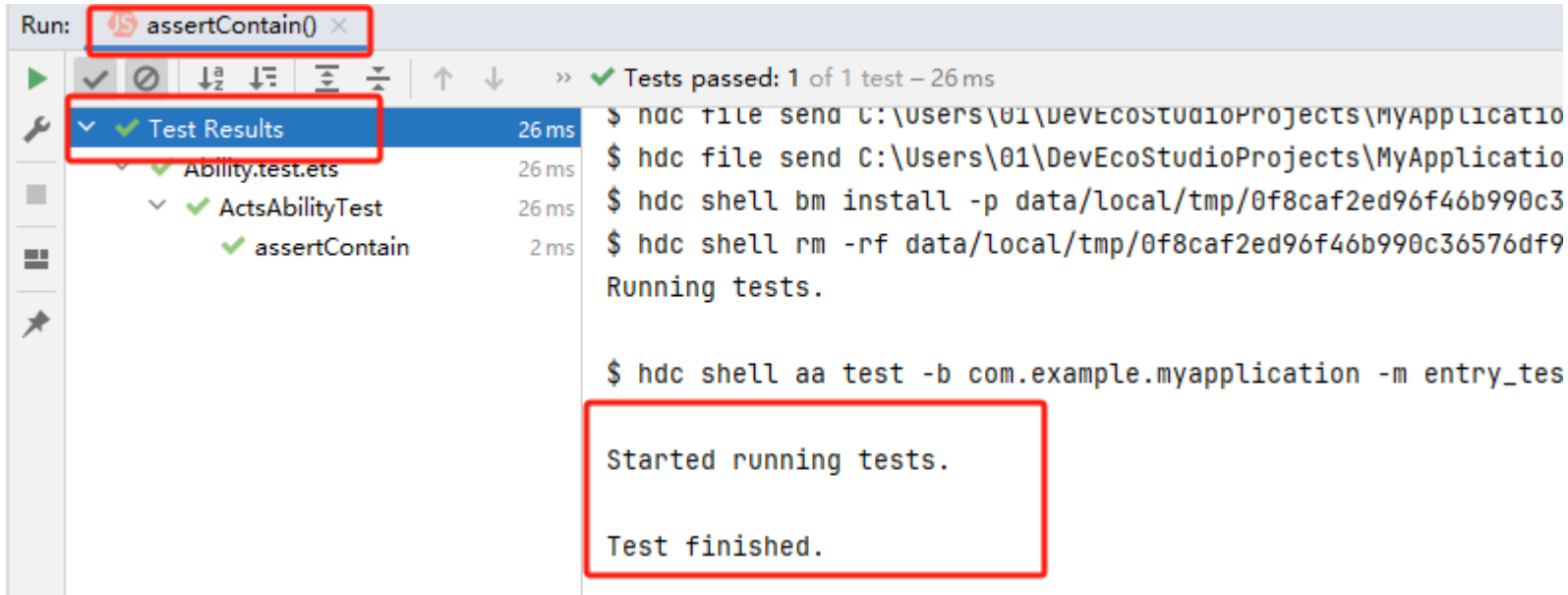
断言，期望两个参数值相等

1.6.1 单元测试

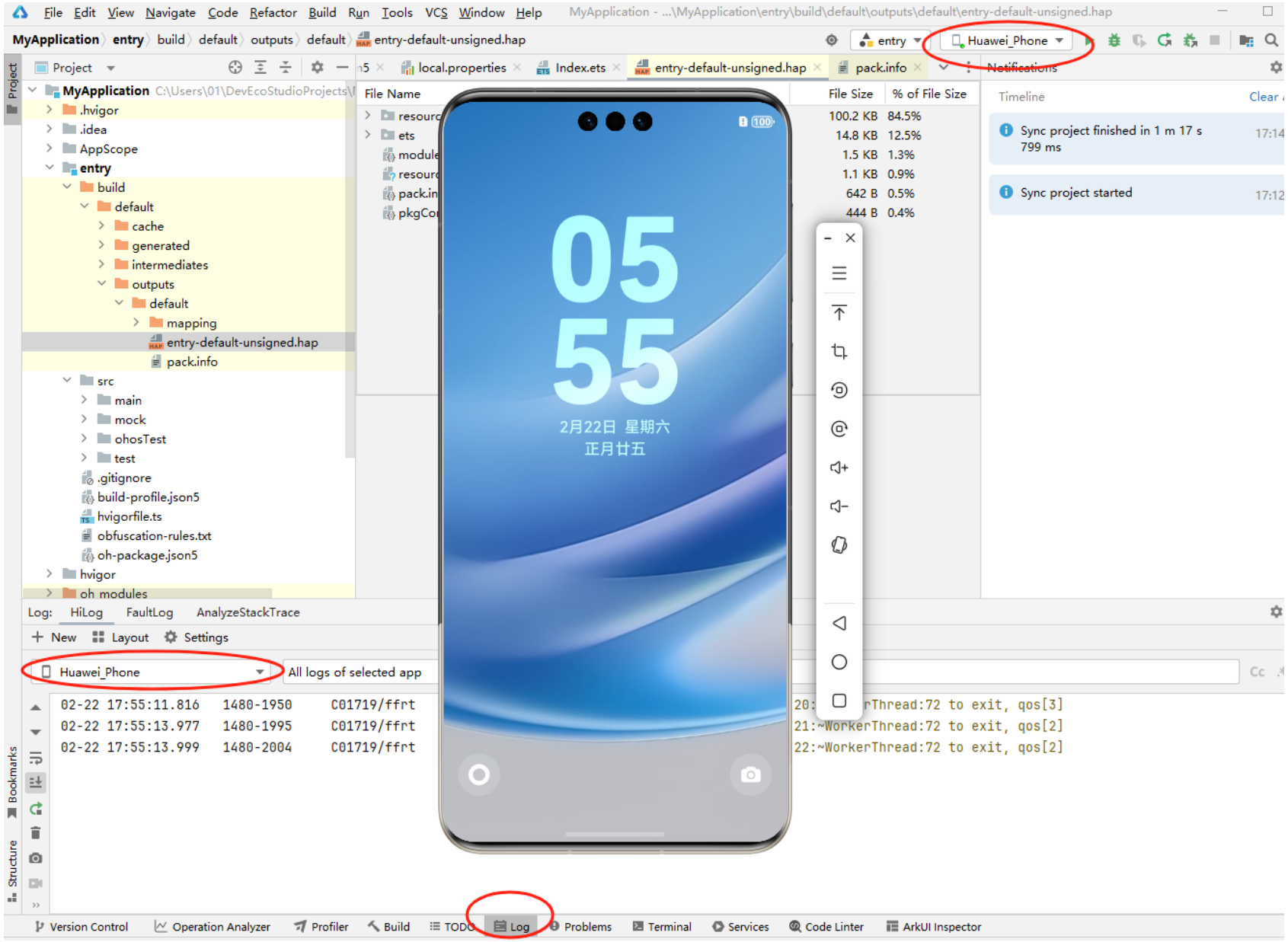
在上一页代码中的方法assertContain()上右击，在弹出框中选择【Run assertContain()】。将程序运行到模拟器后，在OpenHarmony底部导航栏中单击“ ”图标查看运行成功的结果。

4: Run

测试结果正常



1.6.2 HiLog的使用



1.6.2 HiLog的使用

HiLog是系统提供的一个日志系统，它允许应用和服务按照指定的级别、标识和格式字符串输出日志内容，帮助开发者了解应用的运行状态，更好地调试程序。

输出的日志内容分为五个级别。

级别	说明
Debug	输出DEBUG级别日志，仅用于应用/服务调试。
Info	输出INFO级别日志，表示普通的信息。
Warn	输出WARN级别日志，表示存在警告。
Error	输出ERROR级别日志，表示存在错误。
Fatal	输出FATAL级别日志，表示出现致命

1.6.2 HiLog的使用

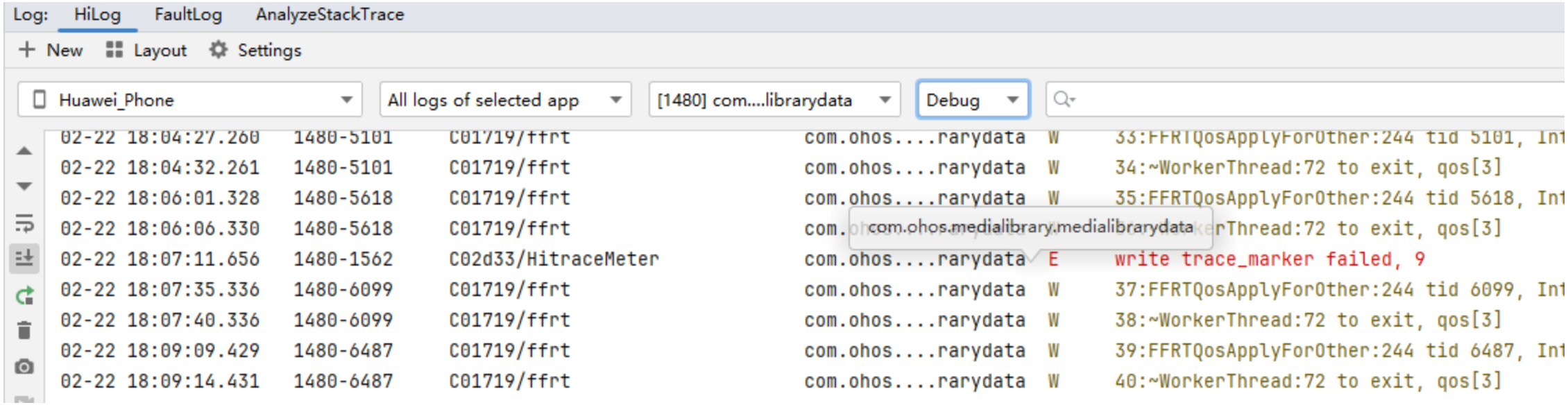
```
    // 打印INFO级别的日志
hilog.info(0xFF00, "LogExample", "This is an info log message.");
}
```

打印信息的标签

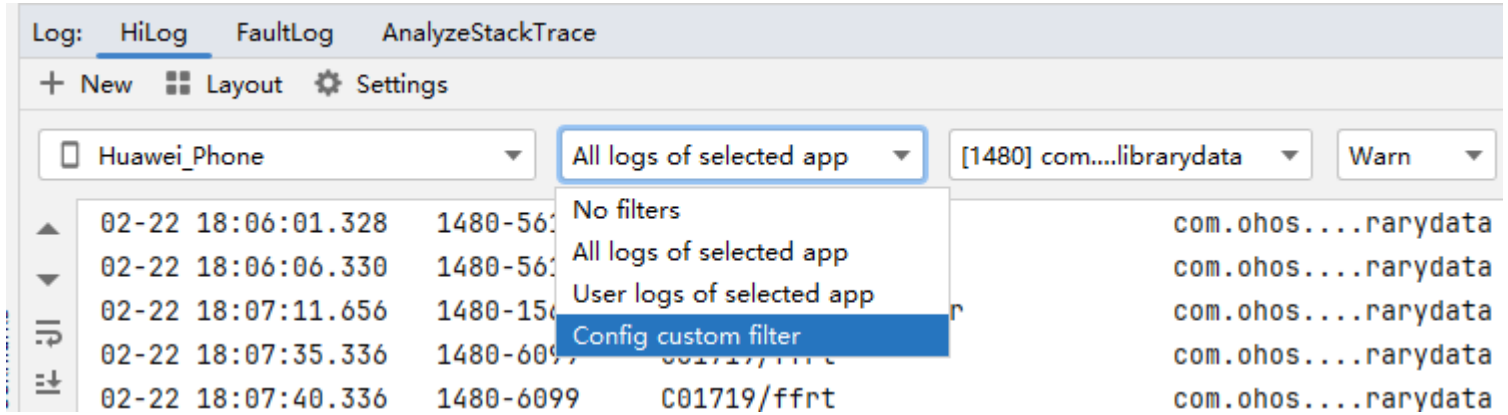
需要打印的信息

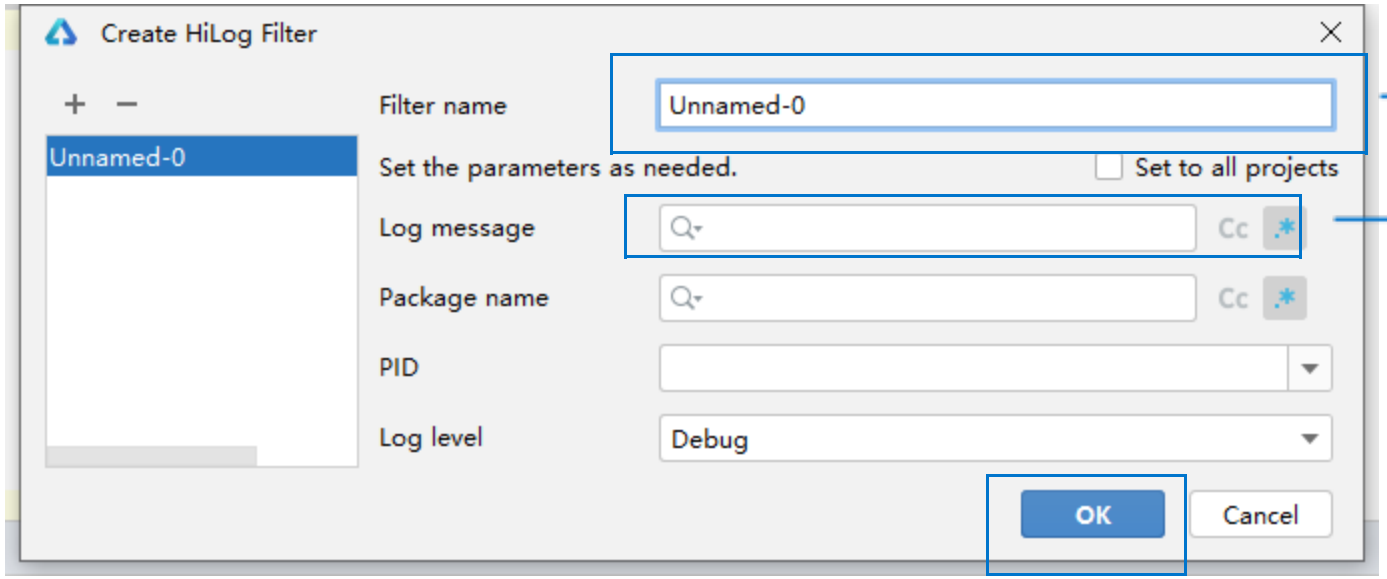
1.6.2 HiLog的使用

运行上一页中的程序，此时HiLog窗口中打印的Log信息，如下图所示。



1.6.2 HiLog的使用



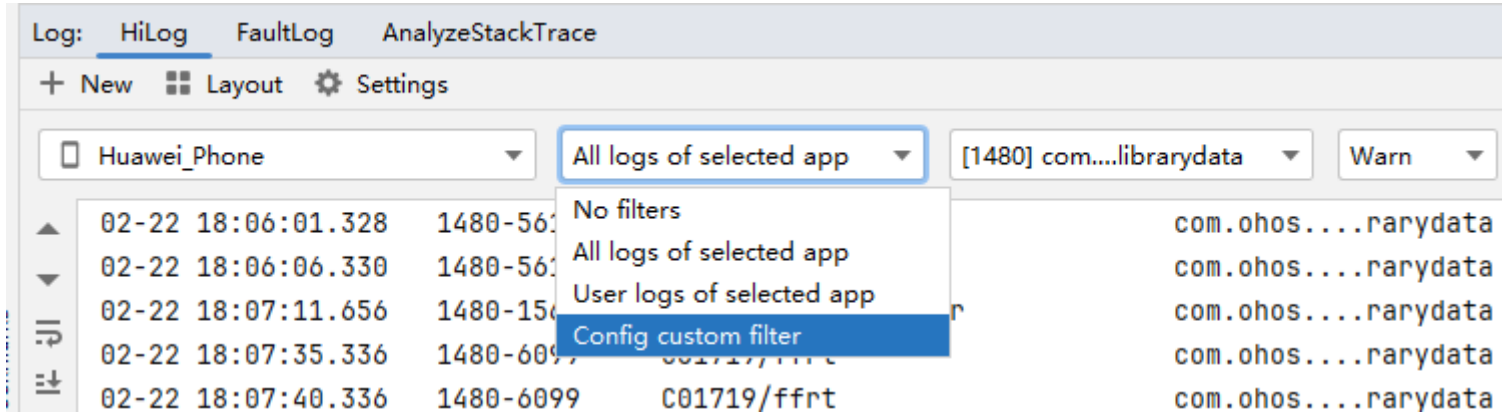


过滤器名称

根据定义的TAG过滤信息

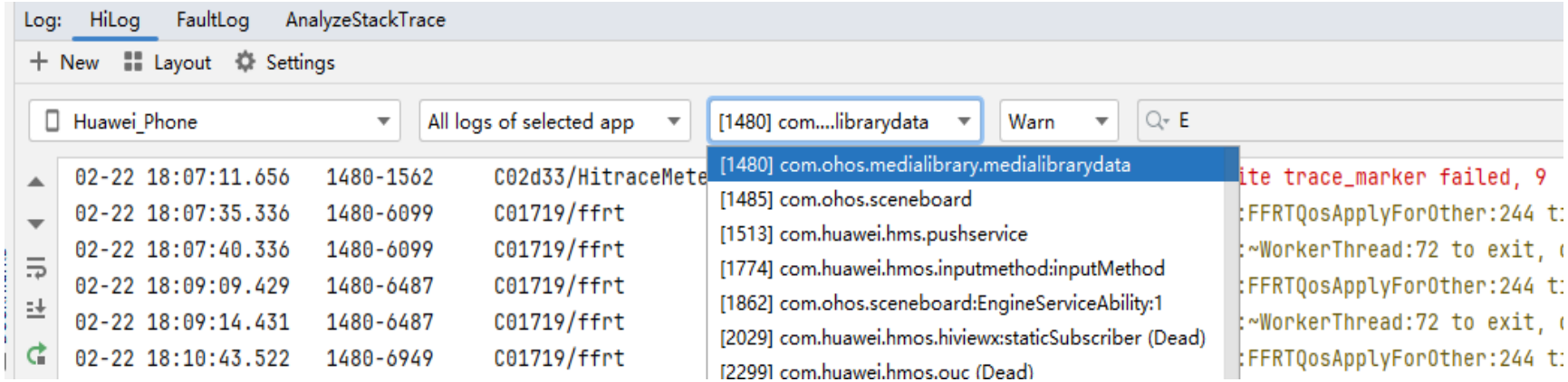
点击创建完成

1.6.2 HiLog的使用



1.6.2 HiLog的使用

还可以选择详细的模块、根据Log级别等方式过滤信息。



级别	显示信息	日志信息颜色
Debug	调试信息	蓝色
Info	全部信息	黑色
Warn	一般信息	绿色
Error	警告信息	红色
Fatal	错误信息	红色

3.1 TypeScrip 快速入门

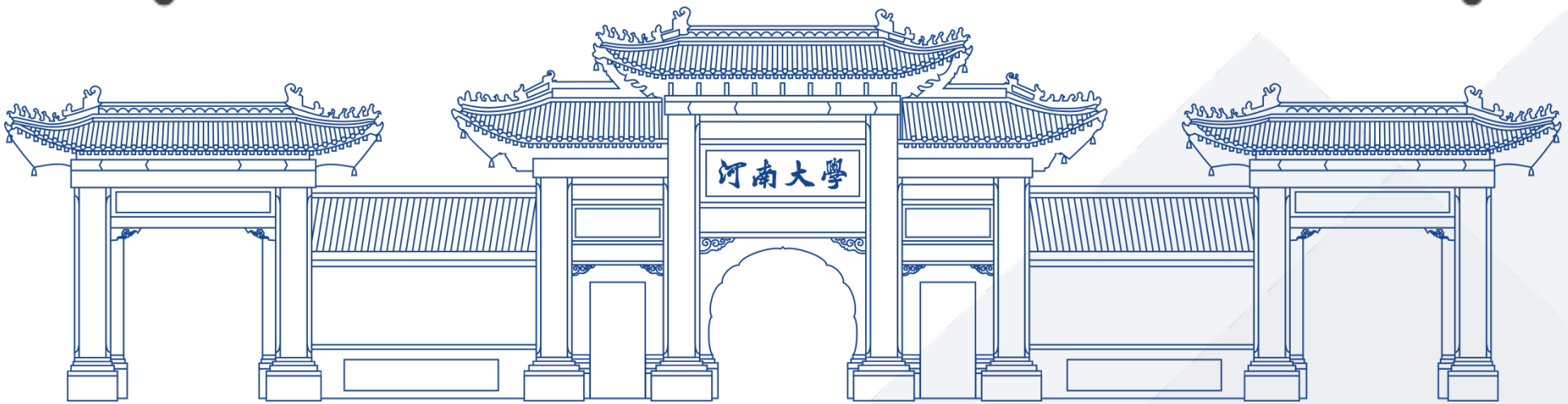
学习 TypeScript 对于 HarmonyOS 应用开发至关重要。在 HarmonyOS 中，主力编程语言为 ArKTS，它是基于 TypeScript 的一种语言，其通过与 ArkUI 框架的匹配，拓展了声明式 UI 和状态管理等能力，使开发者能够以更简洁自然的方式开发跨端应用。TypeScript 本身是 JavaScript 的超集，通过引入静态类型定义等特性，提高了代码的可维护性和可读性，有助于在编码阶段检测潜在错误，提高开发效率另外，学习 TypeScript 还为处理 HarmonyOS 应用中的 UI 和应用状态提供了更强大的支持，在并

ARKTS开发语言

河南大學
Henan University

第2章 TypeScript面向对象

软件学院 · 吕贺贺



TypeScript面向对象语法

01 任务描述

本任务主要学习TypeScript开发语言的面向对象中的类与接口的使用，以支撑OpenHarmony基于ArkTS的声明式UI开发需求。

02 TypeScript面向对象

1. 认识和使用TypeScript的类与对象
2. 认识和使用TypeScript的接口声明与实现

河南大學 Henan University

TypeScript面向对象

01

认识和使用TypeScript的类与对象

认识TypeScript的类与对象

TS支持面向对象的所有特性，在TS中使用类描述了所创建的对象共同的属性和方法。

定义类-语法

- 定义类的关键字是class，类中可以有属性和方法，属性可以定义时初始化，也可以通过构造方法初始化。

```
class 类名{
 属性;
 方法;
}
```

认识TypeScript的类与对象

定义类-示例

```
class Car {
 private color: string = '红色'; //属性，可以定义时初始化
 private type: number = 1;      //属性
 //使用构造方法给属性做数据初始化
 public constructor(color: string,type: number) {
   this.color = color;
   this.type = type;
 }
 public getColor() {return this.color; } //获取属性color的值
 public setColor(color: string) {this.color = color; }//设置
//type的get()和set()方法如下
public show(): string {return "Car show() is running..."; }//普通方法
}
```

认识TypeScript的类与对象

创建并使用对象

- 创建类的实例化对象的关键字是new

```
let car :Car = new Car("红色",2);
console.log(car.getColor());//答案是什么？
```

认识TypeScript的类与对象

类的继承

- 类的继承用关键字extends，子类继承自父类后，可以重写从父类继承下来的方法，也可以新增新的方法。

```
class Bus extends Car{
  //增加子类的方法略
  //重写父类的show()方法
  show(): string {
    return "Bus show() is running.....";
  }
}
```

认识TypeScript的类与对象

类的继承

```
class Bus extends Car{
 private name:string;
 constructor(color: string,type: number, name: string){
  //调用父类的构造函数
       super(color, type);
        this.name = name;
 }
 show(): string {
   return this.getColor()+`,他的名字${this.name}`;
 }
}
let car1 = new Bus("红色", 2, "保时捷")
console.log(car1.show())   //答案是什么?
```

认识TypeScript的类与对象

继承中的多态

- 多态是指由继承而产生了相关的不同的类，对同一个方法可以有不同的响应。多态是有前提的，在子类继承父类并重写父类的方法后，使用父类引用指向子类对象，访问的是子类经过重写的方法，从而呈现子类的状态。

认识TypeScript的类与对象

```typescript
class Animal {
  name: string;

  constructor(name: string) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound`);
  }
}
```

```typescript
class Dog extends Animal {
  speak() {
    console.log(`${this.name} says: Woof!`);
  }
}

class Cat extends Animal {
  speak() {
    console.log(`${this.name} says: Meow!`);
  }
}
```

```typescript
function makeAnimalSpeak(animal: Animal) {
  animal.speak();
}

const myDog = new Dog("Buddy");
const myCat = new Cat("Whiskers");

makeAnimalSpeak(myDog); // 输出: Buddy says: Woof!
makeAnimalSpeak(myCat); // 输出: Whiskers says: Meow!
```

认识TypeScript的类与对象

```
class Animal {
  speak(): void {
    console.log("The animal makes a sound")
  }
}

class Dog extends Animal {
  speak(): void {
    console.log("Woof!");
  }
}
```

```
const myDog: Animal = new Dog();
myDog.speak(); // 输出: Woof!
```

认识TypeScript的类与对象

```
class Animal {
  speak(): void {
    console.log("The animal makes a sound");
  }
}

class Dog extends Animal {
  speak(): void {
    console.log("Woof!");
  }

  // Dog类特有的方法
  fetch(): void {
    console.log("Dog is fetching the ball");
  }
}
```

```
const myDog: Animal = new Dog();
myDog.speak(); // 输出: Woof!
```

```
myDog.fetch();
```

认识TypeScript的可迭代对象

- 当对象实现Symbol.iterator属性时，认为它是可迭代的。
- 内置的Array, Map, Set, String， Int32Array， Uint32Array等都具有可迭代性。

for…of 语句

for…of 会遍历可迭代的对象，调用对象上的 symbol.iterator 方法。

在数组上使用for…of的简单例子定义 数组，使用 for-of 语句进行循环遍历，可以打印数组中的元素。

认识TypeScript的可迭代对象

```
//可迭代性，可迭代对象
let str1: any = "abcdefg";
console.log(str1[Symbol.iterator]);
// for.. of 遍历可迭代对象中的每个元素，i代表其中的每一个元素
for (let i of str1){
 console.log(i)
 // 不是console.log(str1[i])
}
// for..in i代表其中的一个下标
for (let j in str1){
 console.log(str1[j])
 console.log(j)
}
```

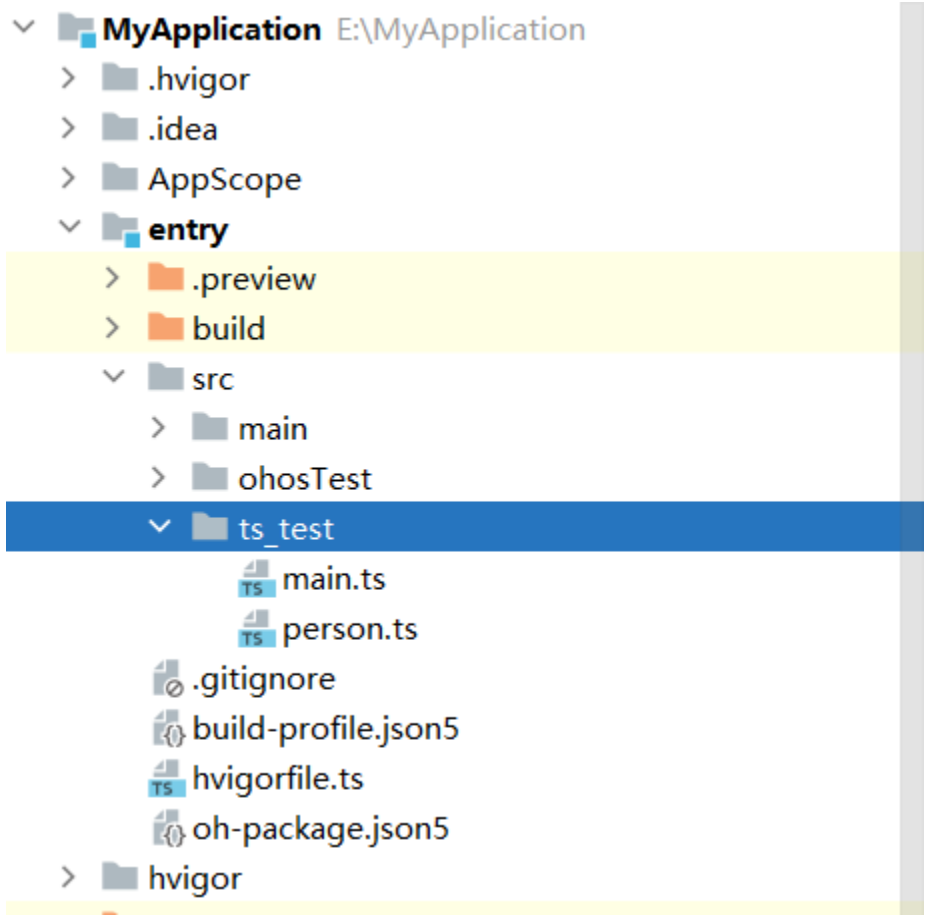
```
//map类型：每个元素都是由两个组成：key，value
let map1 = new Map();
map1.set('a', 1);
map1.set('b', 2);
map1.set('c', 3);
for (let k of map1){
 console.log(k)
 console.log(k[0])
}
```

认识TypeScript的模块

模块及导入

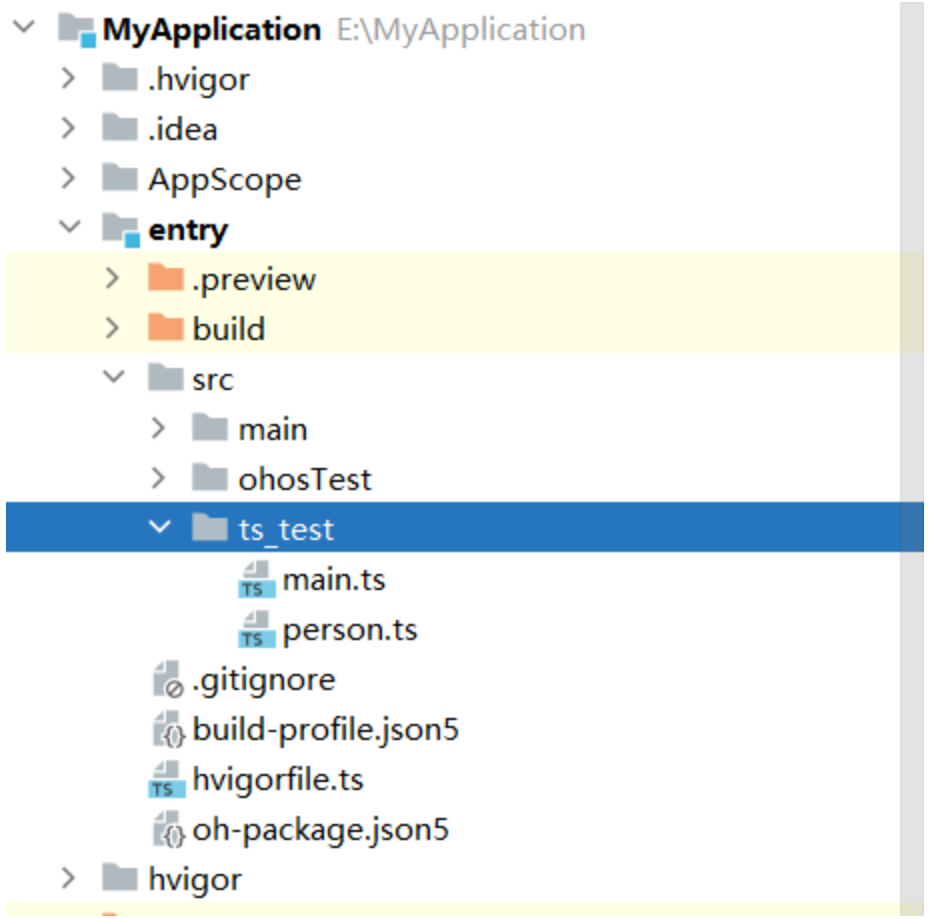
- 随着应用增加，通常要将代码拆分成多个文件，即所谓的模块(module)。模块可以相互加载，并可以使用特殊的指令**export**和**import**来交换功能，从另一个模块调用一个模块的函数。
- 两个模块间的关系是通过在文件级别上使用**import**和**export** 建立的。模块里面的变量、函数和类等在模块外部是不可见的，除非明确地使用export导出它们。类似地，我们必须通过 import 导入其他模块导出的变量、函数、类等。

认识TypeScript的模块



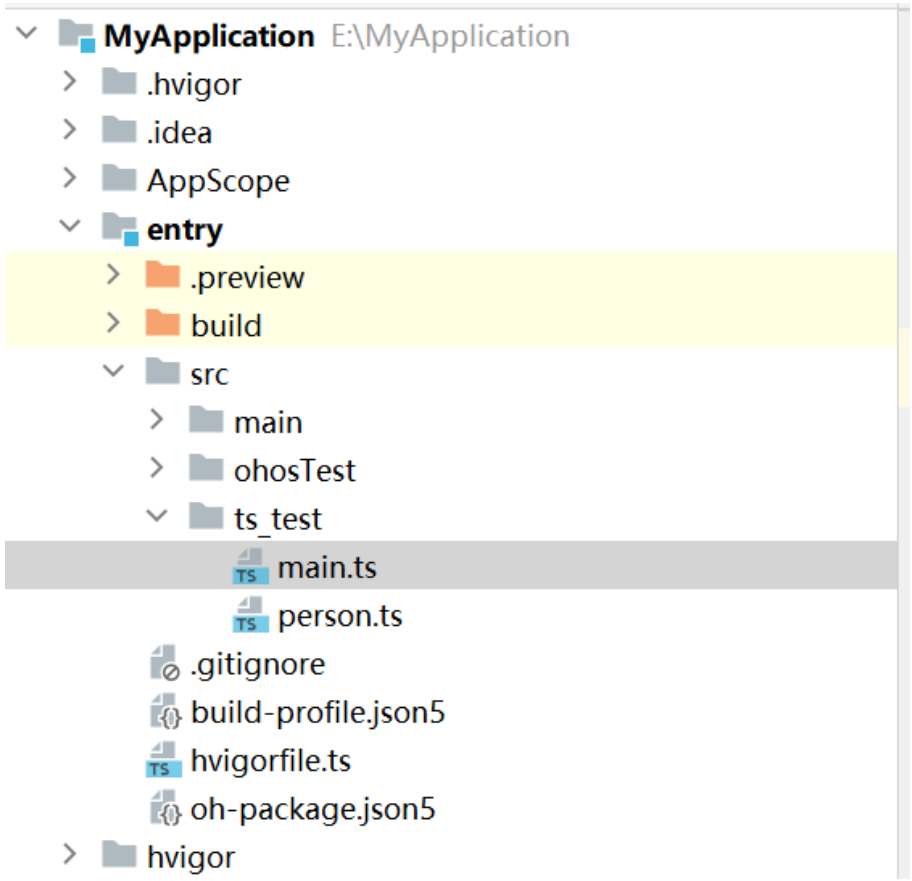

```ts
// 定义一个Person类
export  class Person{
 // 私有属性
  private name: string;
 private age: number;
 //定义构造函数(方法)
 constructor(name: string, age: number){
  this.name = name;
  this.age = age;
 }
 //公共成员函数（方法）
  public getPersonInfo():string{
  return `我的名字是：${this.name}，年龄${this.age}`
 }
}
//构造Person类的实例
let p = new Person('张三', 45)
console.log(p.getPersonInfo())
```

认识TypeScript的模块



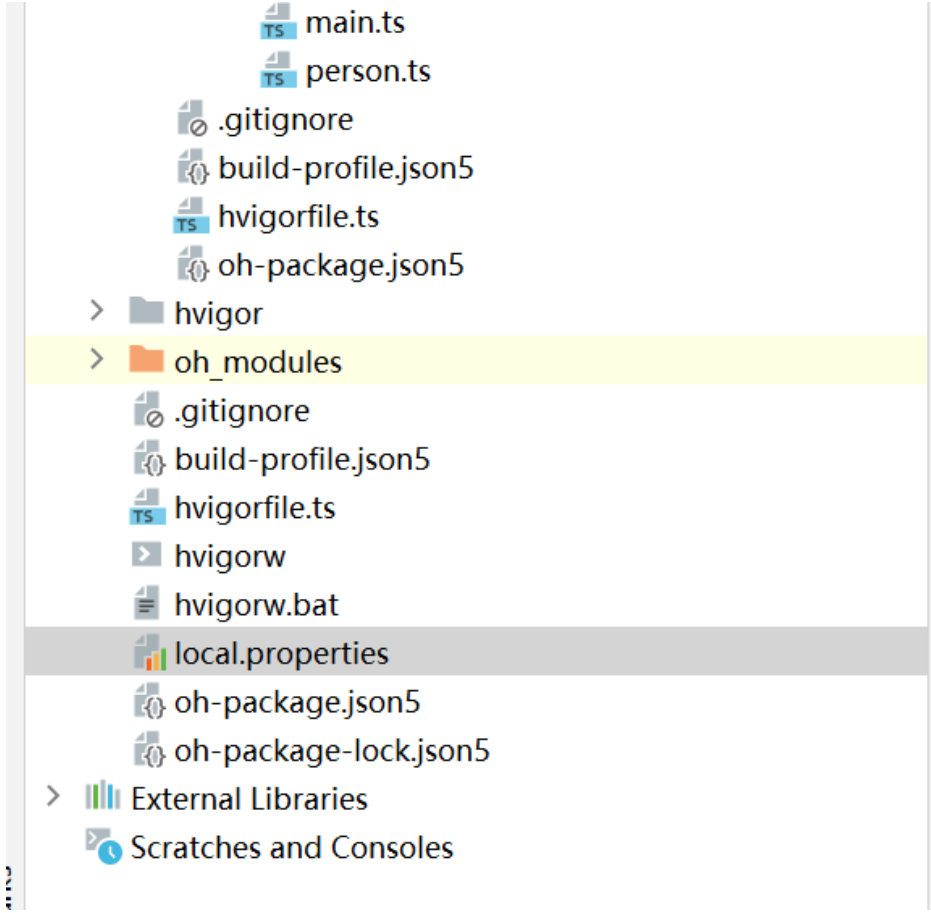

```
//定义一个类继承Person
export class Employee extends Person{
 private deparment:string;
 constructor(name: string, age: number,
deparment: string){
  //调用父类的构造函数
      super(name, age)
  this.deparment = deparment;
 }
 public getEmpInfo():string{
  return this.getPersonInfo() + `， 我工作的部门
${this.deparment}`
 }
}

let emp = new Employee('王五', 45, '行政部')
console.log(emp.getEmpInfo());
console.log(emp.getPersonInfo())
```

认识TypeScript的模块



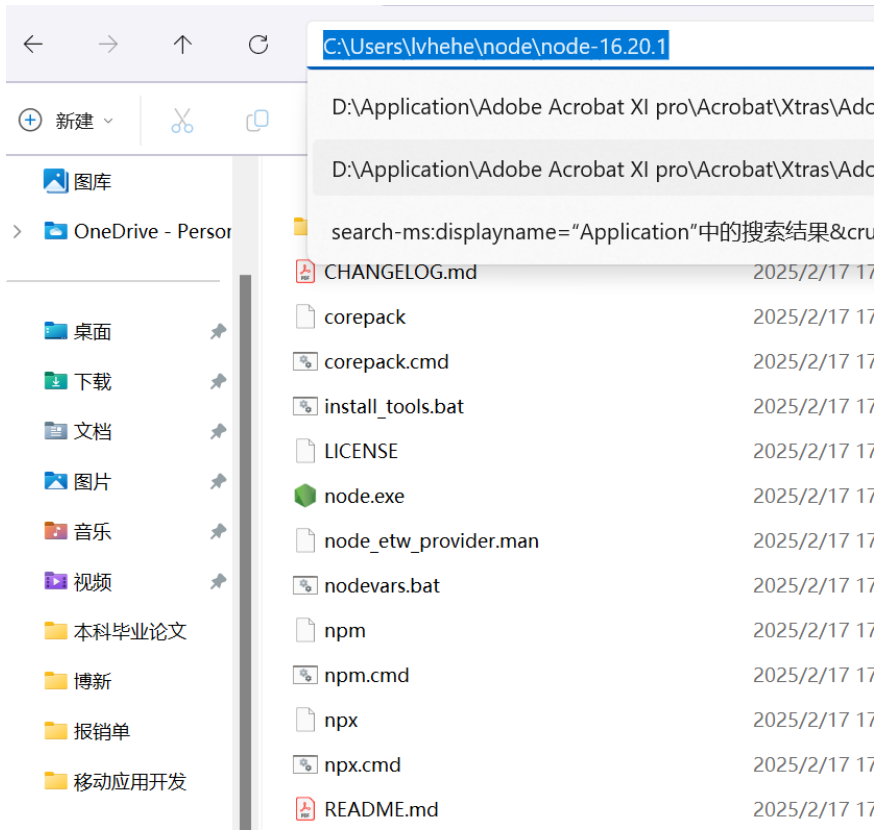

```
import {Employee} from "./person"
// .是当前目录
let emp = new Employee("张三", 22, "技
术开发部")
console.log(emp.getPersonInfo())

//编译ts文件命令:tsc
```

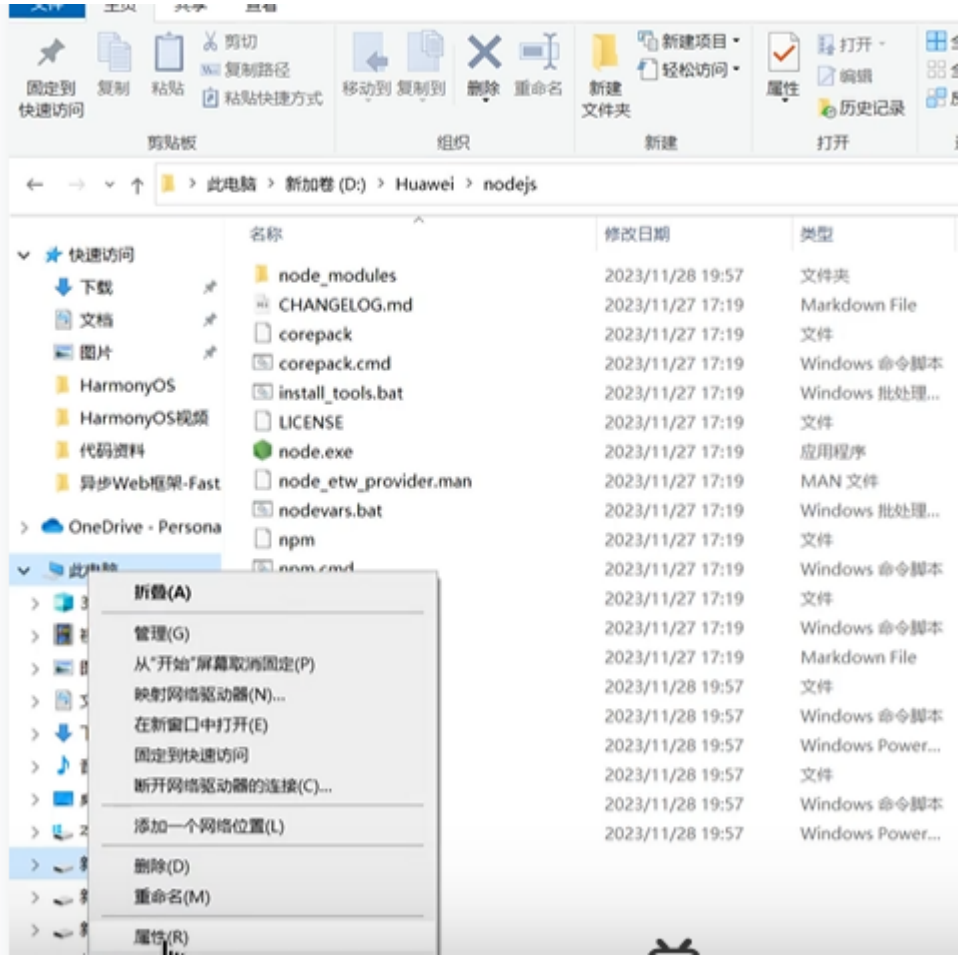
编译执行ts文件




```
node# This file is automatically generated by DevEco Studio.
# Do not modify this file -- YOUR CHANGES WILL BE ERASED!
#
# This file should *NOT* be checked into Version Control Systems,
# as it contains information specific to your local configuration.
#
# For customization when using a Version Control System, please
read the header note.
hwsdk.dir=C:/Users/lvhehe/AppData/Local/Huawei/Sdk
nodejs.dir=C:/Users/lvhehe/node/node-16.20.1
```

编译执行ts文件

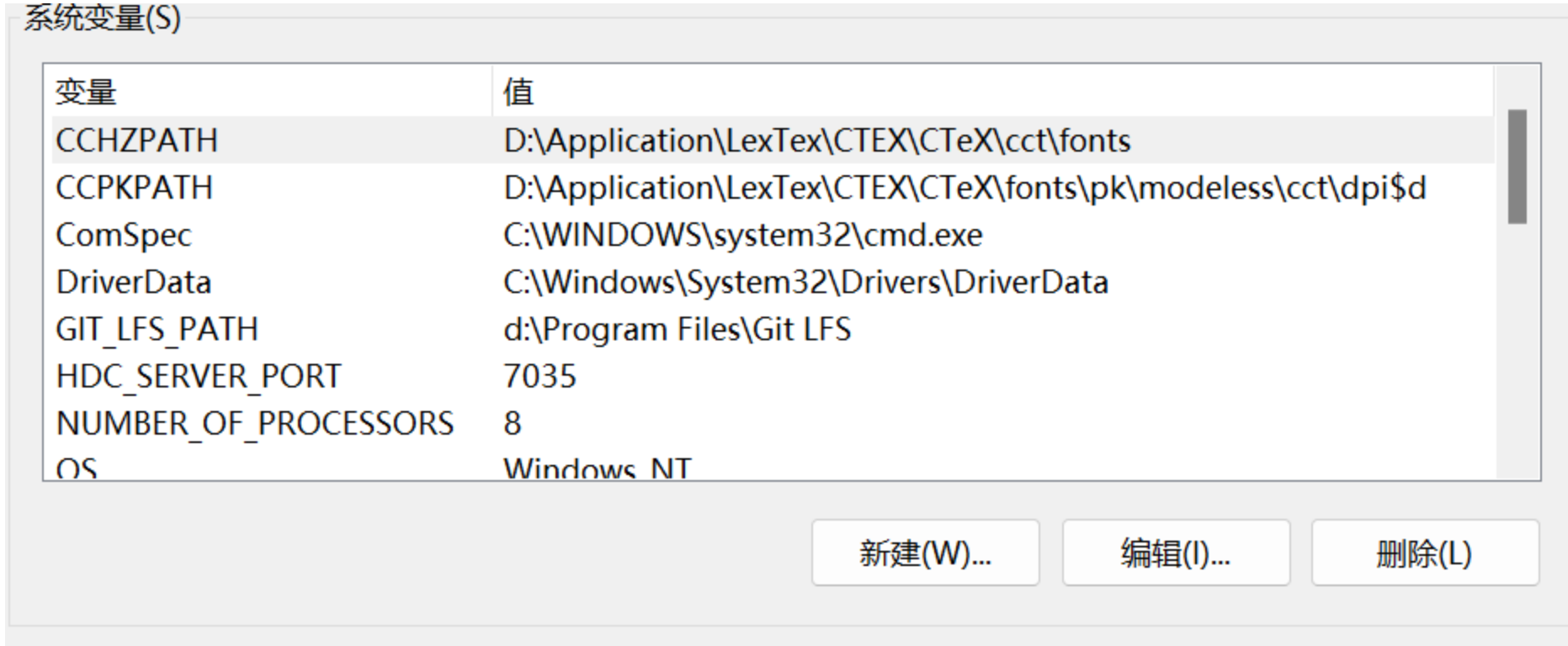

C:\Users\lvhehe\node\node-16.20.1
D:\Application\Adobe Acrobat XI pro\Acrobat\Xtras\Adc
D:\Application\Adobe Acrobat XI pro\Acrobat\Xtras\Adc
search-ms:displayname="Application"中的搜索结果&cru
新建
图库
OneDrive - Persor
桌面
下载
文档
图片
音乐
视频
本科毕业论文
博新
报销单
移动应用开发
CHANGELOG.md 2025/2/17 17
corepack 2025/2/17 17
corepack.cmd 2025/2/17 17
install_tools.bat 2025/2/17 17
LICENSE 2025/2/17 17
node.exe 2025/2/17 17
node_etw_provider.man 2025/2/17 17
nodevars.bat 2025/2/17 17
npm 2025/2/17 17
npm.cmd 2025/2/17 17
npx 2025/2/17 17
npx.cmd 2025/2/17 17
README.md 2025/2/17 17


固定到快速访问
复制
粘贴
剪切
复制路径
粘贴快捷方式
移动到
复制到
删除
重命名
新建文件夹
新建项目
轻松访问
属性
打开
编辑
历史记录
剪贴板
组织
新建
打开
此电脑 › 新加卷 (D:) › Huawei › nodejs
名称
修改日期
类型
快速访问
下载
文档
图片
HarmonyOS
HarmonyOS视频
代码资料
异步Web框架-Fast
OneDrive - Persona
node_modules 2023/11/28 19:57 文件夹
CHANGELOG.md 2023/11/27 17:19 Markdown File
corepack 2023/11/27 17:19 文件
corepack.cmd 2023/11/27 17:19 Windows 命令脚本
install_tools.bat 2023/11/27 17:19 Windows 批处理...
LICENSE 2023/11/27 17:19 文件
node.exe 2023/11/27 17:19 应用程序
node_etw_provider.man 2023/11/27 17:19 MAN 文件
nodevars.bat 2023/11/27 17:19 Windows 批处理...
npm 2023/11/27 17:19 文件
折叠(A)
管理(G)
从"开始"屏幕取消固定(P)
映射网络驱动器(N)...
在新窗口中打开(E)
固定到快速访问
断开网络驱动器的连接(C)...
添加一个网络位置(L)
删除(D)
重命名(M)
属性(R)

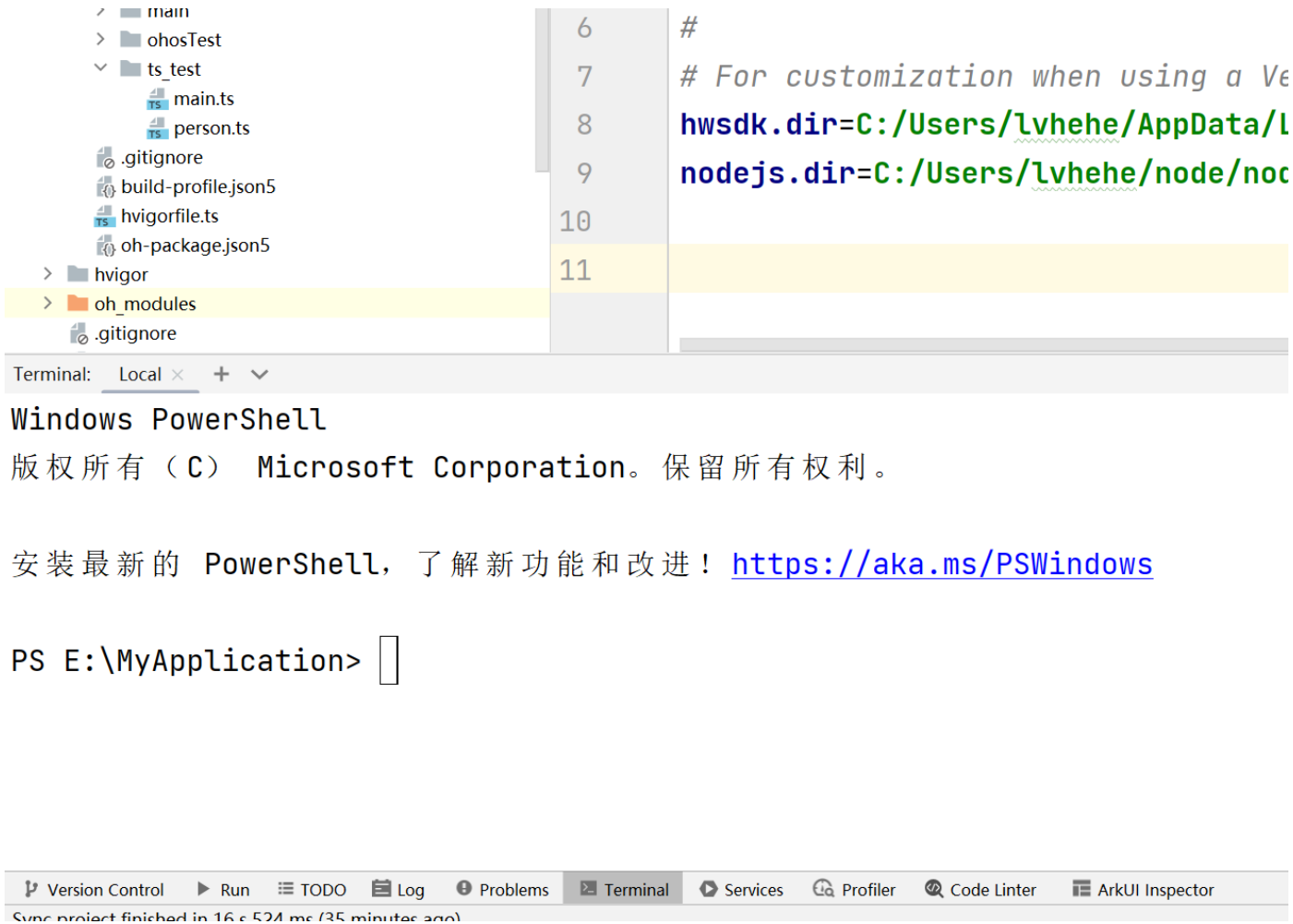
编译执行ts文件

系统 › 系统信息
LAPTOP-2LBVHELL
XiaoXinPro 14ITL 2021
设备规格
设备名称 LAPTOP-2LBVHELL
处理器 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz 2.42 GHz
机带 RAM 16.0 GB (15.8 GB 可用)
设备 ID 6D3487EF-3F69-4541-86A2-E3154569F225
产品 ID 00342-36088-66291-AAOEM
系统类型 64 位操作系统, 基于 x64 的处理器
笔和触控 没有可用于此显示器的笔或触控输入
相关链接 域或工作组 系统保护 高级系统设置
系统属性
计算机名 硬件 高级 系统保护 远程
要进行大多数更改，你必须作为管理员登录。
性能
视觉效果，处理器计划，内存使用，以及虚拟内存
设置(S)...
用户配置文件
与登录帐户相关的桌面设置
设置(E)...
启动和故障恢复
系统启动、系统故障和调试信息
设置(T)...
环境变量(N)...
确定
取消
应用(A)

编译执行ts文件


系统变量(S)
变量
值
CCHZPATH
D:\Application\LexTex\CTEX\CTeX\cct\fonts
CCPKPATH
D:\Application\LexTex\CTEX\CTeX\fonts\pk\modeless\cct\dpi$d
ComSpec
C:\WINDOWS\system32\cmd.exe
DriverData
C:\Windows\System32\Drivers\DriverData
GIT_LFS_PATH
d:\Program Files\Git LFS
HDC_SERVER_PORT
7035
NUMBER_OF_PROCESSORS
8
OS
Windows_NT
新建(W)...
编辑(I)...
删除(L)

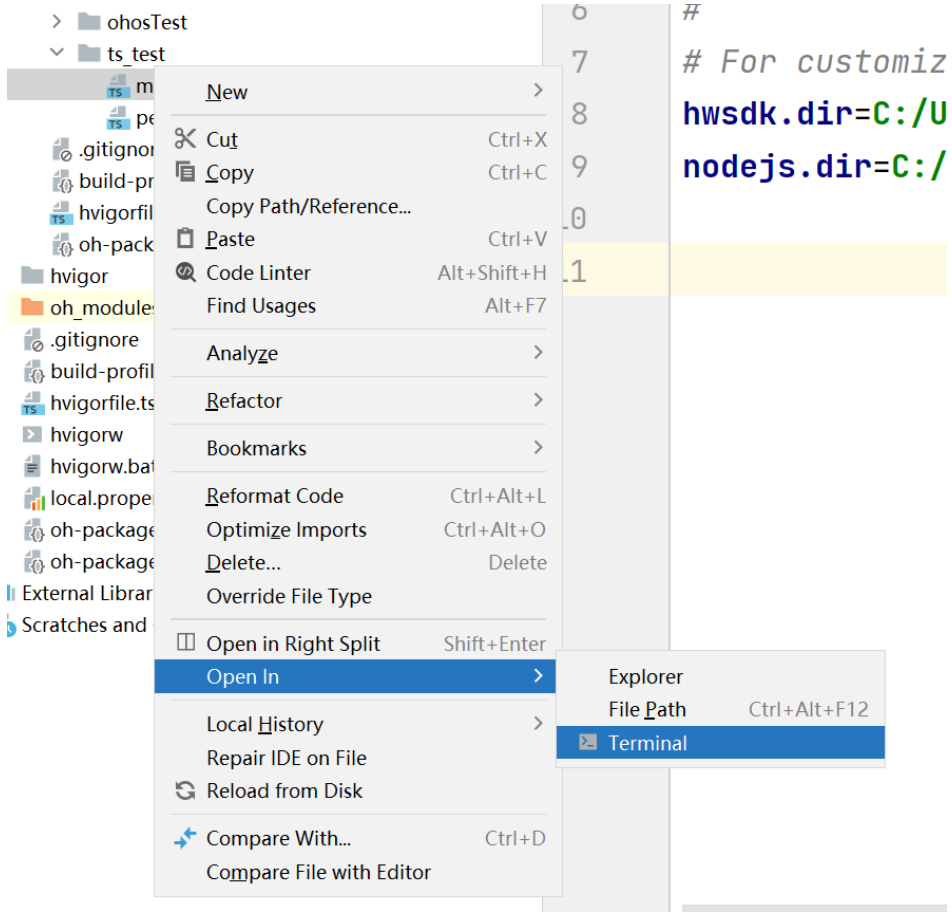
编译执行ts文件

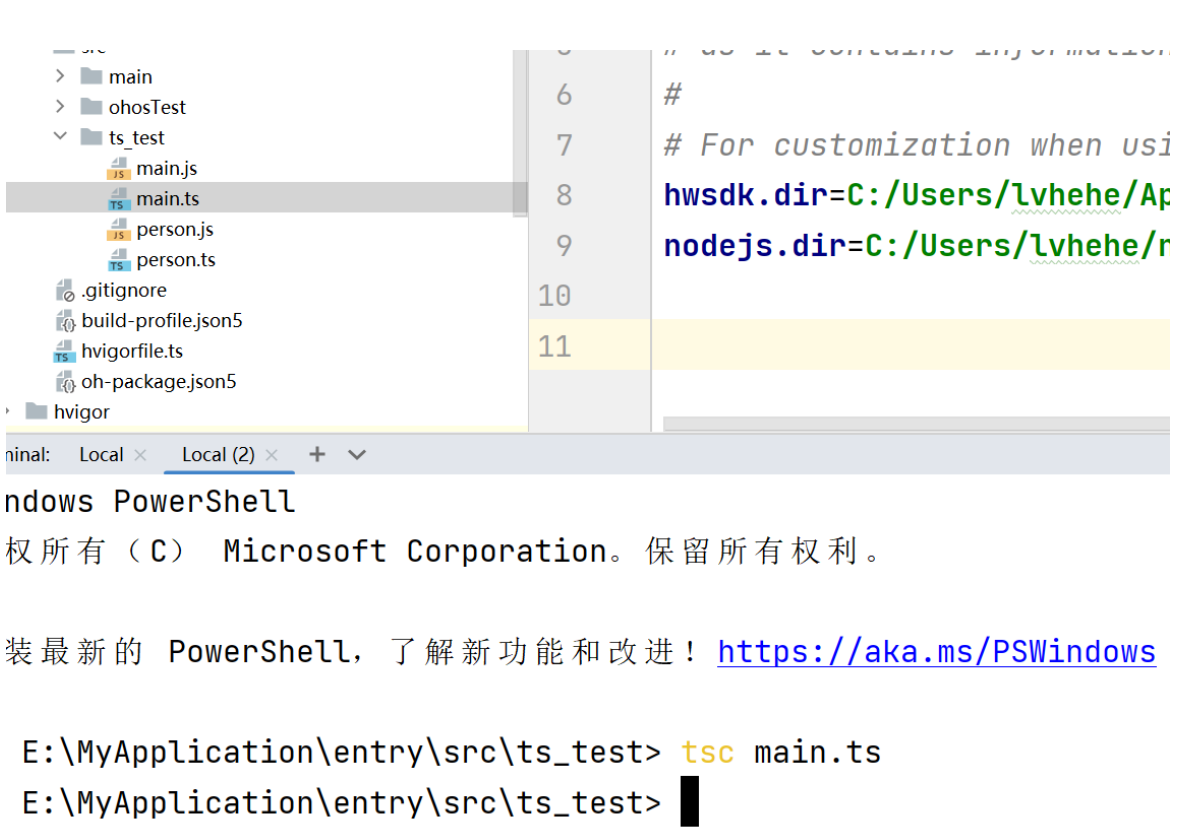

main
ohosTest
ts_test
main.ts
person.ts
.gitignore
build-profile.json5
hvigorfile.ts
oh-package.json5
hvigor
oh_modules
.gitignore
6 #
7 # For customization when using a Ve
8 hwsdk.dir=C:/Users/lvhehe/AppData/l
9 nodejs.dir=C:/Users/lvhehe/node/noc
10
11
Terminal: Local
Windows PowerShell
版权所有（C） Microsoft Corporation。保留所有权利。
安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows
PS E:\MyApplication>
Version Control
Run
TODO
Log
Problems
Terminal
Services
Profiler
Code Linter
ArkUI Inspector

编译执行ts文件

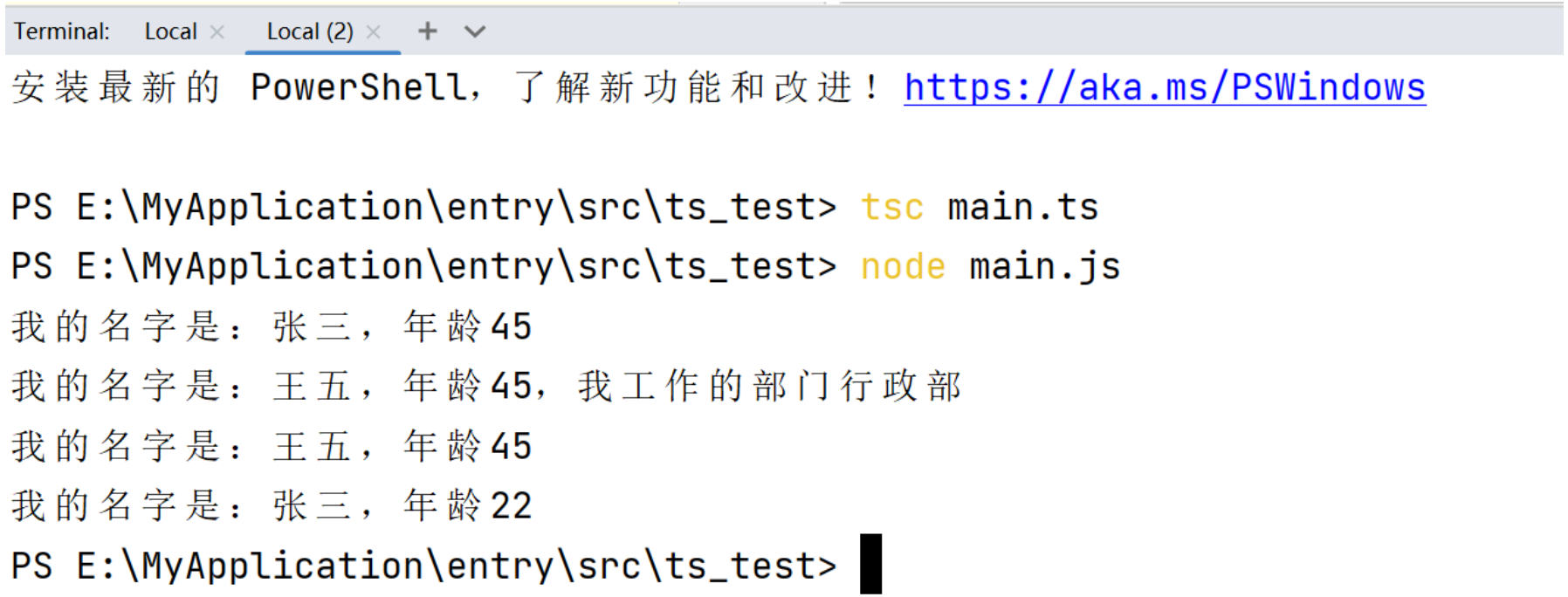
Terminal: Local
Windows PowerShell
版权所有（C） Microsoft Corporation。保留所有权利。
安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows
PS E:\MyApplication> node -v
v16.20.1
PS E:\MyApplication> npa install -g typescript
Version Control
Run
TODO
Log
Problems
Terminal
Services
Profiler
Code Linter
ArkUI Inspector

编译执行ts文件


ohosTest
ts_test
.gitignor
build-pr
hvigorfil
oh-pack
hvigor
oh_module
.gitignore
build-profil
hvigorfile.ts
hvigorw
hvigorw.bat
local.prope
oh-package
oh-package
External Librar
Scratches and
New
Cut Ctrl+X
Copy Ctrl+C
Copy Path/Reference...
Paste Ctrl+V
Code Linter Alt+Shift+H
Find Usages Alt+F7
Analyze
Refactor
Bookmarks
Reformat Code Ctrl+Alt+L
Optimize Imports Ctrl+Alt+O
Delete... Delete
Override File Type
Open in Right Split Shift+Enter
Open In
Local History
Repair IDE on File
Reload from Disk
Compare With... Ctrl+D
Compare File with Editor
Explorer
File Path Ctrl+Alt+F12
Terminal
6 #
7 # For customiz
8 hwsdk.dir=C:/U
9 nodejs.dir=C:/
10
11

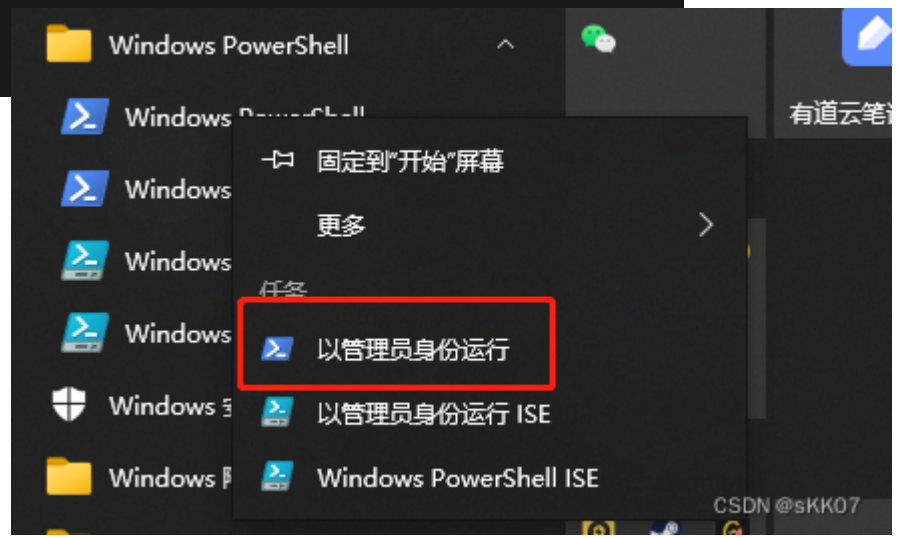

main
ohosTest
ts_test
main.js
main.ts
person.js
person.ts
.gitignore
build-profile.json5
hvigorfile.ts
oh-package.json5
hvigor
6 #
7 # For customization when usi
8 hwsdk.dir=C:/Users/lvhehe/Ap
9 nodejs.dir=C:/Users/lvhehe/r
10
11
inal: Local Local (2)
ndows PowerShell
权所有（C） Microsoft Corporation。保留所有权利。
装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows
E:\MyApplication\entry\src\ts_test> tsc main.ts
E:\MyApplication\entry\src\ts_test>

编译执行ts文件


Terminal: Local Local (2)
安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows
PS E:\MyApplication\entry\src\ts_test> tsc main.ts
PS E:\MyApplication\entry\src\ts_test> node main.js
我的名字是：张三，年龄45
我的名字是：王五，年龄45，我工作的部门行政部
我的名字是：王五，年龄45
我的名字是：张三，年龄22
PS E:\MyApplication\entry\src\ts_test>

编译执行ts文件

```
PS D:\text\TS\ts自动编译> tsc --init
tsc : 无法加载文件 C:\Users\Administrator\AppData\Roaming\npm\tsc.ps1，因为在此系统上禁止运行脚本。有关详细信息，
wlink/?LinkID=135170 中的 about_Execution_Policies。
所在位置 行:1 字符: 1
+ tsc --init
+ ~~~
    + CategoryInfo          : SecurityError: (:) [], PSSecurityException
    + FullyQualifiedErrorId : UnauthorizedAccess
```




```
PS C:\Windows\system32> set-ExecutionPolicy RemoteSigned

执行策略更改
执行策略可帮助你防止执行不信任的脚本。更改执行策略可能会产生安全风险，如 https:/go.microsoft.com/fwlink/?LinkID=135170
中的 about_Execution_Policies 帮助主题所述。是否要更改执行策略?
[Y] 是(Y)  [A] 全是(A)  [N] 否(N)  [L] 全否(L)  [S] 暂停(S)  [?] 帮助 (默认值为“N”): Y
PS C:\Windows\system32>
```

河南大學 Henan University

TypeScript面向对象

02

认识和使用TypeScript的接口声明与实现

认识TypeScript的接口

接口是特殊的类，使用interface表示，接口用来限定属性和规定标准行为。

声明接口

- 声明接口时，属性不能被初始化，行为是没有方法体的抽象方法，接口中的属性和方法默认都是public。

```
interface ICar {
    color: string; //定义接口的一个属性，不能初始化
    //sayHi是函数名，该函数没有参数，返回值类型为string
    sayHi: () => string; //定义接口的抽象方法，不能有方法体
}
```

认识TypeScript的接口

实现接口

- 实现接口，接口声明后可以由具体的类实现（implements）接口中定义的方法和给属性赋值。

```
class Bus implements ICar {
    color: string='红色'                    //可以初始化
    sayHi(): string {                  //实现接口的方法
        return `Hi,${this.color}`;
    }
}
```

认识TypeScript的接口

接口多态

- 接口多态，当接口的实现类实现了接口中的方法后，接口的引用指向实现类对象，访问的是实现类的方法，从而呈现多种状态。

```
let car: ICar = new Bus();//接口的引用指向实现类的对象
```

认识TypeScript的接口

```typescript
interface Shape {
  draw(): void;
}

class Circle implements Shape {
  draw() {
    console.log("Drawing a circle");
  }
}

class Square implements Shape {
  draw() {
    console.log("Drawing a square")
  }
}

function drawShape(shape: Shape) {
  shape.draw();
}

const myCircle = new Circle();
const mySquare = new Square();

drawShape(myCircle); // 输出: Drawing a circl
drawShape(mySquare); // 输出: Drawing a squar
```

认识TypeScript的接口

```typescript
interface Animal {
  speak(): void;
}

class Dog implements Animal {
  speak(): void {
    console.log("Woof!");
  }

  fetch(): void {
    console.log("Dog is fetching the ball")
  }
}
```

```typescript
const myDog: Animal = new Dog();
myDog.speak(); // 输出: Woof!
```

```typescript
myDog.fetch();
```

使用TypeScript的接口

01 任务描述

以Vehicle（车辆）接口为例，练习接口的声明、实现和接口多态的使用。

02 实施步骤

1. 声明接口
2. 实现接口□
3. 实现接口多态

使用TypeScript的接口-任务实施

接口的声明与使用

1、声明了一个Vehicle（车辆）接口，具有启动start()和停止stop()方法；

2、创建了Car和Bike类，并实现了Vehicle接口中的方法；

3、编写showCarState()方法，使用多态的概念传入Vehicle接口的不同实现类对象，通过Vehicle类型的变量vehicle来存储Car和Bike的实例，并通过调用vehicle的start()和stop()方法来启动和停止车辆，展示了接口的多态性，使得能够在运行时动态地调用不同类中的方法。

代码演示

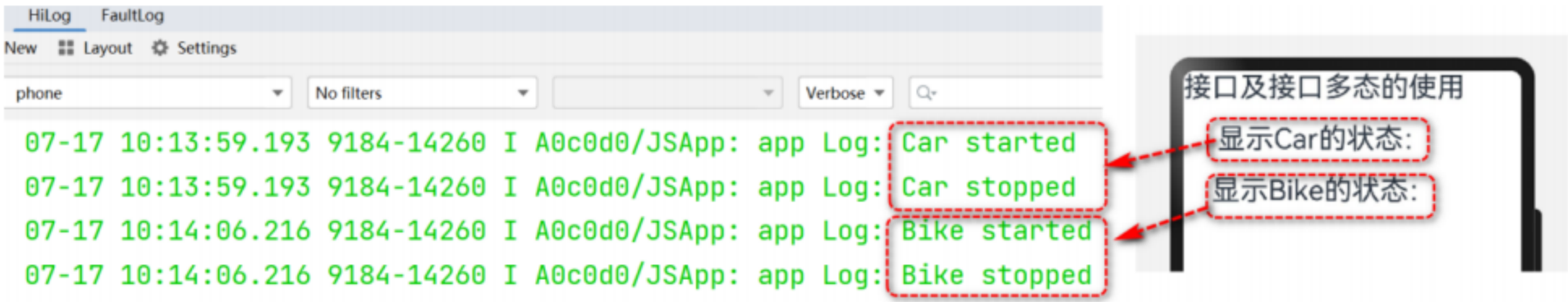
```
1. interface Vehicle { //声明车辆接口
2.    start(): void;     //声明启动的方法
3.    stop(): void;      //声明停止的方法
4. }
5. class Car implements Vehicle { //创建Car 类实现 Vehicle 接口
6.    public start(): void {        //实现start()方法
7.       console.log("Car started");
8.    }
9.    public stop(): void {        //实现 stop()方法
10.       console.log("Car stopped");
11.    }
```

```
12. }
13. class Bike implements Vehicle {//创建Bike 类实现Vehicle 接口
14.   public start(): void {
15.     console.log("Bike started");
16.   }
17.   public stop(): void {
18.     console.log("Bike stopped");
19.   }
20. }
21.
```

```
22. @Entry
23. @Component
24. struct TestInterface{
25.   //使用多态的方式接收不同实现类的对象
26.   showCarState( vehicle :Vehicle)
27.   {
28.     vehicle.start(); //调用实现类的start()方法
29.     vehicle.stop(); // 调用实现类的stop()方法
30.   }
```

```
31.   build(){
32.     Column({space:20})
33.     {
34.       Text("接口及接口多态的使用").fontSize(30)
35.       Text("显示 Car 的状态： ").fontSize(30)
36.         .onClick(()=>{
37.           this.showCarState(new Car())//呈现 Car 类的状态
38.         })
39.       Text("显示 Bike 的状态： ").fontSize(30)
40.         .onClick(()=> {
41.           this.showCarState(new Bike())//呈现 Bike 类的状态
42.         })
43.     }
44.   }
45. }
```

预览应用，单击不同的Text组件后，在预览日志中查看接口的多态使用的效果，如图所示。



任务小结

本任务使用TS中面向对象的语法，初步完成了针对设置页的类与数据源的设计和使用，并学会了声明接口和创建接口的实现类，利用接口的多态性进行了数据的展示。在ArkTS的开发中，组件的声明都使用到了接口，组件的属性都继承自公共的类，因此，读者需要重点掌握类与接口的语法，才能进行后续的ArkTS组件的应用开发。

河南大學
Henan University

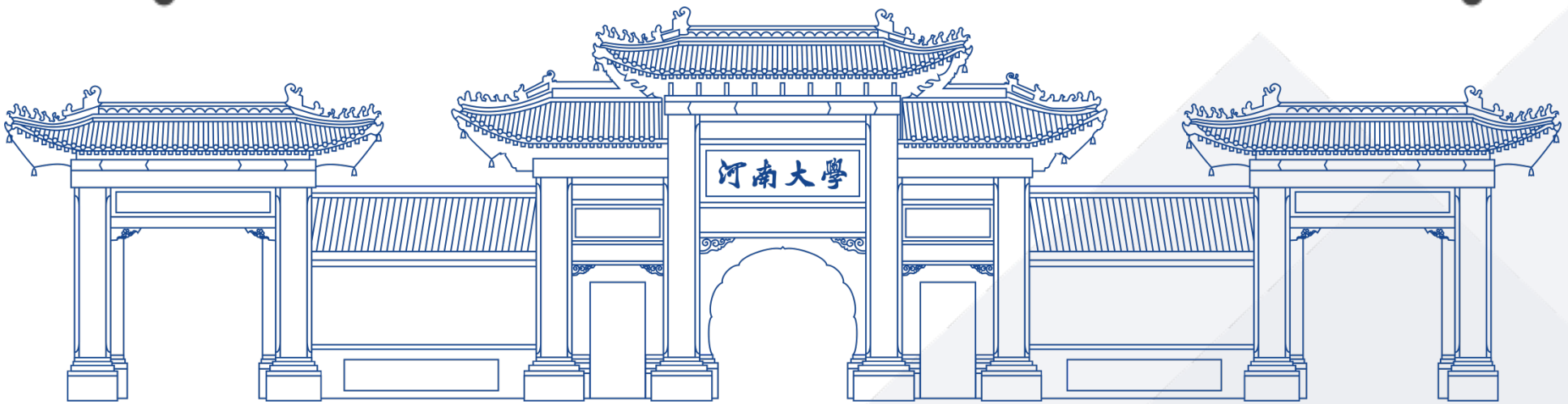
感谢聆听！

Thank you for listening

河南大學
Henan University

第3章 ArkTS与声明式UI

软件学院



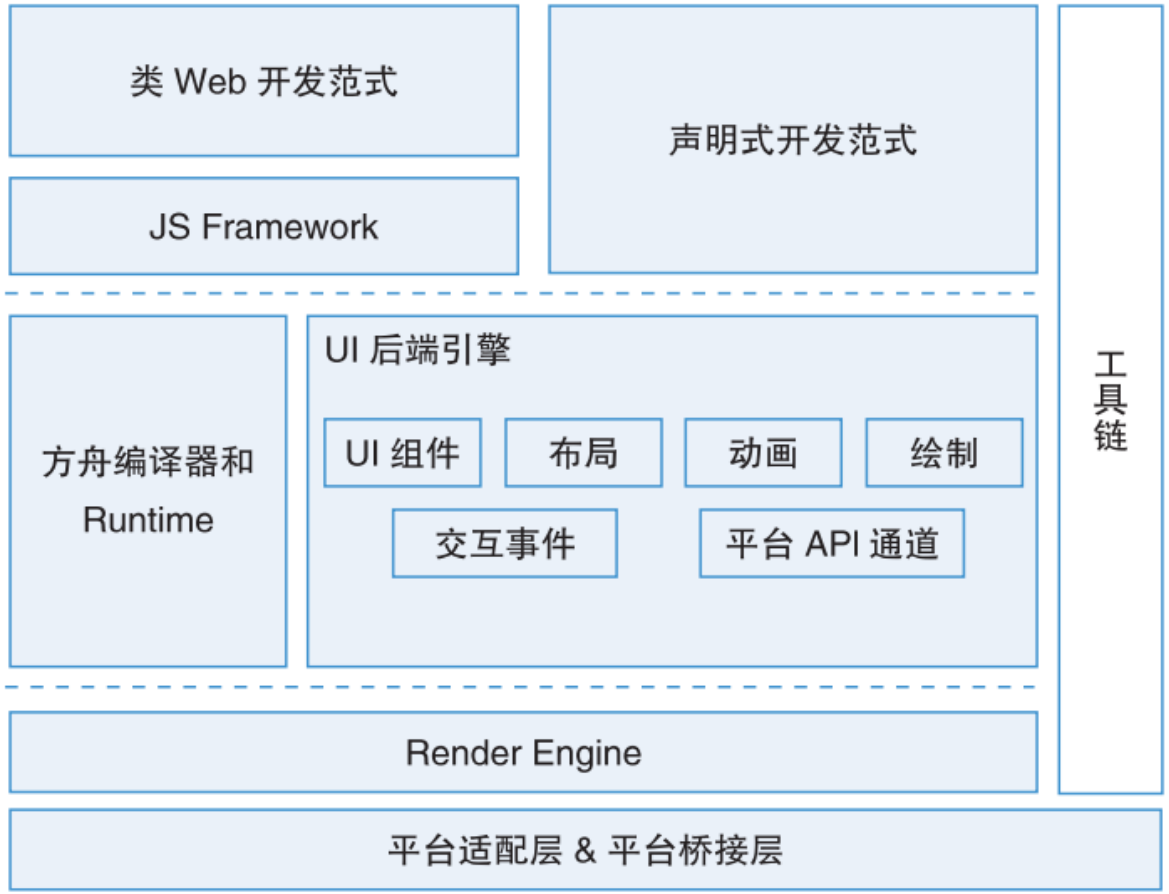
河南大學
Henan University

01

ArkTS介绍与UI页面组成

方舟开发框架

- 方舟开发框架(ArkUI)是一套UI开发框架，旨在为开发者提供必要的应用UI开发能力。该框架提供了两种开发范式，分别是基于TypeScript(TS)扩展的声明式开发范式和基于JavaScript(JS)扩展的类Web开发范式，以满足不同目的和技术背景的开发者的需求。ArkUI架构图如3-1所示。



从图可以看出，类Web开发范式和声明式开发范式共用UI后端引擎和语言运行时。

其中，UI后端引擎实现了方舟开发框架的六种基本能力。相比之下，声明式开发范式无需使用JS Framework进行页面DOM管理，渲染更新链路更为精简，内存占用更少。

因此，建议开发者使用声明式开发范式来构建应用的UI界面。

方舟开发框架

HarmonyOS

ArkUI：为应用的UI开发提供了完整的基础设施

ArkUI（方舟UI框架）为应用的UI开发提供了完整的基础设施，包括简洁的UI语法、丰富的UI功能（组件、布局、动画以及交互事件），以及实时界面预览工具等，可以支持开发者进行丰富页面的开发。

方舟开发框架

ArkUI：基于关键的应用UI开发底座，深度融合语言/编译器/图形

HarmonyOS

开发模型层

提供了UI开发范式的基础语言规范，并提供内置的UI组件、布局和动画，提供了多种状态管理机制，为应用开发者提供一系列接口支持。

引擎层

后端引擎提供了兼容不同开发范式的UI渲染管线，提供多种基础组件、布局计算、动效、交互事件，提供了状态管理和绘制能力。提供了高效的绘制能力，将渲染管线收集的渲染指令，绘制到屏幕能力

平台抽象层

提供了针对不同操作系统渲染层的适配，可抹平不同平台的接口差异，实现ArkUI框架的多平台一致性

方舟开发框架

HarmonyOS

ArkUI：极简开发、高性能、跨设备、跨平台

丰富的UI组件

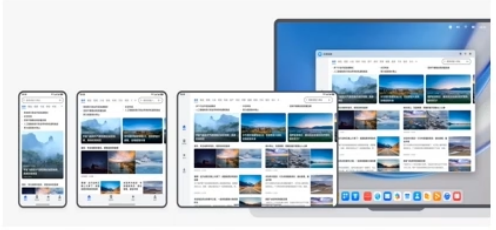
- HarmonyOS Design UI组件
- 平滑自然的内置控件动画
- 100+组件和样式

极简的UI语法

- 声明式UI
- 提升30%开发效率

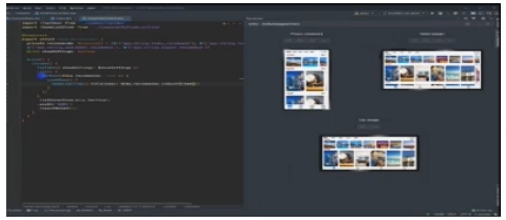
ArkTS/C++混合开发

- 独立渲染能力
- 相机输入接入



易用的UI一多能力

- 响应式布局
- 多态控件
- Syscap能力



实时开发预览

- 所见所得
- 支持多种设备类型

跨平台开发

- 鸿蒙开发框架跨平台
- API桥接
- 原生渲染性能

声明式开发范式

HarmonyOS

声明式开发范式：基于ArkTS的声明式开发范式的方舟开发框架是一套开发极简、高性能、支持跨设备的UI开发框架，提供了构建HarmonyOS应用UI所必需的能力。

声明式UI是系统选用的新一代开发范式，通过数据驱动UI变化，UI逻辑分离，更直观，更高效。

HarmonyOS

声明式UI和命令式UI对比

命令式开发范式

- 开发者需要手动编写UI渲染的每一个细节,包括创建、修改和删除操作;
- UI状态和操作逻辑通常混杂在一起;
- 代码量较大且易出错,但可以精确控制UI渲染的每个细节;
- 适合面向框架性的应用;

```
<DirectionalLayout …>
  <Text …
    ohos:text='Hello World'/>
</DirectionalLayout>

// Imperative style
View b = findViewById(title)
b.setOnClickListener(() => {
  b.setColor('#f1f511')
});
```

基于ArkTS的声明式开发范式

- 开发者只需描述UI的最终状态,而不关注具体的渲染过程;
- 框架负责处理UI的创建、更新和删除等细节;
- 代码中直接描述UI的结构,与状态或数据的关系较为隐式;
- 适合面向业务型的应用;

```
Column({ space: 20 }) {
  Text('其它登录方式')
    .fontSize(16)
    .fontColor(this.myColor)

  Button('修改颜色').onClick(() => {
    this.myColor = '#f1f511'
  })
}
```

ArkTS语言介绍

- ArkTS是OpenHarmony优选的主力应用开发语言。ArkTS基于TS语言扩展而来，是TS的超集。当前，ArkTS继承了TS的所有特性。ArkTS在TS的基础上主要扩展了声明式UI能力，让开发者能够以更简洁、更自然的方式开发高性能应用。当前扩展的声明式UI能力包括如下特性。

基本UI描述：ArkTS定义了各种装饰器、自定义组件、UI描述机制，再配合UI开发框架中的内置组件及其相关的事件方法、属性方法等共同构成了UI开发的主体。

状态管理：ArkTS提供了多维度的状态管理机制，在UI开发框架中，和UI相关联的数据，不仅可以在组件内使用，还可以在不同组件层级间传递。另外，从数据的传递形式来看，可分为只读的单向传递和可变更的双向传递。开发者可以灵活利用这些能力来实现数据和UI的联动。

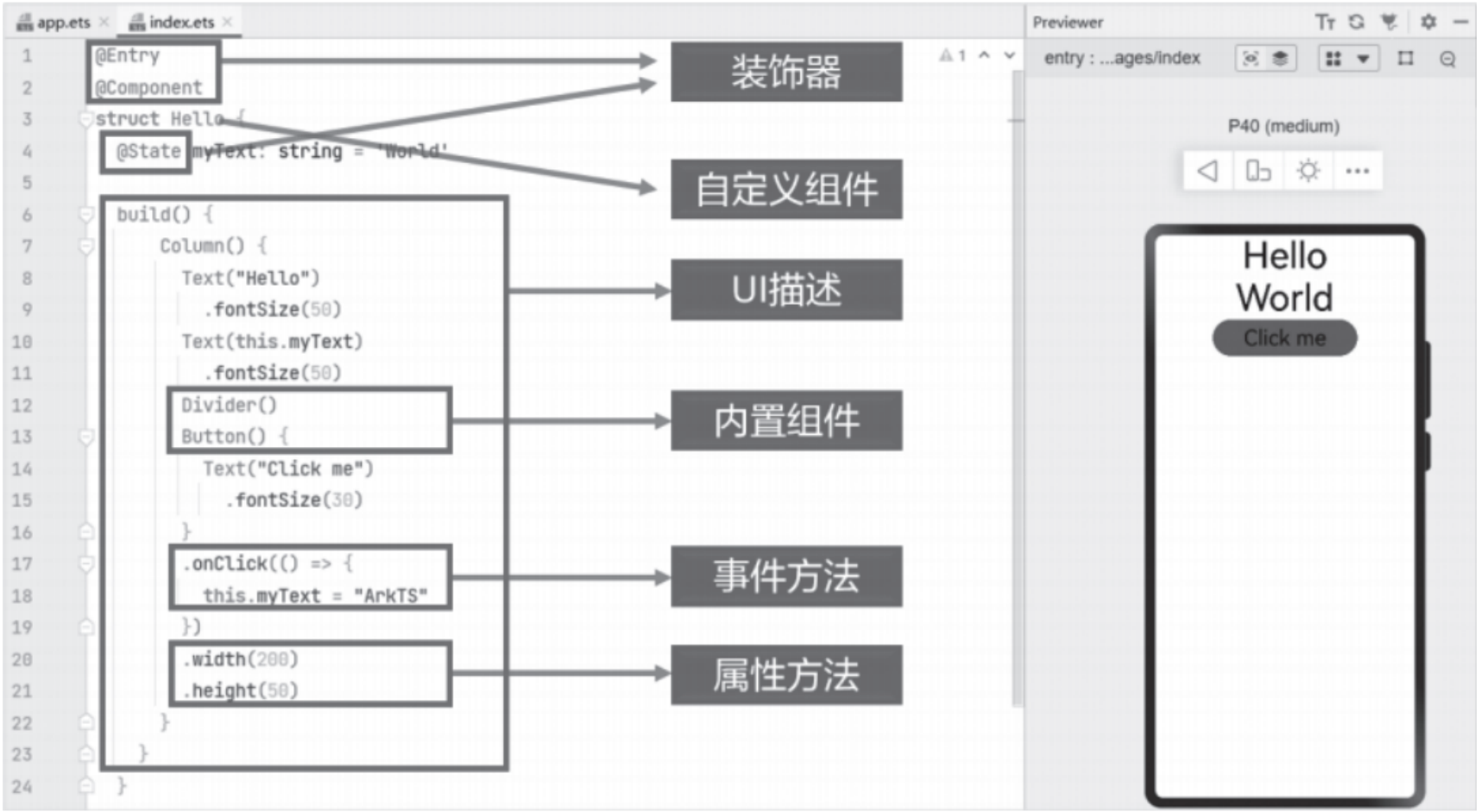
动态构建UI元素：ArkTS提供了动态构建UI元素的能力，不仅可以自定义组件内部的UI结构，还可以复用组件样式，扩展原生组件。

渲染控制：ArkTS提供了渲染控制的能力。条件渲染可根据应用的不同状态，渲染对应状态下的UI内容。循环渲染可从数据源中迭代获取数据，并在每次迭代过程中创建相应的组件。

使用限制与扩展：ArkTS在使用过程中有在限制与约束，同时也扩展了双向绑定等能力

ArkTS语言介绍

ArkTS声明式开发范式基本组成：现以图3-2所示的代码示例来说明ArkTS声明式开发范式的基本组成。UI界面包含两段文本、一条分割线和一个按钮，当开发者点击按钮时，文本内容会从Hello World变为Hello ArkUI。



页面与组件

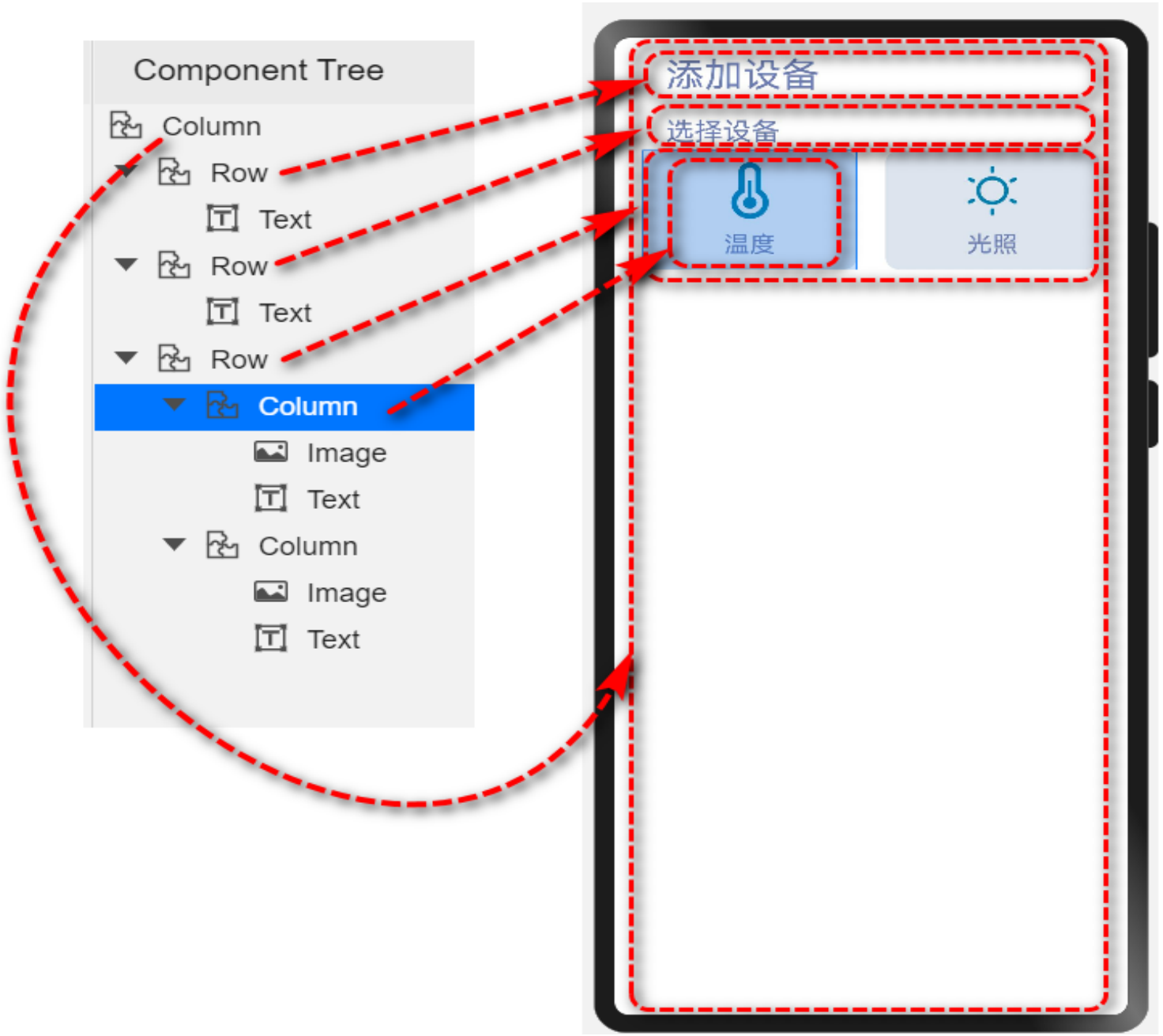
App中的一个页面是一个具有特定作用和功能的用户界面，页面的目的是在用户与计算机之间建立交互，并提供一种方式让用户使用软件。

页面通常包括一个或多个组件，例如文本框、按钮、图片等等，这些组件用于展示数据、收集用户输入、执行操作等等。

页面与组件

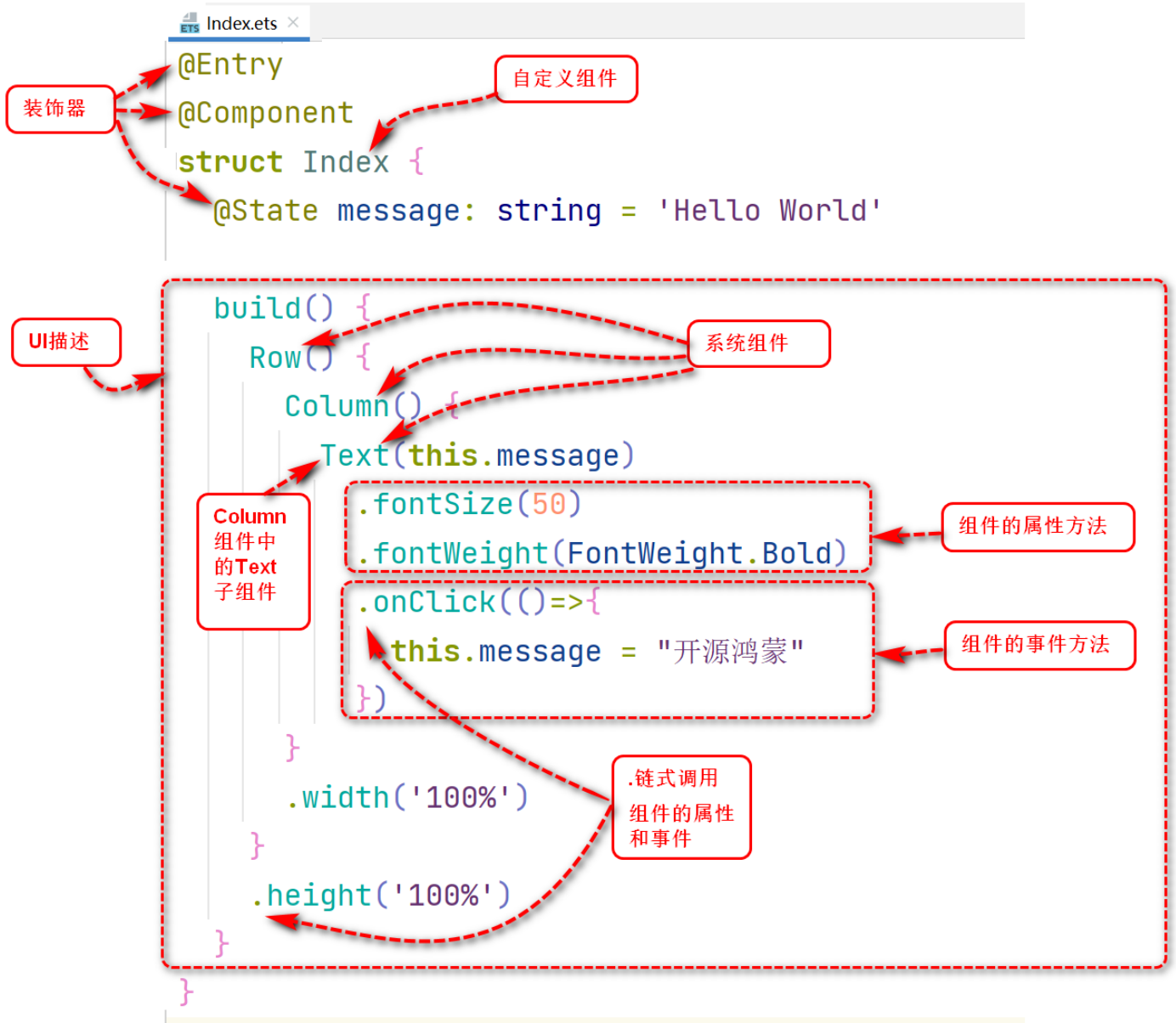
App中的一个页面是一个具有特定作用和功能的用户界面，页面的目的是在用户与计算机之间建立交互，并提供一种方式让用户使用软件。

页面通常包括一个或多个组件，例如文本框、按钮、图片等等，这些组件用于展示数据、收集用户输入、执行操作等等。



ArkTS的UI页面组成

ArkTS以声明方式组合和扩展组件来描述应用程序的UI，同时还采用了链式方式调用组件的属性和事件，提供子组件的配置方法，帮助开发者高效实现应用交互逻辑。



装饰器：用来装饰类、结构体、方法以及变量，并赋予其特殊的含义，如上述示例中@Entry、@Component、@State都是装饰器。
具体而言，@Component表示这是个自定义组件；@Entry表示这是一个页面入口组件，一个页面只能有一个@Entry;@State表示组件中的状态变量，这个状态变化会触发UI刷新。

自定义组件：可复用的UI单元，可组合其他组件，如上述被@Component装饰的structHello。

UI描述：以声明式的方式来描述UI的结构，如上述build()方法内部的代码块。

内置组件：ArkTS中默认内置的基础组件、容器组件、媒体组件、绘制组件、画布组件等，开发者可以直接调用，如示例中的Column（水平排列）、Text（文本）、Divider（分割）、Button（点击）等。

属性方法：用于设置组件的属性，如fontSize()、width（)、height()、color()等。可通过链式调用的方式设置多项属性。

事件方法：用于添加组件对事件的响应逻辑，统一通过事件方法进行设置，如跟随在Button后面的onClick()。同样可以通过链式调用的方式设置多个事件的响应逻辑。

ArkTS通过装饰器@Component和@Entry装饰struct关键字声明的结构，构成一个自定义组件。自定义组件中提供了一个build函数，开发者需在该函数内以链式调用的方式进行基本的UI描述。

1.基本概念

struct: 结构体，自定义组件可以基于struct实现，不能有继承关系，对于struct的实例化，可以省略new。

示例：定义结构体。

```
@Entry @Component struct MyComponent {
}
```

build函数：自定义组件必须定义build函数，并且禁止自定义构造函数。build函数满示例：Builder构造接口定义。满足Builder构造器接口定义，用于定义组件的声明式UI描述。

```
interface Builder {
    build: () => void
}
```

@Entry: 装饰struct, 组件被装饰后作为页面的入口，页面加载时将被渲染显示。

@Component: 装饰struct, 结构体在装饰后具有基于组件的能力，需要实现build方法来创建UI。

@Preview: 装饰struct, 用@Preview装饰的自定义组件可以在DevEco Studio的预览器上进行实时预览，加载页面时，将创建并显示@Preview装饰的自定义组件，一个页面最多可以使用10个@Preview装饰。

链式调用：以"."链式调用的方式配置UI组件的属性方法、事件方法等。

2. ArkTS基本语法

(1) 变量声明和数据类型

在使用ArkTS语言编写应用程序之前，需先了解一下ArkTS在变量声明、支持的数据类型、属性的调用、事件方法、无参数和必选参数的构造配置、子组件配置的相关注意事项。进行变量声明定义名字时，有如下限制：

①只能包含数字、字母、下划线以及$符号；

②首个符号，不能以数字开头，只能允许字母或者下划线以及$符号开头；

③次数有限制：同一个名字的变量在相同作用域中只能声明一次。

ArkTS支持的数据类型包括number(数字、二进制、八进制、十进制、十六进制),以及string、boolean、数组、元组、any。

示例：变量声明和数据类型案例介绍。

```
@Entry @Component struct Index {
_name$123:string // 名字 : 数据类型
// _name$123:string="123" // 名字 : 数据类型 = 值
count1:number = 0b110011 // 二进制
count2:number = 0o12345670 // 八进制
count3:number = 1234567890 // 十进制
count4:number = 0x0123456789abcdef // 十六进制
name1:string = "HarmonyOS 学习 666" // 双引号、单引号均可
name2:string = "^H^ 开发 "
stateON:boolean = true // 布尔型 true、false
stateOFF:boolean = false
names:string[] = ["json","123"," 小鸿 "] // 数组
counts:number[] = [5,0b01,0xa2,123]
person:[string,number,number] = [" 小鸿 ",20,180] // 元组
data:any=123 // 任意类型
build(){
}
}
```

(2) 参数构造配置

如果组件的接口定义中不包含必选构造参数，那么组件后面的()中不需要配置任何内容。

示例：构造参数传值、不传值使用方式。

```
Column() {
    Text("item 1")
    Divider()
    Text("item 2")
}
```

(2) 参数构造配置

如果组件的接口定义中包含构造参数，则在组件后面的“（）”中必须配置相应参数，参数可以使用常量进行赋值。

示例：Image组件的必选参数src。

```
Image("https://xyz/test.jpg")
```

变量或表达式也可以用于参数赋值，其中表达式返回的结果类型必须满足参数类型要求。

示例：设置变量或表达式来构造Image和Text组件的参数。

```
Image(this.imagePath)
Image("https://"+this.imageUrl)
Text(count: ${this.count})
```

(3) 属性

属性方法用于配置组件的属性，如fontSize（)、width（)、height（)、color()等。

在DevEcoStudio中可以在组件后面通过“.”链式调用的方式配置UI的属性。

示例：配置Text组件的字体大小属性。

```
Text("test")
  .fontSize(12)
```

示例：使用”.”运算符进行链式调用并同时配置组件的多个属性。

```
Image("test.jpg")
 .alt("error.jpg")
 .width(100)
 .height(100)
```

除了直接传递常量参数外，还可以传递变量或表达式。

示例：传递变量或表达式。

```
Text("hello")
  .fontSize(this.size)
Image("test.jpg")
    .width(this.count % 2 === 0 ? 100 : 200)
    .height(this.offset + 100)
```

对于系统内置组件，框架还为其属性预定义了一些枚举类型供开发人员调用，枚举类型可以作为参数传递，且必须满足参数类型要求。

示例：使用枚举类型作为参数配置Text组件的颜色和字体属性。

```
Text("hello")
  .fontSize(20)
  .fontColor(Color.Red)
  .fontWeight(FontWeight.Bold)
```

(4) 事件

在事件方法的回调中添加组件响应逻辑。例如，为Button组件添加onClick方法，在onClick方法的回调中添加点击响应逻辑。通过事件方法可以配置组件支持的事件，事件方法紧随组件，并用“.”运算符连接。

示例：使用lambda表达式配置组件的事件方法。

```
Button("add counter")
  .onClick(() => {
    this.counter += 2
  })
```

使用匿名函数表达式配置组件的事件方法，要求使用bind, 以确保函数体中的this引用包含的组件。

示例：使用匿名函数表达式配置组件的事件方法。

```
Button("add counter")
    .onClick(function () {
        this.counter += 2
    }.bind(this))
```

示例：使用组件的成员函数配置组件的事件方法。

```
myClickHandler(): void {
    this.counter += 2
}
...
Button("add counter")
    .onClick(this.myClickHandler.bind(this))
```

(5) 子组件配置

对于支持子组件配置的组件，例如容器组件，在“{...}”里为组件添加子组件的UI描述。Column、Row、Stack、Grid、List等组件都是容器组件。

示例：简单的Column使用。

```
Column() {
  Text("Hello")
  .fontSize(100)
Divider()
Text(this.myText)
  .fontSize(100)
  .fontColor(Color.Red)
}
```

容器组件之间也可以互相嵌套，实现相对复杂的多级嵌套效果。

示例：容器组件之间互相嵌套使用。

```
Column() {
  Row() {
    Image("test1.jpg")
      .width(100)
      .height(100)
    Button("click +1")
      .onClick(() => {
        console.info("+1 clicked!")
      })
  }
  Divider()
  Row() {
    Image("test2.jpg")
      .width(100)
      .height(100)
    Button("click +2")
      .onClick(() => {
        console.info("+2 clicked!")
      })
  }
  Divider()
  Row() {
    Image("test3.jpg")
      .width(100)
      .height(100)
    Button("click +3")
      .onClick(() => {
        console.info("+3 clicked!")
      })
  }
}
```

河南大學
Henan University

02

ArkTS声明式UI的用法

ArkTS声明式UI的用法

组件的参数

组件包含有参数和无参数两种方式，查看组件的声明文件时，可以看到该组件的接口定义是否有包含参数。

ArkTS声明式UI的用法

组件参数用法

Column组件的接口定义中包含的参数space?，后面有个问号?说明该参数是可选的。

在ArkTS中，创建一个组件不需要使用new关键字。

column.d.ts

```
interface ColumnInterface {
  (value?: {
    space?: string | number;
  }): ColumnAttribute;
}
```

组件的参数

组件的属性

```
Column({space:20}) {  //有参,参数用number类型描述
}
Column({space:"20"}){//有参,参数用string类型描述
}
Column() {//无参
}
```

ArkTS声明式UI的用法

组件属性

每个组件都有属性， 以Column组件为例，跟踪组件的声明文件.d.ts，可以看到Column除了有自身在水平和垂直方向上的私有对齐属性外，还继承了CommonMethod类的公共属性。

```
interface ColumnInterface {
  (value?: {
    space?: string | number;
  }): ColumnAttribute;
}
declare class ColumnAttribute extends CommonMethod<ColumnAttribute> {
  alignItems(value: HorizontalAlign): ColumnAttribute;
  justifyContent(value: FlexAlign): ColumnAttribute;
}
```

Column组件的属性

公共属性

水平方向上的对齐属性

垂直方向上的对齐属性

ArkTS声明式UI的用法

组件常用的公共属性

每个组件的属性都继承了CommonMethod类中的公共属性，常见的公共属性说明如下。

```
declare class CommonMethod<T> {
 constructor();
 width(value: Length): T;                    //组件的宽度
 height(value: Length): T;                   //组件的高度
 size(value: SizeOptions): T;                //组件的大小
 layoutWeight(value: number | string): T;  //组件的权重
 padding(value: Padding | Length): T;        //组件的内边距
 margin(value: Margin | Length): T;          //组件的外边距
 backgroundColor(value: ResourceColor): T; //组件的背景色
 onClick(event: (event?: ClickEvent) => void): T;//组件的点击事件
 ...}
```

ArkTS声明式UI的用法

配置子组件

在ArkTS中，有的组件是容器组件，可以包含其他子组件。Column就是一个容器组件，如果容器组件需要配置子组件时，则需在尾随闭包“{...}”中为组件添加子组件的UI描述。

```
Column({space:20}) {  //有参,参数用number类型描述
 Text('跳转')     //子组件
   ...
}
```

ArkTS声明式UI的用法

组件的点击事件

点击事件是组件的公共属性，每个组件都可以处理点击事件。事件方法以“.”链式调用的方式配置系统组件支持的事件。

```
//第一种：使用lambda表达式配置组件的事件方法
Text('事件测试')
  .onClick(() => {
    //在这里写业务逻辑
  })
```

```
//第二种：使用组件的成员函数配置组件的事件方法
private myClick() {
  //在这里写业务逻辑
}
Text('事件测试')
    .onClick(() => {
      this.myClick()
    })
```

河南大學
Henan University

03

ArkTS中的尺寸单位

ArkTS中的尺寸单位

Length类

从组件的公共属性可以看到，宽度、高度、大小等尺寸单位用Length类描述。

名称	类型定义	描述
Length	string|number	用于描述尺寸单位，输入为number类型时，使用vp单位；输入为string类型时，需要显式指定像素单位，如’20px’，也可设置百分比字符串，如’90%’、’100%’等

ArkTS中的尺寸单位

像素单位

有 4 种像素单位供开发者使用，分别是 px 、 vp 、 fp 和 lpx。

单位名称	描述
px	屏幕物理像素单位
vp	屏幕密度相关像素单位，根据屏幕像素密度转换为屏幕物理像素
fp	字体像素，与vp类似适用于屏幕密度变化，随系统字体大小设置变化
lpx	视窗逻辑像素单位，lpx单位为实际屏幕宽度与逻辑宽度（在 config.json 中配置的 designWidth ）的比值，如配置 designWdith 为 720 时，在实际宽度为 1440 物理像素的屏幕上， 1px 为 2px

河南大學
Henan University

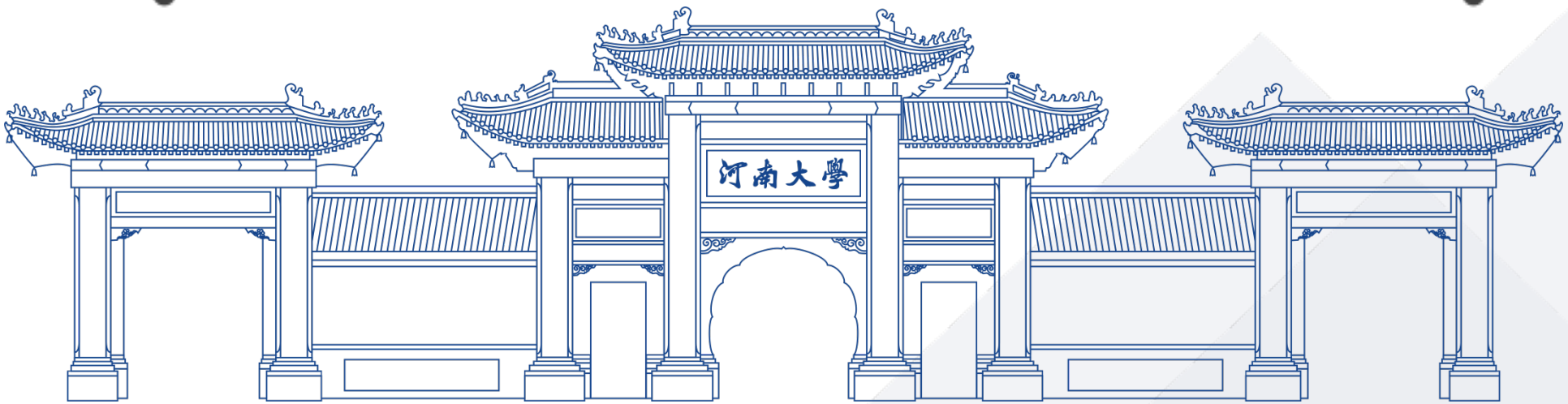
感谢聆听！

Thank you for listening

河南大學
Henan University

第3章 常见布局

软件学院



布局概述

丰富的页面需要很多组件组成，那么，我们如何才能让这些组件有条不紊地在页面上布局呢?

这就需要借助容器组件实现。容器组件是一种比较特殊的组件，它可以包含其他的组件，并按照一定的规律布局，帮助开发者生成精美的页面。

容器组件除了放置基础组件外，也可以放置容器组件，实现多层布局的嵌套，从而布局出更丰富的页面。

容器组件

- 容器组件 vs 一般组件
- 容器组件是一种特殊的组件。
- 容器组件可以包含其他组件，并按一定规律布局。
- 容器组件中可以包含一般组件，也可以包含容器组件
- Row：表示沿垂直方向布局的容器
- Column：表示沿水平方向布局的容器
- Flex（弹性布局）：可以通过direction参数改变主轴的方向
- Stack： 容器中没有明确的主轴与交叉轴，通过设置 alignContent参数来改变容器内组件的对齐方式

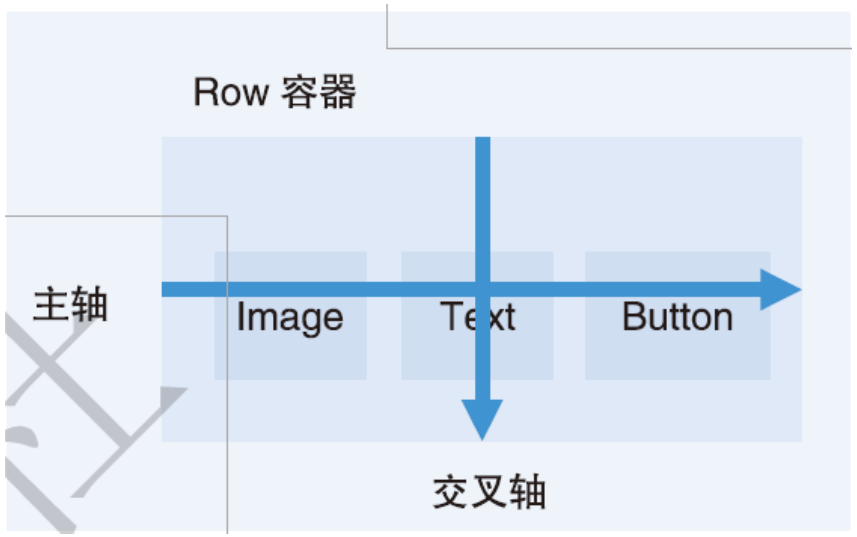
主轴和交叉轴

- 布局容器中，默认存在两个轴
- 主轴
- 交叉轴
- 主轴和交叉轴互相垂直
- Stack比较特殊，它没有明确的主轴与交叉轴

Row（水平布局）

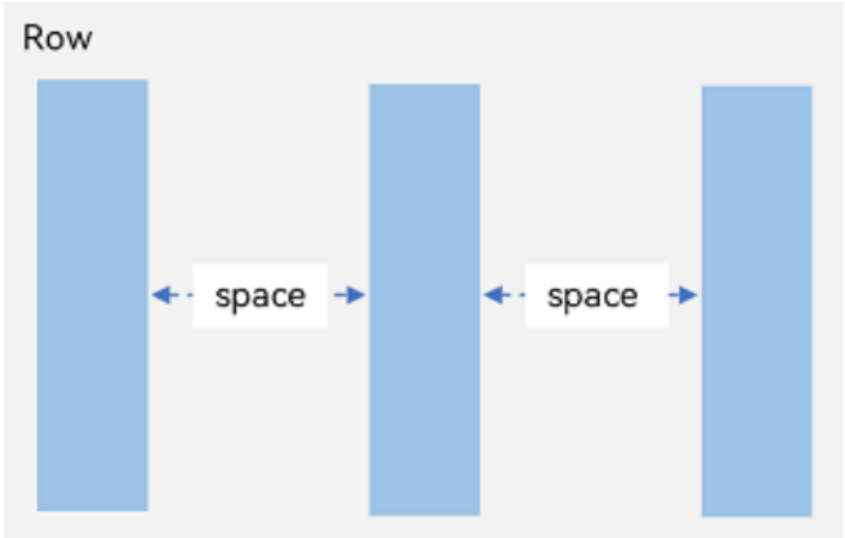
- Row：按照水平方向布局子组件，主轴为水平方向，交叉轴为垂直方向。

```
Row({space: number})
```



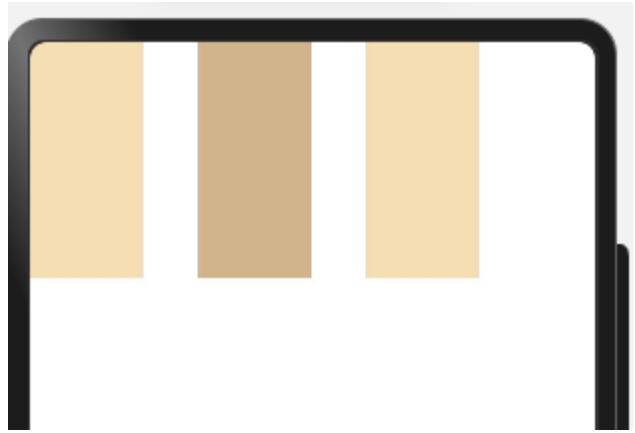
➢ 可选参数：

- space: 子组件的主轴方向上的间距



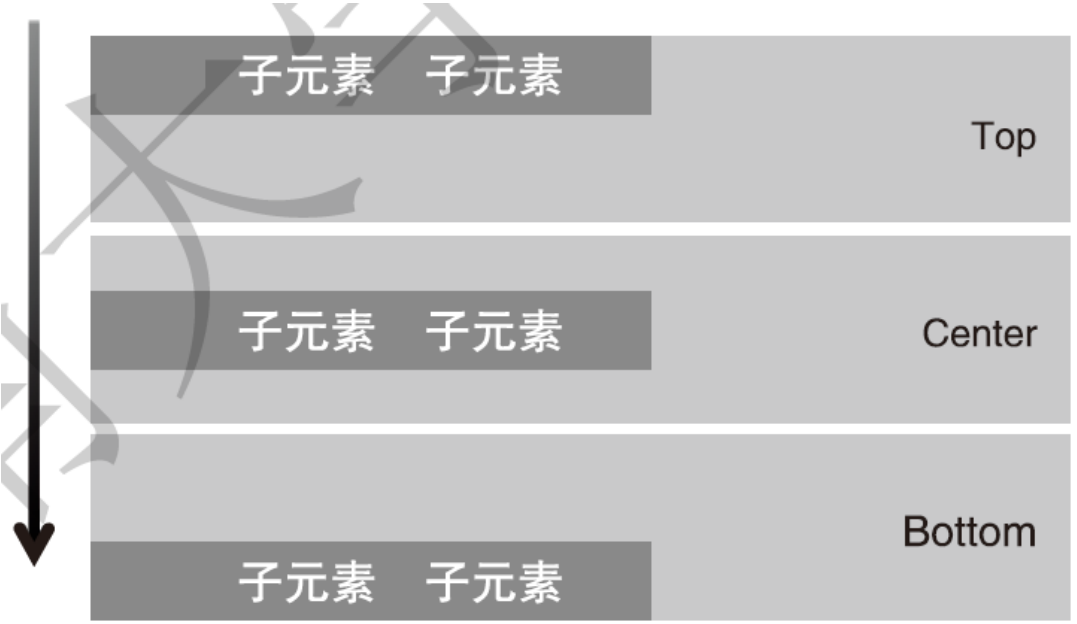
Row（水平布局）

```
@Entry @Component struct Test {
  build() {
    Row({ space: 35 }) {
      Row().width('10%').height(150).backgroundColor(0xF5DEB3)
      Row().width('10%').height(150).backgroundColor(0xD2B48C)
      Row().width('10%').height(150).backgroundColor(0xF5DEB3)
    }.width('200%')
    }}
```



Row（水平布局）

- 主要属性
- alignItems: 子组件在交叉轴方向的对齐方式
- VerticalAlign.Top：顶部对齐
- VerticalAlign.Center：居中对齐
- VerticalAlign.Bottom：底部对齐



名称	描述
Top	设置子组件在竖直方向上居顶部对齐
Center	设置子组件在竖直方向上居中对齐
Bottom	设置子组件在竖直方向上居底部对齐

Row（水平布局）

```
@Entry @Component struct Test {
  build() {
    Row({space: 35}) {
      Row().width('10%').height(30).backgroundColor(0xF5DEB3)
      Row().width('10%').height(30).backgroundColor(0xD2B48C)
      Row().width('10%').height(30).backgroundColor(0xF5DEB3)
    }.width('200%').height(200).alignItems(VerticalAlign.Center)
    }}
```

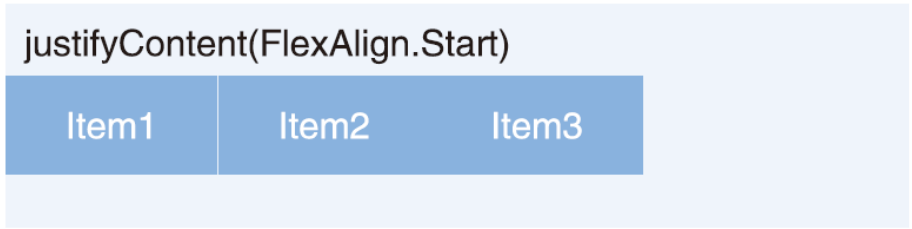
Row（水平布局）

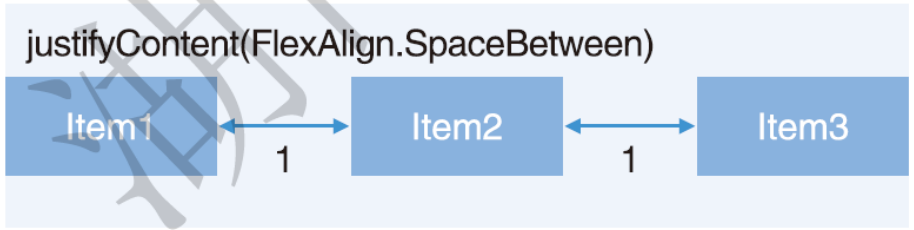
- justifyContent: 子组件在主轴方向的对齐方式
- FlexAlign.Start：左对齐
- FlexAlign.Center：居中对齐
- FlexAlign.End：右对齐
- FlexAlign.SpaceBetween：两端对齐
- FlexAlign. SpaceAround：不均分对齐。首位两端间距为中间组件间距的一半
- FlexAlign. SpaceEvenly：均分对齐

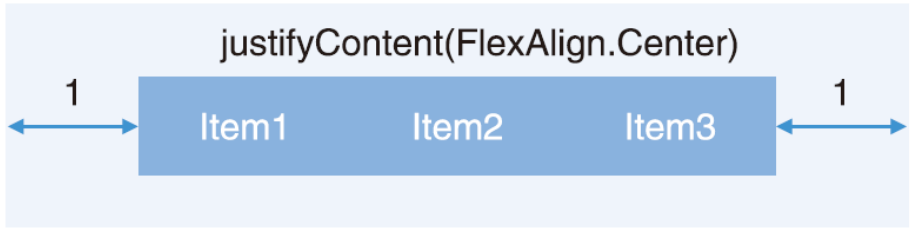
- 使用后三种时，space参数不再起作用

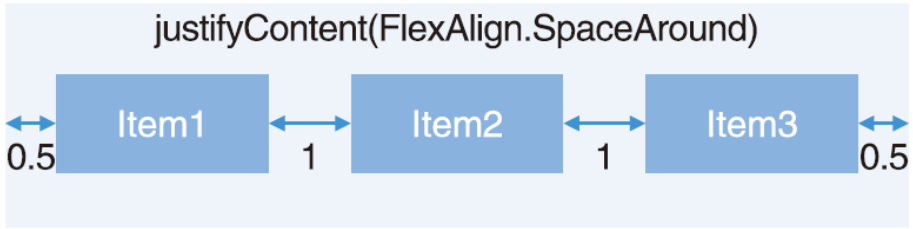
Row（水平布局）

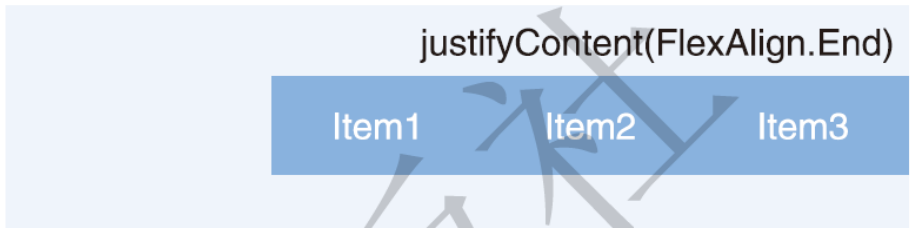
名称	描述
Start	左对齐
Center	水平居中
End	右对齐
SpaceBetween	两端对齐
SpaceAround	不均分对齐，首尾两端间距为中间组件间距的二分之一
SpaceEvenly	均分对齐

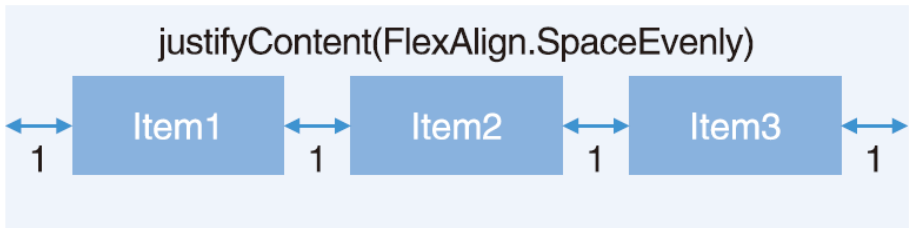






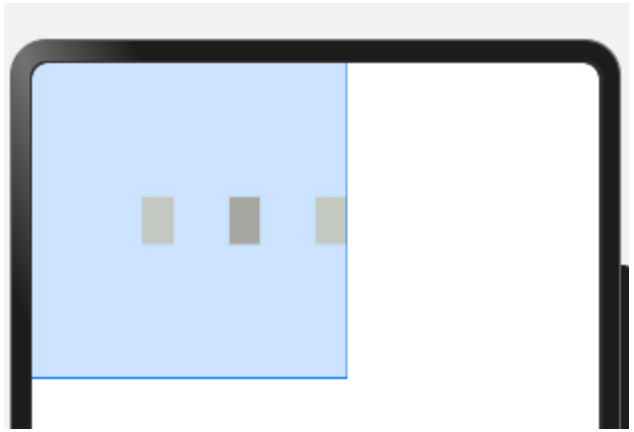






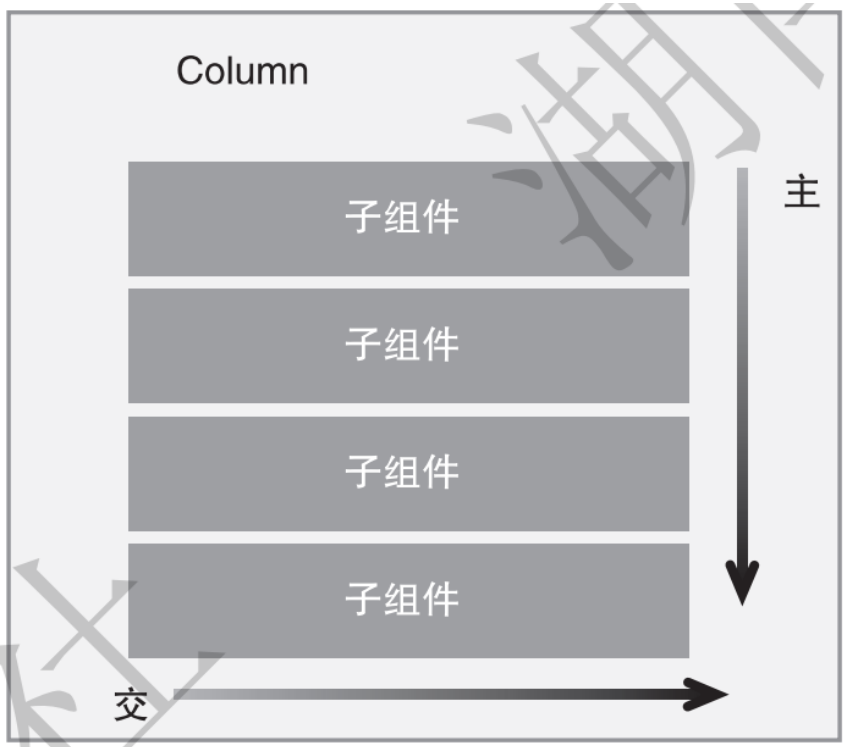
Row（水平布局）

```
@Entry @Component struct Test {
  build() {
    Row({space: 35}) {
      Row().width('10%').height(30).backgroundColor(0xF5DEB3)
      Row().width('10%').height(30).backgroundColor(0xD2B48C)
      Row().width('10%').height(30).backgroundColor(0xF5DEB3)
    }.width(200).height(200).justifyContent(FlexAlign.End)
    }}
```



Column（垂直布局）

- Column: 按照竖直方向布局子组件, 主轴为垂直方向，交叉轴为水平方向。



Column（垂直布局）

Column定义: Column({ space: number })

- 可选参数：
- space: 子组件的主轴方向上的间距

```
@Entry @Component struct Test {
  build() {
    Column({ space: 20 }) {
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
      Row().width('90%').height(50).backgroundColor(0xD2B48C)
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
    }.width('100%')
    }}
```

Column（垂直布局）

- 主要属性
- alignItems: 子组件在交叉轴方向的对齐方式
- HorizontalAlign. Start：左对齐
- HorizontalAlign.Center：居中对齐
- HorizontalAlign. End：右对齐

名称	描述
Start	设置子组件在水平方向上左对齐
Center	设置子组件在水平方向上居中对齐
End	设置子组件在水平方向上右对齐

Column（垂直布局）

```
@Entry @Component struct Test {
  build() {
    Column({ space: 20 }) {
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
      Row().width('90%').height(50).backgroundColor(0xD2B48C)
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
    }.width('100%').alignItems(HorizontalAlign.End)
    }}
```

Column（垂直布局）

- 主要属性

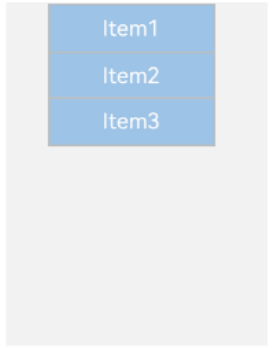
- justifyContent: 设置子组件在主轴方向上的对齐方式。

  - justifyContent(FlexAlign.Start)：元素在垂直方向首端对齐，第一个元素与行首对齐，同时后续的元素与前一个对齐。
  - justifyContent(FlexAlign.Center)：元素在垂直方向中心对齐，第一个元素与行首的距离与最后一个元素与行尾距离相同。
  - justifyContent(FlexAlign.End)：元素在垂直方向尾部对齐，最后一个元素与行尾对齐，其他元素与后一个对齐。
  - justifyContent(FlexAlign.SpaceBetween)：垂直方向均匀分配元素，相邻元素之间距离相同。第一个元素与行首对齐，最后一个元素与行尾对齐。
  - justifyContent(FlexAlign.SpaceAround)：垂直方向均匀分配元素，相邻元素之间距离相同。第一个元素到行首的距离和最后一个元素到行尾的距离是相邻元素之间距离的一半。
  - justifyContent(FlexAlign.SpaceEvenly)：垂直方向均匀分配元素，相邻元素之间的距离

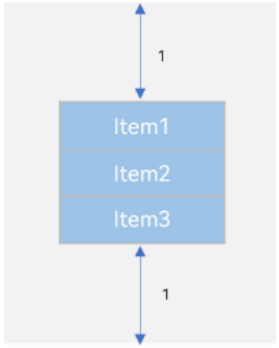
Column（垂直布局）

- 主要属性

- justifyContent: 设置子组件在主轴方向上的对齐方式。

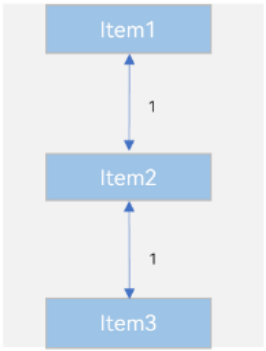


justifyContent(FlexAlign.Start)

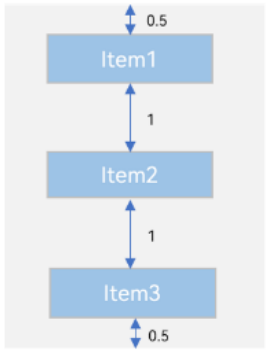


justifyContent(FlexAlign.Center)

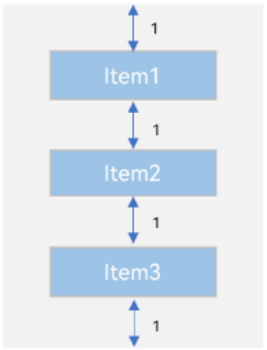
justifyContent(FlexAlign.End)



justifyContent(FlexAlign.SpaceBetween)



justifyContent(FlexAlign.SpaceAround)

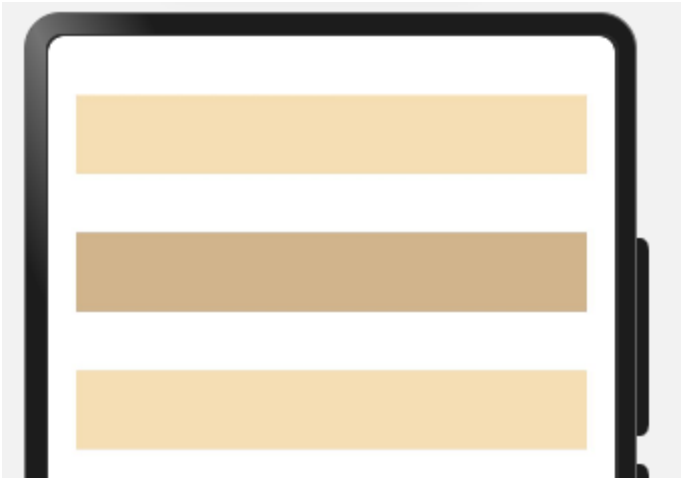


justifyContent(FlexAlign.SpaceEvenly)

- **注意：**Column 设置 space 参数，justifyContent 参数不起作用

Column（垂直布局）

```
@Entry @Component struct Test {
  build() {
    Column({ space: 20 }) {
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
      Row().width('90%').height(50).backgroundColor(0xD2B48C)
      Row().width('90%').height(50).backgroundColor(0xF5DEB3)
    }.width('100%').height(300).justifyContent(FlexAlign.SpaceEvenly)
    }}
```



Flex（弹性布局）

- Flex 是 Flexible 的缩写，意为“弹性布局”，用来为盒状模型提供最大的灵活性。Flex和Row、Column 组件一样，也有主轴和交叉轴之分，不同的是它可以自定义主轴方向。可以定义主轴方向。

- 参数配置：

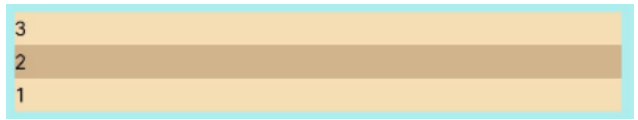
参数名	参数类型	必填	默认值	参数描述
direction	FlexDirection	否	FlexDirection.Row	子组件在Flex容器上排列的方向，即主轴的方向。
wrap	FlexWrap	否	FlexWrap.NoWrap	Flex容器是单行/列还是多行/列排列。 说明： 在多行布局时，通过交叉轴方向，确认新行堆叠方向。
justifyContent	FlexAlign	否	FlexAlign.Start	所有子组件在Flex容器主轴上的对齐格式。
alignItems	ItemAlign	否	ItemAlign.Start	所有子组件在Flex容器交叉轴上的对齐格式。
alignContent	FlexAlign	否	FlexAlign.Start	交叉轴中有额外的空间时，多行内容的对齐方式。仅在wrap为Wrap或WrapReverse下生效。

- 注意：Flex组件在渲染时存在二次布局过程。在对性能有严格要求的场景下建议使用

API Reference

Flex({ direction: FlexDirection})

- FlexDirection.Row（默认值）：主轴为水平方向，子元素从起始端沿着水平方向开始排布。
- FlexDirection.RowReverse：主轴为水平方向，子元素从终点端沿着FlexDirection.Row相反的方向开始排布。
- FlexDirection.Column：主轴为垂直方向，子元素从起始端沿着垂直方向开始排布。
- FlexDirection.ColumnReverse：主轴为垂直方向，子元素从终点端沿着FlexDirection. Column相反的方向开始排布。



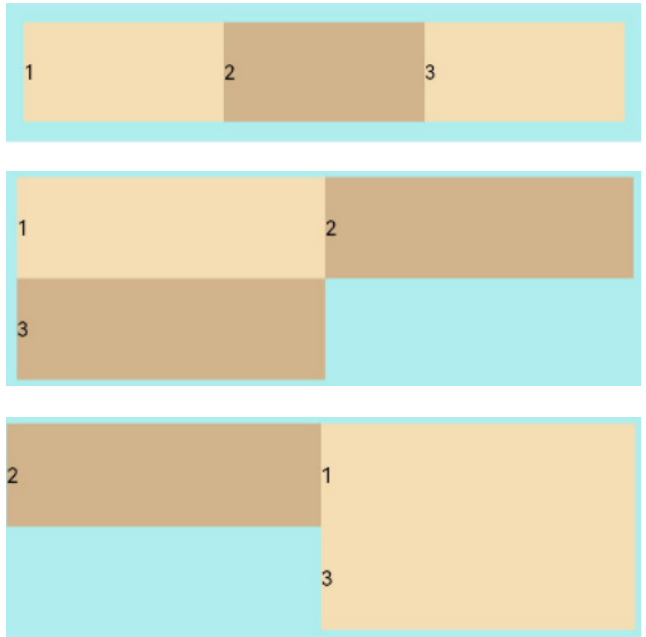
Flex（弹性布局）

```
@Entry @Component struct Test {
  build() {
    Flex({ direction: FlexDirection.ColumnReverse }) {
      Text('1').width('100%').height(50).backgroundColor(0xF5DEB3)
      Text('2').width('100%').height(50).backgroundColor(0xD2B48C)
      Text('3').width('100%').height(50).backgroundColor(0xF5DEB3)
    }
    .height(70)
    .width('90%')
    .padding(10)
    .backgroundColor(0xAFEEEE)
    }}
```

Flex（弹性布局）

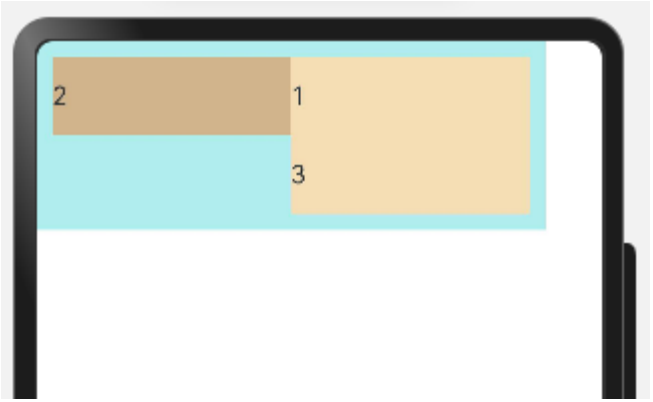
弹性布局分为单行布局和多行布局。默认情况下，Flex容器中的子元素都排在一条线（又称“轴线”）上。wrap属性控制当子元素主轴尺寸之和大于容器主轴尺寸时，Flex是单行布局还是多行布局。在多行布局时，通过交叉轴方向，确认新行排列方向。

- FlexWrap.NoWrap（默认值）：不换行。如果子元素的宽度总和大于父元素的宽度，则子元素会被压缩宽度。
- FlexWrap. Wrap：换行，每一行子元素按照主轴方向排列。
- FlexWrap. WrapReverse：换行，每一行子元素按照主轴反方向排列。



Flex（弹性布局）

```
@Entry @Component struct Test {
  build() {
    Flex({ wrap: FlexWrap.WrapReverse}) {
      Text('1').width('50%').height(50).backgroundColor(0xF5DEB3)
      Text('2').width('50%').height(50).backgroundColor(0xD2B48C)
      Text('3').width('50%').height(50).backgroundColor(0xF5DEB3)
    }
    .width('90%')
    .padding(10)
    .backgroundColor(0xAFEEEE)
    }}
```



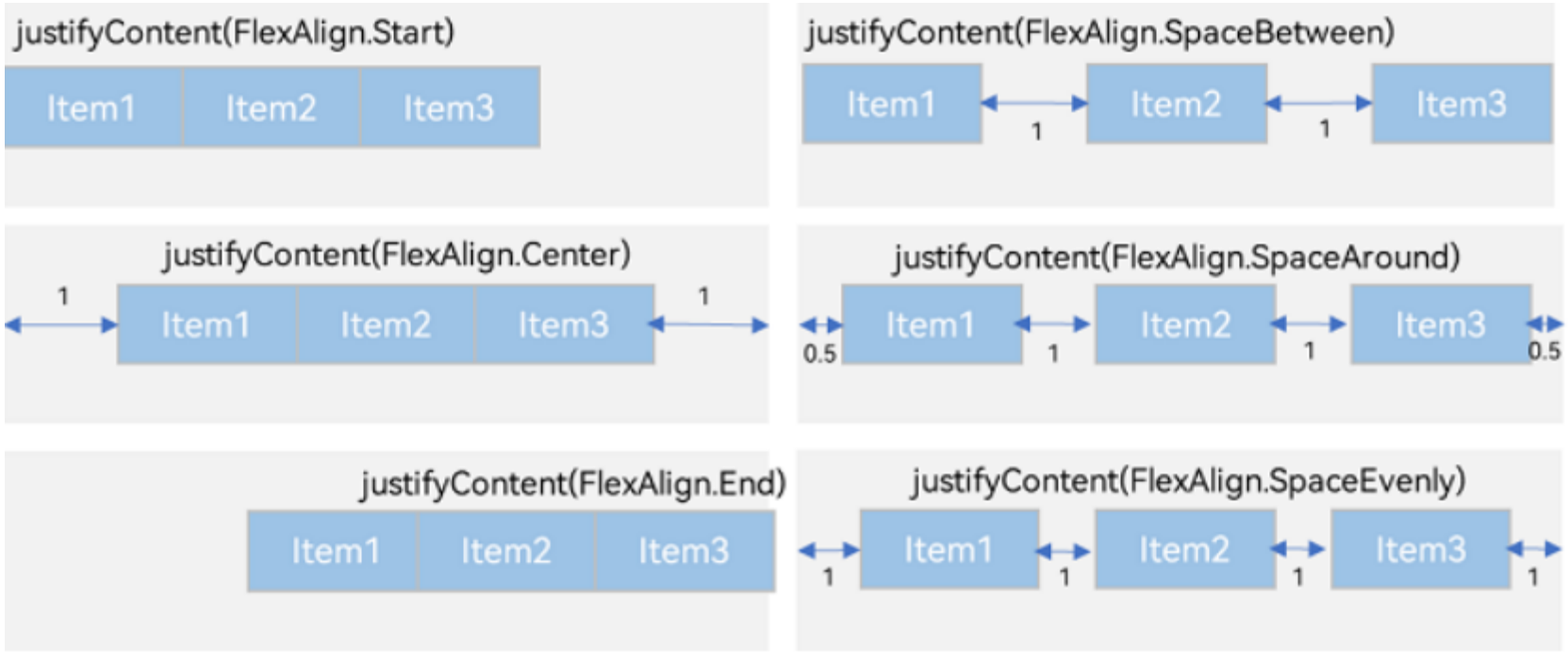
Flex（弹性布局）

主轴对齐方式：通过justifyContent参数设置子元素在主轴方向的对齐方式。

- FlexAlign.Start（默认值）：子元素在主轴方向起始端对齐， 第一个子元素与父元素边沿对齐，其他元素与前一个元素对齐。
- FlexAlign.Center：子元素在主轴方向居中对齐。
- FlexAlign.End：子元素在主轴方向终点端对齐，最后一个子元素与父元素边沿对齐，其他元素与后一个元素对齐。
- FlexAlign.SpaceBetween：Flex主轴方向均匀分配弹性元素，相邻子元素之间距离相同。第一个子元素和最后一个子元素与父元素边沿对齐。
- FlexAlign.SpaceAround：Flex主轴方向均匀分配弹性元素，相邻子元素之间距离相同。第一个子元素到主轴起始端的距离和最后一个子元素到主轴终点端的距离是相邻元素之间距离的一半。
- FlexAlign.SpaceEvenly：Flex主轴方向元素等间距布局，相邻子元素之间的间距、第一个子元素与主轴起始端的间距、最后一个子元素到主轴终点端的间距均相等。

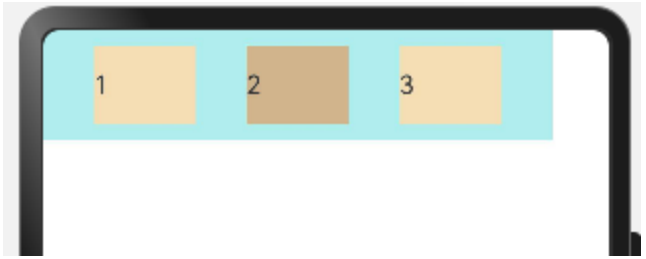
Flex（弹性布局）

主轴对齐方式：通过justifyContent参数设置子元素在主轴方向的对齐方式。



Flex（弹性布局）

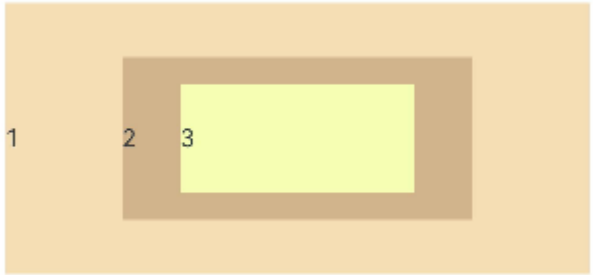
```
@Entry @Component struct Test {
  build() {
    Flex({ justifyContent: FlexAlign.SpaceEvenly }) {
      Text('1').width('20%').height(50).backgroundColor(0xF5DEB3)
      Text('2').width('20%').height(50).backgroundColor(0xD2B48C)
      Text('3').width('20%').height(50).backgroundColor(0xF5DEB3)
    }
    .width('90%')
    .padding({ top: 10, bottom: 10 })
    .backgroundColor(0xAFEEEE)
    }}
```



Stack（堆叠布局）

- 没有明确的主轴与交叉轴
- 把子组件按照设置的对齐方式顺序依次堆叠。默认后一个组件放在前一个组件上面。
- 在Stack布局下，子组件的zIndex属性生效。可以通过zIndex属性设置堆叠顺序，zIndex值越大，组件越在上层。
- Eg.

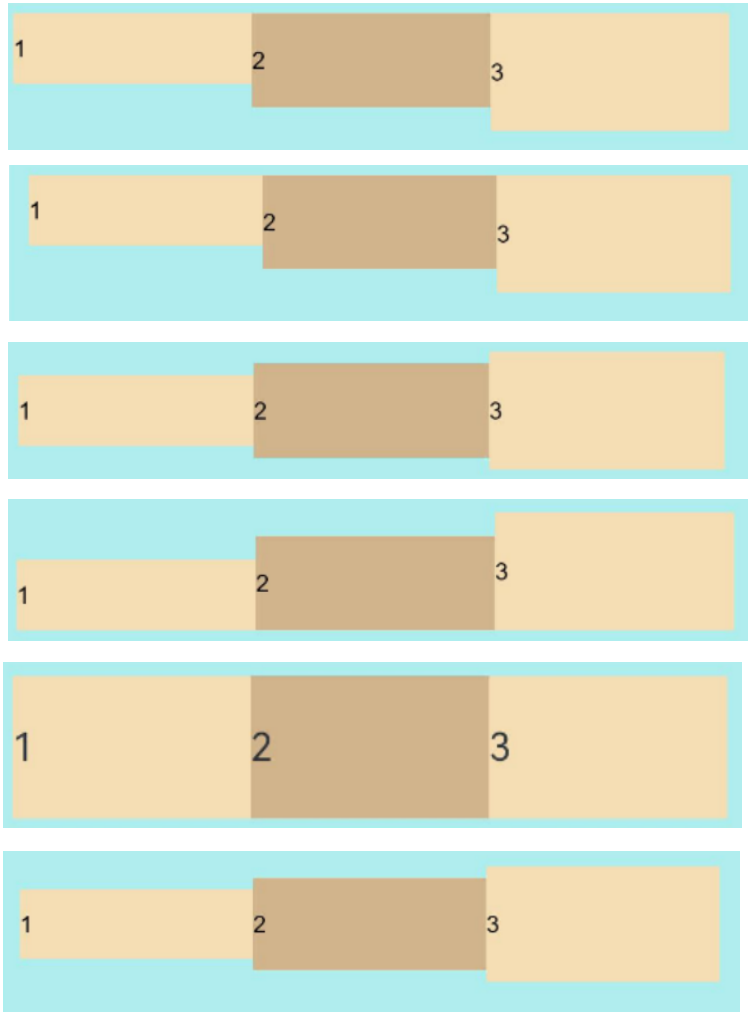
```
Stack() {
  Text('1').width('50%').height("50%").backgroundColor(0xF5DEB3).zIndex(1)
  Text('3').width('20%').height("20%").backgroundColor(0xF5fEB3).zIndex(3)
  Text('2').width('30%').height("30%").backgroundColor(0xD2B48C).zIndex(2)
}
```



Flex（弹性布局）

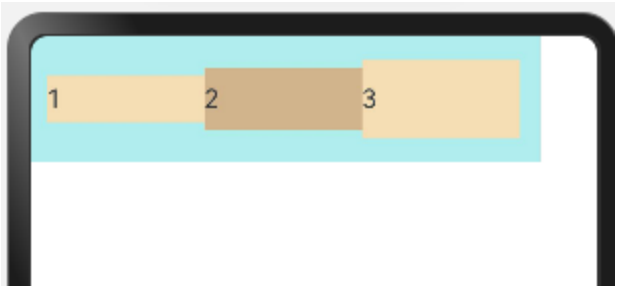
交叉轴对齐方式：可以通过Flex组件的alignItems参数设置子元素在交叉轴的对齐方式。

- ItemAlign.Auto：使用Flex容器中默认配置。
- ItemAlign.Start：交叉轴方向首部对齐。
- ItemAlign.Center：交叉轴方向居中对齐。
- ItemAlign.End：交叉轴方向底部对齐。
- ItemAlign.Stretch：交叉轴方向拉伸填充，在未设置尺寸时，拉伸到容器尺寸。
- ItemAlign. Baseline：交叉轴方向文本基线对齐。



Flex（弹性布局）

```
@Entry @Component struct Test {
  build() {
    Flex({ alignItems: ItemAlign.Center}) {
      Text('1').width('33%').height(30).backgroundColor(0xF5DEB3)
      Text('2').width('33%').height(40).backgroundColor(0xD2B48C)
      Text('3').width('33%').height(50).backgroundColor(0xF5DEB3)
    }
    .size({ width: '90%', height: 80 })
    .padding(10)
    .backgroundColor(0xAFEEEE)
    }}
```



RelativeContainer（相对布局）

- 在应用的开发过程中，经常需要设计复杂界面，此时涉及到多个相同或不同组件之间的嵌套。如果布局组件嵌套深度过深，或者嵌套组件数过多，会带来额外的开销。如果在布局的方式上进行优化，就可以有效的提升性能，减少时间开销。

- RelativeContainer: 一种采用相对布局的容器。

  - 支持容器内部的子元素设置相对位置关系，适用于处理界面复杂的场景，对多个子元素进行对齐和排列。

  - 子元素可以指定兄弟元素或父容器作为锚点，基于锚点进行相对位置布局。

  - 在使用锚点时，需注意子元素的相对位置关系，以避免出现错位或遮挡的情况。

RelativeContainer（相对布局）

基本概念

参考边界：设置当前组件的哪个边界对齐到锚点。

锚点：通过锚点设置当前元素基于哪个元素确定位置。

对齐方式：通过对齐方式，设置当前元素是基于锚点的上中下对齐，还是基于锚点的左中右对齐。

RelativeContainer（相对布局）

- 通过alignRules设置子组件的位置和大小：
  - 容器内子组件区分水平方向，垂直方向：
    - 水平方向可设置left， middle， right
    - 垂直方向可设置top， center， bottom
  - 子组件可以将容器或者其他子组件设为锚点：
    - 参与相对布局的容器内组件必须设置id，不设置id的组件不显示，容器id固定为__container__。
    - 此子组件某一方向上的三个位置可以将容器或其他子组件的同方向三个位置为锚点。如果同方向上设置两个以上锚点，水平方向以Start和Center为准，垂直方向以Top和Center为准。
      - 不建议在同方向上设置三个位置属性
    - 子组件某个方向上设置两个或以上alignRules时，可以决定组件的宽度/高度。此时height和width属性设置无效。

RelativeContainer（相对布局）

- 特殊情况
- 互相依赖，环形依赖时容器内子组件全部不绘制。
- 同方向上两个以上位置设置锚点但锚点位置逆序时此子组件大小为0，即不绘制。

Grid&Griditem（网格布局）

- 网格布局：基于“行”和“列”分割的单元格所组成的布局方式，通过指定“项所在的单元格，实现各种各样的页面布局。

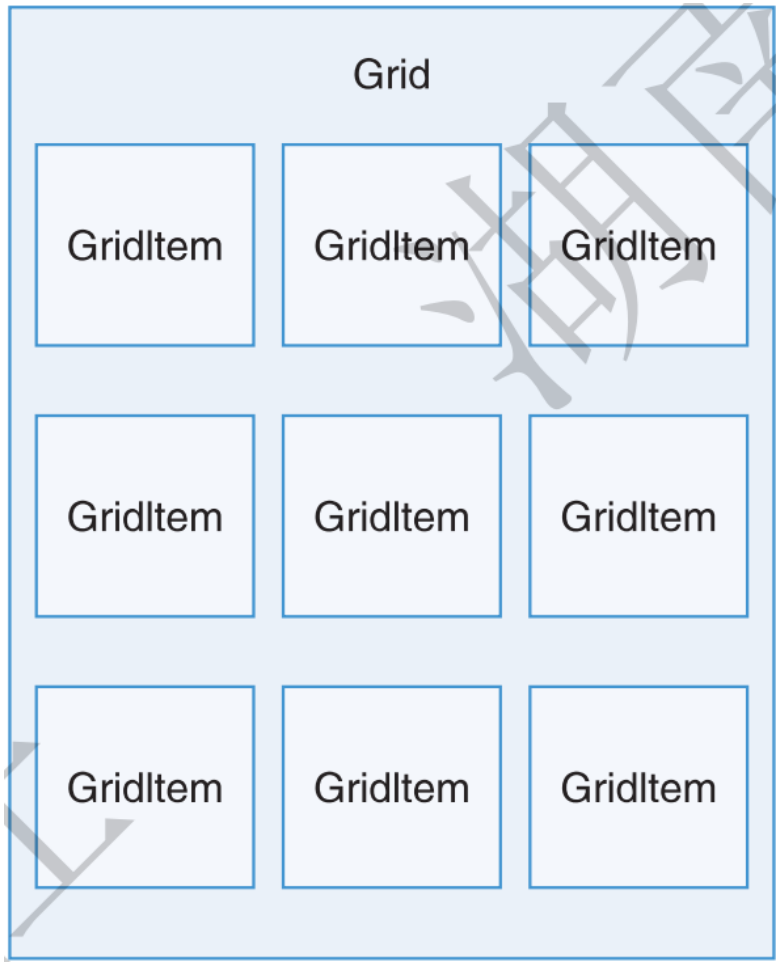
- 特点：

  - 相较于传统的布局方式，网格布局具有较强的页面均分能力和子组件占比控制能力。

  - 一种重要的自适应布局方式，广泛应用于九宫格图片展示、日历、计算器等应用场景中。

Grid&Griditem（网格布局）

- Grid组件用于设置网格布局相关参数，包括行数、列数、单元格大小等;
- GridItem 则用于定义子组件的相关特征，例如子组件所在的单元格、占据的行列数等；
- Grid与 Gridtem 组件的关系如所示。



Grid&Griditem（网格布局）

- 此外，Grid 组件还支持使用条件渲染、循环渲染、懒加载等方式生成子组件，提高开发效率。

- 网格布局是一种实用的布局方式，ArkUI提供了完善的网格布局组件，帮助开发者更加轻松地实现自适应布局。

Grid&Griditem（网格布局）

- Grid 属性介绍：除支持通用属性外，Grid 还支持以下属性

  - columnsTemplate: 设置当前网格布局列的数量，不设置时默认1列。例如，“1fr 1fr2fr”是将父组件分3列，将父组件允许的宽分为4等份，第一列占1份，第二列占1份，第三列占2份。
  - rowsTemplate: 设置当前网格布局行的数量，不设置时默认1行。例如，“1fr 1fr 2fr”是将父组件分3行，将父组件允许的高分为4等份，第一行占1份，第二行占1份，第三行占2份。设置列与列的间距。默认值为0
  - rowsGap: 设置行与行的间距。默认值为 0。
  - scrollBar: 设置滚动条状态。默认值为BarState.Off。
  - cachedCount: 设置预加载的GridItem的数量。默认值为1。

Grid&Griditem（网格布局）

- Grid 属性介绍：除支持通用属性外，Grid 还支持以下属性

➢ layoutDirection: 设置布局的主轴方向。默认值为 GridDirection.Row。

名称	描述
Row	主轴布局方向沿水平方向布局，即自左往右先填满一行，再去填下一行
Column	主轴布局方向沿垂直方向布局，即自上往下先填满一列，再去填下一列
RowReverse	主轴布局方向沿水平方向反向布局，即自右往左先填满一行，再去填下一行
ColumnReverse	主轴布局方向沿垂直方向反向布局，即自下往上先填满一列，再去填下一列

➢ maxCount: 当 layoutDirection 是 Row/RowReverse 时，表示可显示的最大列数。当 layoutDirection 是 Column/ColimnReverse 时，表示可显示的最大行数。默认值为 Infinity。

Grid&Griditem（网格布局）

- Grid 属性介绍：除支持通用属性外，Grid 还支持以下属性

➢ maxCount: 当 layoutDirection 是 Row/RowReverse 时，表示可显示的最大列数。当 layoutDirection 是 Column/ColimnReverse 时，表示可显示的最大行数。默认值为 Infinity。

➢ minCount: 当layoutDirection是 Row/RowReverse 时，表示可显示的最小列数。当 lay-outDirection是 Column/ColumnReverse 时，表示可显示的最小行数。默认值为 1

➢ cellLength: 当layoutDirection是Row/RowReverse 时，表示一行的高度。当 layoutDirec-tion 是 Column/ColumnReverse 时，表示一列的宽度。默认值为 0。

Grid&Griditem（网格布局）

- Grid 事件介绍：除支持通用事件外，Grid 还支持以下事件

名称	功能描述
onScrollIndex(event:(first:number) => void)	当前网格显示的起始位置 item 发生变化时触发。 first：当前显示的网格起始位置的索引值
onItemDragStart(event:(event:ItemDragInfo, itemInde: number)=>(() =>any)|void)	开始拖拽网格元素时触发。 event：见 ItemDragInfo 对象说明； itemIndex：被拖拽网格元素索引值
onItemDragEnter(event:(event:ItemDragInfo) => void)	拖拽进入网格元素范围内时触发。 event：见 ItemDragInfo 对象说明
onItemDragMove(event:(event:ItemDragInfo, itemIndex: number,insertIndex:number)=> void)	拖拽在网格元素范围内移动时触发。 event：见 ItemDragInfo 对象说明； itemIndex：拖拽起始位置； insertIndex：拖拽插入位置
onItemDragLeave(event:(event:ItemDragInfo, itemIndex: number)=>void)	拖拽离开网格元素时触发。 event：见 ItemDragInfo 对象说明； itemIndex：拖拽离开的网格元素索引值
onItemDrop(event:(event:ItemDragInfo, itemIndex:num-ber,insertIndex:number, isSuccess:boolean)=>void)	绑定该事件的网格元素可作为拖拽释放目标，当在网格元素内停止拖拽时触发。 event：ItemDragInfo 对象； itemIndex：拖拽起始位置； insertIndex：拖拽插入位置； isSuccess：是否成功释放

Grid&Griditem（网格布局）

- GridItem 属性介绍：

  - rowStart: 指定当前元素起始行号。
  - rowEnd:指定当前元素终点行号。
  - columnStart: 指定当前元素起始列号。
  - columnEnd.指定当前元素终点列号。
  - forceRebuild:设置在触发组件 build 时是否重新创建此节点。默认值为 false
  - Selectable: 当前 GridItem 元素是否可以被鼠标框选。默认值为 true。注意，外层 Grid容器的鼠标框选开启时，GridItem的框选才生效。

Grid&Griditem（网格布局）

- GridItem 事件介绍：

➢ onSelect(event:(isSelected: boolean)=>void): GridItem 元素被鼠标框选的状态改变时触发回调。

➢ isSelected: 进入鼠标框选范围即被选中返回 tue，移出鼠标框选范围即未被选中返回 false。

Grid&Griditem（网格布局）

- 循环渲染：通过循环渲染(ForEach)从数组中获取数据，并为每个数据项创建相应的组件，可降低代码复杂度，参数介绍如下。

➢ anr: 必须是数组，允许设置为空数组，空数组场景下将不会创建子组件。同时允许设置返回值为数组类型的函数。

➢ itemGenerator: 生成子组件的 lambda 兩数，为数组中的每一个数据项创建一个或多个子组件，单个子组件或子组件列表必须包括在大括号中。

➢ keyGenerator: 匿名函数，用于给数组中的每一个数据项生成唯一旦固定的键值。当数据项在数组中的位置更改时，其键值不得更改;当数组中的数据项被新项替换时，被替换项的键值和新项的键值必须不同。键值生成器的功能是可选的，但是，为了使开发框架能够更好地识别数组更改，提高性能，建议提供键值生成器。将数组反向时，如果没有提供键值生成器，那么 ForEach 中的所有节点都将重建。

GridRow&GridCol（栅格布局）

- 栅格布局：一种通用的辅助定位工具，对移动设备的界面设计有较好的借鉴作用。
- 优势：
  - 提供可循的规律
  - 统一的定位标注
  - 灵活的间距调整方法
  - 自动换行和自适应

GridRow&GridCol（栅格布局）

GridRow 为栅格容器组件，需与栅格子组件 GridCol在栅格布局场景中联合使用。

- GridRow构造参数：参数 GridRowOptions 属性如下 。

➢ gutter: 设置子元素之间的间距，决定内容间的紧密程度

➢ columns: 栅格布局的主要定位工具，设置栅格布局的总列数，默认为 12。

➢ Breakpoints: 以设备宽度为基准，将应用宽度分成了几个不同的区间，即不同的断点。

➢ direction: 设置栅格子组件在栅格容器中的排列方向。

GridRow&GridCol（栅格布局）

- 栅格系统断点：断点以设备宽度为基准，将应用宽度分成了几个不同的区间，即不同的断点。

➢ 开发者根据需求在不同的区间实现不同的页面布局效果。

➢ 栅格系统默认断点将设备宽度分为xs、sm、md、lg 四类。

➢ 在 GridRow新栅格组件中，允许开发者使用 Breakpoints 自定义修改断点的取值范围，最多支持6个断点。

➢ 除了默认的四个断点外，还可以启用x1、xx1两个断点支持六种不同尺寸(xs, sm, md, lg, xl, xxl)设备的布局设置。

GridRow&GridCol（栅格布局）

- 栅格系统断点

断点名称	取值范围
xs	0 vp < 水平宽度 < 320 vp
sm	320 vp ≤水平宽度 < 520 vp
md	520 vp ≤水平宽度 < 840 vp
lg	840 PPL ≤水平分辨率 <1080 PPL
xl	自定义
xxl	自定义

- 注意：栅格子组件，必须作为栅格容器组件(GridRow)的子组件使用。

Grid&Griditem（网格布局）

- GridCol 属性介绍：

➢ span: 占用列数。span为0表示该元素不参与布局计算，即不会被染。默认值为1。

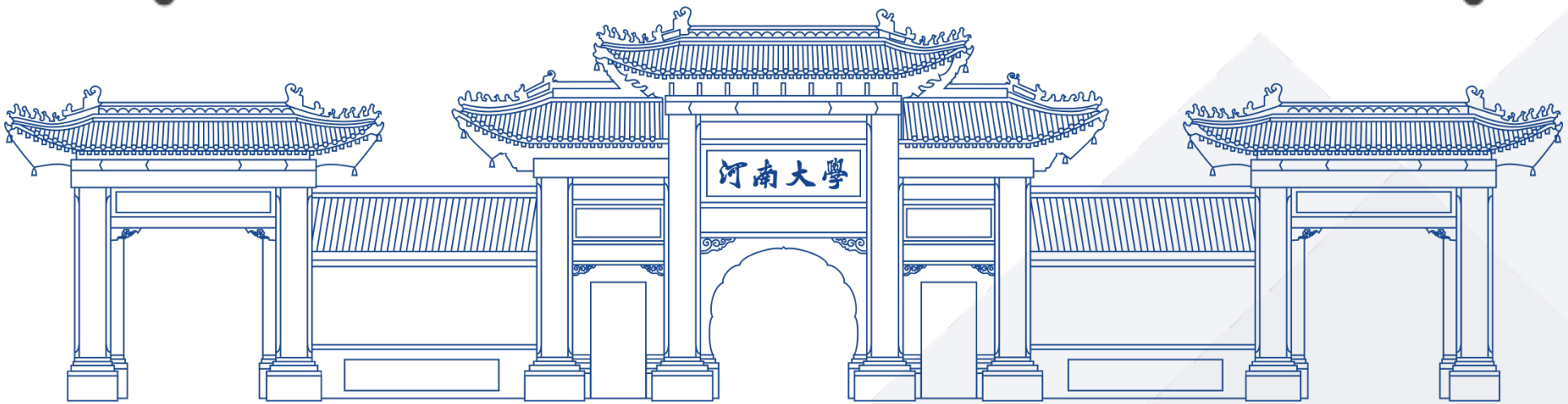
➢ offset: 相对于前一个栅格子组件偏移的列数。默认值为 0。]

➢ order:元素的序号，根据栅格子组件的序号，从小到大对栅格子组件排序。默认值为 0。

河南大學
Henan University

第五章 基础组件

软件学院



基础组件

- 主要介绍了OpenHarmony 中常用的基础组件如

  - 输入框、单选框、复选框、下拉框等。

- 介绍通过这些基础组件实现和用户交互的方法，如

  - 获取用户输入的数据、响应用户的选择等。

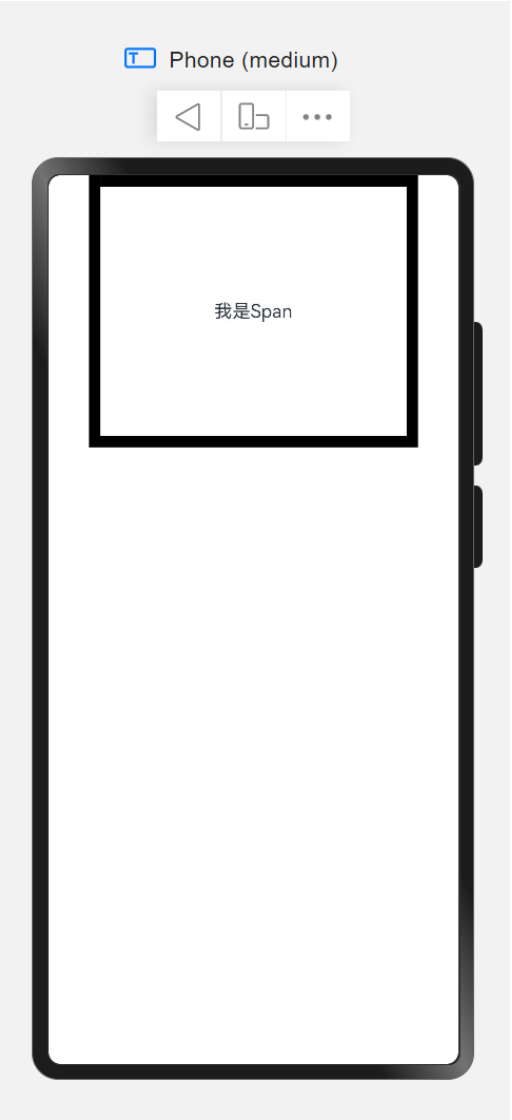
文本组件–Text

- 文本(Text)组件：主要作用是在用户界面上展示一段文本信息。

Text(content?: String|Resource)

content: 文本内容。包含子组件 Span 时不生效，显示Span 内容。

```
@Entry
@Component
struct Index {
 build() {
  Column() {
    Text('我是Text') {
      Span('我是Span')
    }
    .padding(100)
    .borderWidth(10)
  }.width('100%').height('100%')  }}
```



文本组件–Text

- 属性介绍

➢ textAlign: 设置文本水平方向对齐方式，只有Text组件本身的宽度大于文本内容长度时，textAlign属性才起作用。默认值为TextAlign.Start。

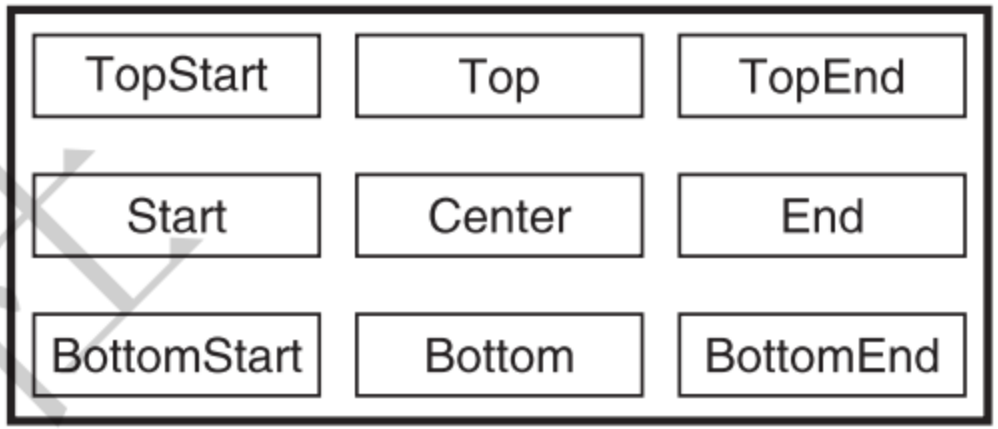
```
@Entry
@Component
struct Index {
 build() {
   Column() {
     Text('默认对齐').width("100%")
     Text('左对齐').textAlign(TextAlign.Start).width("100%")
     Text('居中对齐').textAlign(TextAlign.Center).width("100%")
     Text('右对齐').textAlign(TextAlign.End).width("100%")
   }}}
```

文本组件–Text

- 属性介绍

➢ align: 可用于控制文本段落在垂直和水平方向上的九种位置，例如

```
.align(Alignment.Start)
```



注意：对齐align和textAlign属性都能设置文字对齐方向，对齐方式align比textAlign优先级高，同时设置时textAlign失效。

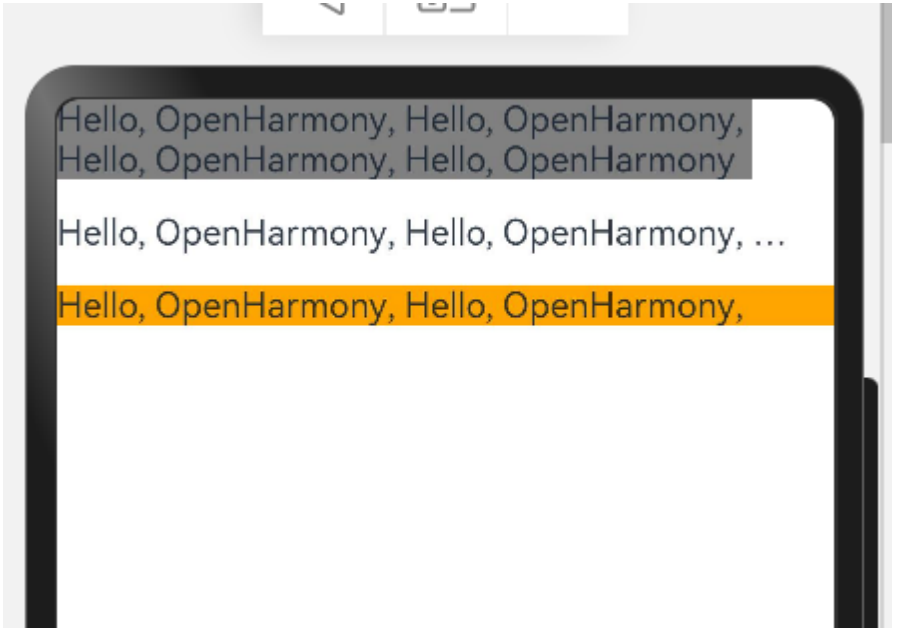
文本组件–Text

- 属性介绍

➢ textOverfow: 当文本内容较多，超出了 Text组件宽度时，可以使用 textOverflow 设置文本截取方式，注意需配合 maxLines 使用，单独设置不生效。maxLines 用于设置文本显示最大行数，默认值为 Imnfnity(无穷)。

文本组件–Text

```
@Entry @Component struct Test {
 build() {
   Column({space:5}) {
     Text("Hello, OpenHarmony, Hello, OpenHarmony, Hello,
OpenHarmony, Hello, OpenHarmony")
       .backgroundColor(Color.Grey)
     Text("Hello, OpenHarmony, Hello, OpenHarmony, Hello,
OpenHarmony, Hello, OpenHarmony")
       .margin({ top: 10 })
       .maxLines(1)
       .textOverflow({overflow:TextOverflow.Ellipsis}) //省略号代替
backgroundColor(Color.Orange)
     Text("Hello, OpenHarmony, Hello, OpenHarmony, Hello,
OpenHarmony, Hello, OpenHarmony")
       .margin({ top: 10 })
       .maxLines(1)
       .textOverflow({overflow:TextOverflow.Clip}) //直接
       .backgroundColor(Color.Orange)
   }.alignItems(HorizontalAlign.Start)
 }}
```



文本组件–Text

- 属性介绍

➢ 属性设置文本的样式，比如 fontSize、fontColor 、 fontStyle 和 fontWeight 分别设置文字的大小、颜色、样式和粗细。

```
@Entry
@Component
struct Index {
 build() {
   Row() {
     Text('绿色')
       .fontColor(Color.Green)
       .fontSize(100)
       .fontStyle(FontStyle.Italic)
       .fontWeight(FontWeight.Bold)
   }}}
```

文本组件–Text

➢ decoration: 设置文本装饰线样式以及颜色，其中包含 type 和 color 两个参数。type 用于设置装饰线样式的参数类型TextDecorationType。而 color 则为装饰线颜色的可选参数。

名称	描述
Underline	文字下划线修饰
LineThrough	穿过文本的修饰线
Overline	文字上划线修饰
None	不使用文本装饰线

文本组件–Text

```
@Entry
@Component
struct Index {
 build() {
   Row() {
     Text('绿色')
       .fontColor(Color.Green)
       .fontSize(100)
       .fontStyle(FontStyle.Italic)
       .fontWeight(FontWeight.Bold)
       .decoration({type:TextDecorationType.Underline,
        color: Color.Orange})
   }}}
```

属性可以组合使用，以创建出更加丰富多彩的文本样式。

文本组件–Text

- TextCase: 设置文本大小写，默认值为TextCase.NormaltextCase。

名称	描述
Normal	保持文本原有大小写
LowerCase	文本采用全小写
UpperCase	文本采用全大写

文本组件–Text

- TextCase: 设置文本大小写，默认值为TextCase.NormaltextCase。

```
@Entry
@Component
struct Index {
 build() {
   Row() {
     Text('aBcDeFg')
       .fontSize(60)
       .textCase(TextCase.LowerCase)
   }
 }
}
```

文本组件–Text

➢ 本垂直显示:Text 组件设置文本垂直显示时，字体大小和组件宽度一致即可，:保证每行只能容纳一个字体，不用设置高度。

```
@Entry @Component struct Test{
 build() {
   Column() {
     Text("开源鸿
蒙").fontSize(50).width(50)
   }.alignItems(HorizontalAlign.Start)
 } }
```

文本组件–TextInput

- TextInput 组件被设计用于输入单行文本，并对输入事件做出响应。
- 其应用范围广泛在登录账号、发送消息等场景中均可使用。
- TextInput组件同样支持文本样式的设置。
- ArkUI开发框架提供了2种类型的输入框，即TextInput和 TextArea。
  - TextInput仅支持单行输人，
  - TextArea支持多行输入。

文本组件–TextInput

➢ TextInput: 可接收参数 TextInputOptions，其对象说明如下：

- text: 设置输入框的初始显示文本，不设置时显示 placeholder 的内容。
- placeholder: 设置无输入时的提示文本，当不设置 text 时，显示该文本。
- controller: 设置 TextInput 控制器。

```
@Entry @Component struct Test {
 build() {
   Column() {
     TextInput({ placeholder: "请输入账号", text:"输入姓名" })
   }
 }
}
```

输入框组件TextInput

单行文本输入框组件TextInput，用于输入文字。

```
TextInput({placeholder:"请输入账户", //设置无输入时的提示文本
  text:""})                          //设置输入框当前的文本内容
  .type(InputType.Normal)            //设置输入框类型
  .onChange( (value: string) =>{
  //输入内容发生变化时，触发该回调，value就是输入的内容
  })
```

文本组件–TextInput

➢ TextInput属性:

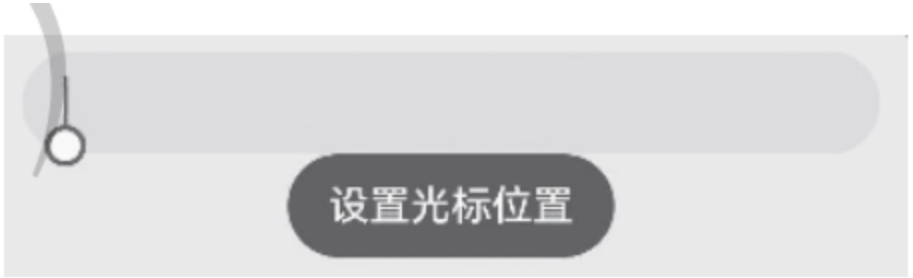
placeholder: 使用输入框时，往往会有一些提示文字。例如登录账号的时候会有“请输人账号”这样的文本提示;当用户输人内容之后，提示文本就会消失，这种提示功能使用placeholder 属性就可以轻松地实现。还可以使用 placeholderColor 和 placeholderFont 分别设置提示文本的颜色和样式。

```
@Entry @Component struct Test {
 build() {
  Column() {
   TextInput({ placeholder:"请输入账号"})
    .placeholderColor(0x999999)
    .placeholderFont({
     size: 20,
     weight: FontWeight.Medium,
     family: "cursive",
     style: FontStyle.Italic })
  }
 }
```

文本组件–TextInput

➢ TextInputController: 光标控制器，可以使用 TextInputController 动态设置光标位置。例如，使用 TextInputController 的caretPosition 方法，将光标移动到了第二个字符后。

```
@Entry @Component struct TextInputDemo {
 controller: TextInputController = new TextInputController()
 build() {
   Column() {
   TextInput({controller: this.controller})
   Button("设置光标位置")
     .onClick(()=>{
       this.controller.caretPosition(3)
       })
       }.height("100%").backgroundColor(0xE6F2FD)
}}
```



输入类型

InputType

- InputType.Normal：基本输入模式。
  支持输入数字、字母、下划线、空格、特殊字符；
- InputType.Password：密码输入模式；
- InputType.Email：邮箱地址输入模式；
- InputType.Number：纯数字输入模式；
- InputType.PhoneNumber：电话号码输入模式。

代码演示

```
.type(InputType.Normal)
```

请输入

```
.type(InputType.Password)
```

···

```
.type(InputType.Email)
```

edu123@edu.com

```
.type(InputType.Number)
```

2111

```
.type(InputType.PhoneNumber)
```

13375917890

文本组件–TextInput

- type: 设置输人框的类型，默认值为 InputType.Normal。

名称	描述
Normal	基本输入模式。支持输入数字、字母、下划线、空格、特殊字符
Password	密码输入模式。支持输入数字、字母、下划线、空格、特殊字符。密码显示小眼睛图标并且默认会将文字变成圆点
E-mail	邮箱地址输入模式。支持数字、字母、下划线，以及 @ 字符（只能存在一个 @ 字符）
Number	纯数字输入模式
PhoneNumber	电话号码输入模式。支持输入数字、+、-、*、#，长度不限

文本组件–TextInput

➢ 例如InputType.Number， 只能输入数字。

```
@Entry @Component struct Test {
 build() {
   Column() {
     TextInput({ placeholder: "请输入账号", text:"输入姓名" })
       .type(InputType.Number)
   }
 }
}
```

文本组件–TextInput

➢ enterKeyType: 用于设置输入法回车键类型，主要控制回车键的显示样式。默认值为EnterKeyType.Done。

名称	
Go	显示为前往样式
Search	显示为搜索样式
Send	显示为发送样式
Next	显示为下一个样式
Done	显示为确认样式

```
TextInput({ placeholder: " 请输入密码 "})
  .type(InputType.PhoneNumber)      // 电话号码输入模式
  .enterKeyType(EnterKeyType.Send)  // 键盘回车文字为发送
```

文本组件–TextInput

maxLength: 设置输人框允许输人多少字符

caretColor: 设置光标的颜色

```
@Entry @Component struct Test {
 build() {
   Column() {
     TextInput({ placeholder: "请输入账号" })
       .type(InputType.Number)
       .maxLength(5)
       .caretColor(Color.Red)
   }
 }
}
```

文本组件–TextArea(了解)

- TextArea 和 Textlnput 是常见的输人框控件，它们广泛应用于用户输入文本、数字、密码等信息的场景。
- TextArea 相对于 TextInput 而言，具有输入多行文本的特性，因此在需要用户输入较多文字的场景下，具有更好的应用效果。
- 这两个控件的属性基本相同，例如宽度、高度、字体、字号等，同时也支持文本的自动换行，以及剪切、复制、粘贴等常见操作。
- 然而，TextArea 在 TextInput 的基础上增加了正则表达式输入验证的功能。正则表达式是一种强大的匹配规则语言，可以用于验证用户输入的合法性，例如邮箱、手机号、密码等。
- 通过使用正则表达式输入验证，可以有效地减少用户输入错误的情况，提高输人数据的准确性和完整性，从而提高系统的稳定性和安全性。因此，在某些需要输入验证的场景下，TextArea相较于 Textlmput 更加实用和可靠。

选择组件

- Checkbox 复选框--可选参数
- CheckboxOptions 定义　　Checkbox(options?: {name?: string, group?: string})
  - name: 复选框名称
  - group: 设置复选框的组
- 属性--支持通用属性
  - select: 设置是否被选中，默认值为false
  - selectColor: 设置复选框选中状态下的颜色
- 事件
  - onChange: 当选中状态发生变化时，触发该回调，监听用户选择的变化

多选框组件CheckBox

多选框组件CheckBox，通常用于某选项的打开或关闭。当选中状态发生变化时，触发onChange回调；回调的value为true时表示已选中，value为false时表示未选中。

```
Row() {
  Checkbox({name: '阅读', group: 'checkboxGroup'})
    .select(true) //默认选中
    .selectedColor(0xed6f21)
    .onChange((value: boolean) => {//事件处理
      console.info('阅读 change is'+ value)
    })
  Checkbox({name: '运动', group: 'checkboxGroup'})
    .select(false)
    .selectedColor(0x39a2db)
    .onChange((value: boolean) => {
      console.info('运动 change is'+ value)
    })
}
```

选择组件

```
@Entry @Component
struct CheckboxExample {
 build() {
   Row() {
     Checkbox({name: ‘checkbox1’,  group: ‘checkboxGroup’})
       .select(true)
       .selectedColor(0xed6f21)
       .onChange((value: boolean) => {
         console.info(‘Checkbox1 change is’+ value)//在日志中打印
       })
     Checkbox({name: 'checkbox2',  group: 'checkboxGroup'})
       .select(false)
       .selectedColor(0x39a2db)
       .onChange((value: boolean) => {

         console.info('Checkbox2 change is'+ value)  //在日志中打印
       })}}}
```

选择组件

- CheckboxGroup 定义

➢ 构造参数 group: 组名称，创建多选框群组，可以控制群组内的Checkbox 全选或者不全选，相同 group 的 Checkbox 和 CheckboxGroup 为同一群组。

- CheckboxGroup 属性

➢ selectA11: 设置复选框组是否全选。若同组的Checkbox显式设置 select，则 Checkbox 的优先级高。

- CheckboxGroup 事件

➢ onChange: CheckboxGroup的选中状态或群组内的Checkbox的选中状态发生变化时，触发回调。

选择组件

- CheckboxGroupReesult 对象说明

➢ CheckboxGroupResult: 回调参数类型定义

➢ names: 群组内所有被选中的多选框名称

➢ status: 选中状态。SelectStatus 具体描述见

名称	描述
All	群组多选择框全部选择
Part	群组多选择框部分选择
None	群组多选择框全部没有选择

选择组件

- Radio 单选框

➢ Radio 单选框是常见的用户交互选择项，

➢ 通常用于让用户在多个选项中选择个

➢ 当用户点击单选框时，它会被选中并显示一个圆形标记，表示用户已经做出了选择。

- 相比于复选框，Radio单选框只允许用户选择一个选项，因此更适用于需要用户做出单一选择的情况。

- 在设计界面时，我们可以通过设置单选框的文本标签和选中状态来让用户清晰地了解每个选项的含义和当前选择的选项。

选择组件

- Radio 定义

Radio(options: {value: string, group: string})

➢ value: 当前单选框的值

➢ group:当前单选框的所属群组名称

- Radio属性

➢ checked: 设置单选框的初始选中状态，默认为false

- Radio事件

➢ onChange: 单选框选中状态改变时触发回调

选择组件

```
@Entry
@Component
struct Test {
 build() {
   Column() {
     Radio({value: 'Radio1', group: 'radioGroup'})
       .checked(false)
       .onChange((isChecked: Boolean)=> {
         if(isChecked){
           console.log("点击成功")
         }//在日志中打印点击成功
               })
     Radio({value: 'Radio2', group: 'radioGroup'})
       .checked(true)
   }}}
```

```
I  app Log: 点击成功
```

选择组件

- Select 下拉选择菜单

➢ 常见的用户界面元素，通常用于让用户在多个选项之间进行选择

➢ 通常包含一个可点击的下拉箭头，当用户点击箭头时，会展开一个下拉列表，其中包含多个选项

➢ 用户可以通过单击选项来进行选择

- 下拉选择菜单通常用于网页表单、设置面板和其他需要用户进行选择的场景中
- 优点：可以节省屏幕空间，并且用户可以轻松地在多个选项之间进行选择，而不必在页面上滚动或搜索

选择组件

- Select 定义

Select(options: Array<SelectOption>)

➢ 构造参数 SelectOption 定义

- value: 下拉选项内容
- group: 下拉选项图片

- Select属性

➢ selected: 设置下拉菜单初始选择项的索引，第一项的索引为0

➢ value: 设置下拉按钮本身的文本显示

➢ font: 设置下拉按钮本身的文本样式

选择组件

- Select属性
  - fontColor: 设置下拉按钮本身的文本颜色
  - selectedOptionBgColor: 设置下拉菜单选中项的背景色
  - selectedOptionFont: 设置下拉菜单选中项的文本样式
  - selectedOptionFontColor: 设置下拉菜单选中项的文本颜色
  - optionBgColor: 设置下拉菜单项的背景色
  - optionFont: 设置下拉菜单项的文本样式
  - optionFontColor: 设置下拉菜单项的文本颜色
- Select事件

选择组件

```
@Entry
@Component struct SelectTest {
 build(){
  Row(){
    Select([{value:"大一", icon: $r("app.media.icon")},
            {value:"大二", icon:$r("app.media.icon")},
            {value:"大三", icon: $r("app.media.icon")},
            {value:"大四", icon: $r("app.media.icon")}])
           .value("请选择")//下拉选项的提示语
            .onSelect((index:number,value:string)=>{
           console.log("Select:" + index+"----"+value)
           })}
   .width("100%")
   .height("100%")
   .alignItems(VerticalAlign.Top)
}}
```

按钮组件–Button

- 按钮组件是一种非常实用的组件，它可以在各种应用场景中使用。
- 在用户界面设计中按钮组件通常被用来触发某些操作或者跳转到其他页面。
- 由于按钮组件的使用频率非常高，因此在设计时需要考虑其使用的便捷性以及外观的美感。
- 在外观方面，按钮组件通常有三种不同的形状: 矩形圆角按钮、胶囊型按钮和圆形按钮。这些不同的形状可以根据具体的设计需求进行选择。
- 按钮组件还可以添加一个子组件，以实现不同的展示样式。例如，可以在按钮上添加一个图标或者文字，以便用户更加清晰地了解按钮的功能。

按钮组件Button

按钮组件，可快速创建不同样式的按钮，使用属性.type()描述按钮样式。

```
Button("登录")
 .type(ButtonType.Normal)//描述按钮显示样式
 //按钮按下时是否开启按压态显示效果,当设置为false时,按压效果关闭
 .stateEffect(true)
 .onClick(()=>{//事件处理
  //todo
 })
```

按钮类型

ButtonType

- Normal： 普通按钮（默认不带圆角）；
- Capsule： 胶囊型按钮（圆角默认为高度的一半）；
- Circle ： 圆形按钮。

```
.type(ButtonType.Normal)
```

登录

```
.type(ButtonType.Capsule)
```

登录

```
.type(ButtonType.Circle)
```

...

代码演示

按钮组件–Button

- 按钮组件Button定义：

```
Button(label?: string, options?: { type?: ButtonType, stateEffect?: boolean })
```

➢ label: 用于设置按钮文本

➢ options: 主要设置按钮的外观风格。默认效果type为胶囊型按钮风格，开启切换效果。除非特意要实现矩形按钮或圆形按钮的效果，否则，建议不设置 options参数。options也由两个可选参数组成。

- type: 设置 Buton 按钮的显示样式，默认值为ButtonType.Capsule。具体见表 。

名称	描述
Capsule	胶囊类型，圆角值默认为高度的一半并且不允许修改 登录
Normal	普通按钮（默认不带圆角），可以通过 borderRadius 设置圆角大小 登录
Circle	圆形按钮，设置该样式时，需要设置 Button 的宽，否则不显示 登录

- stateEffect:按钮按下时是否开启按压态显示效果，当设置为false时，按压效果关闭。默认值为

按钮组件–Button

- 按钮组件Button定义：

```
Button(label?: string, options?: { type?: ButtonType, stateEffect?: boolean })
```

➢ label: 用于设置按钮文本

➢ options: 主要设置按钮的外观风格。默认效果type为胶囊型按钮风格，开启切换效果。除非特意要实现矩形按钮或圆形按钮的效果，否则，建议不设置 options参数。options也由两个可选参数组成。

- type: 设置 Buton 按钮的显示样式，默认值为ButtonType.Capsule。具体见表 。

名称	描述
Capsule	胶囊类型，圆角值默认为高度的一半并且不允许修改 登录
Normal	普通按钮（默认不带圆角），可以通过 borderRadius 设置圆角大小 登录
Circle	圆形按钮，设置该样式时，需要设置 Button 的宽，否则不显示 登录

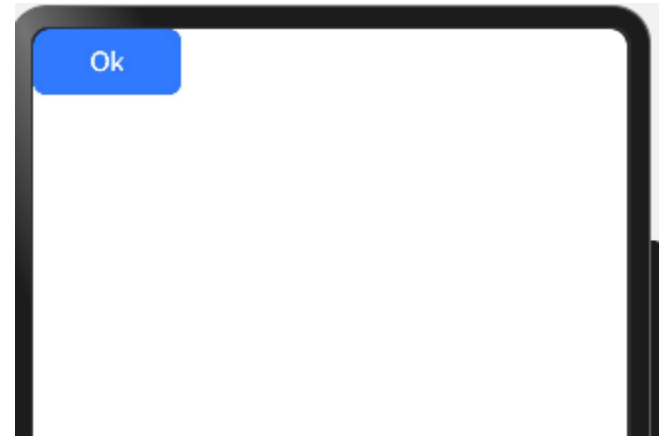
- stateEffect: 按钮按下时是否开启按压态显示效果，当设置为false时，按压效果关闭。默认值为

按钮组件–Button

- 按钮组件Button定义：

```
Button(label?: string, options?: { type?: ButtonType, stateEffect?: boolean })
```

```
@Entry @Component struct Test {
 build() {
   Button('Ok', { type: ButtonType.Normal, stateEffect: true })
     .borderRadius(8)  //设置元素的边框圆角半径
     .backgroundColor(0x317aff)
     .width(90)
     .height(40)
 }
}
```



按钮组件–Button

- Button事件：通过绑定 Button的 onClick 事件实现，当用户点击 Button 时，系统会回调执行onClick 方法，并触发其中的逻辑代码。

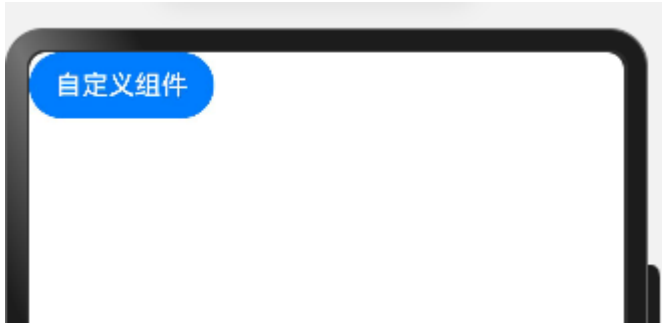
```
@Entry @Component struct Test {
 build() {
   Button('Ok', { type: ButtonType.Normal, stateEffect: true })
     .backgroundColor(0x317aff)
     .width(90)
     .height(40)
     .onClick(()=>{
       console.log("按钮触发")//日志中打印
     })
 }
}
```

按钮组件–Button

- Button自定义样式：

➢ Button 组件的容器特性赋予其嵌套其他组件的能力，进而实现更加灵活和多样化的Button开发策略。例如，例如通过borderRadius属性设置按钮的边框弧度

```
@Entry @Component struct Test {
  build() {
    Button('自定义组件', { type: ButtonType.Normal })
      .borderRadius(20)
      .height(40)
  }
}
```



按钮组件–Button

- Button的Text子组件：

➢ 通过设置 Button 的Text子组件达到文本修改的效果

```
@Entry @Component struct Test {
  build() {
    Button({type: ButtonType.Normal }){
      Text("文本组件")
        .fontColor(Color.Orange)
        .fontSize(80)
    }
  }
}
```

按钮组件–Button

- Button的Image子组件：

➢ 通过设置 Button 的Image子组件达到图片修改的效果

```
@Entry @Component struct Test {
  build() {
    Button({type: ButtonType.Normal, stateEffect: true }){
      Image($r("app.media.app_icon"))
        .width(30)
        .height(30)
    }}}
```

Image组件

- 在移动应用开发领域，Image组件是一种广泛使用的工具，因为图片作为一种直观生动的表现形式，能够更好地吸引用户的关注。

- 通过1mage组件，开发者可以将各种图片展示在应用中，例如应用的logo、产品图片、用户头像等。

- 展示图片非常简单，只需在组件中指定图片的地址、宽和高，即可控制图片的大小和位置，让其展示更加美观。

- 此外Image 组件还支持从**内存**、**本地**和**网络加载**图片，以满足不同场景下的需求。但需要注意的是，若从网络加载图片，需申请网络访问权限，以确保用户授权后进行网络加载操作。

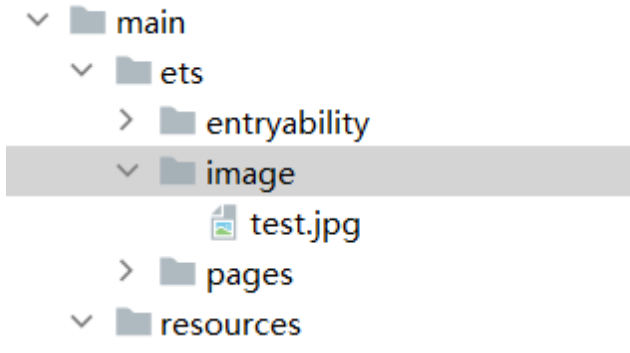
Image组件

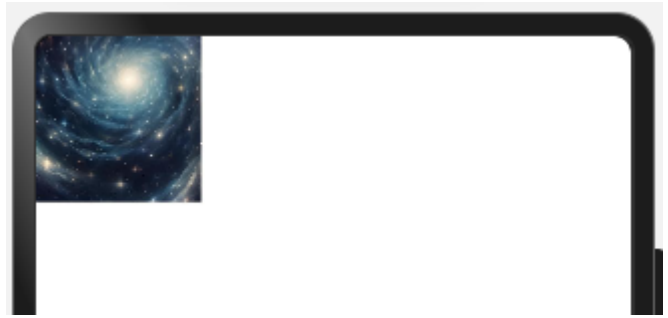
- Image定义　Image(src: string | Resource | media.PixelMap)

➢ src: 必选参数，设置要加载的图片资源，支持从本地、网络和内存中加载图片。支持的图片格式包括 png、jpg、bmp、svg 和 gif。

1）本地资源：创建文件夹，将本地图片放入ets文件夹下的任意位置

```
@Entry @Component struct Test {
 build() {
   Column(){
   Image("image/test.jpg")
      .width(100)
      .height(100)
   }}}
```





Image组件

2）Resource资源：使用资源格式可以跨包/跨模块引入图片，resources文件夹下的图片都可以通过$r资源接口读取到并转换到Resource格式。

```
@Entry @Component struct Test {
 build() {
   Column(){
   Image($r("app.media.app_icon"))
       .width(100)
       .height(100)
   }}}
```

Image组件

3）网络资源：引入网络图片需申请权限ohos.permission.INTERNET。

```
Image("https://openatom.cn/static/img/logo1.8f19f0a8.png")
  .size({width:400,height:100})
  .backgroundColor(Color.Blue)
  .objectFit(ImageFit.Contain)
```

Image组件

- Image属性

➢ renderMode: 设置图片的渲染模式。

ImageRenderMode.Original为默认值，ImageRenderMode.Template为黑白效果

➢ alt: 加载时显示的占位图，支持本地图片。

➢ matchTextDirection: 设置图片是否跟随系统语言方向，在RTL语言环境下显示镜像翻转效果，默认值为 false。

➢ fillOriginalSize: 图片组件尺寸未设置时，其显示尺寸是否跟随图源尺寸，默认值为false 。

➢ fillColor: 填充颜色。设置的填充颜色会覆盖在图片上，仅对svg图源生效，设置后会替换 svg 图片的fill颜色。

Image组件

- Image属性

➢ objectFit: 设置图片的缩放类型。当Image 组件大小和图片大小不同时指定图片的放缩类型。默认值为 ImageFit.Cover。具体描述见表 。

名称	描述
Cover	超出部分会裁去。保持宽高比进行缩小或者放大，使得图片两边都大于或等于显示边界
Contain	等比例缩放。保持宽高比进行缩小或者放大，使得图片完全显示在显示边界内
Fill	不保持图片宽高比显示，使图片完全充满显示边界
None	保持图片原有尺寸显示，通常配合 objectRepeat 属性一起使用
ScaleDown	保持宽高比显示，图片缩小或者保持不变
Auto	自适应显示

➢ objectRepeat: 当图片小于组件大小时，设置图片的重复样式。默认值为ImageRepeat.NoRepeat.

➢ autoResize: 是否需要在图片解码过程中对图源做resize操作。该操作会根据显示区域的尺寸决定用于绘制的图源尺寸，有利于减少内存占用。默认值为

Image组件

- Image属性

➢ interpolation: 设置图片的插值效果，即减轻低清晰度图片在放大显示的时候出现的锯齿问题，仅针对图片放大插值。默认值为ImageInterpolation.None。

➢ sourceSize: 指定图片解码大小，让原始图片被解码为指定大小的图片。数字类型的单位是 px。

➢ syncload: 设置是否同步加载图片，默认是异步加载。同步加载时阻塞UI线程，不会显示占位图。

➢ overlay: 在当前组件上，增加遮罩文本。

Image组件

- Image属性

例如，指定图片解码大小。

```
@Entry @Component struct Test {
 build() {
  Column(){
  Image(“image/test.jpg”)
      .width(100)
      .height(100)
      .sourceSize({
        width:10,
        height:10,
      })
  }}}
```

Image组件

- Image事件

➢ onComplete: 图片成功加载时触发该回调，返回成功加载的图片尺寸。

- width: 图片的宽，单位为像素。
- height: 图片的高，单位为像素。
- componentWidth: 组件的宽，单位为像素。
- componentHeight: 组件的高，单位为像素。
- loadingStatus: 图片加载成功的状态。
- onError: 图片加载出现异常时触发该回调。
- onFinish: 当加载的源文件为带动效的svg 图片时，当svg 动效播放完成时会触发这个回调；若为无限循环动效，则不会触发这个回调。

Image组件

- Image事件

例如打印返回图片的宽

```
@Entry @Component struct Test {
 build() {
   Column(){
   Image("image/test.jpg")
       .width(100)
       .height(100)
       .onComplete((msg) =>{console.log(msg.width)})

   }}}
```

```
I  app Log: 1440
```

开关组件Toggle

Toggle组件提供勾选框样式、状态按钮样式及开关样式；开关的样式用枚举ToggleType来描述；开关状态切换时触发onChange()事件，回调的isOn为true时，代表状态从关切换为开，isOn为false时，代表状态从开切换为关。

```
Toggle({ type: ToggleType.Switch,//开关样式
 isOn: false })              //开关的状态
//当开关样式为Button时，才可以使用子组件
//{Text('Toggle开关').fontColor('#ff5990cb').fontSize(12)}
 .selectedColor('#007DFF') //设置组件打开状态的背景颜色
  //设置Switch类型的圆形滑块颜色,仅对type为ToggleType.Switch生效
 .switchPointColor('#ffefb1b1')
 .onChange((isOn: boolean) => { //开关事件
  console.info('开关状态:' + isOn)
 })
 .width("30%").height(30)
```

开关类型

ToggleType

- ToggleType.Checkbox：提供单选框样式；
- ToggleType.Switch：提供开关样式
- ToggleType.Button：提供状态按钮样式。

代码演示

消息提示模块

@ohos.promptAction (弹窗)是ArkTS提供的API，用于创建并显示文本提示框，一般称为消息提示模块，使用时先导入模块，再使用showToast()创建并显示文本提示框。

```
import promptAction from '@ohos.promptAction'; //导入模块
promptAction.showToast({
 message:"提示的消息内容",  //提示内容
 duration:1000, //消息显示时长为1秒
 bottom:50     //消息提示框距离底部50
})
```

代码演示

任务 使用交互类组件开发登录页

完成任务达成的技能目标

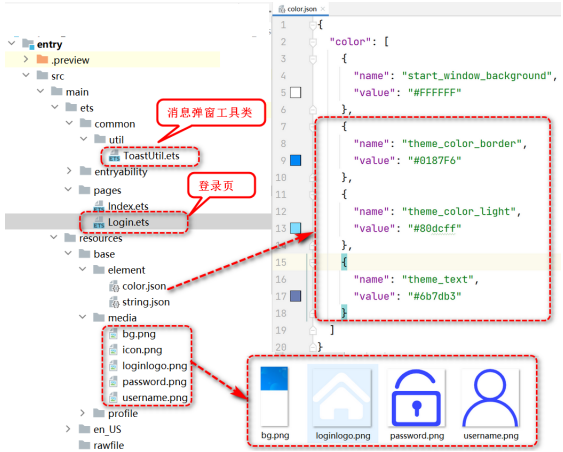
01 任务描述

本任务讲解登录页的设计和开发过程，使用输入组件、按钮组件、开关组件和提示框模块，完成登录页的开发。

02 技能目标

1. 能正确设置输入组件的属性
2. 能使用合适的按钮组件并处理按钮的事件
3. 能使用开关组件并处理开关的事件
4. 能正确导入消息提示模块并完成消息提示
5. 能完成登录页面的开发

任务实施步骤



1 整理工程资源

2 设计登录页布局

3 实现最外层的背景区

4 实现登录功能区的布局设计

5 实现用户名/密码的输入区设计

6 实现记住密码和登录按钮区的设计

7 封装消息弹窗工具类

8 处理点击事件

STEPS

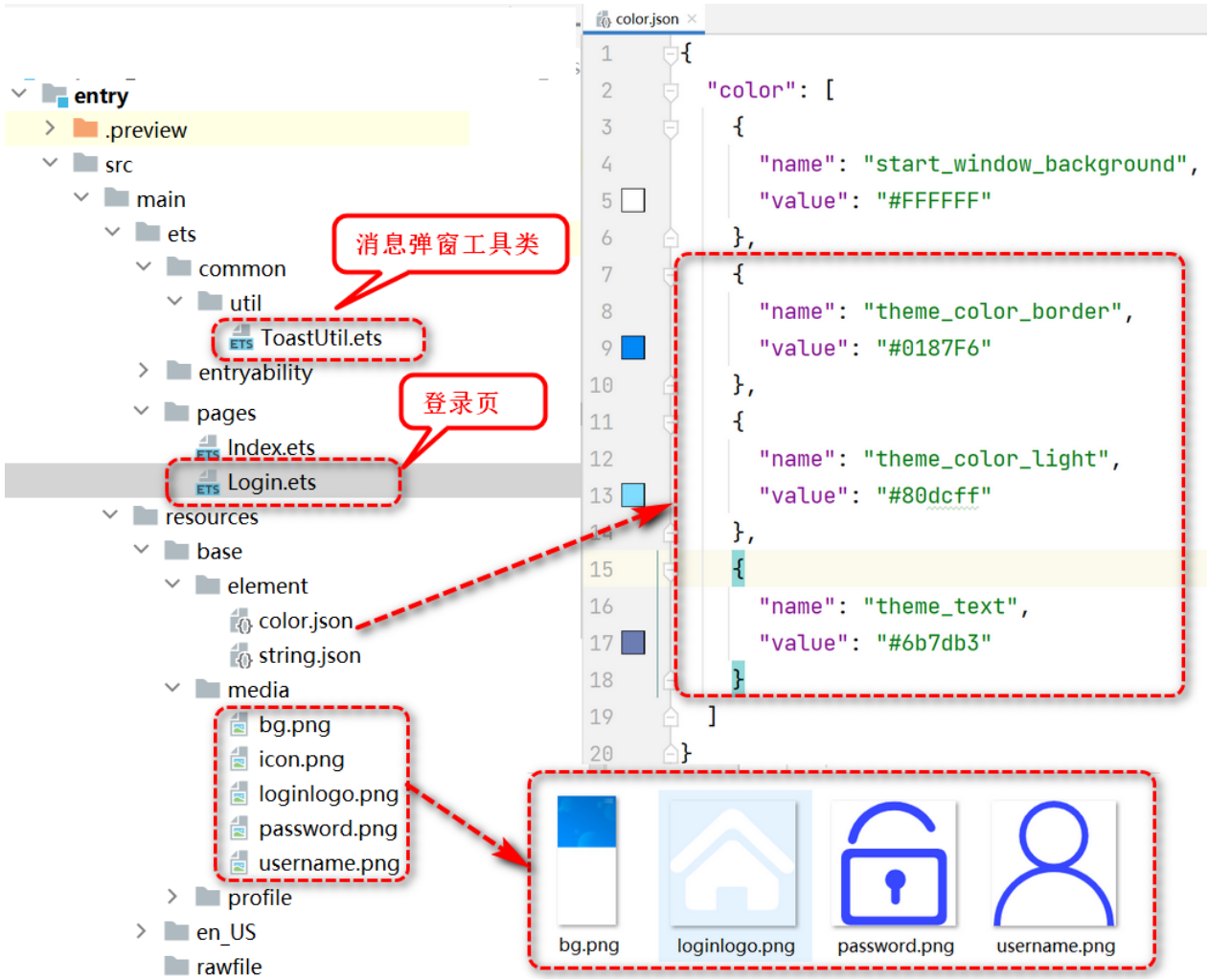
河南大學
Henan University

任务实施

01 整理工程资源

在pages目录下新建Login.ets页面；将任务需要的图片资源拷贝到media目录下；在color.json中添加任务需要的颜色资源；在ets目录下创建common/util目录，并在该目录下创建ToastUtil.ets文件。

代码演示



河南大學 Henan University

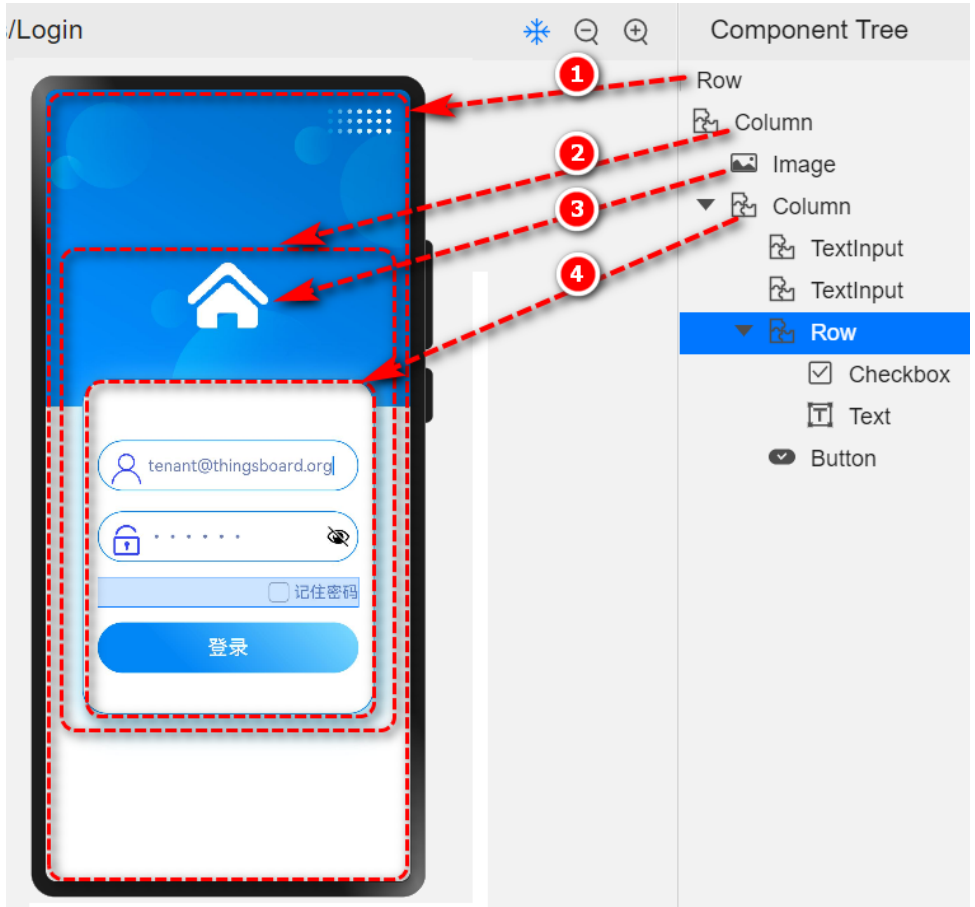
任务实施

02

设计登录页布局

组件树

登录页的最外层是用Row布局，设置了一个背景图片；在Row内部嵌套了一个Column布局，用于处理登录功能区；登录功能区有Logo、用户名和密码输入区、记住密码区和登录按钮区。



河南大學 Henan University

任务实施

03

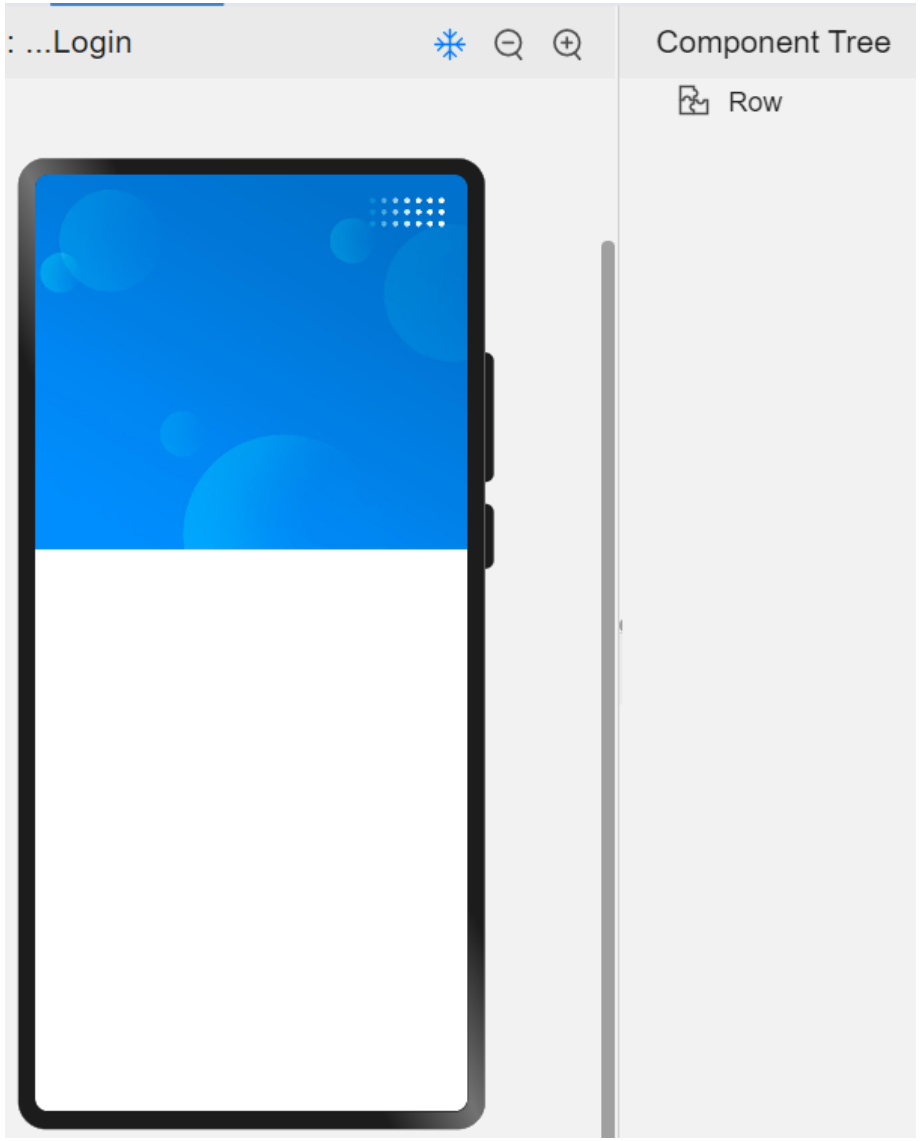
实现最外层的背景区

实现最外层的背景区

组件树

在Login.ets中编写代码，实现登录页面的背景设置。登录页面最外层用一个Row布局，设置背景图片和宽高属性。

代码演示



河南大學 Henan University

任务实施

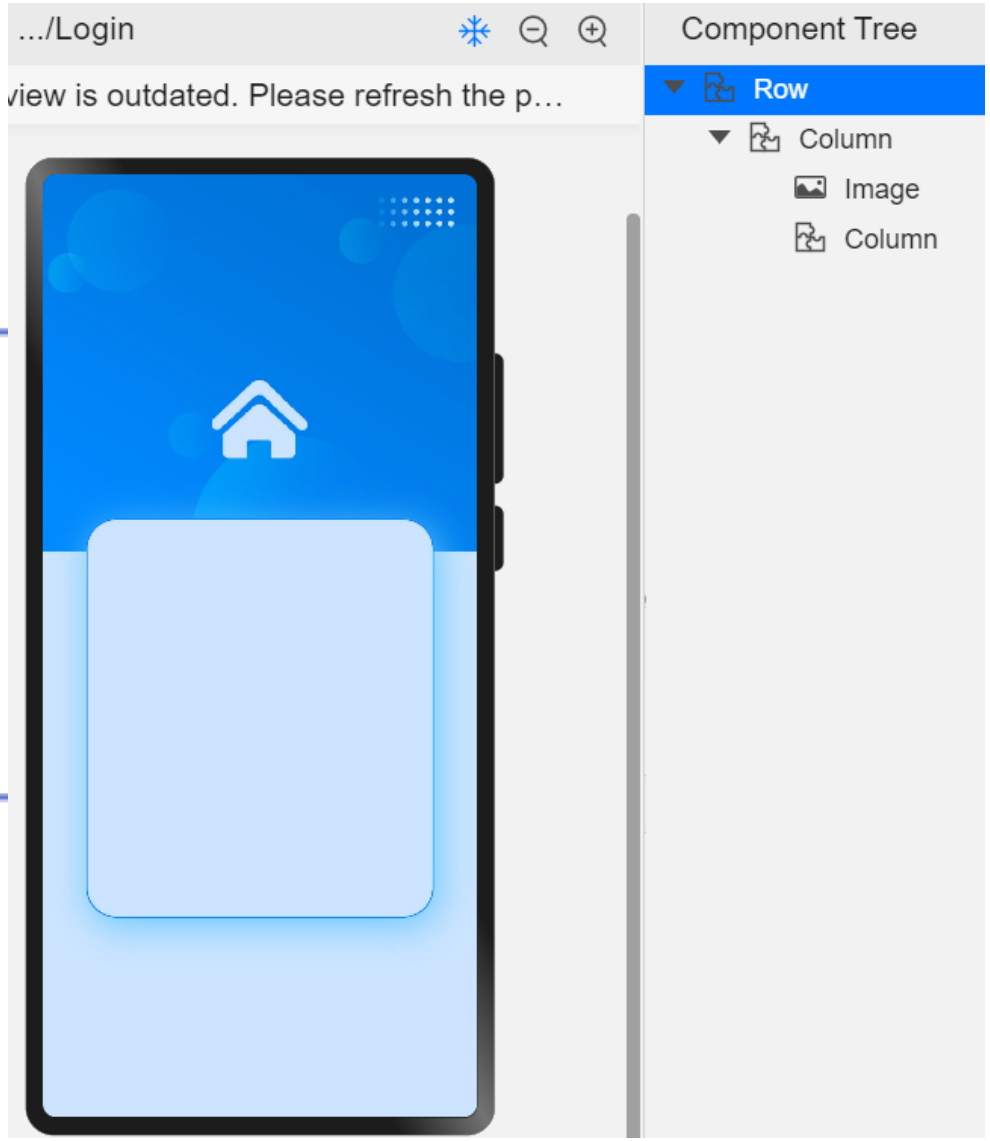
04

实现登录功能区的布局设计

实现登录功能区的布局设计

[redacted]部是一个Logo，用户
[redacted]名和密码以及按钮信息放在一个Column布局中。

代码演示



河南大學 Henan University

任务实施

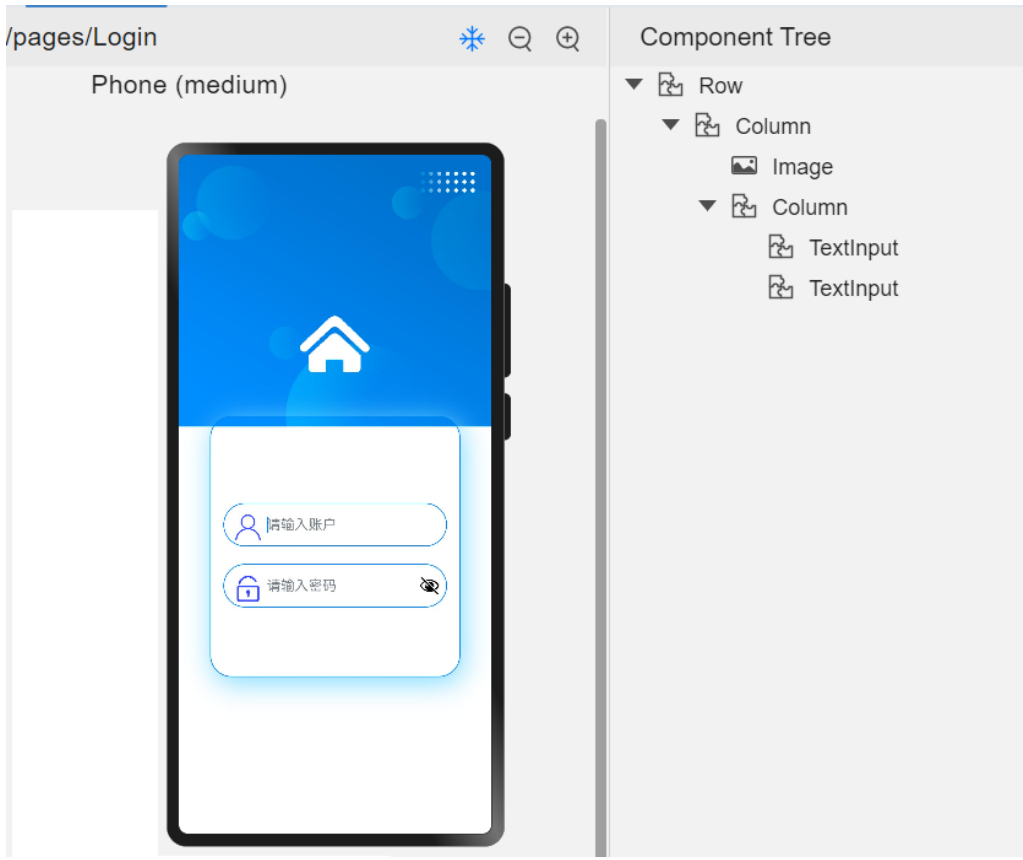
05

实现用户名/密码的输入区设计

实现用户名/密码的输入区设计

组件树

用户名和密码输入都使用TextInput组件，用户名的输入类型为Normal，密码的输入类型为Password。



代码演示

河南大學
Henan University

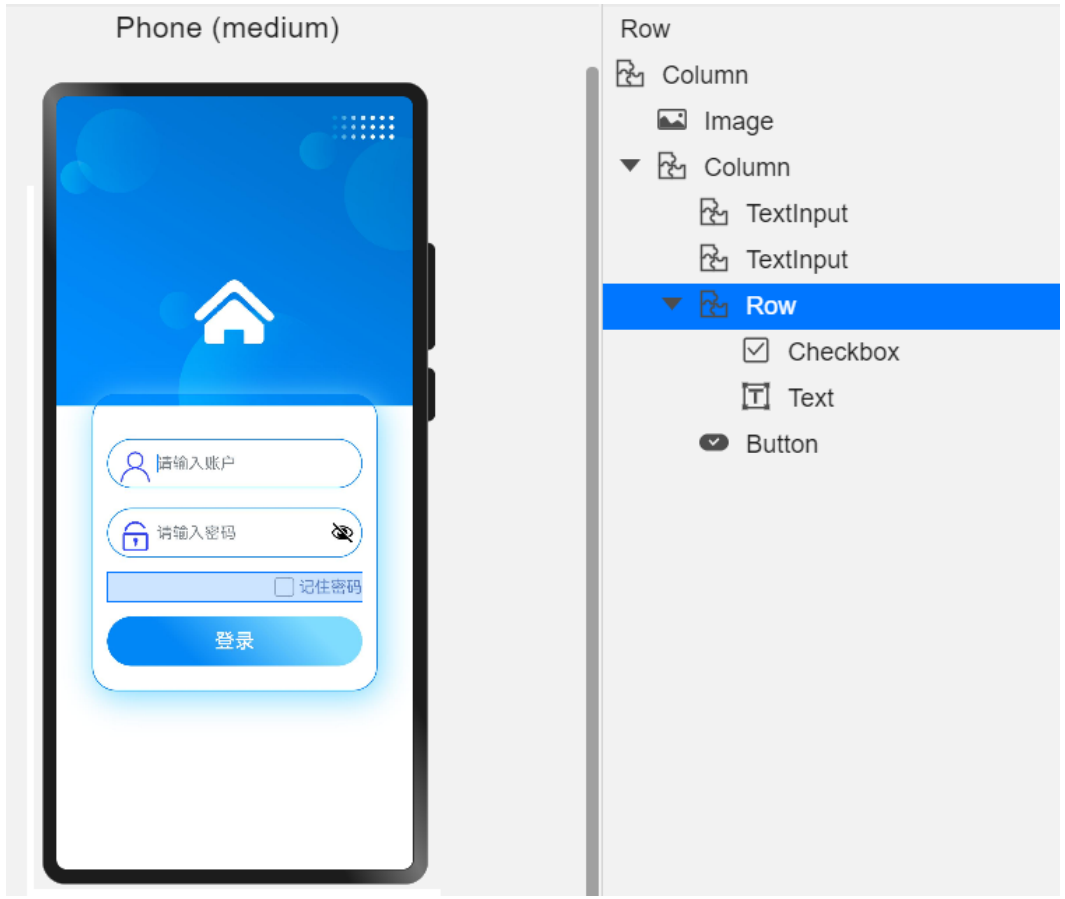
任务实施

06

实现记住密码和登录按钮区的设计

组件树

记住密码区用Row组件包裹住CheckBox组件和Text组件，在水平方向上居末端对齐；登录按钮设置了渐变色属性。



代码演示

07

封装消息弹窗工具类

封装消息弹窗工具类

ToastUtil

在ArkTS的API中，有提供了文本的消息弹窗'@ohos.promptAction，使用时需要用import进行模块的导入，再利用提供的showToast()方法进行消息弹窗提示，因此可以定义一个工具类ToastUtil，内部定义一个方法show()，参数是要显示的消息内容。

代码演示

河南大學
Henan University

任务实施

08

处理点击事件

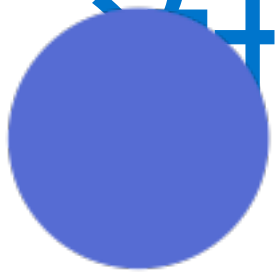
当想获取输入的用户名和密码，想知道记住密码的多选框的状态，都可以通过给组件添加相关的事件来获取；获取到相关信息后，可以使用消息弹窗进行提示。

在Login.exs文件的第一行导入消息弹窗工具类，给用户名和密码输入组件、记住密码组件、按钮组件添加事件。

代码演示

任务小结

本任务主要讲解输入组件、按钮组件、开关组件和消息提示函数的使用，并综合使用布局和组件，开发了登录页面，处理了按钮的事件。通过本任务，读者可以完整的了解App的登录页面的开发过程。

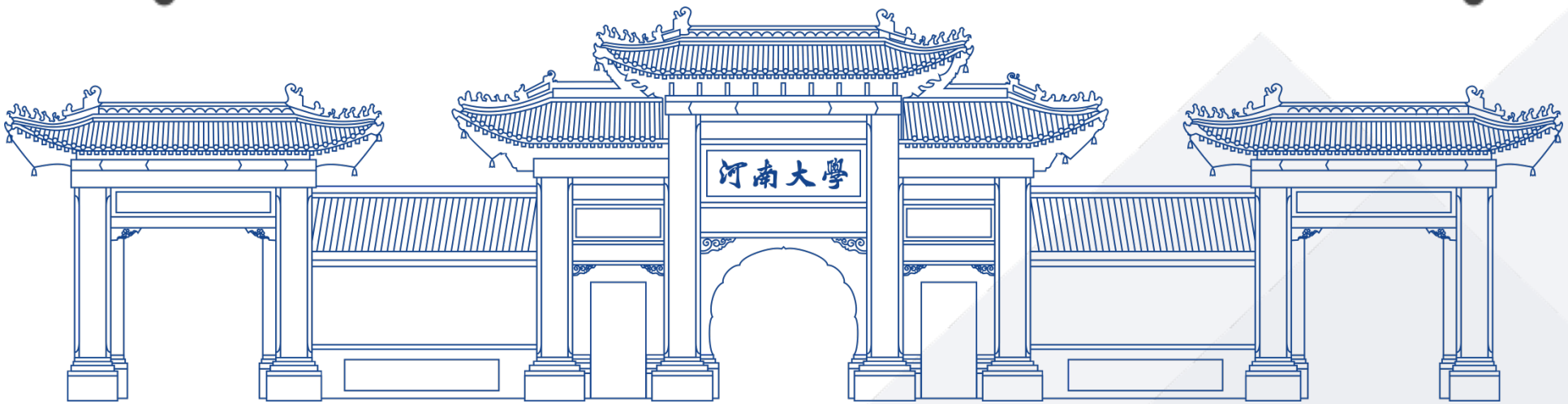


谢聆听

河南大學
Henan University

特殊组件

软件学院



特殊组件

- 主要介绍现代移动应用中不可或缺的的基本要素如

  - 执行进度、加载进度、常用选择器、九宫格密码锁、二维码、弹窗等。

- 介绍通过这些基础组件实现和用户交互的场景，如

  - 购物应用、社交应用、游戏应用、金融应用等。

特殊组件：进度条

- Progress定义参数。

Progress(options: ProgressOptions)

options：参数内容。

```
@Entry
@Component
struct Index {
 build() {
   Column() {
Progress({
      value: this.value, // 设置当前进度
      total: 100, // 设置进度总量
    })
}.width('100%').height('100%')  }}
```

特殊组件：进度条

- Progress定义参数。

  ➢ value:表示当前进度。

  ➢total:表示进度条总长，默认值为100。

  ➢type:设置进度条的样式，默认值为ProgressType.Linear。

名称	描述
Linear	线性样式。高度大于宽度的时候自适应垂直显示
Ring	环形无刻度样式，环形圆环逐渐显示至完全填充效果
Eclipse	日食进度条。类似月圆月缺的进度展示效果，从月牙逐渐变化至满月
ScaleRing	环形有刻度样式，显示类似时钟刻度形式的进度展示效果。刻度外圈出现重叠的时候自动转换为环形无刻度进度条
Capsule	胶囊样式，头尾两端圆弧处的进度展示效果与 Eclipse 相同；中段处的进度展示效果与 Linear 相同。高度大于宽度的时候自适应垂直显示
it	定义一条测试用例，支持三个参数：用例名称，过滤参数和用例函数
expect	支持 bool 类型判断等多种断言方法

知识储备

01

文本选择器TextPicker

文本选择器TextPicker

文本选择器TextPicker是滑动选择文本内容的组件。

```
private flowers: string[] = ['玫瑰','郁金香','百合','康乃馨']
```

TextPicker文本选择器

示例

```
private flowers: string[] = ['玫瑰','郁金香','百合','康乃馨']
TextPicker({
 range: this.flowers, //选择器的数据选择列表
 selected: 0,  //设置默认选中项在数组中的索引值,默认值：0
 value:'郁金香'//设置默认选中项的值，优先级低于selected,默认值：第一个元素值
})
 .width(300).height(200).backgroundColor('#ffe7ceee')
 .defaultPickerItemHeight(30)//设置TextPicker各选择项的高度
 .onChange( //滑动选中TextPicker文本内容后，触发该回调
   (value: string | string[], //当前选中项的文本
    index: number | number[] //当前选中项的索引值
   ) => {
   console.info('您选中的文本是, value: ' + value + ', index: ' + index)  })
```

知识储备

02

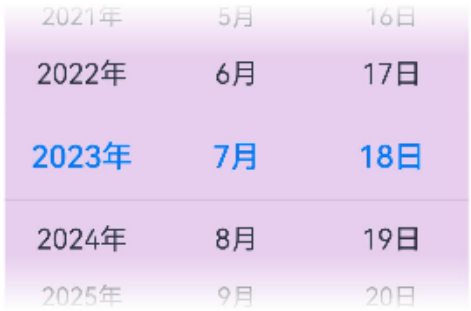
日期选择器DatePicker

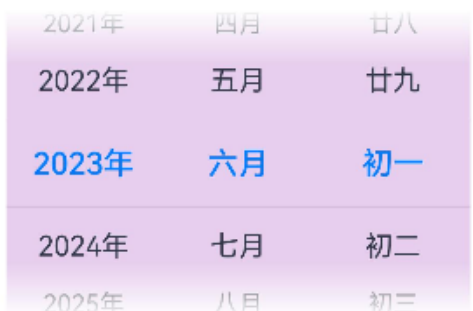
日期选择器DatePicker

日期选择器DatePicker是选择年月日的滑动选择器组件，默认以 1970-1-1 至 2100-12-31 的日期区间创建滑动选择器。

不显示农历

```
.lunar(false)
```





显示农历

```
.lunar(true)
```

日期选择器DatePicker

示例

```
DatePicker({
 start:new Date("2020-1-1"),
 end:new Date("2040-1-1"),
 selected:new Date()
})
 .lunar(false)
 .width(300).height(200).backgroundColor('#ffe7ceee')
 .onDateChange((date:Date)=>{
   console.log("您选择的是:"+date.getFullYear()+"年 "+(date.getMonth()+1)+"月
"+date.getDate()+"日")
 })
```

代码演示

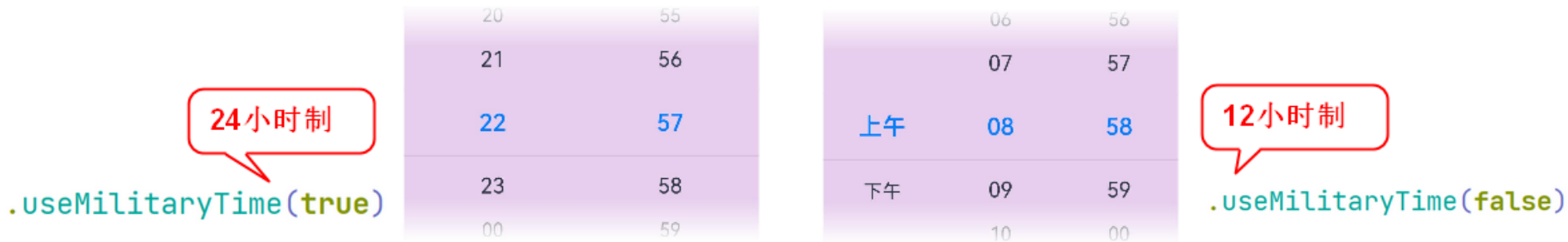
知识储备

03

时间选择器TimePicker

时间选择器TimePicker

时间选择器TimePicker是选择时和分的滑动选择器组件。



时间选择器TimePicker

示例

```
TimePicker({
 selected: new Date() //设置选择器的默认选中时间，设置默认当前时间
})
 .width(300) .height(200).backgroundColor('#ffe7ceee')
 .useMilitaryTime(true)   //展示时间是否为 24小时制，默认值为false。
  .onChange((time:TimePickerResult) => {    //选择器的事件回调，当时间变化时会触发该回调
      console.log(`您选择的是:${time.hour} : ${time.minute}`)
 })
```

代码演示

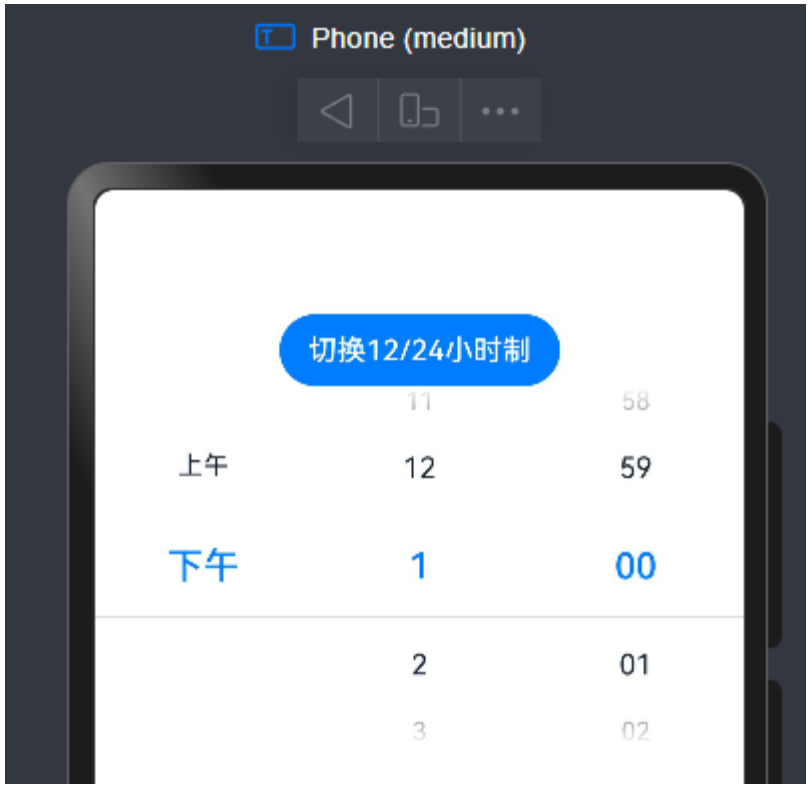
特殊组件：选择器组件

- TimePicker时间选择器：

  useMilitaryTime：展示实践是否为24小时制，默认false，即时间区间为12小时

  onChange：选择器的事件回调，当时间变化会触发该回调

```
@Entry
@Component
struct Index {
 build() {
   Column() {TimePicker({selected:this.selectedTime})
      .useMilitaryTime(this.isMilitaryTime)
      .onChange((value:TimePickerResult) => {
        this.selectedTime.setHours(value.hour, value.minute)
        console.info('select current date is: ' + JSON.stringify(value))
      })
}
```

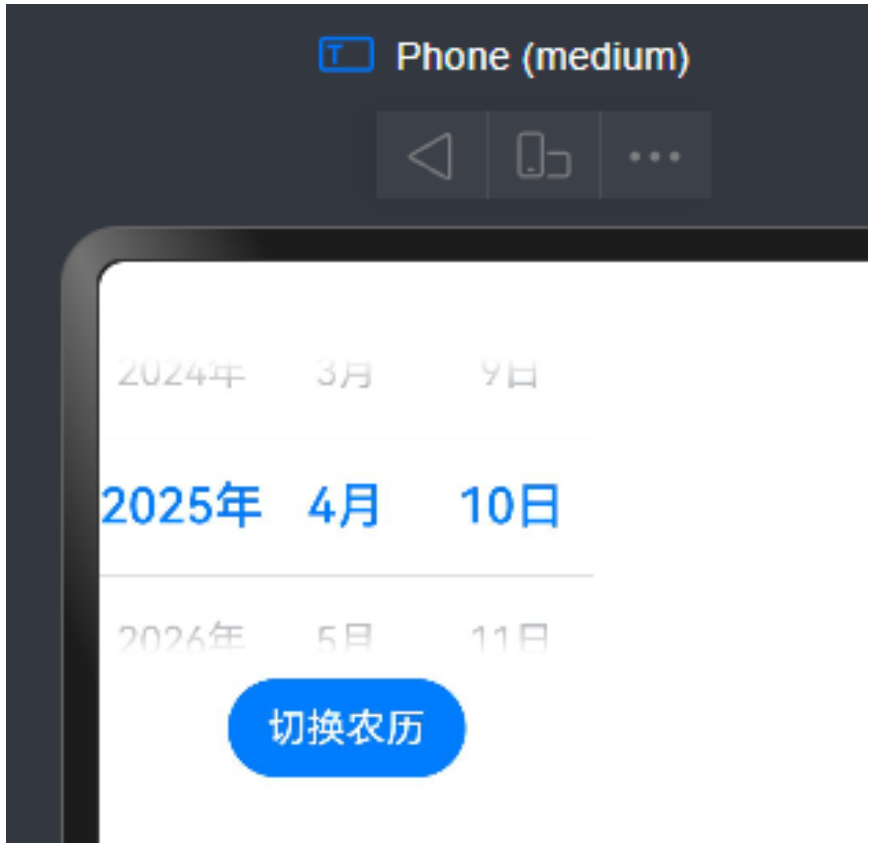


特殊组件：选择器组件

- DatePicker构造函数。
  - ➢start:设置选择器的开始时间。
  - ➢end:设置选择器的结束时间。
  - ➢selected:设置选中项的日期，默认当前系统日期。
- DatePicker属性。
  - ➢lunar:日期是否以农历的形式显式，默认值为false。
- DatePicker事件。
  - ➢onChange:选择器的事件回调，当日期变化时会触发该回调。

特殊组件：选择器组件

```
@Entry
@Component
struct Index {
 build() {
   Column() {
DatePicker({
       start:new Date('2000-1-1'),
       end: new Date('2030-1-1'),
       selected: new Date()
     })
       .lunar(this.flag)
       .width(200)
       .height(120)

     Button(this.flag?"切换公历":"切换农历")
       .onClick(() =>{
         this.flag = !this.flag
       })
}
```



特殊组件：二维码组件

1.QRCode构造参数

value:设置将要生成二维码的内容。

2.QRCode属性

color:设置二维码颜色，默认为黑色。

backgroundColor:设置二维码背景色。

特殊组件：二维码组件

```
Stack() {
 QRCode(this.info)//二维码内容字符串
       .color(this.color)//设置二维码颜色
       .width(this.qrsize).height(this.qrsize)
 Image($r("app.media.startIcon"))//在二维码上叠加一个头像
       .width(this.qrsize * 0.2).height(this.qrsize * 0.2).border({
width: 2, color: Color.White }).borderRadius(5)
}.padding(10).backgroundColor(Color.White)
.borderRadius(5)
```

特殊组件：DataPanel组件

1. 构造参数

values:数据值列表，最多包含9个数据，大于9个数据则取前9个数据。若数据值小于0则置为0。

max:数据的最大值。max等于value数组各项的和，默认值为100。

type:数据面板的类型(不支持动态修改),默认值DataPanelType.Cirde。

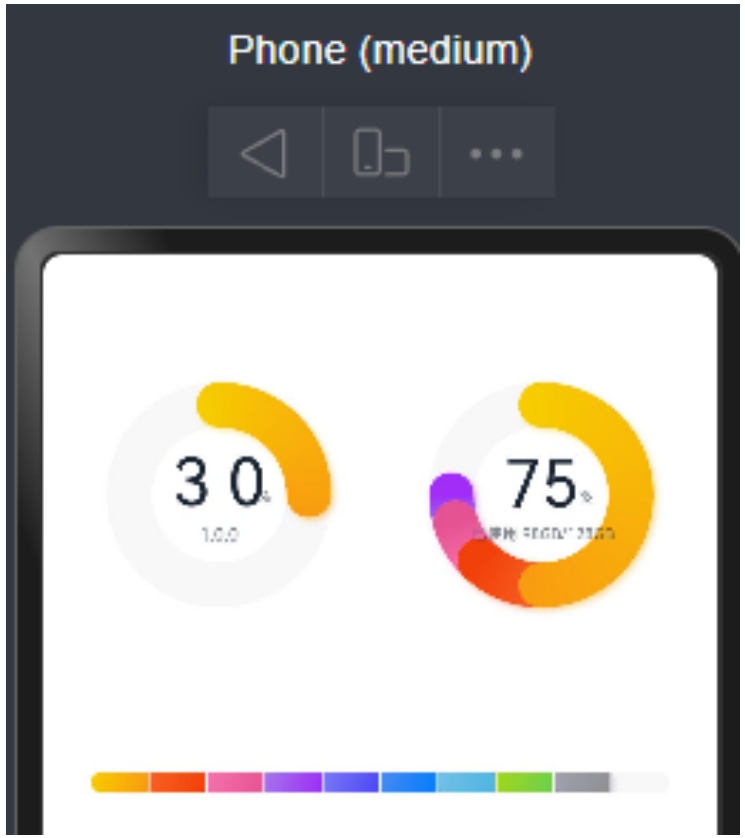
名称	描述
Line	线型数据面板。
Circle	环形数据面板。

特殊组件：DataPanel组件

2. 属性

closeEffect:禁止数据面板动画效果，默认true

```
Stack() {
 DataPanel({ values: [25], max: 100, type: DataPanelType.Circl
 }).width(168).height(168).closeEffect(true)
 Column() {
   Text( " 3 0 " ).fontSize(35).fontColor( " #182431 " )

 Text('1.0.0').fontSize(9.33).lineHeight(12.83).fontWeight(500).opacity(0.6)
 }
 Text("%")
   .fontSize(9.33)
   .lineHeight(12.83)
   .fontWeight(500)
   .opacity(0.6)
   .position({ x: 104.42, y: 78.17 })
}.margin({ right: 4 })
```



特殊组件：弹窗组件

警告弹窗

1. AlertDiaog构造函数

show:定义并显示AlertDialog组件。该方法被static修饰使用时不需要new对象

2.单按钮AlertDialogParamWithConfirm对象

title:弹窗标题。

message:弹框内容。

autoCancel:点击遮障层时，是否关闭弹窗。默认值为true。

cancel:点击遮障层关闭dialog时的回调。

alignment:弹窗在垂直方向上的对齐方式。

offset:弹窗相对alignment所在位置的偏移量。

gridCount:弹窗容器宽度所占用的栅格数。当gridCount小于等于0时，弹窗宽度是固定的；大于0时，按照设置的数值显示宽度。gridCount最大值为4,若值为小数，则向下取整。

confirm:确认按钮的文本内容、文本色、按钮背景色和点击回调。

警告弹窗

ArkTS API 提供了全局的AlertDialog，通过show（）方法定义并显示警告弹窗组件，可设置文本内容与响应回调。

示例

```
AlertDialog.show({                    //弹出一个对话框
  title: "温度告警",                  //设置标题
  message: "温度超过38°C",            //设置内容
  alignment: DialogAlignment.Center, //设置弹窗的对齐方式
  confirm: {                          //单个按钮的配置文本
      value: "确定",
   fontColor: Color.Red,
   backgroundColor: "#aabbcc",
   action: () => {
     console.log("点击了")
    }
  }
})
```

代码演示

特殊组件：弹窗组件

警告弹窗

```
Button(" 单按钮弹框")
  .onClick(() => {
    AlertDialog.show({
      title: " 系统提示",
      message: " 您确定要删除这条数据吗？ ",
      cancel: () => {
        console.log(" 用户点击了遮罩层")
      },
      confirm: { // 设置弹窗按钮
        value: " 确定",
        fontColor:Color.Red,
        backgroundColor:Color.Orange,
        action: () => {
          console.log(" 用户点击了确定按钮")
        }
      }
    })
  })
```

列表选择弹窗

ActionSheet构造参数如下：

title:弹窗标题。

message:弹框内容。

confirm:确认按钮的文本内容、文本色、按钮背景色和点击回调。

cancel:点击遮障层关闭dialog时的回调。

autoCancel:点击遮障层时，是否关闭弹窗。默认值true。

alignment:弹窗在垂直方向上的对齐方式。

offset:弹窗相对alignment所在位置的偏移量。

sheets:设置选项内容，每个选择项支持设置图片、文本和选中的回调。

特殊组件：弹窗组件

警告弹窗

```
Button(" 列表弹框 ")
  .onClick(()=>{
    ActionSheet.show({
      title:" 班级信息 ",
      message:" 请选择 ",
      confirm:{
        value:" 取消 ",
        action:()=>{
          console.log(" 用户点击了按钮 ")
        }
      },
      sheets:[
        {
          title:" 一班 ", // 项文字
          icon:$r("app.media.app_icon"), // 项图标
          action:()=>{
            console.log(" 一班被选中 ")
          }
        },
        {
          title:" 二班 ",
          icon:$r("app.media.app_icon"),
          action:()=>{
            console.log(" 二班被选中 ")
          }
        }],
      cancel:()=>{
        console.log(" 点击了遮罩层 ")
      },
      // alignment:DialogAlignment.Bottom
    })
  })
```

特殊组件：弹窗组件

日期选择弹窗

DatePickerDialog构造参数如下

start:设置选择器的起始日期，默认值为Date(“1970–1–1”)。

end:设置选择器的结束日期，默认值为Date(“2100–12–31”)。

selected:设置当前选中的日期，默认值为系统当前时间。

lunar:日期是否显示为农历，默认值为false。

onAccept:点击弹窗中的“确定”按钮时触发该回调。

onCancel:点击弹窗中的“取消”按钮时触发该回调。

onChange:滑动弹窗中的滑动选择器使当前选中项改变时触发该回调。

特殊组件：弹窗组件

警告弹窗

```
            DatePickerDialog.show({
              start: new Date("2020-1-1"),  // 开始时间
              end: new Date("2024-1-1"),// 结束时间
              selected:new Date(),// 系统当前时间
              onAccept: value => { // 确定按钮回调
                      console.log(` 年 :${value.year} 月 :${value.month + 1}
日 :${value.day}`)
              },
              onCancel: () => { // 取消按钮回调
                console.log("取消选择")
              },
              onChange: value => { // 选项改变回调
                      console.log(` 年 :${value.year} 月 :${value.month + 1}
日 :${value.day}`)
```

自定义弹窗

CustomDialogController 构造参数：

CustomDialogController 的创建需要 CustomDialogControllerOptions 对象：

builder：自定义弹窗内容构造器。

cancel()：点击遮障层退出时的回调。

autoCancel：是否允许点击遮障层退出，默认值为 true。

alignment：弹窗在竖直方向上的对齐方式。

offset：弹窗相对 alignment 所在位置的偏移量。

customStyle：弹窗容器样式是否自定义。

gridCount：弹窗宽度占栅格宽度的个数。

open()：显示自定义弹窗。

close()：关闭显示的自定义弹窗

自定义弹窗

CustomArkDialog 的布局很简单，仿照的是 iOS 的低电量弹窗。需要注意的是，用 @CustomDialog 修饰符创建的组件，内部必须要定义一个 CustomDialogController 类型的属性，该属性的初始化由 ArkUI 框架自动完成，不需要开发者手动进行初始化操作。这样做的目的是方便开发者控制弹窗的显示和关闭，另外对属性的访问权限不做要求，如果不定义该属性，在项目编译的时候会出现如下异常：

```
[Compile Result]  @CustomDialog component should have a property of the CustomDialogController type.
```

特殊组件：弹窗组件

定义完CustomArkDialog后，使用CustomDialogController来创建自定义弹框。示例代码如下所示：

```
@Entry @Component struct Index {
  // 创建一个 controller
  controller: CustomDialogController = new CustomDialogController({
    builder: CustomBatteryDialog(), // 弹框的构造器
    cancel: () => {
      console.log("cancel")          // 点击蒙层的回调
    },
    autoCancel: true,                // 允许点击蒙层关闭弹窗
    customStyle: true                // 使用自定义样式
  });
  build() {
    Column({space: 10}) {
      Button("按钮")
        .size({width: 160, height: 50})
        .onClick(function() {
          this.controller.open()     // 打开弹窗
        }.bind(this))
    }
    .padding(10)
    .size({width: "100%", height: "100%"})
  }
}
```

滚动类组件

主要介绍滑动条组件、评分条组件、滚动组件、下拉列表、轮播组件和下拉刷新，这些组件可以让页面内容更加丰富。

- 滑动条组件是常用的交互组件，可以让用户调整音量、亮度、选择年龄、价格等。
- 评分条组件是常见的评价和打分工具，用户可以通过点击星星或其他图标进行评分。
- 滚动组件可以让页面内容在有限的空间内进行滚动展示，节省页面空间，如展示长文本、图片、表格等内容。
- 下拉列表是常见的选择工具，用户可以从多个选项中选择一个或多个选项，如选择城市、日期、颜色等。
- 轮播组件可以让多张图片或内容在同一位置进行轮流展示，如展示广告、新闻、产品等内容。

这些组件的设计和使用需要考虑到各种因素，如评分方式、滚动方式、

Slider构造参数

options:Slider接收一个SliderOptions类型的可选参数，SliderOptions参数说明如下：

value:滑动条当前进度值，默认值为0。

min:设置滑动条设置最小值，默认值为0。

max:设置滑动条设置最大值，默认为为100。

step:设置Slider滑动步长，默认值为1。

style:设置滑动条的滑块样式，默认值为yle.OutSet,具体描述见表。

direction:设置滑动条滑动方向为水平或垂直方向，默认值为Axis.Horizontal。

reverse:设置滑动条取值范围是否反向，横向Slider默认为从左往右滑动，竖向Slider默认为从上往下滑动，默认值为false。

滚动类组件

名称	描述
Linear	线性样式。高度大于宽度的时候自适应垂直显示
Ring	环形无刻度样式，环形圆环逐渐显示至完全填充效果

名称	描述
vertical	方向为垂直。
Horizontal	方向为水平。

滚动类组件

Slider属性

blockColor:设置滑块的颜色。
trackColor:设置滑轨的背景颜色。
selectedColor:设置滑轨的已滑动部分颜色。
minLabel:设置滑动条最小值处的标签。
maxLabel:设置滑动条最大值处的标签。
showSteps:设置当前是否显示步长刻度值，默认值为false。

show Tips:设置滑动时是否显示百分比气泡提示，默认值为false。
trackThickness:设置滑动条粗细。

滚动类组件

Slider事件

onChange:滑动条滑动时触发事件回调。

value:表示当前进度值回值有小数，可使用Math.toFixed()方法将数据处理为预期的精度。

mode:表示滑动条的拖动状态。

名称	值	描述
Begin	0	用户开始拖动滑块
Moving	1	用户拖动滑块中
End	2	用户结束拖动滑块
Click	3	用户点击滑动条使滑块位置移动

1.Rating构造参数

rating:表示设置当前评分值，默认值为0。

indicator:表示是否可以操作，设置为true表示评分条是一个指示条，不可操作，false表示可以操作的评分条。

2.Rating属性介绍

stars:设置星星的总数，默认值为5。

stepSize:设置操作评级的步长，默认值为0.5。

starStyle:设置星星的样式，各参数说明如下：

backgroundUri:自定义未选中时星级的图片路径，仅支持本地图片。

foregroundUri:自定义选中时星级的图片路径，仅支持本地图片。

secondaryUri:自定义部分选中时星级的图片路径，仅支持本地图片。

3.Rating事件介绍

Rating组件

```
@Entry @Component struct RatingTest {
  @State rating: number = 0
  build() {
    Column({ space: 10 }) {
      Rating({
        rating: this.rating,
        indicator: false
      })
        .width(220).height(40)
        .stars(10)
        .stepSize(0.5)
        .onChange((value) => {
          this.rating = value;
        })
      Text(` 总分数 :${this.rating}`)
        .fontSize(22).width(220)
    }
    .width("100%").height("100%").padding(10)
  }
}
```

scroll组件

1.scroll构造参数

scroller:给Scroll绑定一个滚动控制器，该控制器可以控制子组件的各种滚动能力，Scroller目前支持绑定到List、Scroll、ScrollBar上。

2.Scroll属性

scrollable:设置Scroll的滚动方向，默认值为ScrollDirection.Vertical。

scrollBar:设置滚动条状态，默认值：BarState.Auto。

scrollBarColor:设置滚动条的颜色。

scrollBarWidth:设置滚动条的宽度。

edgeEffect:设置滑动到边缘后效果。默认值为EdgeEffect.None。

scroll组件

ScrollDirection 枚举说明

名称	描述
Horizontal	水平方向滚动
Vertical	垂直方向滚动
None	不可滚动

BarState 枚举说明

名称	描述
Off	不显示滚动条
On	一直显示滚动条
Auto	按需显示（触摸时显示，2s 后消失）

EdgeEffect 枚举说明

名称	描述
Spring	弹性物理动效，滑动到边缘后可以根据初始速度或通过触摸事件继续滑动一段距离，松手后回弹（弹簧效果）
Fade	阴影效果，滑动到边缘后会有圆弧状的阴影
None	滑动到边缘后无效果

3.Scroll事件

onScroll:滚动事件回调，返回滚动时水平、竖直方向的偏移量。

onScrollEdge:滚动到边缘事件回调。

onScrollEnd:滚动停止事件回调。

onScrollBegin:滚动开始事件回调。事件参数如下：

dx:即将发生的水平方向滚动量。

dy:即将发生的竖直方向滚动量。

dxRemain:水平方向滚动剩余量。

dyRemain:垂直方向滚动剩余量。

4.Scroller控制器

Scroller作为滚动容器组件的控制器，它提供了滚动子组件的各种能力，比如设置子组件滚动指定位置、滚动到底部以及滚动到下一页、上一页等能力。目前Scroller只支持绑定到List、Scroll、ScrollBar上。

scrollTo:设置子组件滚动到指定位置，在滚动的时候还可以根据animation参数设置动画。

scrollEdge:滚动到边缘，Edge提供了多种类型。

scrollPage:滚动到上一页或者下一页。

currentOffset:获取当前滚动的偏移量。

scrollToIndex:滚动到指定下标，仅支持Grid、list组件。

scroll组件

```
@Entry @Component struct ScrollExample {
  scroller: Scroller = new Scroller()
  private arr: number[] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
  build() {
    Stack({ alignContent: Alignment.TopStart }) {
      Scroll(this.scroller) {
        Column() {
          ForEach(this.arr, (item) => {
            Text(item.toString())
              .width("90%")
              .height(150)
              .backgroundColor(0xFFFFFF)
              .borderRadius(15)
              .fontSize(16)
              .textAlign(TextAlign.Center)
              .margin({ top: 10 })
          }, item => item)
```

List下拉列表

1.List构造参数

initiallndex:设置当前List初次加载时视口起始位置显示的item的索引值。如果设置的值超过了当前List最后一个item的索引值，则设置不生效，默认值为0。

space:列表项间距，默认值为0。

scroller:可滚动组件的控制器，用于与可滚动组件进行绑定。

2.List属性

listDirection:设置List组件排列方向，默认值为Axis.Vertical。

scrollBar:设置滚动条状态。

edgeEffect:设置组件的滑动效果，默认值为EdgeEffect.Spring。

divider:设置分割线样式，默认无分割线。

editMode:声明当前List组件是否处于可编辑模式，默认值为filse

3.List事件

onScroll:列表滑动时触发。

onScrollndex:列表滑动时触发。计算索引值时，ListItemGroup作为一个整体占一个索引值，不计算ListltemGroup内部Listltem的索引值。

onReachStart:滚动到起始位置时触发该回调。

onReachEnd:滚动到底部触发该回调。

onltemDelete:当List组件在编辑模式时，点击Listltem右边出现的删除按钮时触发。

onScrollStop:列表滑动停止时触发。

onltemMove:列表元素发生移动时触发。

List下拉列表

4.ListItem

ListItem用来展示列表具体item,它的宽度默认充满List的宽度，它必须配合List使用。构造参数为value:标记位，标记当前item的flag。

5.ListItem属性

sicky:设置ListItem的吸顶效果，默认值为Sicky.None。

editable:当前ListItem元素是否可编辑，进入编辑模式后可删除或移动列表项，默认值为filse。

名称	描述
None	无吸顶效果。
Normal	当前 item 吸顶。
Opacity	当前 item 吸顶显示透明度变化效果。

List下拉列表

```
@Entry @Component struct ListDemo {
  private scroller: Scroller = new Scroller(); // 创建一个滚动控制器
   private items: number[] = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1
14]
  @State editFlag: boolean = false
  build() {
    Column({space:3}) {
      List({initialIndex:0,space: 0,scroller:this.scroller}) {
           ForEach(this.items,(item:any)=>{
             if(item==0){
           ListItem() {                    // List 的子组件只能是 ListItem
             Text("Text:"+item)
               .size({width: "100%", height: 60})
               .fontSize(26).backgroundColor("#aa00ee")
           }.sticky(Sticky.Normal)         // 设置为吸顶效果
         }else{
           ListItem() {                    // List 的子组件只能是 ListItem
             Text("Text:"+item)
               .size({width: "100%", height: 60})
               .fontSize(26).backgroundColor("#aabbcc")
           }.editable(true)
         }
       })
```

完成任务达成的技能目标

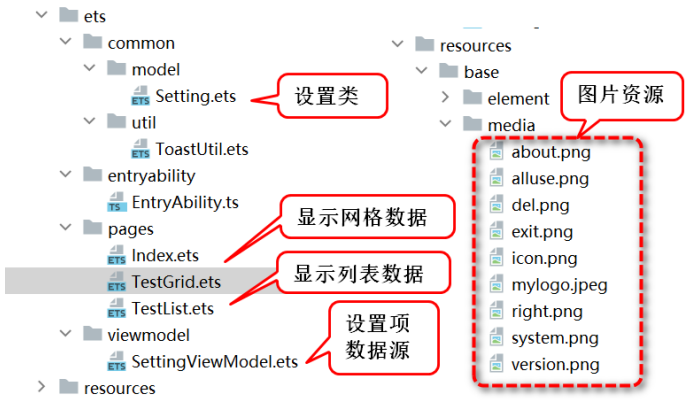
01 任务描述

本任务使用List列表组件开发常见的列表数据展示。

02 技能目标

1. 能使用List组件展示列表数据
2. 能删除列表项的数据

任务实施步骤



1 整理工程资源

2 处理数据源

3 使用List显示列表数据

STEPS

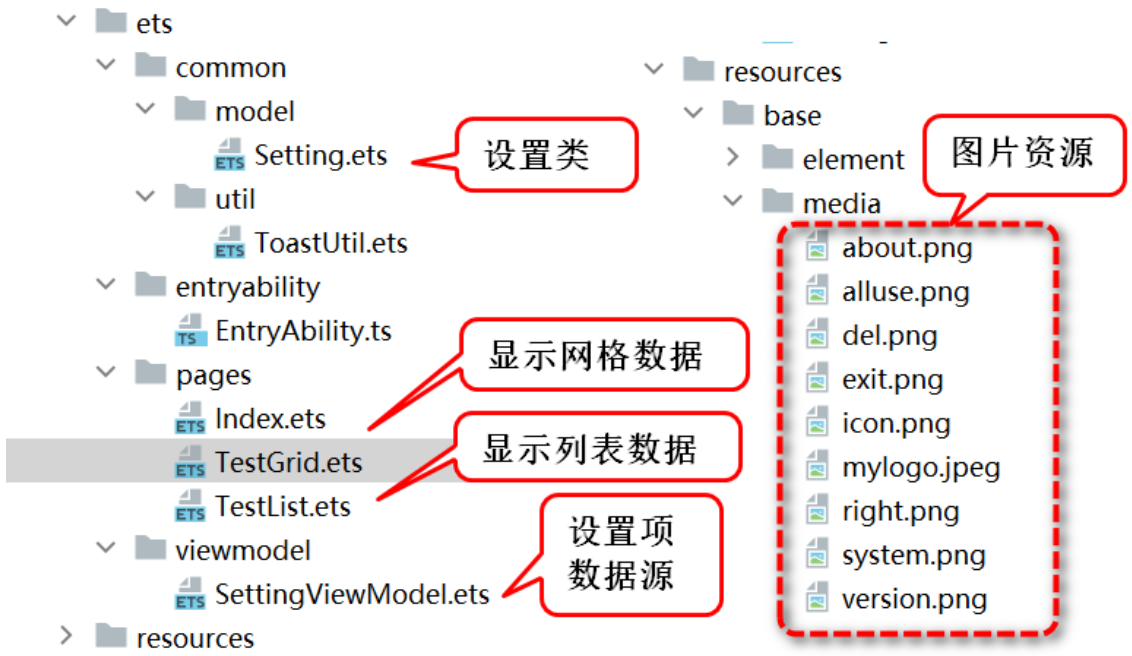
任务实施

01

整理工程资源

整理工程资源

将任务需要的图片资源放到media目录下，创建任务所需的类和文件，将任务4的工具类ToastUtil.ets文件复制并粘贴到工程中。



代码演示

任务实施

02

处理数据源

处理数据源

在ets/common/model/Setting.ets中编写Setting类，类中属性有icon和text，分别用来描述图片和文字，提供构造方法用于初始化数据。

在ets/viewmodel/SettingViewModel.ets中，初始化数据源并用export default进行导出。

代码演示

任务实施

03

使用List显示列表数据

认识List

特点

List是很常用的滚动类容器组件之一，列表包含一系列相同宽度的列表项，适合连续、多行呈现同类数据。

List的子组件

说明

使用List时，通常将List要展示的数据放在数组中，List的子组件必须是ListItem，它的宽度默认充满List的宽度，通常配合ForEach循环从数组中取出数据渲染ListItem子组件。

语法

```
List()
{
 ForEach(数据存放的数组或集合, (item:数组或集合中成员的类型) => {
   ListItem() { //List的子组件
     // ListItem布局
   }…
 })
}
```

List的属性

说明

List列表组件按照水平或者竖直方向线性排列子组件，常用的属性有方向、分割线等。

常用属性

```
List()
 {ListItem子组件}
 .listDirection(Axis.Vertical)//设置子组件竖直方向排列
 .edgeEffect(EdgeEffect.Spring)//滑动到边缘时的弹性物理动效
 .divider({
  strokeWidth: 2,   //设置分割线宽度
  color: Color.Green //设置分割线颜色
 })
```

List的事件

常用事件

```
declare class ListAttribute<T> extends CommonMethod<T> {
 …
 onReachStart(event: () => void): T;//滚动到顶部触发
 onReachEnd(event: () => void): T;//滚动到底部触发
}
```

使用List展示列表数据

要求

在TestList.ets页面中编写代码，实现List列表数据展示和列表项的删除效果。

代码演示

滑块视图容器组件Swiper

Swiper是滑块视图容器，提供子组件滑动轮播显示的能力，Swiper包含的每一个子组件就表示一个页面，Swiper可以绑定一个SwiperController控制器，用来控制上一页或者下一页的翻页，并在事件的处理方法中进行翻页控制。

```
private swiperController: SwiperController = new SwiperController()
Swiper(this.swiperController) {
 //子组件1
 //子组件2
 //子组件3
}
//控制翻页
this.swiperController.showNext()//下一页
this.swiperController.showPrevious()//上一页
```

Swiper轮播

1.Swiper构造参数

controller:给Swiper绑定一个控制器，用来控制页面翻页。

2.Swiper属性

index:设置当前在容器中显示的子组件的索引值，默认值为0。

autoPlay:子组件是否自动播放，自动播放状态下，导航点不可操作，默认为false。

interval:使用自动播放时播放的时间间隔，单位为ms,默认值为3000。

indicator:是否启用导航点指示器，默认为true。

loop:是否开启循环。设置为true时表示开启循环，在LazyForEach懒循环加载模式下，加载的组件数量建议大于5个。默认值为true

duration:子组件切换的动画时长，单位为ms,默认值400。

vertical:是否纵向滑动，默认是false。

itemSpace:设置子组件与子组件之间间隙，默认值为0。

cachedCount:设置预加载子组件个数，默认值为1。

disableSwipe:禁用组件滑动切换功能。默认为false

curve:设置Swiper的动画曲线，默认为淡入淡出曲线，默认值为Curve.Ease。

displayCount:设置一页内元素显示个数，默认值为1

effectMode:滑动效果，默认值为EdgeEffect.Spring。

Curve 枚举说明

名称	描述
Linear	表示动画从头到尾的速度都是相同的
Ease	表示动画以低速开始，然后加快，在结束前变慢，CubicBezier(0.25, 0.1, 0.25, 1.0)
EaseIn	表示动画以低速开始，CubicBezier(0.42, 0.0, 1.0, 1.0)
EaseOut	表示动画以低速结束，CubicBezier(0.0, 0.0, 0.58, 1.0)
EaseInOut	表示动画以低速开始和结束，CubicBezier(0.42, 0.0, 0.58, 1.0)
FastOut-SlowIn	标准曲线，cubic-bezier(0.4, 0.0, 0.2, 1.0)
LinearOut-SlowIn	减速曲线，cubic-bezier(0.0, 0.0, 0.2, 1.0)
FastOut-LinearIn	加速曲线，cubic-bezier(0.4, 0.0, 1.0, 1.0)
Extreme-Decelera-tion	急缓曲线，cubic-bezier(0.0, 0.0, 0.0, 1.0)
Sharp	锐利曲线，cubic-bezier(0.33, 0.0, 0.67, 1.0)
Rhythm	节奏曲线，cubic-bezier(0.7, 0.0, 0.2, 1.0)
Smooth	平滑曲线，cubic-bezier(0.4, 0.0, 0.4, 1.0)
Friction	阻尼曲线，CubicBezier(0.2, 0.0, 0.2, 1.0)

Swiper的属性

常用属性

```
Swiper() {    //子组件...  }
.index(0) //默认显示第几页，默认0
.autoPlay(true) //是否自动播放，默认false ，当设置为自动播放时，导航点无法点击
.interval(1000) //设置自动播放时，播放的时间间隔，单位毫秒，默认 3000
.indicator(true) //是否显示导航点指示器，默认显示
.loop(true) //是否开启循环显示，默认开启
.duration(100) //页面切换的动画时长，单位为毫秒，默认是 400
.vertical(false) //是否竖直翻页，默认是 false
.itemSpace(1) //设置子组件与子组件之间间隙，默认是 0
//页面切换时的回调方法，index显示当前第几页
.onChange((index: number) => {//事件处理  })
//其他属性略
```

代码演示

3.Swiper事件

onChange:当前显示的子组件索引变化时触发该事件，返回值为当前显示的子组件的索引值。Index:当前显示元素的索引。

4.SwiperController控制器

SwiperController是Swiper的页面切换控制器，可以将此对象绑定至Swiper组件上，然后通过它控制翻页。提供方法如下。

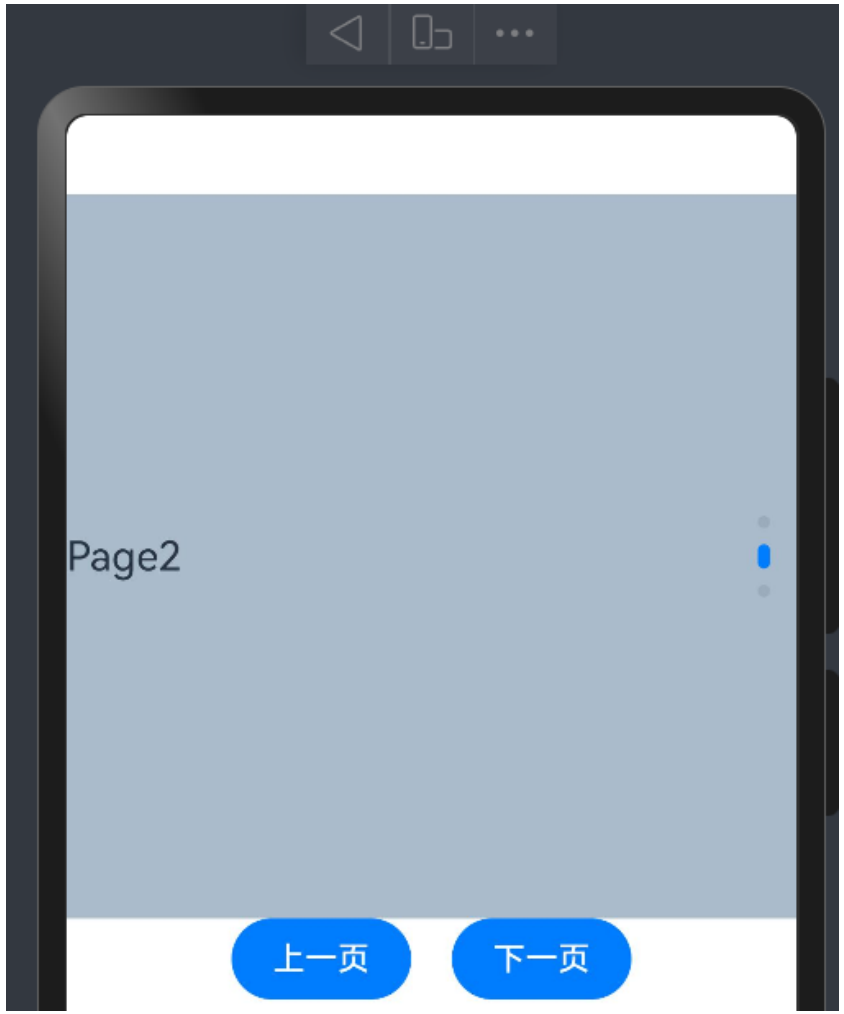
showNext：翻至下一页。

showPrevious：翻至上一页。

finishAnimation：停止播放动画。

Swiper轮播

```
@Entry @Component struct SwiperDemo {
  private controller:SwiperController=new SwiperController()
  build() {
    Column() {
      Swiper(this.controller) {
        Text("Page1")
          .fontSize(20)
          .size({ width: "100%", height: "100%" })
          .backgroundColor("#aabbcc")
        Text("Page2")
          .fontSize(20)
          .size({ width: "100%", height: "100%" })
          .backgroundColor("#aabbcc")
        Text("Page3")
          .fontSize(20)
          .size({ width: "100%", height: "100%" })
          .backgroundColor("#aabbcc")
      }
```



河南大學
Henan University

使用Swriper实现轮播页面

完成任务达成的技能目标

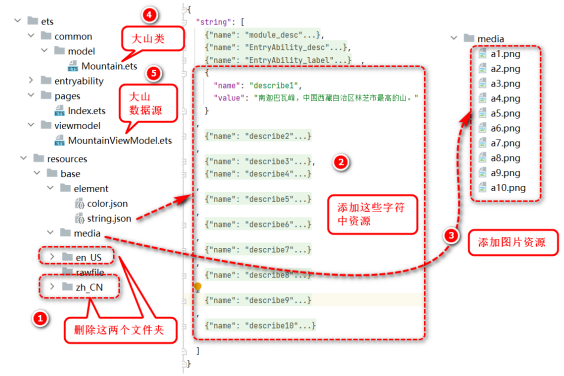
01 任务描述

本任务使用滑块视图容器组件Swiper，配合层叠布局和页面路由，完成页面的轮播功能和跳转到指定的详情页。

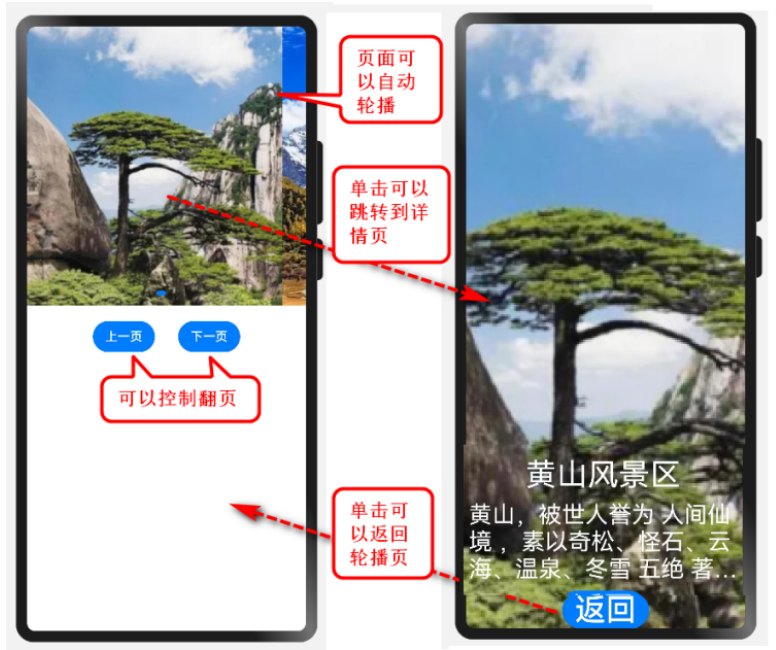
02 技能目标

1. 能正确使用滑块视图容器Swiper
2. 能开发轮播页

任务实施步骤



1 整理工程资源



2 创建名山类

3 处理名山的数据源

4 实现名山图片的轮播功能

5 从轮播页跳转到指定的名山详情页

STEPS

河南大學
Henan University

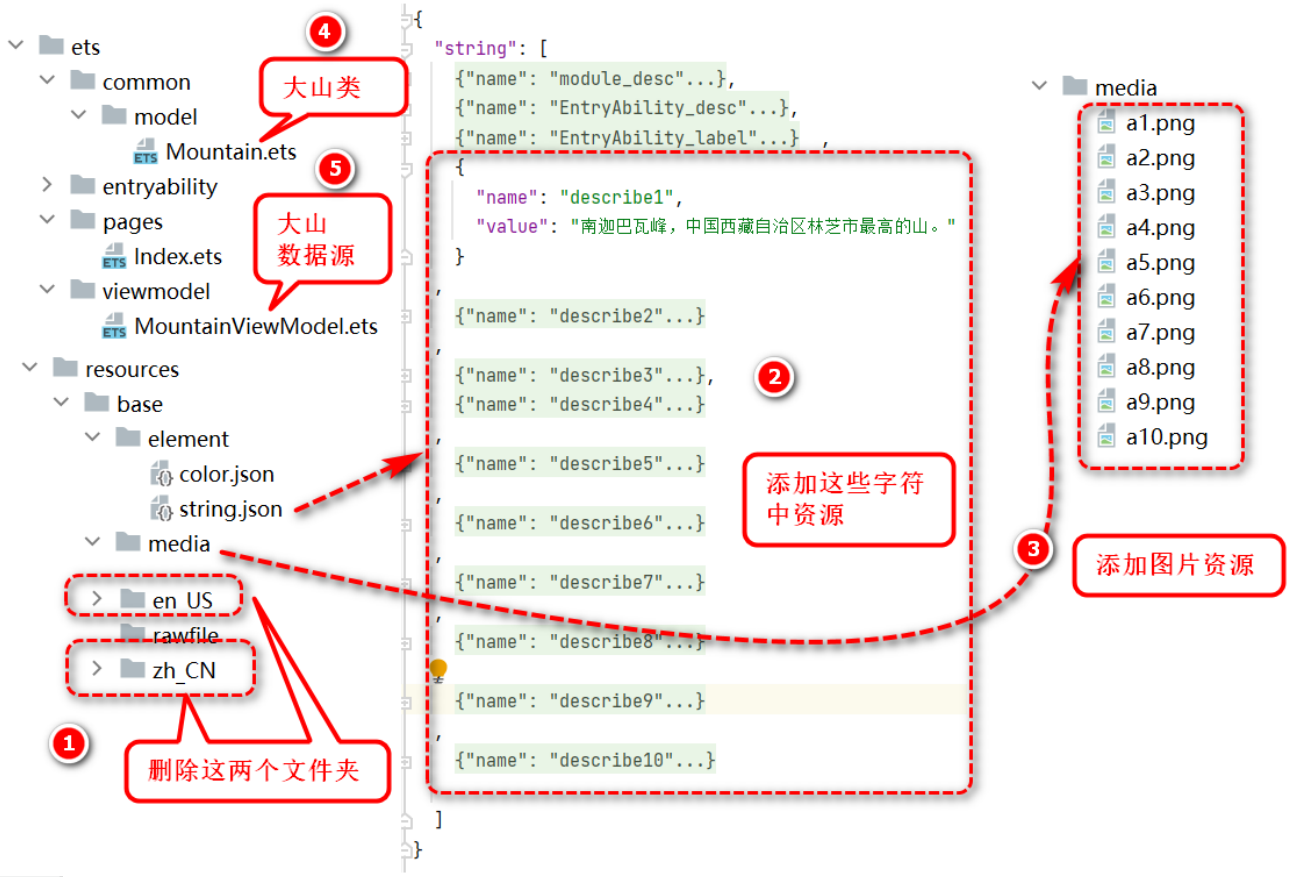
任务实施

01

整理工程资源

整理工程资源

将任务需要的图片资源放到media目录下，在string.json中添加本任务需要的字符串资源，创建大山类Mountain和数据源MountainViewModel。



代码演示

河南大學
Henan University

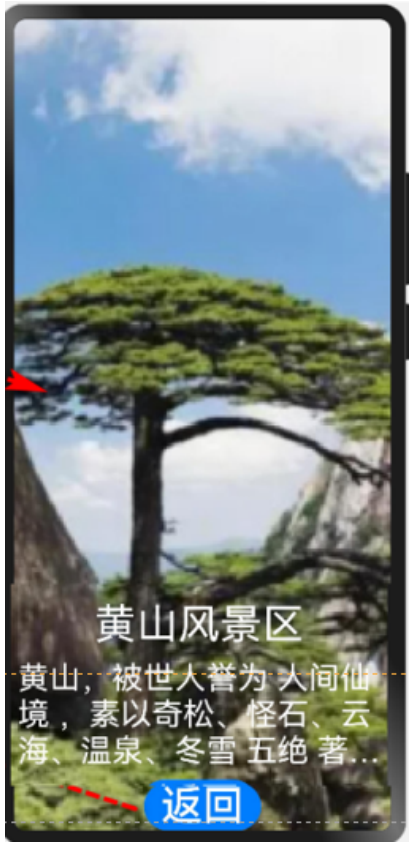
02

创建名山类

创建名山类

分析

Mountain类用来描述山的数据，由山的名字、描述和对应的图片构成，用构造方法初始化数据，用export default导出类。



代码

```
export default class Mountain
{
 public name:string     //名称
 public icon:Resource   //图片
 public describe:Resource //描述
 constructor(name:string,icon:Resource,describe:Resource) {
   this.name = name;
   this.icon = icon;
   this.describe = describe
 }
}
```

河南大學 Henan University

任务实施

03

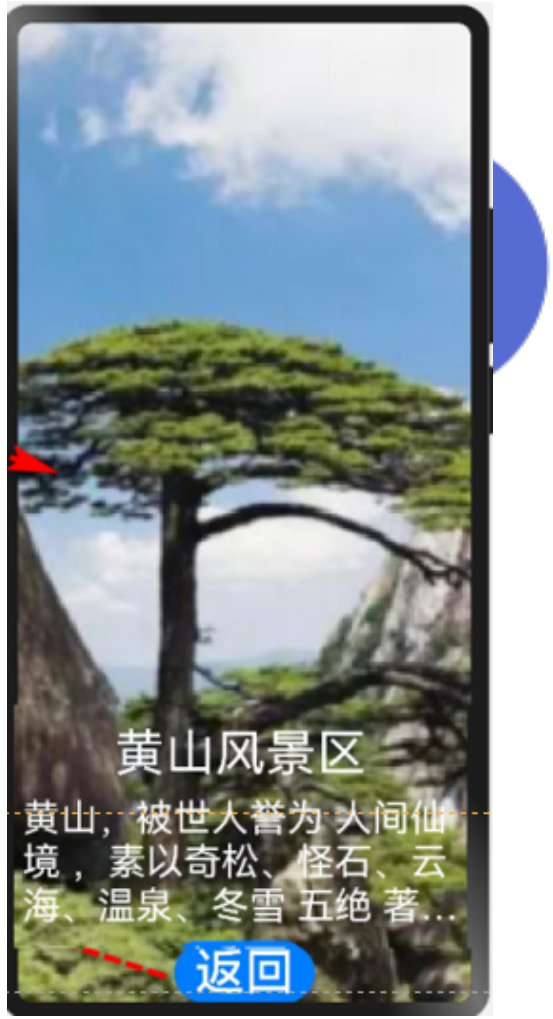
处理名山的数据源

处理名山的数据源

说明

每一个名山都有名字、描述和图片，很多个名山项的数据可以通过数据源进行提供。

定义类MountainViewModel，在类内的构造方法里初始化每个名山对象，并将对象放入数组mountains中，并提供方法getMountains()用于对外提供名山的数据源(就是mountains数组)，最后用export default 导出数据源对象。



代码演示

河南大學
Henan University

任务实施

04

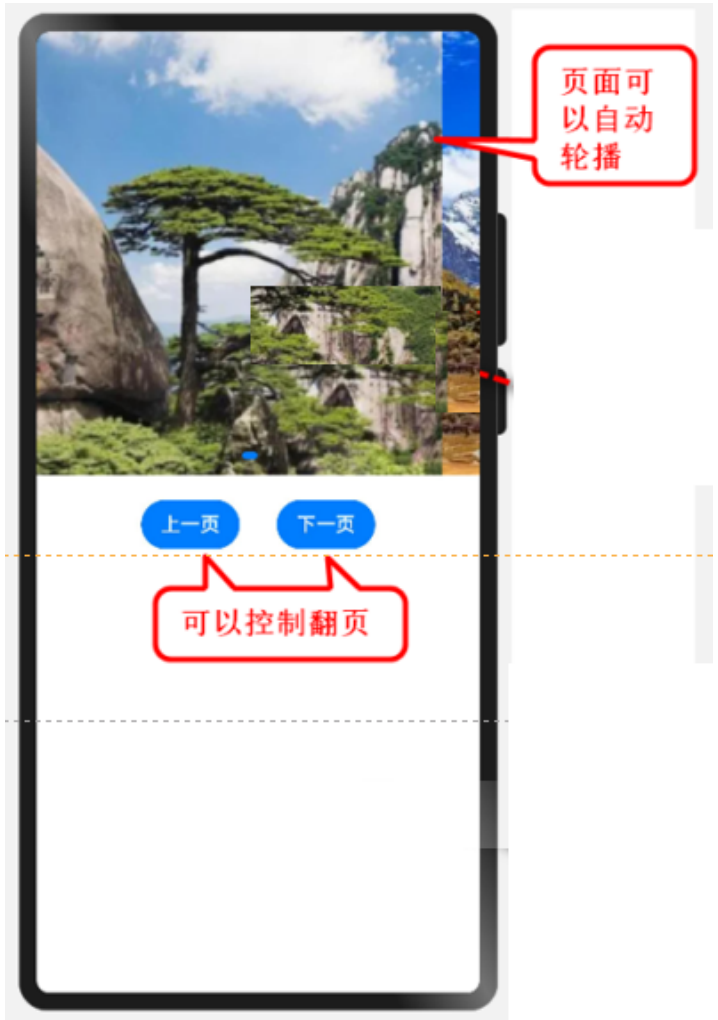
实现名山图片的轮播功能

实现名山图片的轮播功能

过程

在Index.ets页面，导入名山类和数据源；使用Swiper组件实现轮播功能，并给Swiper绑定控制器，实现上下翻页；设置Swiper的属性，实现自动播放等。

代码演示



河南大學
Henan University

05

从轮播页跳转到指定的名山详情页

从轮播页跳转到指定的名山详情页

过程

在Index.ets中导入路由模块，给循环渲染中的Image组件添加事件处理，使用路由模块跳转到pages/Detail详情页，跳转时传递当前轮播页对应的名山对象item。

代码演示

任务小结

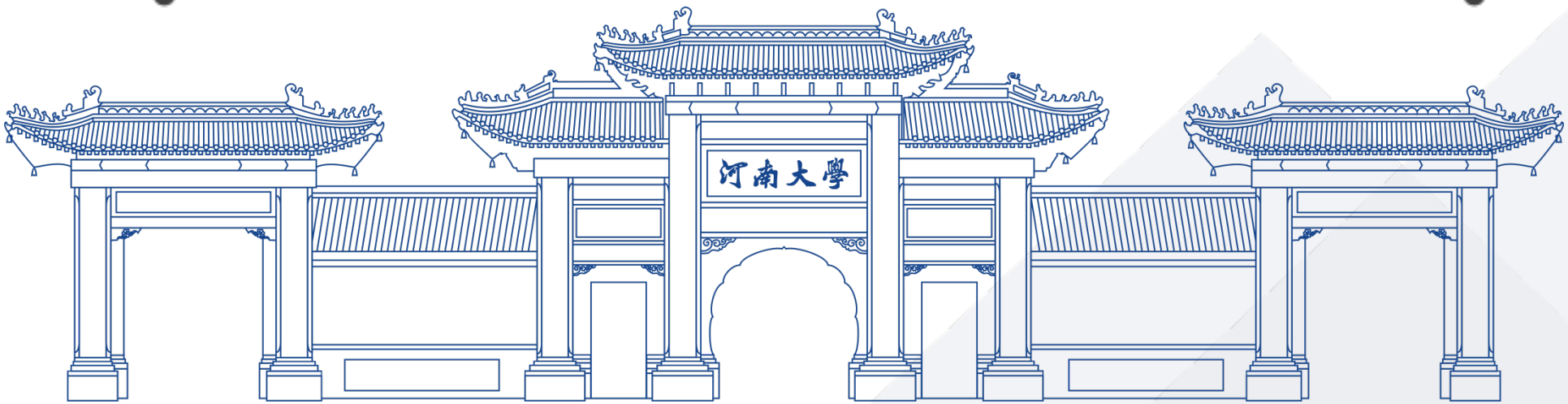
本任务学习了滑块视图容器Swiper的使用，并从类的设计、数据源的设计、路由的跳转和参数传递开始讲起，按正常开发流程进行轮播页面的开发。

轮播页一般用于向用户突出介绍产品的功能和特性，在App的引导页中，也可以使用轮播功能，有兴趣的读者可以自行实现。

河南大學
Henan University

自定义组件

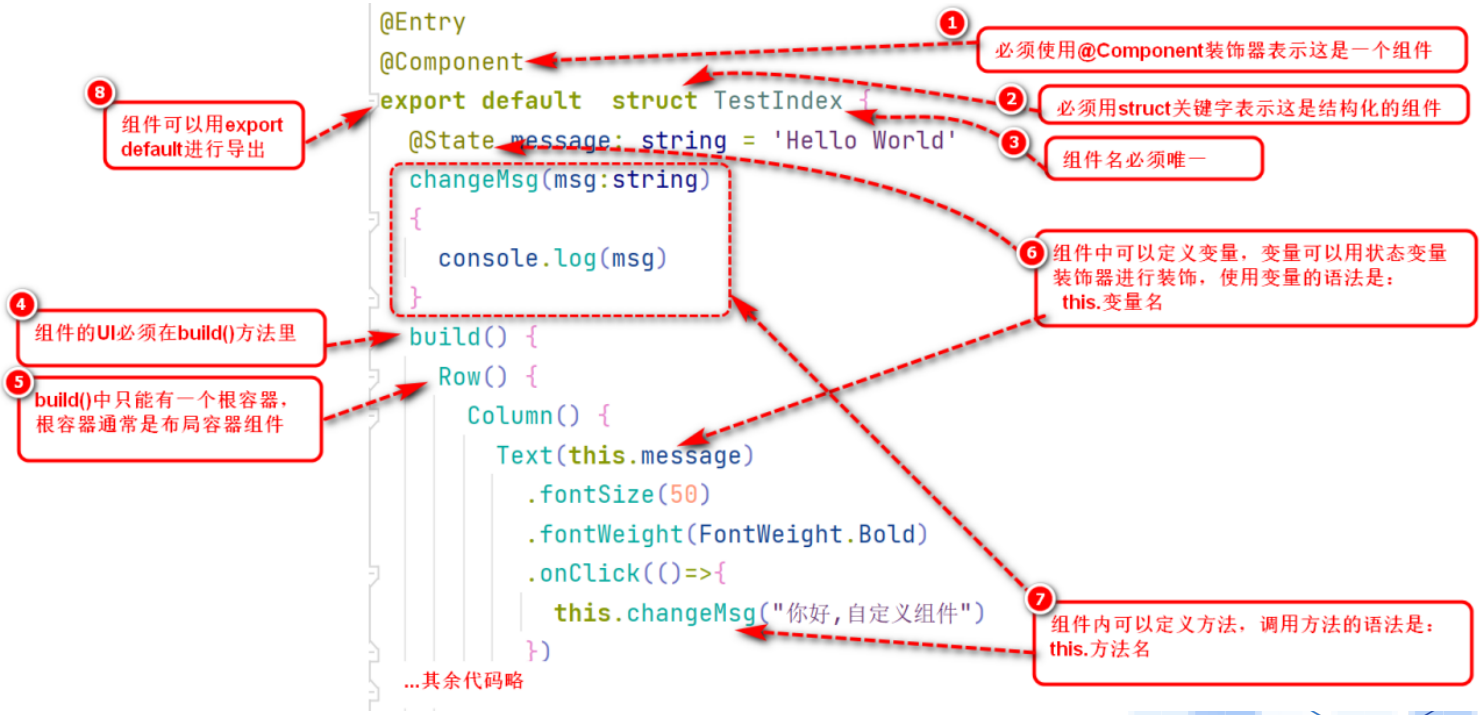
软件学院



自定义组件的使用规则

自定义组件是一个可组合的组件，可以组合使用内置组件、其他组件、公共属性和方法。

页面可以由一个或多个自定义组件组成，一个自定义组件如果用了@Entry装饰，则这个组件是页面的入口组件。在单个UI页面中，最多可以使用@Entry装饰一个自定义组件；如果用了@Preview，则说明这个组件可以预览。



自定义组件的导出和导入

自定义组件可以用export default进行导出，在需要使用自定义组件的页面或其他组件中，使用import 组件名 from ‘带路径的组件名’导入该组件。

```
@Component
export default  struct TestIndex {
```

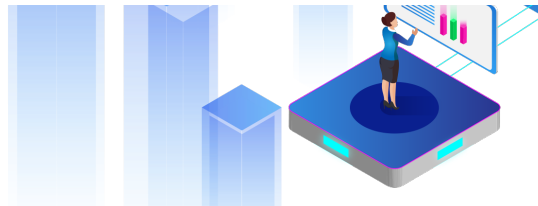
导出自定义组件 ①

```
1  //导入自定义组件,路径要依据实际情况进行修改
2  import TestIndex from './TestIndex'
3  @Entry
4  @Component
5  struct Index {
6    @State message: string = 'Hello Wor
7
8    build() {
9      Row() {
10       TestIndex()//使用自定义组件
11
```

② 导入自定义组件

③ 使用自定义组件

代码演示

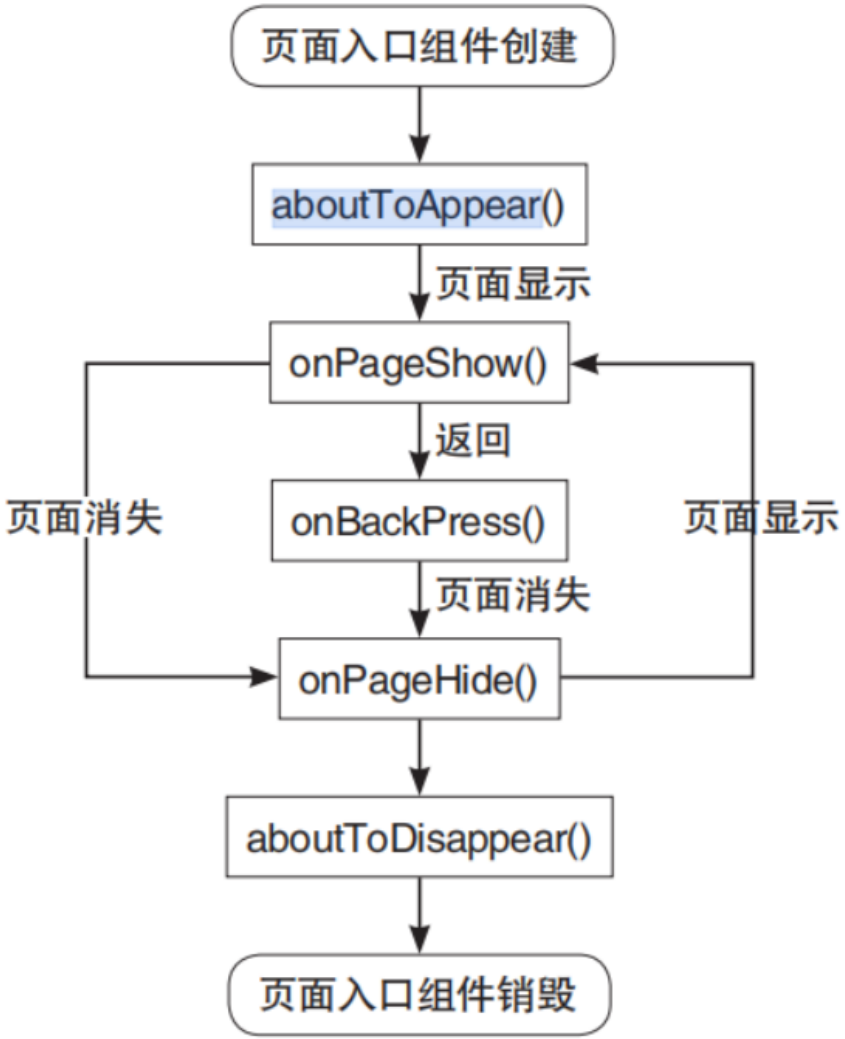


知识储备

02

组件和页面的生命周期

自定义组件插槽

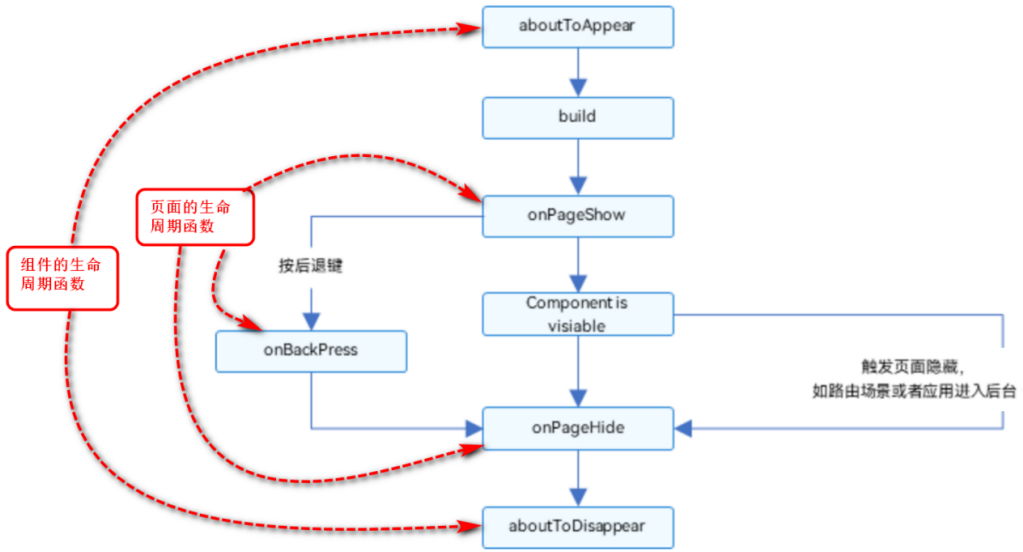


生命周期

组件和页面的生命周期

组件的生命周期函数

- aboutToAppear()：在创建自定义组件的新实例后，回调该接口,在执行其build()函数之前执行。
- aboutToDisappear()：在自定义组件即将析构销毁时执行。



页面的生命周期函数

- onPageShow()：页面每次显示时触发, 包括路由过程、应用进入前后台等场景。
- onPageHide()：页面每次隐藏时触发一次, 包括路由过程、应用进入前后台等场景。
- onBackPress()：当用户点击返回按钮时触发。

知识储备

03

条件渲染语法

条件渲染语法

在ArkTS声明式开发范式中，组件可以使用if/else条件渲染，满足条件就渲染对应状态下的UI内容，条件渲染使用说明如下：

- if/else条件语句可以使用状态变量，并且在更新场景下必须使用状态变量。
- 使用if/else可以使子组件的渲染依赖条件语句。
- 必须在容器组件内使用。
- 某些容器组件限制子组件的类型或数量。

```
@State count: number = 1
...
  if (this.count % 2 == 0) {
    Text("我是自定义组件")
      .fontSize(30)
  }
```

代码演示

知识储备

04

循环渲染语法

循环渲染语法

在ArkTS声明式开发范式中，重复出现的组件可以用ForEach循环来迭代渲染，组件需要的数据放在数组中，ForEach在迭代数组时利用每一数据项创建子组件。

```
ForEach(
 //参数1： arr必须是数组
        arr: any[],
 //参数2：生成子组件的箭头函数，为数组中的每一个数据项创建一个或多个子组件
        itemGenerator: (item: any, index?: number) => void,
 //参数3：匿名函数，用于给数组中的每一个数据项生成唯一且固定的键值
        keyGenerator?: (item: any, index?: number)=>string
)
```

循环渲染示例

```
@State arr: number[] = [10, 20, 30]
...
    ForEach(this.arr,(item:number)=>{
      Text(`item value: ${item}`).fontSize(18)
      Divider().strokeWidth(2).color(Color.Black)
    }, (item: number) => item.toString())
```

代码演示

自定义组件

介绍在ArkUI开发框架中自定义组件的语法规则

先通过示例分别介绍如何使用circle自定义圆和canvas自定义圆，

介绍如何自定义标题栏以及如何使用自定义插槽容器组件。

自定义圆环

自定义圆环的实现非常简单，可以通过组合现有控件或者使用Canvas自绘制的方式来实现一个自定义组件。

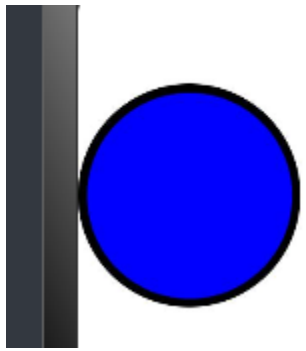
```
@Component export struct Ring {
  build() {
  }
}
```

自定义圆环

实现Ring的布局：

目前通过堆叠的方式实现了Ring组件，但是这个圆环不能设置颜色和大小，因此接下来我们需要定义一下圆环具有的属性。

```
Circle()
 //大圆充满父布局
  .width("100%")
 .height("100%")
Circle()
 //小圆居中显示
  .width(83)
 .height(83)
 .fill(Color.Blue)
```



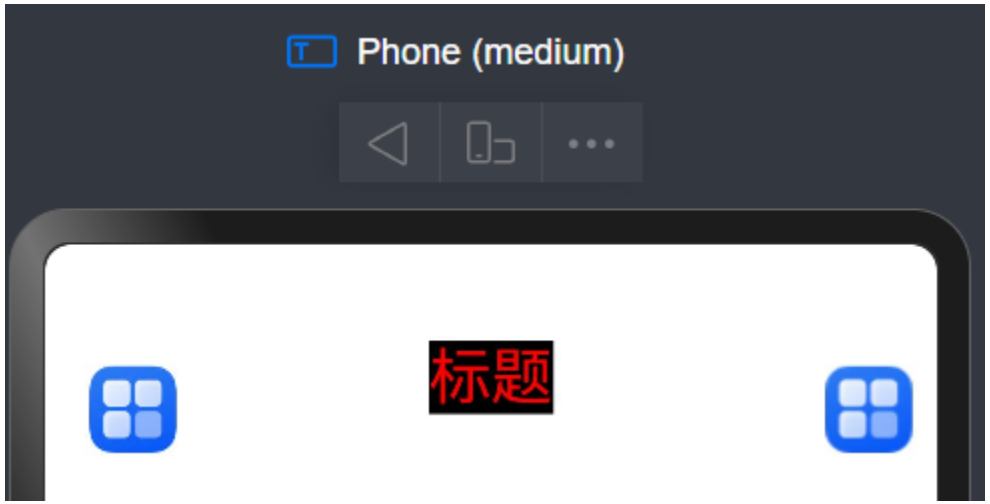
自定义标题栏TitleBar

标题栏在移动端开发中是一个很常用的控件，带有标题栏的页面一般都是分成两部分，上边是标题栏，下边是页面内容；而标题栏的样式一般都是左边是一个返回键，中间是标题，右边是菜单，如图所示。

自定义标题栏TitleBar

为了简单实现这一布局，直接使用Flex组件把它在主轴方向上均分成3份

```
Stack({alignContent:Alignment.Start}){//返回键左对齐
  Image($r("app.media.startIcon"))
   .width(35)
   .height(55)
   .objectFit(ImageFit.Contain)
   .margin({left:18})}
.layoutWeight(1)//使用权重属性均分父布局S
 Stack({alignContent:Alignment.Center}){//标题居中对齐
       Text('标
题').fontSize(25).fontColor(Color.Red).backgroundColor(Color.
用权重属性均分父布局
  .layoutWeight(1)
 Stack({alignContent:Alignment.End}){ //菜单右对齐
       Image($r("app.media.app_icon"))
     .width(35)
     .height(55)
     .objectFit(ImageFit.Contain)
     .margin({        :10})
```



自定义组件插槽

1.@BuilderParam定义

BuilderParam定义在common.d.ts文件中，是一个属性修饰符，修饰的属性类型为0=>0,因此需要在自定义组件CustomWidget内部声明一个属性，比如chid

```
@Component struct CustomWidget {
  @BuilderParam child: () => {}; // 声明属性 child，该属性返回一个对象。
  // 省略部分代码
}
```

自定义组件插槽

2.@BuilderParam使用介绍

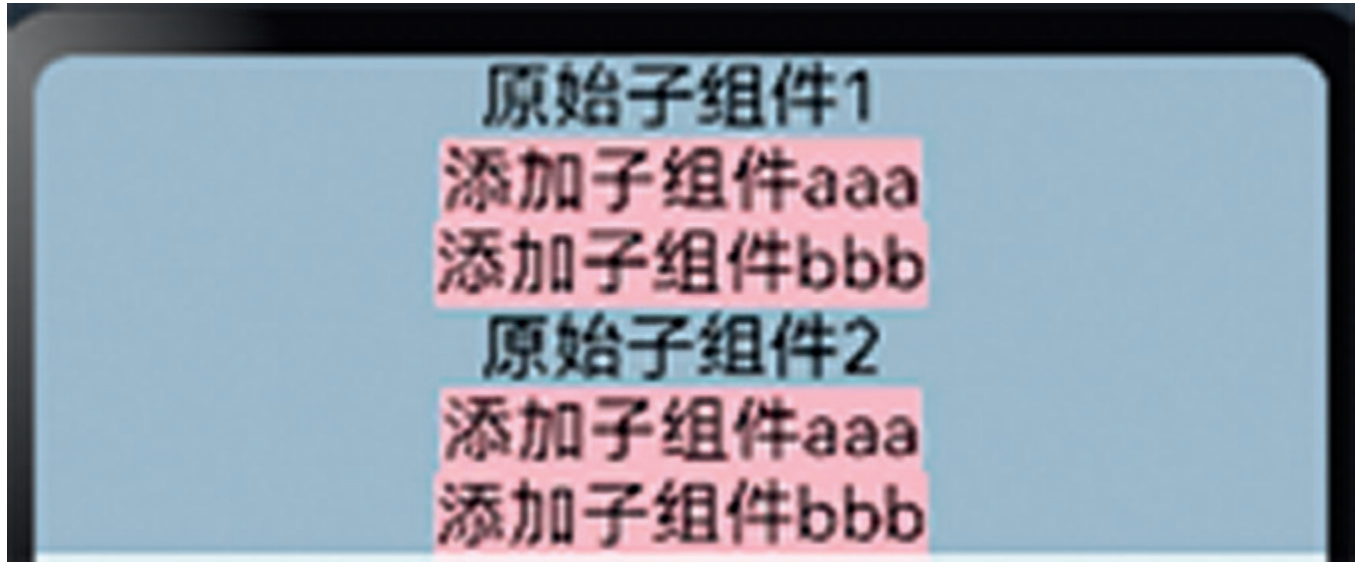
CustomWidget组件添加dhild属性后，表示该组件具备了额外添加子组件的能力，接下来在需要添加子组件的地方使用child属性做占位即可

```
@Component export struct CustomWidget {
 @BuilderParam child: () => {}
 build() {
   Column() {
     Text(" 原始子组件 1")
       .fontSize(20)
     this.child() // 在这里插入额外的子组件
           Text(" 原始子组件 2")
       .fontSize(20)
     this.child() // 在这里插入额外的子组件
       }
   .width("100%")
   .backgroundColor("#aabbcc")
 }
}
```

自定义组件插槽

CustomWidget 组件添加完占位后，就可以直接给其添加子组件了

```
import {CustomWidget} from
"../common/CustomWidget"
@Entry @Component struct CustomWidgetTest
{
 build() {
  Column() {
    CustomWidget() { // 使用 CustomWidget 组
件并给它添加了两个子组件 Text
      Text(" 添加子组件 aaa") // 添加子组件
                .fontSize(20)
        .backgroundColor(Color.Pink)
      Text(" 添加子组件 bbb") // 添加子组件
                .fontSize(20)
        .backgroundColor(Color.Pink)
    }}.width("100%").height("100%")
 }
}
```



河南大學
Henan University

使用自定义组件和渲染组件开发设置页

完成任务达成的技能目标

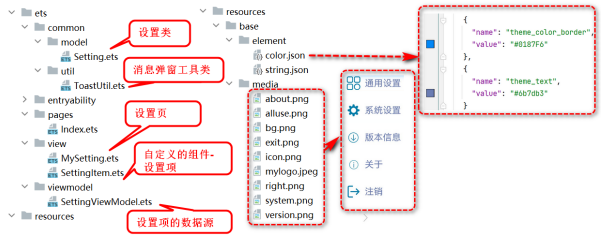
01 任务描述

本任务讲解自定义组件的规则和组件渲染，并进行自定义组件的创建、导出和导入，完成App中常见的设置页的设计与开发。

02 技能目标

1. 能分析需要自定义的组件
2. 能创建自定义组件
3. 能导出和导入组件
4. 能正确传递参数给自定义组件
5. 能正确的使用条件和循环渲染
6. 能完成设置页的设计与开发

任务实施步骤



1 整理工程资源

ST

EP

S

2 开发自定义的设置项组件

3 用循环渲染设置项

4 验证组件和页面的生命周期

河南大學
Henan University

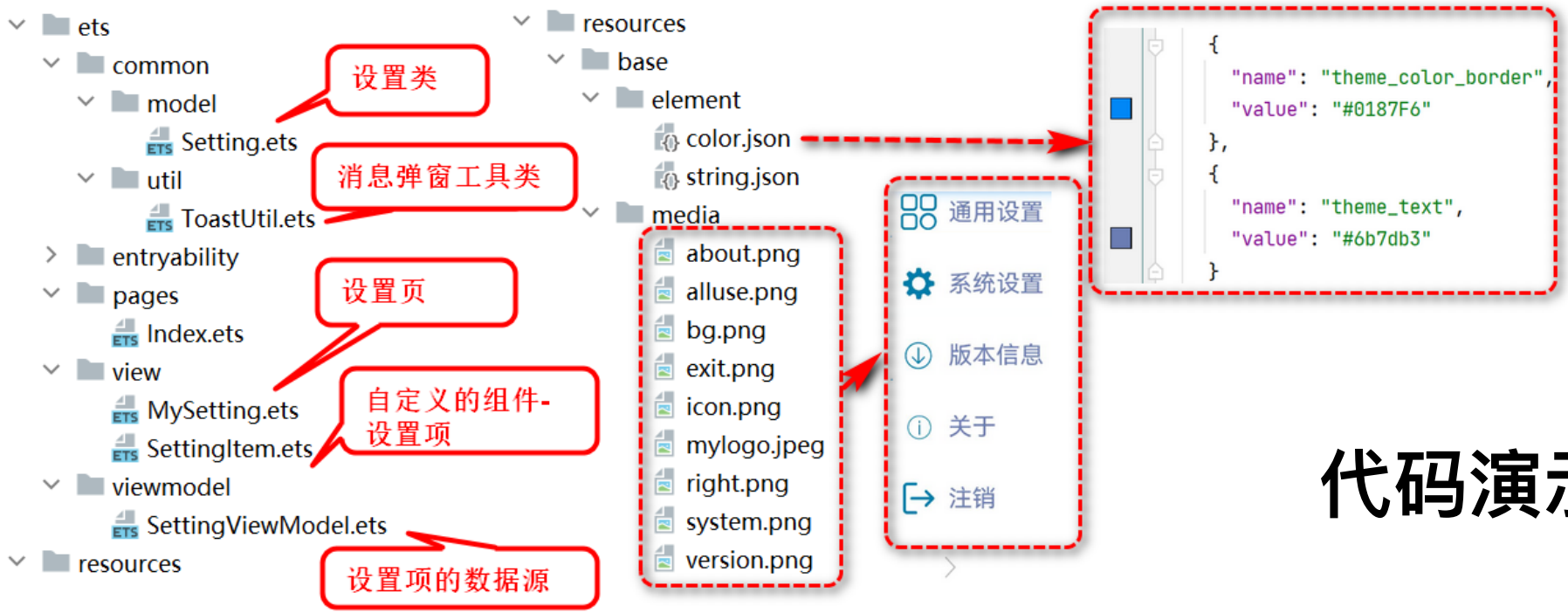
任务实施

01

整理工程资源

整理工程资源

将项目1任务4的工程Project1_Task4进行复制，并重命名为Project2_Task5；在工程Project2_Task5中的color.json中创建颜色资源，将任务需要的图片资源整理到media目录下；在ets目录下创建设置页涉及到的目录和文件，将项目2任务5中的ToastUtil工具类复制到util目录下。



代码演示

河南大學
Henan University

任务实施

02

开发自定义的设置项组件

开发自定义的设置项组件

组件树

每一个设置项的UI，由左边图片、左边文字和右边图片构成，每一项间用分隔线组件隔开。

在ets/view/SettingItem.ets中编写代码，实现自定义的设置项组件SettingItem，变化的左边图片和文字用@State状态变量修饰，用export default导出设置项组件，当文字为注销时，用条件渲染不出现分隔线。

代码演示

河南大學 Henan University

任务实施

03

用循环渲染设置项

用循环渲染设置项

过程

在设置页，导入设置类、设置项的数据源、自定义的设置项组件和消息弹窗工具类；在组件的生命周期函数中获取数据源；用循环渲染自定义的设置项组件，并使用数据源给自定义的设置项组件传递参数。

代码演示

河南大學 Henan University

任务实施

04

验证组件和页面的生命周期

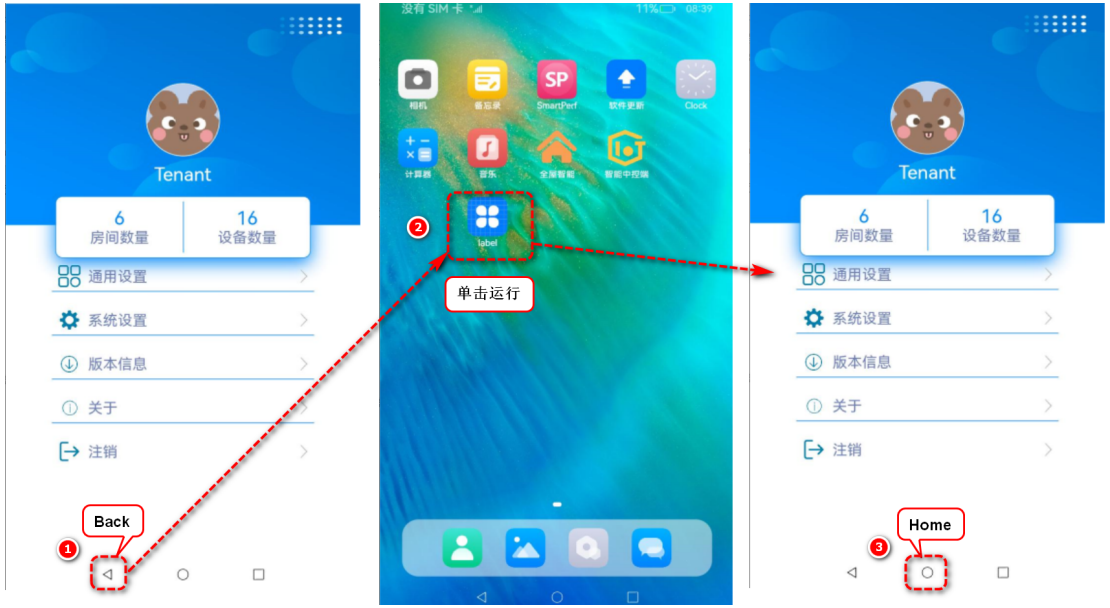
验证组件和页面的生命周期（需修改）

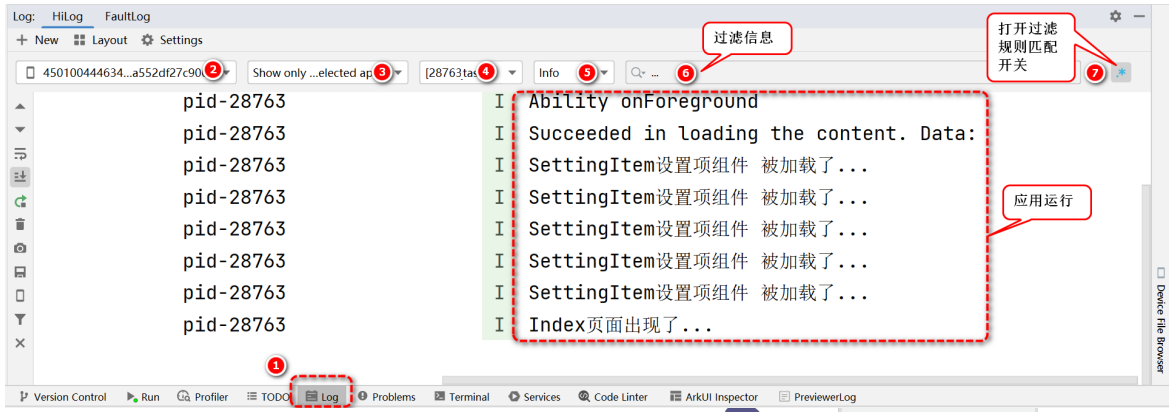
生命周期

修改SettingItem组件的代码，添加组件的生命周期函数；修改页面Index的代码，添加页面的生命周期函数。

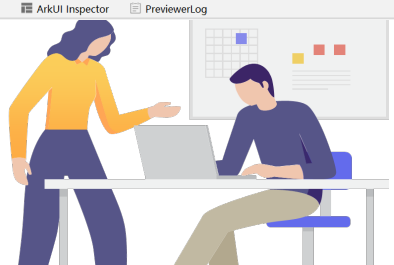
```
SettingItem.ets
@Preview      //预览
@Component
export default  struct SettingItem { //设置项组件
  @State icon: Resource = $r("app.media.version")
  @State name: string = "版本" //默认名称
  aboutToAppear(){
    console.log("SettingItem设置项组件 被加载了...")
    this.icon = $r("app.media.version")
  }
  aboutToDisappear(){
    console.log("设置项组件SettingItem 被卸载了...")
  }
  build() {
```

```
Index.ets
import MySetting from '../view/MySetting'
@Entry
@Component
struct Index {
  onPageShow()
  {
    console.log("Index页面出现了...");
  }
  onPageHide(){
    console.log("Index页面隐藏了...")
  }
  onBackPress(){
    console.log("Index页面,用户按了Back键...");
  }
  build() {
```

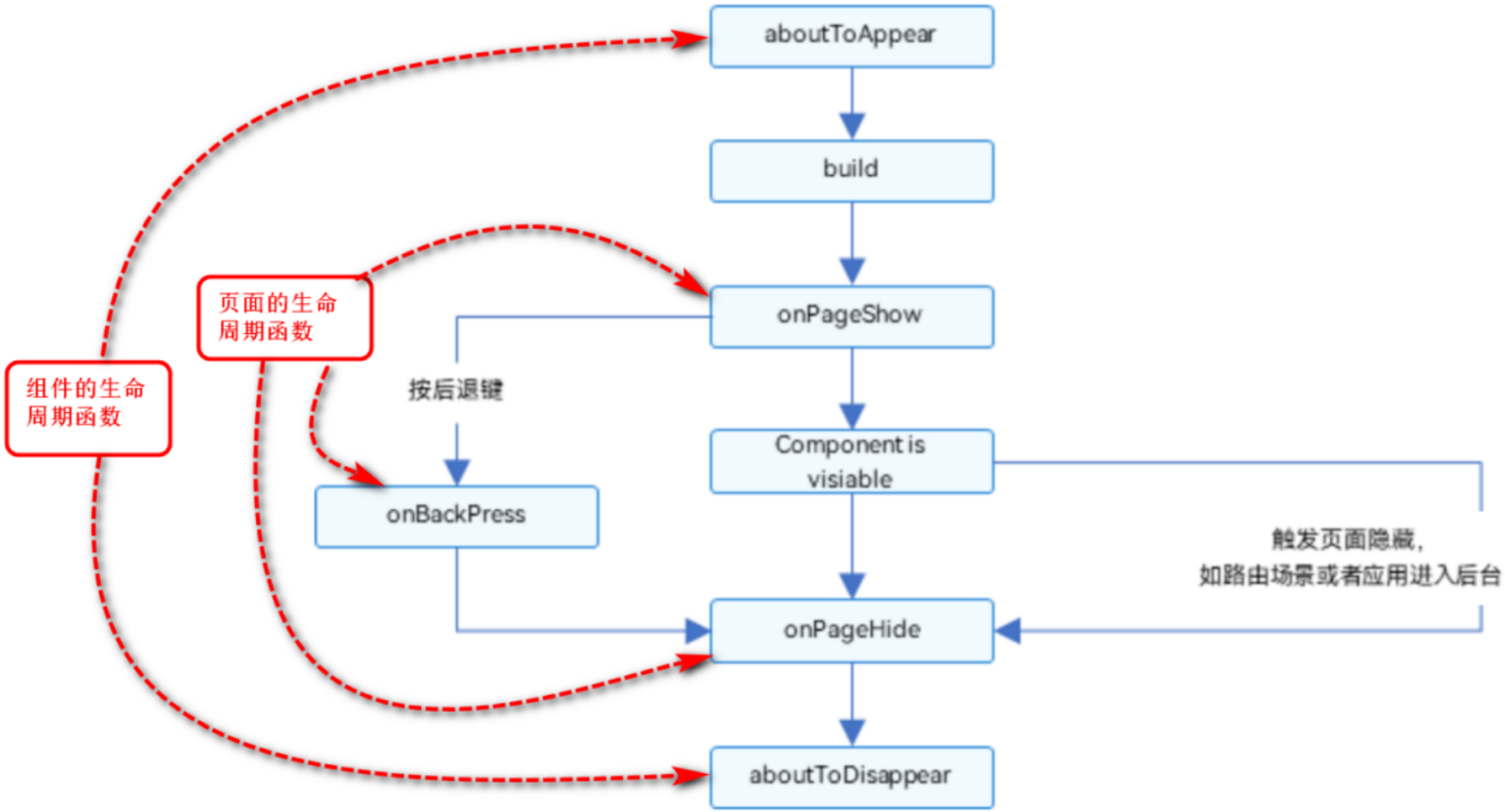




代码演示



验证组件和页面的生命周期



任务小结

本任务主要讲解了ArkTS开发过程中很重要的自定义组件的相关操作，从分析需要自定义的组件开始，到学会抽取自定义组件需要的参数，再到导出和导入组件、调用自定义组件并正确传递参数，同时使用条件和循环进行组件的渲染，同时验证了组件的生命周期函数，完整清晰的把自定义组件的开发流程讲明白了，读者需要好好揣摩其中用法，并在后续的UI设计中进行应用。

组件通用信息

1 常用属性设置

width:设置组件自身的宽度，缺省时使用元素自身内容需要的宽度。若子组件的宽度大于父组件的宽度，则会画出父组件的范围。

height:设置组件自身的高度，缺省时使用元素自身内容需要的高度。若子组件的高度大于父组件的高度，则会画出父组件的范围。

size:设置高宽尺寸。

clip:参数为相应类型的组件，按指定的形状对当前组件进行裁剪；参数为boolean类型时，设置是否按照父容器边缘轮廓进行裁剪。默认值为false。

backgroundColor:设置组件的背景色。

需要注意的是，在设置组件的宽高时，若未指定具体数值，则组件会自动采用其自身内容的宽高。例如，若希望组件充满父组件，可以设置其宽高为”100%"。此外，当同时设置了size和width/height时，组件会以最后设置的值为准。值得注意的是，若子组件的宽高超过了父组件的宽高，子组件会默认绘制出父组件的可视范围。为了避免这种情况，可以使用clip(rue)方法限制子组件不超出父组件的范围。

aspectRatio:指定当前组件的宽高比，aspectRatio=width/height。

padding:设置内边距属性。

margin:设置外边距属性。
请留意，在设置组件的内边距和外边距时，若只设置了一个值，则该值会同时应用于四个方向的边距。此外，当使用PaddingMargin参数类型时，可以单独设置边距。若使用百分比进行设置，则上下左右内外距均以父容器的宽度作为基础值。

组件通用信息

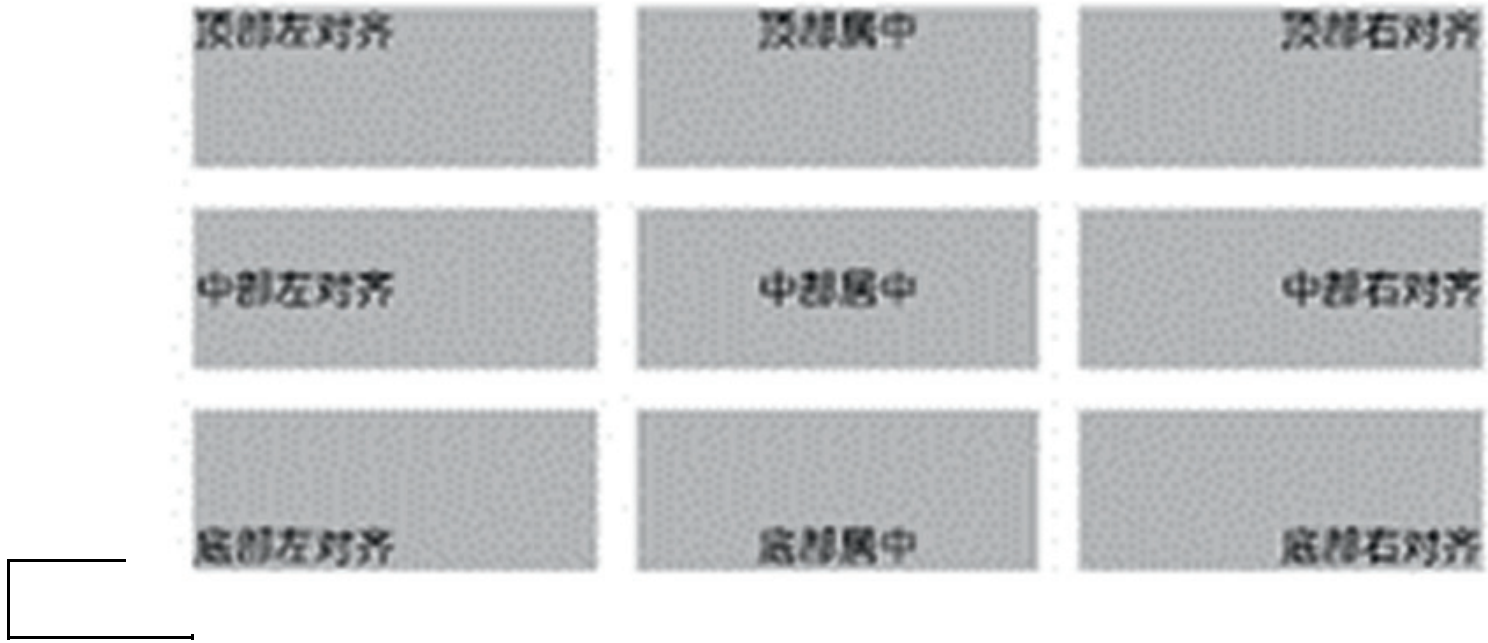
layoutWeight:在父容器尺寸确定的情况下，若子元素设置了layoutWeight属性，则该子元素与其他兄弟元素在主轴尺寸上按照其权重进行分配，而不考虑元素本身的尺寸设置。
这表示该子元素会自适应地占据剩余的空间。需要注意的是，该属性仅在 Row/Column/Flex 布局中生效。

constraintSize：设置约束尺寸，组件布局时，进行尺寸范围限制。constraintSize 的优先级高于 Width 和 Height。若设置的 minWidth 大于 maxWidth，则 minWidth 生效，minHeight与 maxHeight 同理。

align：设置元素内容的对齐方式，只有当设置的 width 和 height 大小超过元素本身内容大小时生效。

组件通用信息

```
Row({space:10}){
 Text(" 顶部左对齐 ")
   .width(100).height(40)
   .backgroundColor("#aabbcc")
   .align(Alignment.TopStart)
 Text(" 顶部居中 ")
   .width(100).height(40)
   .backgroundColor("#aabbcc")
   .align(Alignment.Top)
 Text(" 顶部右对齐 ")
   .width(100).height(40)
   .backgroundColor("#aabbcc")
   .align(Alignment.TopEnd)
}
```



2 其他属性

OpenHarmony还提供其他通用属性，简介如下。

markAnchor:设置元素在位置定位时的锚点，以元素左上角作为基准点进行偏移。通常配合position和offset属性使用，单独使用时，效果类似offset。

Border:统一边框样式设置接口。

backgroundImage:设置背景图片。

backgroundImageSize:设置背景图像的高度和宽度。当输入为{width:Length,height:Length}对象时，如果只设置一个属性，则第二个属性保持图片原始宽高比进行调整。默认保持原图的比例不变，默认值为ImageSize.Auto。

backgroundlmagePosition:设置背景图在组件中显示位置，即相对于组件左上角的坐标。

backgroundBlurStyle:为当前组件提供一种在背景和内容之间的模糊能力，入参为模糊材质。

borderStyle:设置元素的边框样式。

borderWidth:设置元素的边框宽度，不支持百分比。

borderColor:设置元素的边框颜色，默认值为Color Black。

borderRadius:设置元素的边框圆角半径，不支持百分比。

borderlmage:设置图像边框。

vsibility:设置组件的显示和隐藏，默认值为Visbiliy.Visible。

linearGradient:设置组件的渐变样式，参数说明如下：

angle:线性渐变的起始角度。0点方向顺时针旋转为正向角度。默认值为180。

direction:线性渐变的方向。设置angle后不生效。默认值为

repeating:为渐变的颜色重复着色，默认值为6ilse。

grayscale:该函数为当前组件添加灰度效果。其参数值定义了灰度转换的比例，当参数值为1.0时，图像将完全转换为灰度图像；当参数值为0.0时，图像不会发生任何变化；当参数值在0.0和1.0之间时，效果呈线性变化。该参数值的单位为百分比。

enabled:值为true表示组件可交互，响应点击等操作。值为fake表示组件不可交互，不响应点击等操作，默认值为true。

touchable:设置当前组件是否可以响应点击事件、触摸事件等手指交互事件。true表示可以触摸，falbe不可以触摸，默认值为true。

opacity:是指元素的不透明度，取值范围在0到1之间。当取值为1时，元素完全不透明；当取值为0时，元素完全透明。通过设置元素的opacity属性，可以实现隐藏组件的效果，但

组件通用信息

opacity:是指元素的不透明度，取值范围在0到1之间。当取值为1时，元素完全不透明；当取值为0时，元素完全透明。通过设置元素的opacity属性，可以实现隐藏组件的效果，但是该组件在布局中仍然占据着相应的空间。需注意的是，组件具备继承其父组件该属性的能力，默认值为1。

focusable:设置当前组件是否可以获焦，支持焦点控制的组件有Button、Text、Image、List、Grid。

rotute:旋转。

translate:平移。

scale:缩放。

direction:设置元素水平方向的布局，默认值为Direction.Auto。

position:绝对定位，设置元素左上角相对于父容器左上角偏移位置。在布局容器中，设置该属性不影响父容器布局，仅在绘制时进行位置调整。

offset:相对定位，设置元素相对于自身的偏移量。设置该属性，不影响父容

通用事件

1 点击事件

onClick:点击动作触发该回调，ClickEvent对象说明如下。

screenX:点击位置相对于应用窗口左上角的x坐标。

screenY:点击位置相对于应用窗口左上角的y坐标。

x:点击位置相对于被点击元素左上角的x坐标。

y:点击位置相对于被点击元素左上角的y坐标。

组件通用信息

通用事件

```
@Entry
@Component
struct Index {
 @State msg:string=""
 build(){
   Column({space:10}){
     Button("点击事件")
       .onClick((event?:ClickEvent)=>{
         this.msg="相对窗口坐
标"+event.screenX+":"+event.screenY+"\n相
对元素坐标"+event.x+":"+event.y})
     Text(this.msg)}
   .padding(20)
   .size({width:"100%",height:'100%'})
}
}
```

通用事件

2 触摸事件

onTouch:手指触摸动作触发该回调，TouchEvent对象说明如下：

type:触摸事件的类型，TouchType枚举说明见表。

Touches:全部手指信息。

changedTouches:当前发生变化的手指信息。

stopPropagation:阻塞事件冒泡。

名称	描述
Down	手指按下时触发
Up	手指抬起时触发
Move	手指按压态在屏幕上移动时触发
Cancel	触摸事件取消时触发

通用事件

3 拖拽事件

拖拽事件包括dragstart、drag、dagend、dragenter、dragover、dragleave、drop等，开发者可以根据自己的需求选择相应的事件并编写相应的代码逻辑。

onDragStart:第一次拖拽此事件绑定的组件时，触发回调。

onDragEnter:拖拽进入组件范围内时，触发回调。

onDragMove:拖拽在组件范围内移动时，触发回调。

onDragLeave:拖拽离开组件范围内时，触发回调。

onDrop:绑定此事件的组件可作为拖拽释放目标，当在本组件范围内停止拖拽行为时，触发回调。

通用事件

4 焦点事件

onFocus:当前组件获取焦点时触发的回调。

onBlur:当前组件失去焦点时触发的回调。

Button、TextInput等组件具备默认的交互逻辑，这些组件默认状态下是可获得焦点的。而相对的，Text、Image等组件在默认状态下是不可获得焦点的。在不可获得焦点的状态下，无法触发焦点事件，此时需要将focusable属性设置为true,方可触发焦点事件。

组件通用信息

通用事件

5 按键事件

onKeyEvent:绑定该方法的组件获焦后，按键动作触发该方法调用。

type:按键的类型，KeyType枚举见表。

keyCode:按键的键码。

keyText:按键的键值。

keySource:触发当前按键的输入设备类型，KeySource枚举见表。

deviceId:触发当前按键的输入设备ID。

metaKey:按键发生时元键(即Windows键盘的WIN键、Mac键盘的Command键)的状态，1表示按压态，0表示未按压态。

Timestamp:按键发生时的时间戳。

stopPropagation:阻塞事件冒泡传递。

组件通用信息

通用事件

5 按键事件

KeyType 枚举说明

名称	描述
Down	按键按下
Up	按键松开

KeySource 枚举说明

名称	描述
Unknown	输入设备类型未知
Keyboard	输入设备类型为键盘

通用事件

6 鼠标事件

onHover:鼠标进入或退出组件时触发该回调。

isHover:表示鼠标是否悬浮在组件上，鼠标进入时为true,退出时为false。

onMouse:当前组件被鼠标按键点击时或者鼠标在组件上悬浮移动时，触发该回调，event返回值包含触发事件时的时间截、鼠标按键、动作、鼠标位置在整个屏幕上的坐标和相对于当前组件的坐标。

MouseEvent对象说明如下。

screenX:鼠标位置相对于应用窗口左上角的x坐标。

screenY:鼠标位置相对于应用窗口左上角的y坐标。

x:鼠标位置相对于当前组件左上角的x坐标。

y:鼠标位置相对于当前组件左上角的y坐标。

组件通用信息

通用事件

button:鼠标按键，MouseButton枚举说明见表。

action:鼠标动作，MouseAction枚举说明见表。

stopPropagation:阻塞事件冒泡。

timestamp:事件时间戳，触发事件时距离系统启动的时间间隔，单位ns(纳秒)。

target:触发手势事件的元素对象显示区域

sc

MouseButton 枚举说明

名称	描述
Left	鼠标左键
Right	鼠标右键
Middle	鼠标中键
Back	鼠标左侧后退键
Forward	鼠标左侧前进键
None	无按键

组件通用信息

通用事件

MouseAction 枚举说明

名称	描述
Press	鼠标按键按下
Release	鼠标按键松开
Move	鼠标移动
Hover	鼠标悬浮

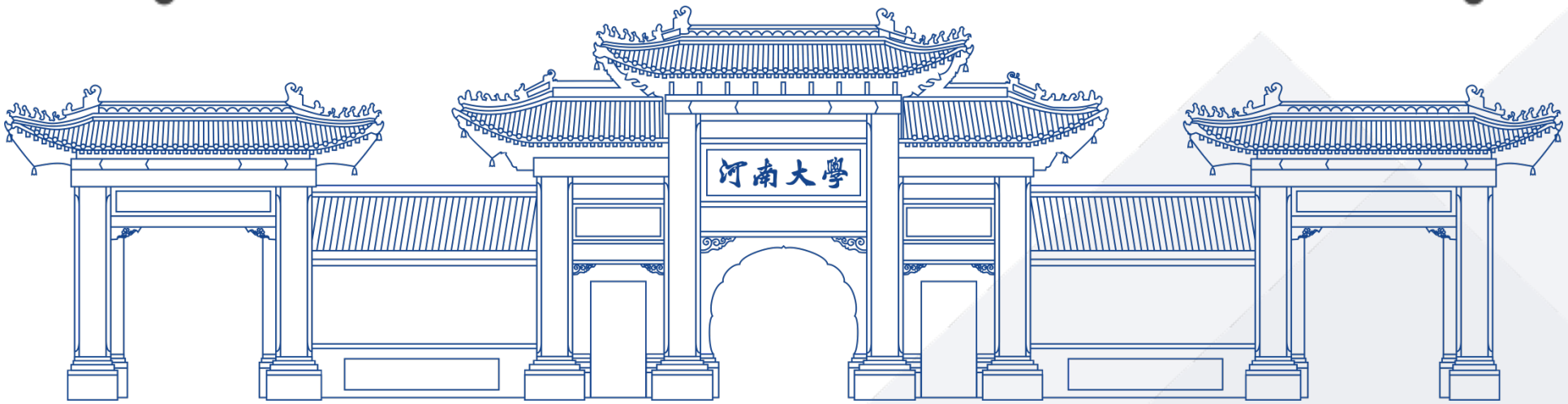
KeyType 枚举说明

名称	描述
Unknown	输入设备类型未知
Mouse	鼠标
TouchScreen	触摸屏

河南大學
Henan University

页面跳转

软件学院



页面跳转

我们将学习arkui框架router页面跳转和传值的方法。本章从定义路由、配置路由、跳转页面以及传递参数等方面进行详细讲解，以帮助读者轻松掌握使用该框架进行页面导航的技巧和方法。在后续的内容中，本章进一步介绍Navigation页面导航的重要性，并讲解使用arkui框架实现导航栏的定义、配置和操作按钮的添加等功能。

页面路由

页面内跳转，直接使用ArkAPI提供的页面路由模块router，使用时需要先导入。

导入

```
import router from '@ohos.router'
```

页面跳转的信息

```
interface RouterOptions {
  //目标页面的路径，该路径必须在pages下配置，否则不起作用
  url: string;
  //可选参数，向目标页面传递参数
  params?: Object;
}
```

页面路由

常用的页面跳转的函数

```
//跳转到应用内的指定页面,异常响应结果为Promise异步方式
 function pushUrl(options: RouterOptions): Promise<void>;
//用应用内的某个页面替换当前页面，并销毁被替换的页面
function replaceUrl(options: RouterOptions): Promise<void>;
//返回上一页面或指定的页面
 function back(options?: RouterOptions): void;
//清空页面栈中的所有历史页面，仅保留当前页面作为栈顶页面
 function clear(): void;
//获取发起跳转的页面往当前页传入的参数
    function getParams(): Object;
//其他函数略
```

页面跳转并传递参数

跳转并传参

```
router.pushUrl({
 url:"pages/DestPage", //目标页面
 params:{              //传递参数
   key1: 'val1',       //参数1
   key2: 'val2'        //参数2
 }
})
```

代码演示

目标页面接收参数

```
@State val1: string = (router.getParams() as Record<string, string>)["key1"]
@State val2: string = (router.getParams() as Record<string, string>)["key2"]
```

知识储备

setInterval()函数

setInterval()函数

定时器模块

ArkTS API中的Timer定时器模块提供基础的定时器能力，支持按照指定的时间执行对应函数。

Timer模块中的setInterval函数用于重复调用一个函数，在每次调用之间具有固定的时间延迟。

setInterval()

```
setInterval(
handler: Function | string, //要重复调用的函数
delay: number, //延迟的毫秒数（一秒等于1000毫秒），函数的调用会在该延迟之后发生
...arguments: any[]) //附加参数，一旦定时器到期，他们会作为参数传递给handler
: number //函数的返回值为intervalID，重复定时器的ID
```

setInterval()函数

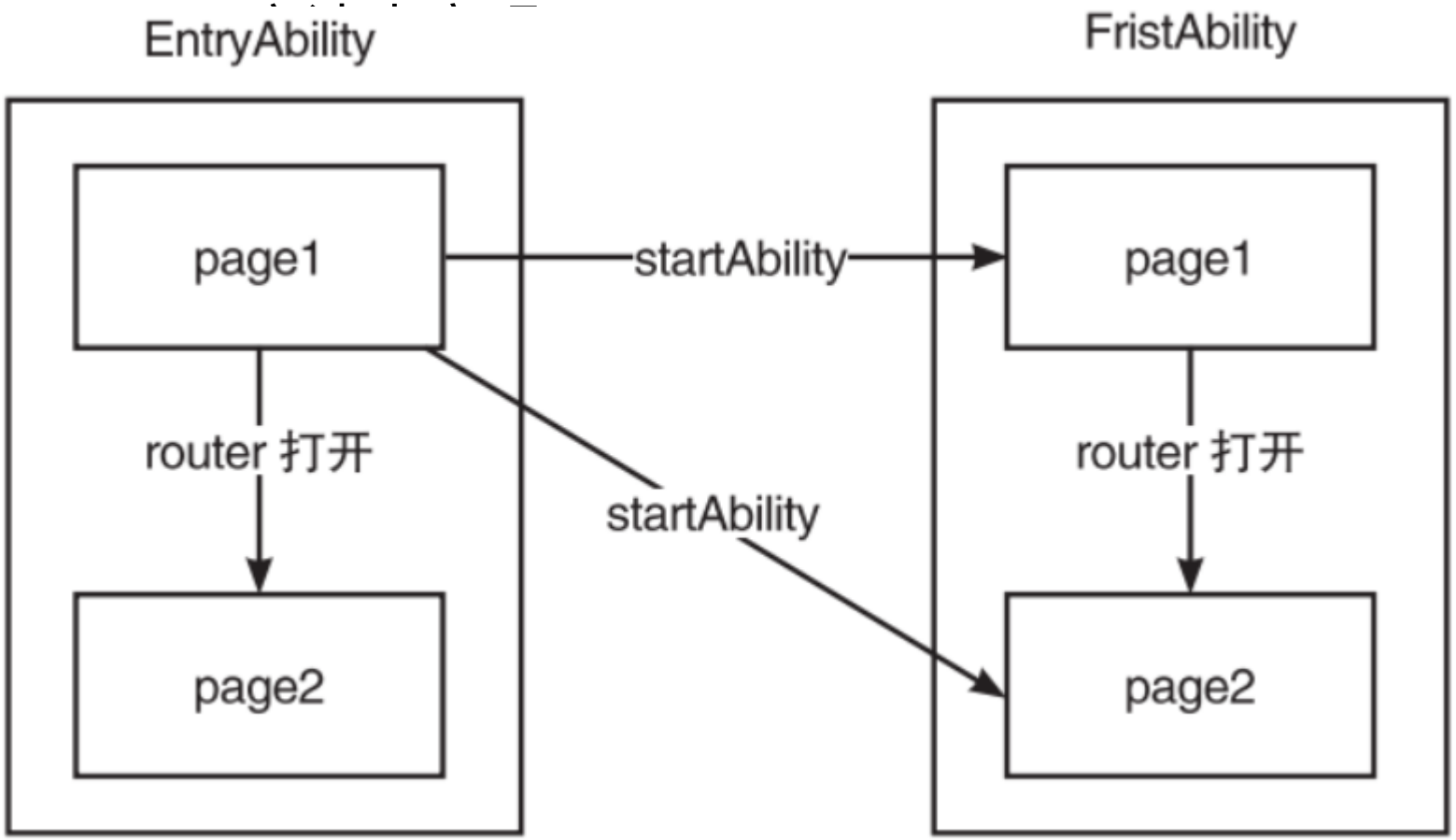
示例

```
//每隔1秒执行一次定时任务
let intervalID =setInterval(()=>{
  //业务代码
},1000)
//停止任务
 clearInterval(this.intervalID)
```

代码演示

页面跳转

在应用程序开发中，页面跳转可以被分为两种类型：页面内跳转和页面间跳转。页面内跳转指的是在同一Ability内部进行的页面跳转，可以通过使用Router或Navigator来实现。而页面间跳转则是指所跳转的页面属于不同的Ability,这种跳转需要借助



页面跳转

```
// 第一个页面
@Entry @Component struct ComponentTest {
  build() {
    Column({space: 10}) {
      Text(" 第一个页面 ")
        .fontSize(30)
        .width("100%")
        .textAlign(TextAlign.Center)
      Button( '打开下一页' )
        .onClick(() => {
          router.pushUrl({
            url: "pages/second",  // 打开新页面
              params: {value: "Hi"} // 给新页面传递一个对象，key 为 value，取值以 .value 的形式
          })
        })
    }
    .size({width: "100%", height: "100%"})
  }
}
```

```
// 第二个页面
@Entry @Component struct Second {
  private value: string = router.getParams()["value"];// 取值
    build() {
    Column({space: 10}) {
      Text(" 第二个页面 :" + this.value) // 使用获取过来的参数
        .fontSize(30)
        .width("100%")
        .textAlign(TextAlign.Center)
      Button(" 返回上一页 ")
        .onClick(() => {
          router.back();              // 返回上一页
        })
    }
    .size({width: "100%", height: "100%"})
  }
}
```

页面跳转

1 Router定义介绍

pushUrl:跳转到应用内的指定页面，新页面在栈顶位置，页面路由栈支持的最大Page数量为32。

replaceUrl:用应用内的某个页面替换当前页面。

back:返回上一页面或指定的页面。

clear:清空页面栈中的所有历史页面，仅保留当前页面作为栈顶页面。

getLength:获取当前在页面栈内的页面数量。

getState:获取当前页面的状态信息。

showAlertBeforeBackPage:开启页面返回询问对话框。

hideAlertBeforeBackPage:禁用页面返回询问对话框。

getParams:获取通过路由传递过来的参数。

页面跳转

参数类型	类型描述	对象属性	属性描述
RouterOptions	跳转页面描述信息。	url	目标页面的路径，该路径必须在 main_pages.json 下配置，否则不起作用
		params	可选参数，向目标页面传递参数
AsyncCallback<void>	异常响应回调。		
RouterMode	跳转页面使用的模式。	Standard	单实例模式。如果目标页面的 url 已经存在于页面栈中，则会将离栈顶最近的同 url 页面移动到栈顶，该页面成为新建页 如果目标页面的 url 在页面栈中不存在同 url 页面，则按照默认的多实例模式进行跳转
		Single	当前页面是第几个打开的。表示当前页面在页面栈中的索引。从栈底到栈顶，index 从 1 开始递增
RouterState	页面状态信息。	name	表示当前页面的名称，即对应文件名
		path	当前页面的路径
EnableAlertOptions	文本弹窗信息描述。	message	询问对话框内容

Router参数对象说明

Navigation页面导航

1 Navigation定义介绍

该组件为路由容器组件，提供路由跳转能力。默认情况下，该组件的导航样式为一个带有返回箭头的标题栏及其子控件组成的组件。

2 Navigation属性介绍

title:设置导航栏的标题，当参数类型为string时，可以直接设置标题，但样式不可修改；当参数为CustomBuilder时，可以自定义标题样式。

subTitle:设置页面的副标题。

hideBackButton:隐藏返回键，若为true则隐藏返回键，若为false则显示返回键。默认值为false。

toolBar:设置工具栏内容。当参数类型为object时，可以直接设置工具栏选项，但样式不支持修改；当参数为CustomBuilder时，可以自定义标题栏。

hideTitleBar:隐藏标题栏。

hideToolBar:隐藏工具栏。

menus:设置标题栏右上角的菜单项，参数类型有NavigationMenuItem和CustomBuilder，当参数为CustomBuilder时可以自定义菜单项。

onTitleModeChange:当titleMode为Navigation TitleMode.Free时，随着可滚动组件的滑动标题栏模式发生变化时触发此回调。

3 Navigation事件介绍

Navigation页面导航

```
@Entry @Component struct ComponentTest {
  @Builder title() {// 通过 Builder 自定义标题栏，可以灵活的设置标题样式
    Row() {
      Text("Builder 标题")
        .fontSize(20)
    }
    .width("100%")
    .height(60)
    .backgroundColor(Color.Yellow)
  }
  build() {
    Navigation() {
      Text("title")
        .textAlign(TextAlign.Center)
        .fontSize(30)
        .size({width: "100%", height: "100%"})
        .backgroundColor("#aabbcc")
    }
    .size({width: "100%", height: "100%"})
    .title(this.title())// 使用自定义的标题栏
  }
}
```